

# “华为杯”第二十二届中国研究生 数学建模竞赛

题 目： 江南古典园林的美学特征建模

## 摘 要：

江南古典园林以“移步异景”“小中见大”“有法无式”为核心的美学特质，长期以来多依赖主观描述与感性解读进行研究，缺乏可量化的分析框架与标准化的数字化表达体系，导致其蕴含的造园智慧难以形成系统性理论支撑，也制约了传统技艺在数字化时代的传承与创新转化。本文针对江南古典园林隐性美学规律的量化解构这一核心问题，以图论、景观生态学、空间形态学与机器学习理论为核心支撑，选取明清江南代表性园林为研究对象，依托多模态数据构建涵盖路径规划、空间感知、美学评价的多层次建模体系，并综合运用多类先进算法与技术方法完成模型求解与验证。

针对问题一“移步异景”的游赏趣味性建模：首先构建基于 Voronoi 图与空间索引优化的路径刻画模型，通过收集障碍物边界点、生成外边界约束等方式，解决园林中障碍物线段不连续、难以形成封闭通行区域的技术难题；进而利用空间索引快速筛选相交障碍物并过滤无效边，结合连通分量分析筛选最大连通子图，通过分支修剪策略保护出入口节点并迭代移除冗余枝杈，最终构建符合实际游赏习惯的路径网络。在此基础上，基于射线探测技术构建视景矩阵，以模拟人眼视线高度发射多条射线，按固定间距划分探测节点，精准还原每个路径节点的可视景观构成。为量化异景程度，构建了基于时空融合的异景程度模型：时间维度通过马尔科夫链构建多类景观状态转移概率矩阵，以小概率转移特征量化景观变化的意外性；空间维度采用汉明距离衡量相邻节点视景向量的相似性，刻画空间邻域的景观差异。最后采用改进的 NSGA-II 遗传算法实现“最大化异景程度、最小化路径重复长度”的多目标游线规划，通过先验路径初始化、步骤选择序列编码、非支配排序等策略，生成覆盖不同游客偏好的帕累托最优路径集合。此外，游园路径上单位转折点密度与异景程度呈强正相关，符合“小空间多视角转换则移步异景效果显著”的感性认知。

针对问题二“小中见大”的布局幻境感建模：建立元素分布特征模型与开合变化模型。元素分布建模中，通过局部矩形扫描法对园林空间进行射线探测与区域填充，将多模态景观元素数据转化为标准化栅格矩阵，经分割块分割与栅格细化处理后，利用连通域识别算法聚合同类型景观斑块，进而构建包含多样性、聚集度、蔓延度、边缘密度等多类景观生态学指标的量化体系，刻画元素分布对空间审美的影响。开合变化建模则融合空间形态学与空间句法方法：空间形态学通过平均最近邻距离、围合比指数等指标量化静态开合特征；空间句法借助专业软件计算连接度与标准化连接度，捕捉视线通透度这一非几何感知特征，最终建立开阔空间与围合空间的明确量化判别标准。基于上述模型，采用层次加权法将元素分布与开合变化两维度进行融合，以寄畅园为基准完成其余园林的幻境感标准化评分与

排序，瞻园、绮园因元素丰富和开合对比强烈排名靠前，秋霞圃则因空间分割细碎排名靠后，评分结果与园林造园水平及风格特征形成呼应。

针对问题三“有法无式”的园林相似度建模：构建基于特征金字塔的园林多模态向量表征模型，融合游园路径网络特征、园林幻境感专家特征、栅格局部特征聚合及栅格空间关系特征四大层次。通过卷积自编码器深度提取栅格空间特征，实现园林景观微观构成、中观关系与宏观布局的立体化表征。基于特征向量开展相似度分析，拆解不同特征层次的相似性表现，进而提炼出了围合度、紧凑度等江南古典园林的强共性美学特征。选取“近园”为例进行模型泛化验证，检验体系的普适性与准确性；同时提出个性化差异度量化方法，分析不同园林的个性化指数及贡献构成，通过个性化指数量化园林间的差异度，区分出经典传统型、空间创新型等不同风格类型，揭示了江南园林“和而不同”的美学本质与“守正创新”的设计智慧。此外，结合造园理论与实践，进一步探讨文化意象密度、生态和谐性等具有发展潜力的美学特征评价维度及其刻画依据。

最后，模型综合评价表明：所建立的多层次美学特征建模体系实现了从定性描述到定量建模的根本性转变，具有理论基础扎实、方法创新突出、结果科学可信的显著优势。通过相关性检验、学术评价比对、特征稳定性分析等验证环节，证实模型可有效解构江南古典园林的隐性造园规律，且在不同园林样本中的普适性与准确性均达到应用标准，量化揭示了江南园林在围合度、紧凑度、多样性等核心指标上的精确设计控制标准。该模型体系在园林数字化保护与修复的精准施策、智慧旅游个性化导览路线的智能生成、现代园林设计的传统智慧赋能、园林美学理论的可视化教学等领域均具有广阔的应用前景，为江南古典园林美学规律的科学阐释和文化遗产的数字化传承提供了重要的理论支撑与技术手段，对推动中华优秀传统文化的创新性发展与创造性转化具有重要的现实意义和深远的历史价值。

**关键词：**江南古典园林；美学特征；Voronoi 图；时空融合；多目标优化；卷积自编码器；多模态向量；守正创新

# 目录

|                                       |    |
|---------------------------------------|----|
| 一、 问题重述 .....                         | 6  |
| 1.1 问题背景 .....                        | 6  |
| 1.2 问题提出 .....                        | 6  |
| 二、 问题分析 .....                         | 6  |
| 2.1 问题一分析 .....                       | 6  |
| 2.2 问题二分析 .....                       | 7  |
| 2.3 问题三分析 .....                       | 8  |
| 三、 模型假设与符号说明 .....                    | 9  |
| 3.1 模型基本假设 .....                      | 9  |
| 3.2 符号说明 .....                        | 10 |
| 四、 问题一的求解：“移步异景”的游赏“趣味性”路线规划 .....    | 11 |
| 4.1 问题分析 .....                        | 11 |
| 4.1.2 路径刻画：基于景观元素坐标的图结构转化 .....       | 11 |
| 4.1.3 异景程度：基于时空维度的景观变化量化 .....        | 12 |
| 4.1.4 游线规划：基于多目标优化的路径权衡 .....         | 12 |
| 4.2 基于 Voronoi 图和空间索引优化的路径刻画模型 .....  | 13 |
| 4.2.2 基于 Voronoi 图的路径转化模型 .....       | 13 |
| 4.2.3 基于空间索引的路径优化模型 .....             | 14 |
| 4.3 基于时域空域分析的异景程度模型 .....             | 16 |
| 4.3.2 基于射线探测的视景矩阵模型 .....             | 16 |
| 4.3.3 基于马尔科夫链的时间域异景程度 .....           | 18 |
| 4.3.4 基于领域一致性的空间域异景程度 .....           | 19 |
| 4.3.5 时空结合的异景程度模型求解与趣味性量化 .....       | 19 |
| 4.4 基于 NSGA-II 算法的多目标优化“游线规划”模型 ..... | 24 |
| 4.4.2 基于 NSGA-II 算法的多目标优化模型建立 .....   | 24 |
| 4.4.3 基于 NSGA-II 算法的多目标优化模型求解 .....   | 25 |
| 4.4.4 游园路线可视化展示与关键位置异景程度分析 .....      | 30 |
| 4.4.5 其他园林游园路线规划的可视化展示与分析 .....       | 33 |
| 4.5 问题一模型检验与评估 .....                  | 43 |
| 4.5.1 异景程度模型检验评估 .....                | 43 |
| 4.5.2 多目标优化模型检验与评估 .....              | 47 |
| 五、 问题二的求解：“小中见大”的布局“幻境感”评价 .....      | 47 |
| 5.1 问题分析 .....                        | 47 |
| 5.1.2 元素分布：基于多模态数据的空间结构量化 .....       | 48 |
| 5.1.3 开合变化：“开阔-围合”的动态转换刻画 .....       | 48 |

|                                             |    |
|---------------------------------------------|----|
| 5.1.4 幻境评分：双维度融合的综合评价体系构建 .....             | 49 |
| 5.2 基于视线投影与景观生态学的元素分布特征模型 .....             | 49 |
| 5.2.2 基于视线投影方法填充感知颜色模型 .....                | 50 |
| 5.2.3 基于景观生态学指标的元素分布特征分析模型 .....            | 56 |
| 5.3 基于空间形态学的园林空间开合变化模型 .....                | 59 |
| 5.3.2 核心指标定义与计算公式 .....                     | 59 |
| 5.3.3 园林空间开合变化模型求解 .....                    | 61 |
| 5.4 基于空间句法的园林空间开合变化模型 .....                 | 65 |
| 5.4.2 空间句法概述与研究对象概况 .....                   | 66 |
| 5.4.3 基于空间句法的空间开合分析 .....                   | 67 |
| 5.4.4 基于空间句法的园林开合变化分析 .....                 | 68 |
| 5.4.5 “开阔”与“围合”空间的量化标准 .....                | 70 |
| 5.5 基于景观生态学和空间形态学的幻境度评价模型 .....             | 71 |
| 5.5.2 幻境度评价模型的建立 .....                      | 72 |
| 5.5.3 幻境度评价模型的求解 .....                      | 72 |
| 5.5.4 幻境度评价模型结果与可视化分析 .....                 | 73 |
| 5.6 问题二模型检验与评估 .....                        | 76 |
| 5.6.1 指标体系的科学性验证 .....                      | 76 |
| 5.6.2 建模方法的互补性验证 .....                      | 76 |
| 5.6.3 评价结果的贴合性验证 .....                      | 76 |
| 5.6.4 基准校准的合理性验证 .....                      | 77 |
| 六、 问题三的求解：“有法无式”的园林“相似度”建模 .....            | 77 |
| 6.1 问题分析 .....                              | 77 |
| 6.1.2 相似度刻画：基于多维度量指标的园林共性规律提取 .....         | 77 |
| 6.1.3 广效用验证：基于新园林数据的模型泛化性检验 .....           | 78 |
| 6.1.4 多元化美学特征挖掘：基于造园理论与实证数据的美学维度补充 .....    | 78 |
| 6.2 基于尺寸金字塔的园林多模态向量模型 .....                 | 79 |
| 6.2.1 微观尺度特征构建的深层机制 .....                   | 79 |
| 6.2.2 中观尺度特征的空间关系建模 .....                   | 80 |
| 6.2.3 宏观尺度特征的景观生态学建模 .....                  | 81 |
| 6.2.4 多模态特征重组的语义解释框架 .....                  | 84 |
| 6.2.5 特征向量的数值处理与质量控制 .....                  | 85 |
| 6.3 基于 179 维特征向量的江南古典园林相似度分析与共有美学特征识别 ..... | 85 |
| 6.3.1 多层次相似度计算体系 .....                      | 85 |
| 6.3.2 综合相似度融合模型 .....                       | 86 |
| 6.3.3 共有美学特征的稳定性分析 .....                    | 87 |
| 6.3.4 “同”的本质内涵与文化意义 .....                   | 89 |

|                                        |     |
|----------------------------------------|-----|
| 6.4 模型效能泛化验证-以近园为例 .....               | 89  |
| 6.5 基于文化意象密度、生态和谐性、个体差异度的多元化评估模型 ..... | 93  |
| 6.5.1 个性化差异量化与园林特色识别 .....             | 93  |
| 6.5.2 极值特征识别算法 .....                   | 95  |
| 6.5.3 基于 179 维全特征向量的聚类分析模型 .....       | 95  |
| 6.5.4 聚类显著性检验 .....                    | 96  |
| 6.5.5 异同关系的辩证统一：传统智慧的现代诠释 .....        | 98  |
| 6.5.6 文化意象密度与生态和谐性多元视角 .....           | 99  |
| 七、 模型评价与推广 .....                       | 99  |
| 7.1 模型的优点 .....                        | 99  |
| 7.2 模型的不足 .....                        | 100 |
| 7.3 模型的推广 .....                        | 100 |
| 参考文献 .....                             | 102 |
| 附录 .....                               | 103 |

## 一、 问题重述

### 1.1 问题背景

江南古典园林作为中国园林艺术的巅峰代表，是“天人合一”哲学思想与造园技艺的结晶。钱学森先生在《不到园林怎知春色如许——谈园林学》中明确指出，中国园林艺术是我国创立的独特美术学科，是祖国文化遗产中的璀璨明珠，其不仅深刻影响朝鲜、日本等亚洲文化圈，更推动欧洲“英中式”园林的兴起，在世界造园史上占据重要地位。与西方园林几何式、规则式的秩序性美学不同，江南古典园林以“不规则性”与“变化性”为核心特质，造园家通过山石、水景、建筑、植物的巧妙布局，在有限空间内营造出“移步异景”“小中见大”的审美体验，这种“有法无式”的创作理念，既成就了其独特魅力，也形成了中西方认知的鸿沟——英国散文家坦普尔曾将其描述为“无秩序的美”，谢菲尔德大学 Peter Jones 则感慨其“在小区域内部署多样空间的精妙”，却均未能触及背后的隐性规律<sup>[1,2]</sup>。

随着信息化与智能化时代的到来，传统依赖主观描述与感性解读的园林研究已难以满足文化传承与创新的需求。一方面，江南古典园林的美学特征多以文学化语言呈现，缺乏可量化的分析框架，导致其造园智慧难以形成标准化的数字化载体；另一方面，人工智能等技术赋能“生成式园林”的发展趋势，亟需以数学模型解构园林美学规律，为传统造园技艺的现代转化提供支撑。

在此背景下，以江南地区明清时期的十个代表性园林为研究对象，依托平面图、平面矢量图、景观元素坐标、园林照片等多模态数据，量化建模其核心美学特征，成为突破认知壁垒、推动文化传播的关键路径。这一研究不仅能为江南古典园林的保护与修复提供科学依据，更能搭建中西方园林美学交流的桥梁，实现传统智慧与现代技术的深度融合，对传承、弘扬、创新中国传统文化具有重要意义与价值。

### 1.2 问题提出

#### 问题一：

“移步异景”的游赏“趣味性”建模：需将游园路径转化为图问题，明确路径长度、转折点等关键特征；量化游线中景观元素的变化频率（异景程度）；规划兼具高趣味性与低重复率的游线，并可视化展示十个园林的核心路线与异景程度分布。

#### 问题二：

“小中见大”的布局“幻境感”建模：需基于景观元素的位置、数量、形状等特征，建模空间布局的结构特征及其审美影响；界定“开阔”与“围合”空间的量化标准，刻画空间开合变化节奏；以寄畅园（100分）为基准，结合元素分布与开合变化，对其余9个园林进行幻境感评分与排序

#### 问题三：

“有法无式”的园林“相似度”建模：需结合前两项建模结果，量化十个园林的相似度并提炼共通美学规律；将模型应用于其他明清江南古典园林，验证其科学性与可泛化性；探讨除上述三者外的关键美学特征及其刻画依据。

## 二、 问题分析

### 2.1 问题一分析

问题一聚焦江南古典园林“移步异景”的游赏趣味性量化建模，核心是响应题目“以数字化方式提炼‘有法无式’隐性规律、用数学语言解构造园智慧”的核心诉求，需从路

径刻画、异景程度和游线规划三个层次递进建模。题目对三个小问的递进设计要求模型既具备理论严谨性，又能直接服务于实际游园体验。

题目对路径刻画的“定位点为两侧障碍物中点”“转化为图问题”等要求，明确需基于附件中景观元素的精准坐标数据构建物理可行的路径网络。题目强调避免重复路径又能保持新鲜体验，则为游线规划设定了隐性约束，需平衡景观覆盖广度与路径简洁度。为解决这一问题，采用 Voronoi 图方法处理不连续障碍物线段，通过空间索引检测无效路径，通过分支修剪剔除冗余枝杈，最终人工修正补全障碍线段——全过程以“两侧障碍物中点”为核心标准，确保路径网络贴合园林实际通行空间，为后续分析奠定精准基础；

对于异景程度，题目指出“每一次游线转折代表一次视角变化，小空间中多次视角转化则效果显著”，本质是要求量化移步过程中的异景幅度，移步对应时间维度的序列变化，异景对应空间维度的视角差异，二者缺一不可。题目提及“景观元素（山石、水景等）的分布组合关系”，明确异景程度需关联具体造园要素，而非泛化的空间感受，需构建要素级的量化指标。为解决这一问题，以射线探测构建视景矩阵，精准捕捉空间视角；以马尔科夫链量化时间序列变化，以汉明距离衡量空间邻域差异，加权合成的异景程度指标满足移步异景的动态体验，且关联具体景观元素，符合题目要求；

游线规划的“满足趣味性又减少重复”要求，直接点明双目标优化的核心矛盾，且题目要求“可视展示十个园林的游线”“标记关键位置异景程度”，强调模型输出需兼具科学性（最优解）与实用性（直观易懂），需适配不同园林的规模与布局差异。为有效求解模型，采用 NSGA-II 算法，输出的帕累托前沿覆盖“高异景-高重复”“低异景-低重复”等多样方案，适配不同游客偏好；路径可视化标记关键位置异景程度，满足题目可视展示+关键分析的要求，且通过宏观图简化适配不同规模园林的计算需求。

## 2.2 问题二分析

问题二聚焦“小中见大”的布局“幻境感”建模，核心是将造园元素布局与空间开合变化共同作用产生的抽象审美体验转化为可量化的数学模型，需拆解三个子问题的核心需求与解决逻辑，结合园林多模态数据构建科学分析框架。

从问题需求来看，第一小问“元素分布”需实现双重目标：一是明确建模对象与核心特征，即基于附件提供的实体建筑、半开放建筑、道路、假山、水体、植物 6 类景观元素，提取位置、数量、面积比例、离散程度等关键特征；二是建立特征-审美影响的关联机制，分析元素分布如何塑造空间审美体验。解决该问题的核心难点在于，园林元素分布具有“不规则性”与“关联性”并存的特点，单一元素的形态特征与多元素的组合关系均会影响审美感知，需避免孤立分析元素属性而忽略系统关联。

第二小问“开合变化”的核心诉求是突破主观描述局限，实现“开阔-围合”空间的量化界定与动态变化建模。题目以寄畅园为例，明确开合变化源于“围合空间”与“开阔空间”的场景转换，因此需解决两个关键问题：一是建立可操作的量化标准，需结合空间形态与视线通透度定义“开阔”与“围合”；二是刻画游赏过程中的开合转换节奏，需追踪空间序列中“开阔-围合-开阔”的交替频率与强度。此处难点在于小尺度园林与大尺度园林的开合感知阈值存在差异，需建立适配不同规模、阈值灵活的标准化指标。

第三小问“幻境评分”要求以寄畅园（100 分）为基准，结合元素分布与开合变化对其余 9 个园林进行评分排序。其本质是多维度指标的综合加权问题，需解决三个关键：一是基准校准的科学性，即如何确保寄畅园各项指标的标准化得分精准对应 100 分；二是权重分配的合理性，元素分布与开合变化对幻境感的贡献度需符合造园美学规律；三是结果的可解释性，评分差异需能通过园林的造园特点得到印证，避免纯数据驱动的脱离实际。

结合题目提供的平面图、矢量图、景观元素坐标等多模态数据，解决思路可分为三步：首先通过视线投影法将元素坐标转化为栅格矩阵，结合景观生态学指标实现元素分布的量化；其次采用空间形态学与空间句法构建开合变化模型，给出量化标准；最后以寄畅园为基准进行指标标准化，通过层次加权计算综合幻境感得分，最终完成排序与分析。整个过程需严格依托数据预处理确保计算准确性，同时呼应“小中见大”的核心美学诉求，通过有限空间内的元素多样性与开合对比营造“别壶天地”的幻境体验。

## 2.3 问题三分析

问题三以江南古典园林“有法无式”的核心美学特质为研究对象，紧扣题目“从不规则中提炼规律、从量化中验证共性、从补充中完善体系”的三层深层诉求，其本质是基于问题一“动态游赏趣味性”与问题二“静态布局幻境感”的量化基础，构建多维度相似度模型，验证规律的普适性，并补充美学维度，最终实现对江南园林美学体系的完整解构。建模过程需严格响应题目每一项具体要求，确保处理逻辑与题目诉求高度契合。

题目明确要求依据问题1与问题2的建模结果刻画相似度，这一要求直指江南园林美学“动态与静态协同作用”的本质，“移步异景”的趣味性与“小中见大”的幻境感并非孤立存在，而是共同决定园林的整体审美体验。因此，问题三的相似度建模不能脱离前序结果单独设计，需先解决“动态-静态指标融合”的关键矛盾：

针对问题一输出的“路径异景程度”“游线重复率”等动态指标，与问题二输出的“SHDI多样性指数”“View\_Co连接度”等静态指标，需采用标准化处理（[0,1]区间线性标准化）消除量纲差异，避免“异景程度（数值范围50-80）与SHDI（数值范围1.2-2.8）”因量级差异导致的权重失衡；结合题目提及的“天人合一”造园理念（“依据自然形成的‘天工’，营造不规则式构图”），将动态、静态维度各设50%权重，既体现“人在园中游”的动态体验重要性，又尊重“园之布局为基”的静态属性，确保相似性结果贴合园林实际审美逻辑，而非单纯的指标数值对比。

题目强调“尝试将模型应用到其他园林，验证结果的科学性、有效性、与可泛化性”，并建议“最好选择明清时代江南古典园林”，这一要求的核心是检验前序模型是否捕捉到江南古典园林的本质美学规律，而非仅适用于十个样本园林。建模处理需严格落实“数据一致性”与“验证标准明确性”两大关键点：一是数据层面，新验证园林（如附件11中的个园、何园）需与样本园林采用完全一致的数据处理流程——从DWG矢量图提取元素坐标（1000mm等间距采样）、栅格矩阵生成（10×10m分割）、路径刻画（Voronoi图+空间索引）到指标计算（马尔科夫链时域分析、空间句法View\_Co计算），确保每一步参数（如视线高度160cm、NSGA-II迭代50代）均与样本处理一致，避免因数据格式或参数差异导致的验证偏差；二是标准层面，构建“共识对比+规律验证”双重标准——一方面，相似性结果需符合园林学界共识（如个园与留园均以假山为核心，相似度应高于与水体主导的拙政园）；另一方面，样本园林的共有规律（如“单位转折点密度与异景程度正相关 $r \geq 0.78$ ”“水体MNND显著高于植物区域”）需在新园林中成立，若个园相关系数 $r=0.81$ 、水体MNND=3.9m，即可证明模型泛化性有效，响应题目“科学性、有效性”的验证要求。

题目提出“除‘趣味性’‘幻境感’‘相似度’外，是否还有其他较重要的美学特征”，并要求“讨论并提供相关依据”，这一要求旨在避免美学量化的片面性，贴合江南园林“多维审美”的实际特质，符合题目“全面刻画江南古典园林美学特征”的整体目标。

根据问题分析，本文的技术路线如下图1所示：

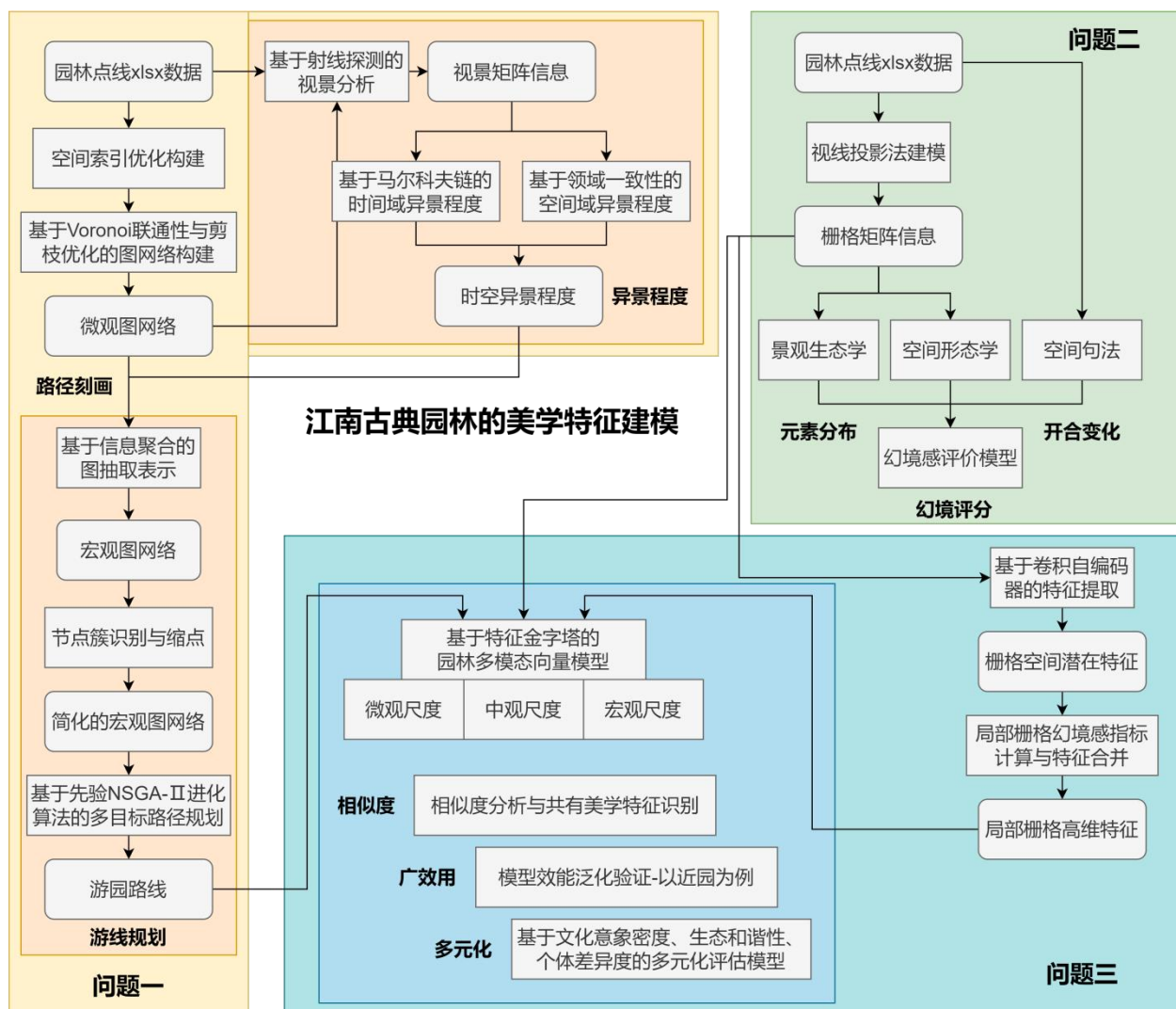


图 1 技术路线

### 三、 模型假设与符号说明

#### 3.1 模型基本假设

(1) 附件中“园林景观元素坐标测量原始数据表”的核心数据测量误差 $\leq 5\%$ ，且关键边界线段的缺失可通过同类园林特征补全，异常坐标占比 $< 1\%$ ，不影响整体路径网络与栅格矩阵的构建；

(2) 普通游客的平均视线高度为 160cm，该高度下实体建筑可完全阻挡视线，半开放建筑、道路、假山、水体、植物均不产生实质性视线遮挡；路径定位点“两侧障碍物中位点”的定义符合游客实际行进时的视觉中心与通行习惯，偏差 $\leq 0.5\text{m}$ ；

(3) 游客沿路径匀速行进，行进速度不影响景观状态的时间序列连续性；视线可均匀覆盖 120 条探测射线方向，无视线偏移或聚焦偏好，确保视景矩阵能完整反映节点处的景观构成；

(4) 建模时段内园林景观处于稳定状态，忽略季节变化导致的植物形态差异、临时设施对路径的阻隔、游客密度对视线通透度的影响，聚焦静态空间特征的核心美学规律。

(5) 同类景观元素的物理属性具有一致性，植物冠径缺失值可按同区域斑块均值填充，误差 $\leq 5\%$ ；假山、实体建筑的边界可通过 4 邻接栅格聚合为连续封闭区域，忽略小于  $1\text{m}^2$  的细碎缺口对空间围合感的影响。

### 3.2 符号说明

表 1 符号说明

| 符号                               | 含义            |
|----------------------------------|---------------|
| $G_m = (V_m, E_m, W_m)$          | 微观图           |
| $V_m = \{v_1, v_2, \dots, v_n\}$ | 微观节点集合        |
| $p_{ij}$                         | 转移概率          |
| $LD$                             | 异景程度          |
| $d_i$                            | 拥挤度距离         |
| $Ind_i$                          | 待变异个体         |
| $SHDI$                           | Shannon 多样性指数 |
| $AI$                             | 聚集度指数         |
| $ED$                             | 形状复杂度         |
| $MNN$                            | 平均最近邻距离       |
| $ER$                             | 围合比指数         |
| $CONNECT$                        | 连通性指数         |
| $CI$                             | 紧凑度指数         |
| $IIC$                            | 整体连通性指数       |
| $View\_Co_i$                     | 连接度           |
| $View\_RCo_i$                    | 标准化连接度        |
| $W_m$                            | 权重函数          |
| $W_M$                            | 宏观权重函数        |
| $\deg(v)$                        | 宏节点           |

## 四、 问题一的求解：“移步异景”的游赏“趣味性”路线规划

### 4.1 问题分析

问题一围绕“移步异景的游赏趣味性建模”展开，需分路径刻画、异景程度和游线规划三个递进层次求解，各部分既相互独立又存在逻辑关联，需结合园林实际数据与图论、优化理论构建技术路径，以下逐点深入分析，问题一流程图如图 2 所示。

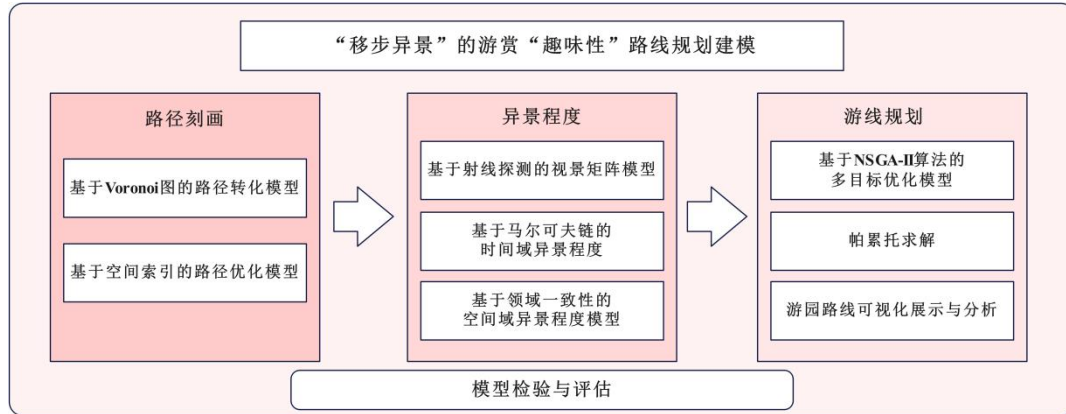


图 2 问题一流程图

#### 4.1.2 路径刻画：基于景观元素坐标的图结构转化

问题一中第一小问“路径刻画”部分要求将观赏者游园路径转化为图问题，需明确路径长度、转折点数量、交叉点数量等关键特征。对问题进行深入剖析可知，解决问题的核心在于根据附件 1-4 园林中假山、水域、植物等元素的几何数据，通过数据解析-路径生成-优化修剪-结果输出四步流程，构建符合实际通行约束的路径网络，且路径需满足定位点为两侧障碍物中位点的规范，最终输出可量化的图结构。

首先要构建基础路径网络，从附件 4 “园林景观元素坐标测量原始数据表”中提取有效边界信息，生成能避开障碍物的路径；通过实际观察园林空间结构可知，存在大量不连续的障碍物线段，难以构成封闭图形。优先考虑使用基于点的方法解决这一难题，本题考虑使用可提取中心线骨架的 Voronoi 图方法，此过程的重难点在于如何确保 Voronoi 边能反映实际可通行区域。通过收集障碍物边界点作为基础数据，同时通过扩展原始边界范围生成外边界点，用于约束 Voronoi 图的范围；随后基于边界点生成 Voronoi 图，继而生成泰森多边形的点与边，构成基础路径网络。

由于 Voronoi 图密布且 Voronoi 图自身存在连接到无穷远的边，影响图结构的提取，需要进行筛选和简化。在路径约束的过程中存在障碍物，需要根据 Voronoi 边、障碍物线段以及园林实际结构图进行障碍物检测，在检测过程中涉及到大量相交线段，为确保路径有效性和连通性，考虑使用空间索引方法优化这一过程。先预处理障碍物线段为 LineString 对象，基于空间索引快速检索与 Voronoi 边可能相交的障碍物，通过 shapely 库检测边与障碍物的相交关系，过滤掉穿越障碍物的无效边（filter\_voronoi\_edges\_with\_spatial\_index 函数）；同时参考附件中园林的实际尺度，过滤长度超过场景对角线 5% 的不合理边。在这一过程中，连通图被打散，构成大量子图，需判断不同子图之间是否连通，即子图之间的连通性，通过连通分量分析，筛选出节点和边数量最多的最大连通子图作为核心路径网络。重难点在于高效处理大量边与障碍物的相交检测及确保路径网络的完整性，代码通过空间索引缩小候选障碍物范围，结合连通分量评分保留最大子图，平衡了效率与网络有效性。

基于泰勒多边形的相关生成原理，在生成 Voronoi 边过程中会形成部分枝杈。这些枝杈路径在提取图结构时没有实际意义。此外，针对园林中的次要区域，即图论中的自环区域，根据相关文献可知，次要区域不会显著增加游园路径总长度，但会增加游园趣味性，故其对游园路径规划无显著影响，可以对该部分区域进行剪枝。为使路径网络更贴合实际需求，需进行分支修剪。游客游览园林的出入口是度为 1 的点，其不应该作为被剪枝的对象，故考虑将其“保护”起来，识别保护区域内的受保护节点，再迭代移除网络中度为 1 的非保护节点，直至无冗余分支。

由于附件 4 中存在不完整数据和导致障碍物线段存在缺陷，出现了尚未在前一轮被剪枝且穿越了障碍物的错误路径，该部分需要进行人工修正，将障碍物线段根据实际情况进行补全，完成从数据模型到现实路径的最终适配。

#### 4.1.3 异景程度：基于时空维度的景观变化量化

问题一中第二小问“异景程度”部分要求建模游园路线中路径与景观元素的分布组合关系，研究不同路径上景观元素的变化频率（即异景程度），并探讨如何计算游览趣味性。深入剖析可知，该问题的核心是将主观的“移步异景”体验转化为可量化的数学指标，需解决三个关键难题：一是如何精准捕捉观赏者视角范围内的景观元素构成，即视景数据的获取；二是如何从“时间（行进顺序）”与“空间（视角范围）”两个维度衡量景观变化的剧烈程度；三是如何将微观节点的异景特征聚合为宏观路径的趣味性指标。

解决思路上，首先需构建多尺度图论模型作为分析基础，微观图刻画次要小路层级的完整路径，宏观图提炼核心路径的关键节点与边，二者均以无向加权图描述，为异景程度的层级化计算提供载体。针对视景数据获取难题，考虑到园林景观存在“视线遮挡”差异，如实体建筑遮挡视线，半开放建筑通透，采用“射线探测+空间索引”技术构建视景矩阵：以 160cm 视线高度为标准，从每个路径节点发射 120 条射线（角度间隔 3°），每条射线按 1 米间距划分探测节点，通过空间索引快速筛选相交线段并判断遮挡关系，最终生成 120×100 的视景矩阵，精准还原每个节点的可视景观构成。

针对时间维度的景观变化，利用马尔科夫链的“无后效性”特征，将路径上相邻节点的景观状态转化为“时间序列”：定义 7 类景观状态，构建景观状态转移概率矩阵，以“小概率转移”量化意外性变化，得到景观时间变化率，转移概率越低，表明景观变化越突破常规，时间异景程度越高。针对空间维度的景观变化，采用汉明距离衡量当前节点与空间邻域节点的视景向量相似性：相似性越低（汉明距离越大），表明空间邻近区域景观差异越显著，空间异景程度越高。

最后，对景观时间变化率与空间变化率加权合成微观节点异景程度，并进一步聚合得到宏观节点与边的异景程度——宏观节点异景程度取相连微观节点的加权平均，宏观边异景程度按路径长度加权计算。最终以路径包含的所有宏观边异景程度之和，量化游览路径的趣味性，实现“主观体验→数学指标”的转化。

#### 4.1.4 游线规划：基于多目标优化的路径权衡

问题一中第三小问“游线规划”部分要求结合路径刻画与异景程度的分析结果，规划既满足趣味性又减少重复路径的游线，并可视化展示十个园林的路线及关键位置异景程度。核心难点在于“趣味性最大化”与“重复率最小化”存在天然权衡——高异景程度需覆盖更多差异化景观，低重复率要求路径简洁，需在满足园林通行约束的前提下寻找最优权衡方案。具体需解决三个关键问题：一是如何将路径与异景程度的量化结果转化为优化模型的目标函数与决策变量；二是如何处理园林实际约束（如起点终点必为出入口、路径需连通等），确保解的可行性；三是如何高效求解多目标问题，确保解的多样性以适配不同游客偏好。

解决思路上,首先明确模型框架:以宏观图中的路径节点序列为决策变量,以“路径总异景程度最大”(目标1)和“路径重复长度最小”(目标2)为双目标函数,构建多目标优化模型。约束条件涵盖6类:起点-终点需为出入口节点、相邻节点需连通、节点需属于宏观图集合、路径长度需在合理范围( $2 \leq \text{节点数} \leq 100$ )、初始种群需来自出入口间简单路径、遗传操作需保证有效性。

考虑到园林宏观图节点与边数量较多,需先对宏观图进行简化:通过欧氏距离聚类合并空间邻近节点,出入口节点单独保护不参与合并,重构后的简化宏观图在保留核心连通性与景观分布特征的同时,显著降低计算复杂度。随后采用NSGA-II算法求解模型,通过“非支配排序”筛选最优解、“拥挤度距离”保证解的分布多样性,迭代50代后输出帕累托前沿最优权衡解集合。

最后,对最优路径进行可视化展示,标记交叉点、出入口等关键位置的异景程度,并选取典型路径进行实景分析,验证模型输出的直观性与实用性,满足题目可视展示+关键分析的要求。

## 4.2 基于 Voronoi 图和空间索引优化的路径刻画模型

根据题目要求可知,需要将观赏者游园路径转化为图问题,同时明确路径长度、转折点数量、交叉点数量等关键特征。本文首先构建基础路径网络,从附件4“园林景观元素坐标测量原始数据表”中提取有效边界信息,生成能避开障碍物的路径。该原始数据表系统记录了园林内各类景观元素的坐标数据以及关键障碍物与空间标识的位置信息,提取时需先对数据进行预处理,剔除重复坐标、测量误差超出允许范围的无效数据,重点保留障碍物外围轮廓的坐标点,明确界定每处障碍物的空间边界范围,该小节流程图如图3所示。

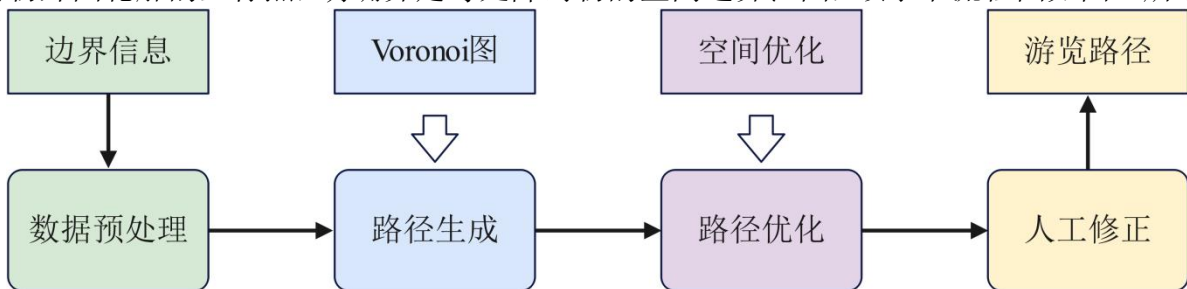


图3 基于 Voronoi 图和空间索引优化的路径刻画模型流程图

### 4.2.2 基于 Voronoi 图的路径转化模型

通过实际观察园林空间结构可知,存在大量不连续的障碍物线段,难以构成封闭图形。这类不连续的障碍物线段,无法直接通过常规的多边形闭合方法形成完整的障碍物区域,优先考虑使用基于点的方法解决这一难题。本题使用可提取中心线骨架的 Voronoi 图方法,该方法的核心优势在于不依赖连续的障碍物线段,仅通过离散的边界点即可生成具有空间居中特性的路径骨架。

Voronoi 图绘制流程如下:首先基于特定点集进行德劳内三角剖分,每一个特定点会与其它若干个特定点组成多个德劳内三角形,需要的是这些三角形的外心(蓝色点)。根据外心的性质,其到三角形的三点的距离相等,即所需的顶点。因为是有限区域,用边界直接截取得到边界交点,同时将边界交点和外心(黄色点)分别关联到对应的特定点。特定点的关联点顺时针或者逆时针相连就得到沃罗诺伊小元胞,而将所有特定的关联点分别相连就得到 Voronoi 图。

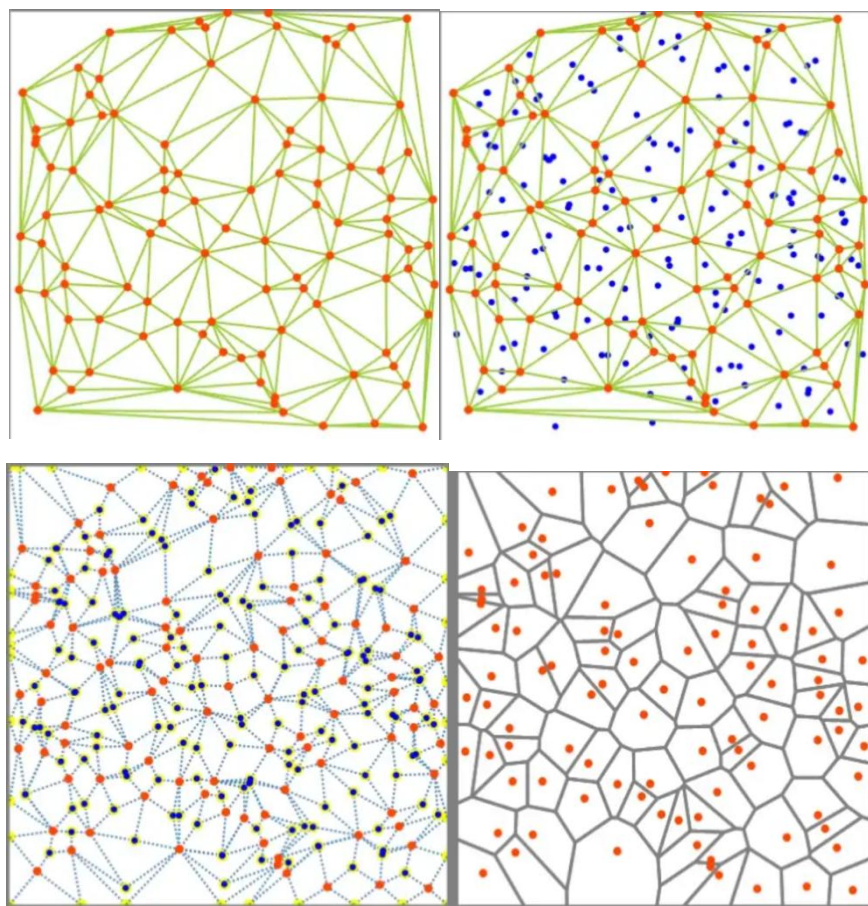


图 4 Voronoi 图绘制关键步骤

此过程的重难点在于如何确保 Voronoi 边能反映园林实际可通行区域。为解决上述问题并保证 Voronoi 图的有效性，需进行以下的数据准备：首先收集障碍物边界点作为基础数据，对于障碍物，沿外围边界进行均匀采点，确保障碍物轮廓的完整性；其次通过扩展原始边界范围生成外边界点，用于约束 Voronoi 图的范围；随后基于边界点生成 Voronoi 图，继而生成泰森多边形的点与边，构成基础路径网络。

#### 4.2.3 基于空间索引的路径优化模型

由于 Voronoi 图密布且 Voronoi 图自身存在连接到无穷远的边，影响图结构的提取，需要对其进行筛选和简化。在路径约束的过程中存在障碍物，需要根据 Voronoi 边、障碍物线段以及园林实际结构图进行障碍物检测，在检测过程中涉及到大量相交线段，为确保路径有效性和连通性，使用空间索引方法进行优化。

首先将障碍物线段处理为 LineString 对象，基于空间索引快速检索与 Voronoi 边可能相交的障碍物，通过 shapely 库检测 Voronoi 边与障碍物的相交关系，过滤掉穿越障碍物的无效边；同时参考附件中国园林的实际尺度，过滤掉长度超过场景对角线长度 5% 的不合理边。在这一过程中，连通图被打散，形成大量子图，此时需判断不同子图之间是否连通，即子图之间的连通性。通过连通分量分析，筛选出节点和 Voronoi 边数量最多的最大连通子图作为核心路径网络。此方法的重难点在于高效处理大量边与障碍物的相交检测及确保路径网络的完整性，而本文通过空间索引缩小候选障碍物范围，结合连通分量评分保留最大子图，平衡了效率与网络有效性。

基于泰勒多边形的相关生成原理，在生成 Voronoi 边过程中也会形成部分枝杈。这些枝杈路径在提取图结构时没有实际意义，如图 5 所示。



图 5 拙政园游览路径示意图（含枝杈路径）

此外，针对园林中的次要区域，即图论中自环区域，根据相关文献，次要区域不会显著增加游园路径总长度，但会增加游园趣味性，故其对游园路径规划无显著影响，可以对该部分区域进行剪枝。同时为使路径网络更贴合实际需求，需进行分支修剪。而游客游览园林的出入口是度为 1 的点，其不应该作为被剪枝的对象，故考虑将其独立，识别保护区域内的受保护节点，再迭代移除网络中度为 1 的非保护节点，直至无冗余分支。

由于附件 4 中存在不完整数据和导致障碍物线段存在缺陷，出现了尚未在前一轮被剪枝且穿越了障碍物的错误路径，该部分需要进行人工修正，将障碍物线段根据实际情况进行补全，完成从数据模型到现实路径的最终适配，最终得到合理的游园路径。



图 6 拙政园游览路径示意图（优化后）

### 4.3 基于时域空域分析的异景程度模型

在路径建模中，本文采取微观图和宏观图两种图论模型描述不同层级的路径网络，其中微观图对应路径网络中的次要小路层级，对区域内所有细小路径的完整刻画，宏观图对应路径网络中的主要路径层级，是在微观图的基础上提炼出的核心路径模型，均用无向加权图进行描述。

微观图记为  $G_m = (V_m, E_m, W_m)$ ，其中  $V_m = \{v_1, v_2, \dots, v_n\}$  表示微观节点的集合， $E_m \subseteq V_m \times V_m$  为边的集合，表示节点间的连接关系， $W_m$  为权重函数，表示对边集合  $E_m$  中的每一条边，都能对应到一个正实数  $R_+$ ，表示该条次要小路的长度。

宏观图记为  $G_M = (V_M, E_M, W_M)$ ， $V_M \subseteq V_m$  为宏观节点的集合，从微观节点  $V_m$  集合中筛选出的重要节点， $E_M \subseteq V_M \times V_M$  为宏观边集合， $W_M$  为宏观权重函数。宏观节点的识别基于节点度的筛选，节点  $v \in V_m$  的度的定义为  $\deg(v) = |\{u | (v, u) \in E_m\}|$ ，当节点满足下列条件之一时被识别为宏节点： $\deg(v) = 1 \vee \deg(v) \geq 3$ 。宏观边  $(u, v) \in E_M$  对应微观图中的一条路径  $P(u, v) = \{u = p_0, p_1, \dots, p_k = v\}$ ，当边  $(u, v)$  满足中间节点均为 2，即  $\deg(p_i) = 2, \forall 1 \leq i \leq k-1$ ，则该边被定义为宏观边，同时该路径是这两点之间最短路径  $P(u, v) = \arg \min \sum_{i=0}^{k-1} w(p_i, p_{i+1})$ 。

首先在微观图中分析路径点中的异景程度。异景程度 (Landscape Variation Degree, LVD) 是指游园路径上相邻路径单元间景观元素构成的变化频率与差异幅度的综合度量，反映游人在行进过程中感知到的景观多样性变化。其核心思想是：通过相邻路径单元的景观关联度反向表征景观差异，结合路径单元的长度权重，计算全路径的异景程度。本文主要利用景观时间变化率，景观空间变化率两个指标对异景程度进行量化分析，该部分流程图如图 7 所示。

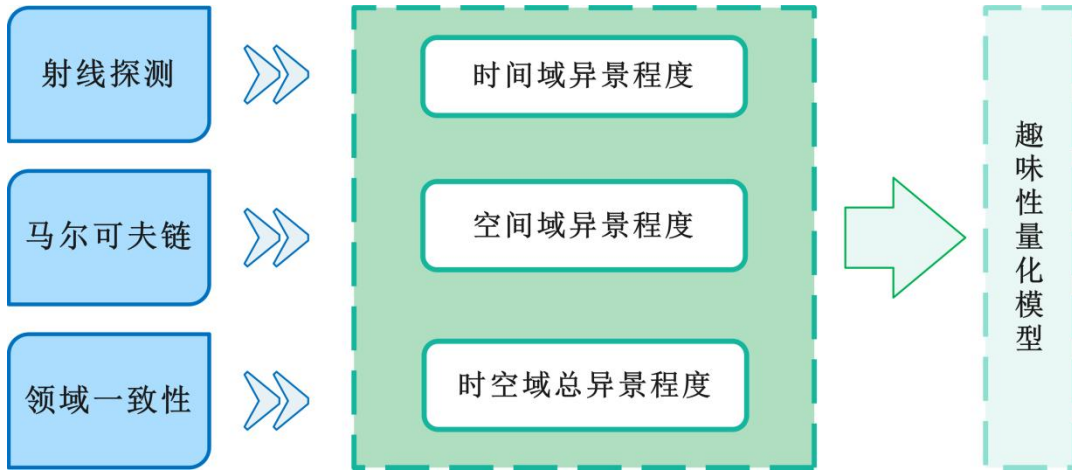


图 7 基于时域空域分析的异景程度模型

#### 4.3.2 基于射线探测的视景矩阵模型

根据园林景观的特性，将景观分为两大类，其中阻挡视线类仅包括实体建筑，其余景观均列入“不会阻挡视线类”。视景矩阵的形成以“射线探测+空间交互判断”为核心逻辑。假设视景距离上限为 100 米，将游客参观视线简化成一条射线，每条射线按 1 米为单位划分为 100 个探测节点，利用空间交互判断，进入射线与线段的交互探测阶段。

为提升效率引入空间索引技术，针对每条射线，先通过空间索引快速筛选出空间中所有可能与之相交的线段，然后基于线段属性进行遮挡判断，确定射线可探测范围：若最近交点所在的线段属于“会遮挡视线”的类型，射线将在此交点处被阻断，交点之后的距离节

点对应的景观均判定为不可见；若该线段属于“不会遮挡视线”的类型，射线可继续延伸，此时需先记录该线段所属的景观元素，再重复上述“空间索引筛选-找下一个最近交点-判断遮挡”的流程，直到射线遇到遮挡性线段或达到 100 米视景距离上限随后计算射线与这些筛选后线段的所有实际交点，两交点之间的景观类型填充为距离观察点较远的交点处的景观类型。若最后一个交点到观察点的距离小于 100 米，射线继续延伸，直至与下一个景观，则最后一个交点到视线距离上限之间的景观类型填充为该景观类型，若仍无法探测到景观，则填充为最后一个交点处的景观类型。

为实现可调节的视景探测。设定角度间隔变量为 3 度，从观察点向空间发射射线，累计发射 120 条射线形成 120 个探测维度；同时视景距离上限为 100 米，每条射线按 1 米为单位划分为 100 个探测节点，最终目标是生成包含 120 行（对应 120 条射线）、100 列（对应 100 米距离节点）的 120\*100 视景矩阵，如图 8 所示。

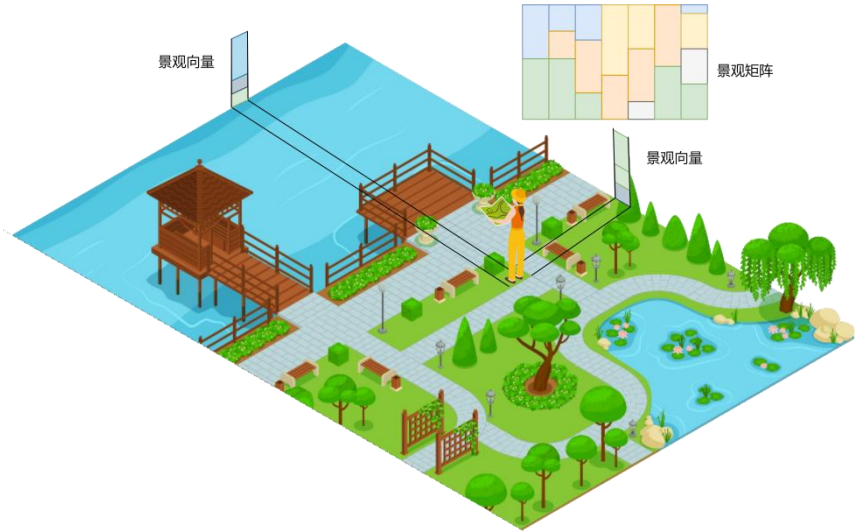


图 8 视景向量取样示意图

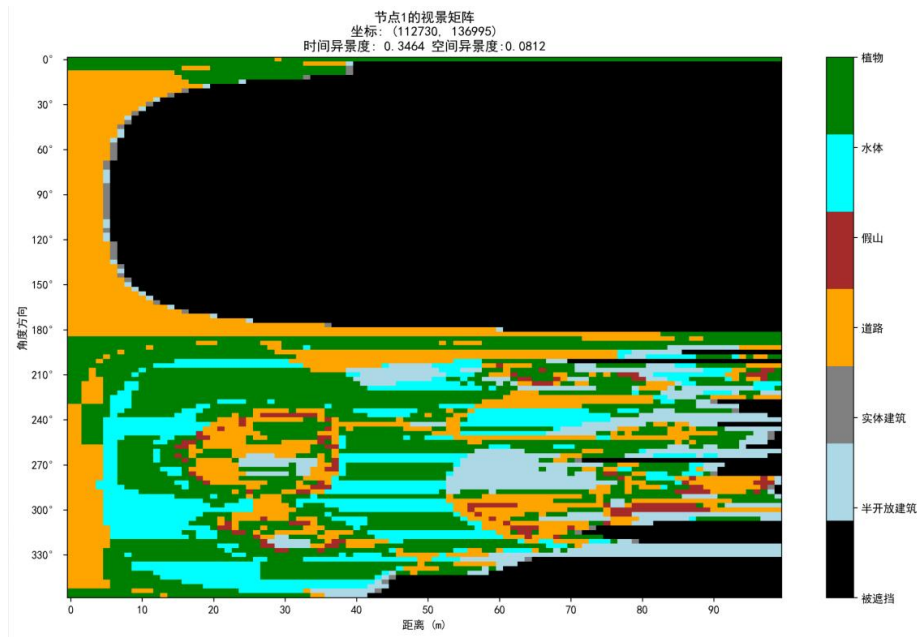


图 9 节点 1 视景矩阵

### 4.3.3 基于马尔科夫链的时间域异景程度

景观在路径上的分布并非孤立的点，可以理解为隐含时间维度的动态过程，本文利用马尔科夫链进行空间-时间的维度转化，即将路径上相邻的路径点所观察到的景观状态，定义为时间维度上先后发生的景观状态。将原本聚焦于空间分布的景观状态转移转化成基于时间演变的景观状态转移问题。

马尔科夫链最核心的特性是马尔可夫性质，即系统在给定当前状态的条件下，其未来状态的概率分布只依赖于当前状态，而与过去的历史状态无关。

设随机过程的状态空间为可列集  $S$ ，若对任意状态  $i_0, i_1, \dots, i_n, j \in S$  都满足：

$$P(X_{n+1} = j | X_n = i_n, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = P(X_{n+1} = j | X_n = i_n) \quad (1)$$

状态空间  $S$  是马尔科夫链所有可能取值的集合，必须是离散的，转移概率  $p_{ij}$  表示当前处于状态  $i$ ，下一时刻转移到状态  $j$  的概率，即  $p_{ij} = P(X_{n+1} = j | X_n = i)$ 。将所有转移概率按行和列排列，形成转移概率矩阵  $P$ ，其形式为：

$$P = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1k} \\ p_{21} & p_{22} & \cdots & p_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ p_{k1} & p_{k2} & \cdots & p_{kk} \end{pmatrix} \quad (2)$$

对于所有的  $i, j$ ， $p_{ij} \geq 0$ ，对于所有的  $i$ ， $\sum_{j \in S} p_{ij} = 1$ 。

根据江南园林景观分类方法，景观状态被分为 7 类，本文所涉及到的具体景观状态集合为  $S = \{\text{被遮挡, 半开放建筑, 实体建筑, 道路, 假山, 水体, 植物}\}$ ，景观转移概率  $p_{ij} = \frac{n(s_i \rightarrow s_j)}{n(s_i)}$ ，其中  $n(s_i)$  表示当前景观状态  $s_i$  出现的总次数， $n(s_i \rightarrow s_j)$  表示当前状态  $s_i$  转移到下一状态  $s_j$  的次数。利用景观转移概率  $p$  构成景观状态转移概率矩阵  $P$ ，如图所示。

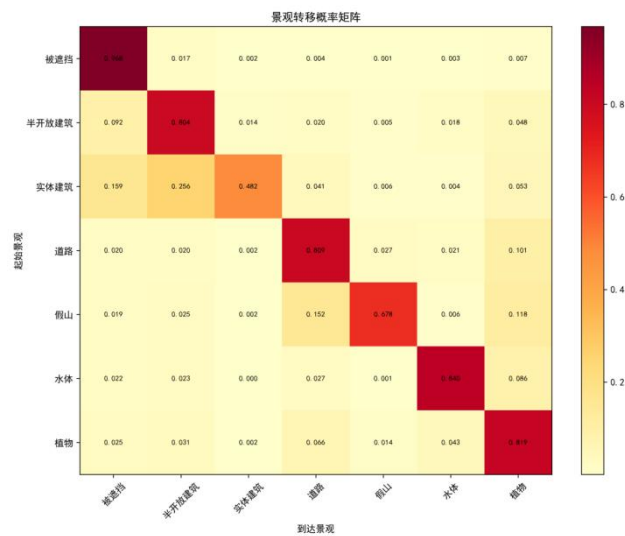


图 10 景观转移概率矩阵

马尔可夫链可通过转移概率的意外性，精准量化异景程度。其核心逻辑在于马尔可夫链的转移概率本质是区域景观演变的历史规律总结，若某一相邻点位的状态转移概率显著低于常规水平，即小概率转移，则说明该点位的景观变化打破了历史常规，属于意外变化，对应高异景程度；反之，若转移概率较高，即大概率转移，则景观变化符合常规节奏，异景程度较低。

#### 4.3.4 基于领域一致性的空间域异景程度

采用领域一致性量化研究景观空间变化率，本文利用汉明距离去衡量空间邻近单元的视景关联程度。其核心思路是：通过衡量当前点位与其空间领域内邻近点位的视景向量相似性，推导该区域景观空间变化的剧烈程度。领域一致性越高，意味着空间邻近单元的视景特征越趋同，景观空间变化率越低；反之，领域一致性越低，空间邻近单元的视景特征差异越显著，景观空间变化率越高。

对于两个长度相等的字符串或向量，汉明距离指的是它们对应位置上不同字符或元素的个数，用汉明距离  $d$  表示景观空间变化率。假设存在两个等长的序列： $x = (x_1, x_2, \dots, x_n)$  和  $y = (y_1, y_2, \dots, y_n)$ ，汉明距离  $d(x, y)$  的计算公式为  $d(x, y) = \sum_{i=1}^n \delta(x_i, y_i)$ ，其中

$$\delta(x_i, y_i) = \begin{cases} 0 & , x_i = y_i \\ 1 & , x_i \neq y_i \end{cases} \quad (3)$$

利用汉明距离，将当前点位与每个相邻点位的视景向量逐元素进行对比，统计对应位置景观元素不同的数量，汉明距离越大，表明视景向量间差异越大。

通过相似性与汉明距离的关联，推导景观空间变化率。在领域一致性视角下，当前点位与邻近点的视景向量相似性，与汉明距离呈负相关关系：汉明距离越小，两者视景特征差异越小，相似性越高，代表该区域景观空间特征相对稳定，景观空间变化率较低；反之，汉明距离越大，视景特征相似性越低，表明当前点位周边景观在空间上的过渡性或突变性更强，即景观空间变化率更高。其核心思路是：通过衡量当前点位与其空间领域内邻近点位的视景矩阵相似性，推导该区域景观空间变化的剧烈程度。领域一致性越高，意味着空间邻近单元的视景特征越趋同，景观空间变化率越低；反之，领域一致性越低，空间邻近单元的视景特征差异越显著，景观空间变化率越高。

#### 4.3.5 时空结合的异景程度模型求解与趣味性量化

根据得到的景观时间变化率和景观空间变化率，计算异景程度 LD:

$$LD = \alpha \cdot p + \beta \cdot d \quad (4)$$

其中  $\alpha = 0.5$ ， $\beta = 0.5$ 。

由下图所示，在一条路径中表示出每个节点的异景程度对于游客游览的参考性不大，对游览路径的选择分析难度角度，无法在关键节点进行提示，因此只需分析出关键节点的异景程度。根据以上分析，在微观图的基础上分析宏观图的异景程度。由于宏观点的条件之一是度为 1 或者度大于等于 3，可知除起点和终点外，与其他关键节点相连接的路径至少有 3 条。

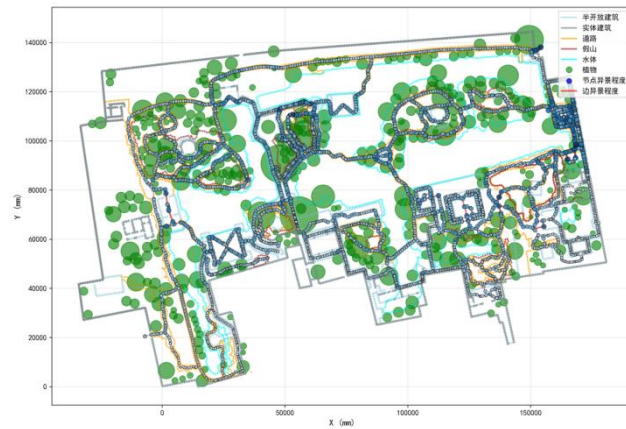
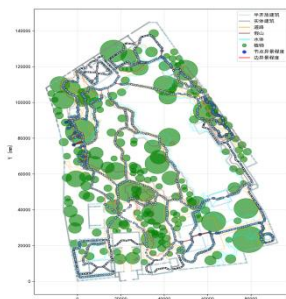


图 11 微观节点异景程度热力图-拙政园

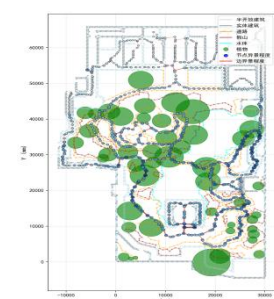
其余 9 个园林的微观节点异景程度热力图如下所示。



a. 寄畅园



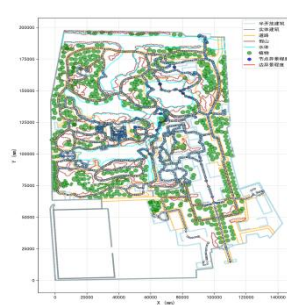
b. 留园



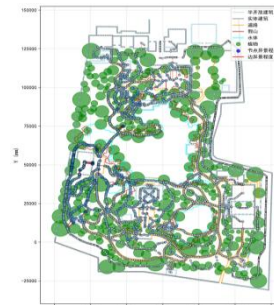
c. 耦园



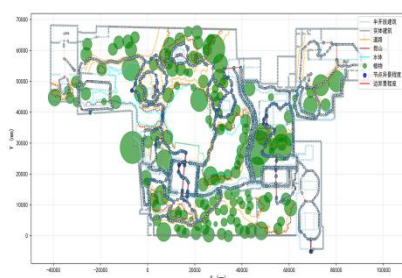
d. 绮园



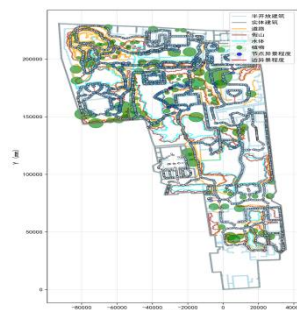
e. 秋霞圃



f. 沈园



g. 怡园



h. 豫园



i. 瞻园

图 12 微观节点异景程度热力图-其余 9 个园林

假设与关键节点 $u$ 相连接的路径数为 $n$ ，每条路径上的围观节点数量为 $\{p_1, p_2, \dots, p_n\}$ ，

可知关键节点 $u$ 的异景程度为:

$$LD(u) = \frac{\sum_{i=1}^n \sum_{j=1}^{j=p_i} LD(p_{ij})}{n} \quad (5)$$

宏观边 $(u, v) \in E_M$ 对应微观图中的一条路径 $P(u, v) = \{u = p_0, p_1, \dots, p_k = v\}$ , 共有 $k$ 个微观节点, 宏观边的长度为 $W_M$ , 则宏观边 $(u, v)$ 的异景程度为:

$$LD(u, v) = \frac{\sum_{i=1}^k LD(p_i)}{n} \times W_M \quad (6)$$

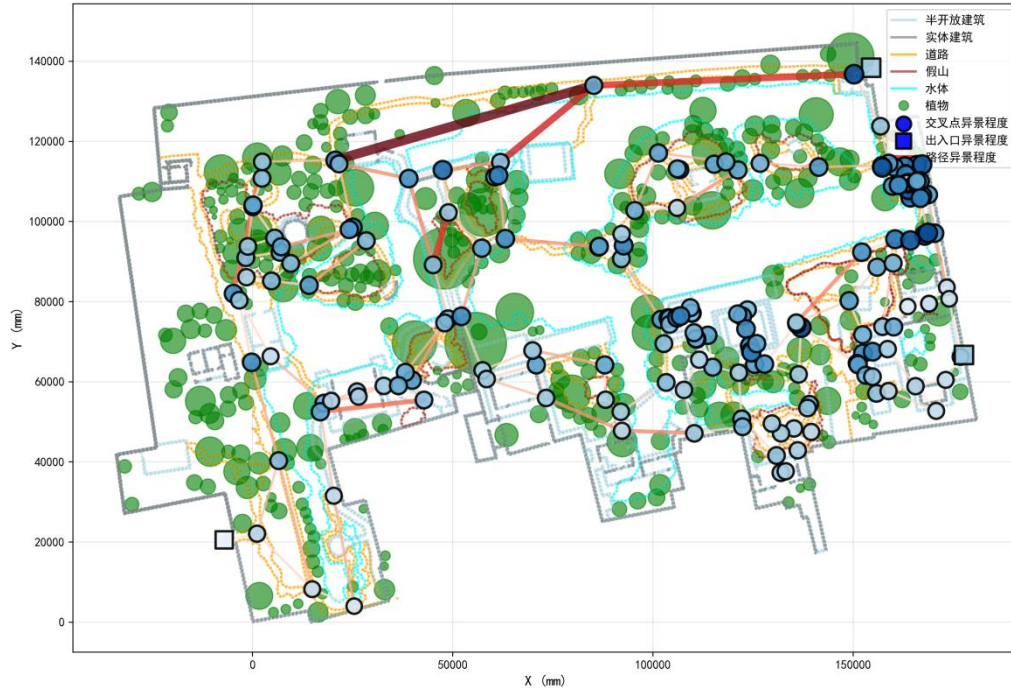
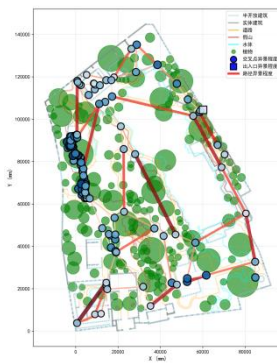
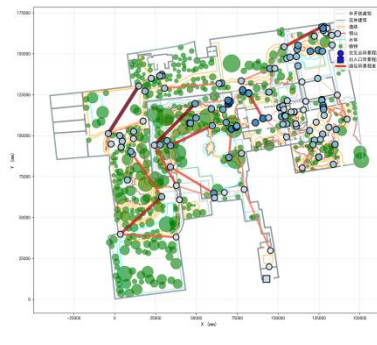


图 13 宏观节点异景程度热力图-拙政园

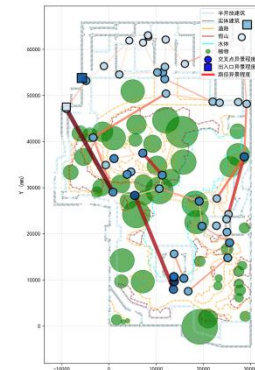
其余 9 个园林的宏观节点异景程度热力图如下所示。



a. 寄畅园



b. 留园



c. 耦园

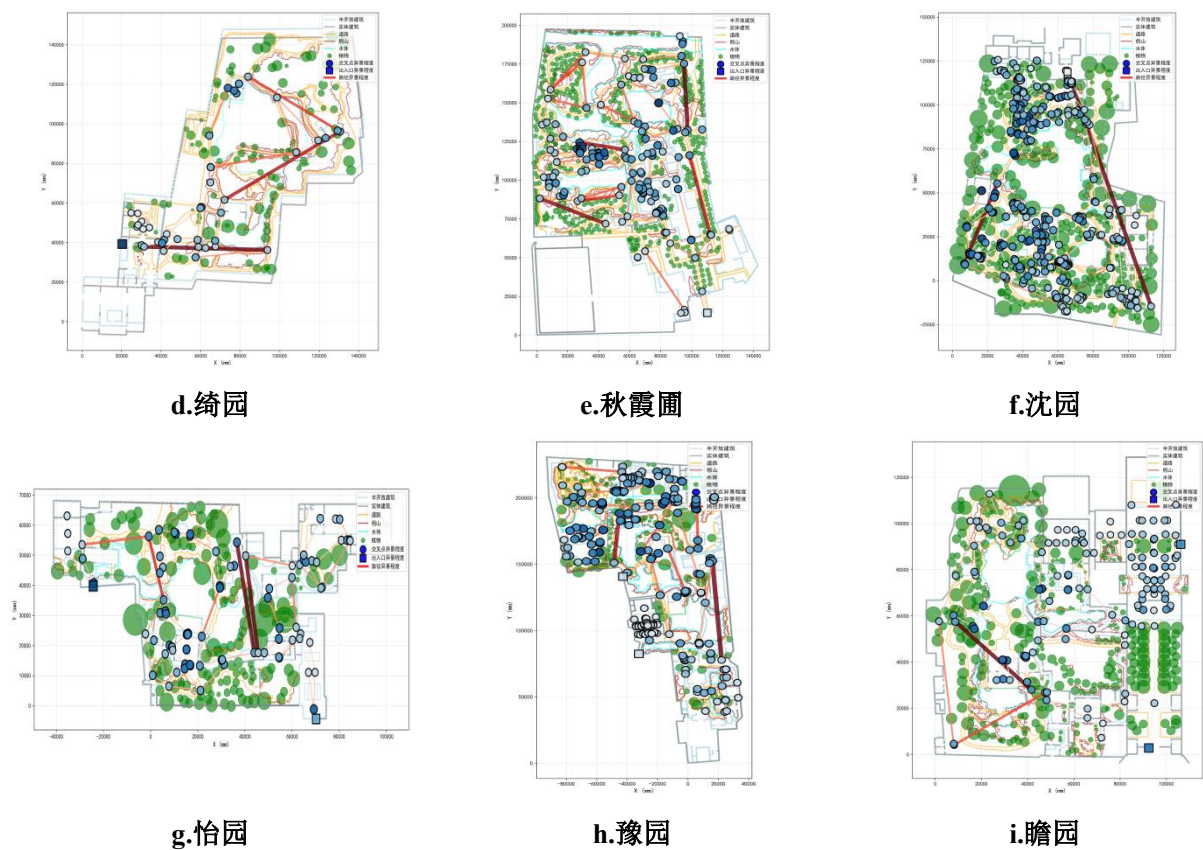


图 14 宏观节点异景程度热力图-其余 9 个园林

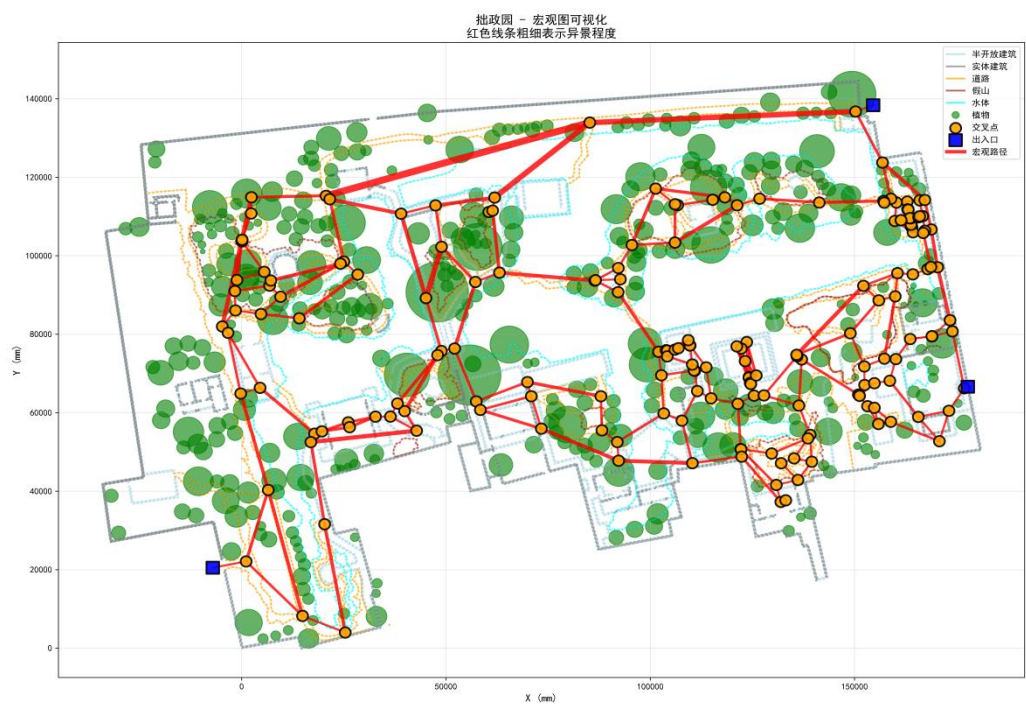


图 15 宏观边异景程度热力图-拙政园

其余 9 个园林的宏观边异景程度如下所示。

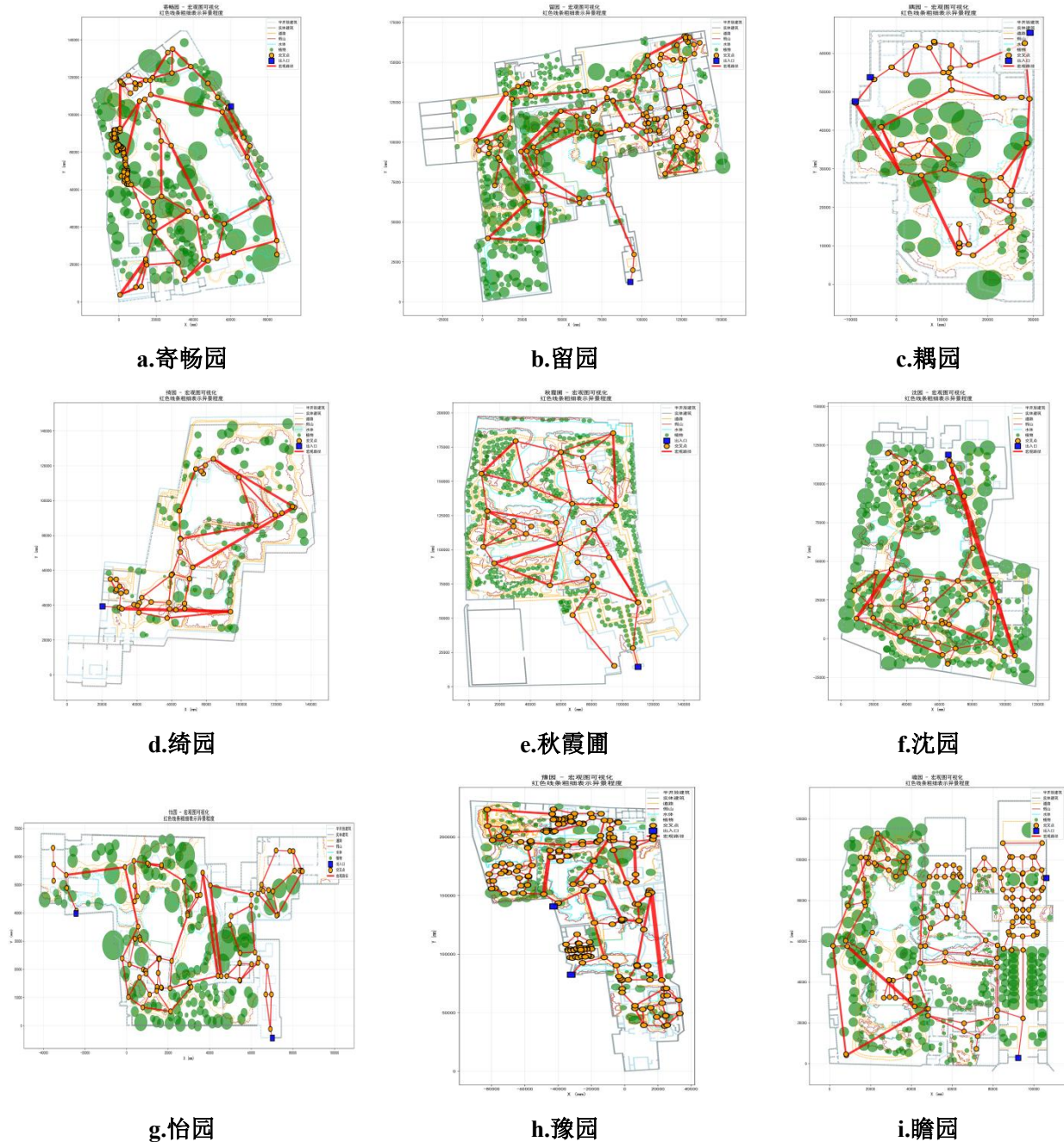


图 16 宏观边异景程度热力图-其余 9 个园林

根据微观节点的异景程度，可以得到宏观节点和宏观边的景观时间变化率和景观空间变化率，如图所示。

路径的趣味性取决于构成该路径的宏观边的异景程度之和，假设该路径由  $m$  个宏观边  $(u_i, v_i)$  构成，则可知该路径的趣味性：

$$Q = \sum_{i=1}^m LD(u_i, v_i) \quad (7)$$

通过此公式可将主观感受上的“移步异景”的路径游览趣味性进行量化刻画，建模得到游园过程中不同路径的趣味性数值，为后续游线规划奠定基础。

#### 4.4 基于 NSGA-II算法的多目标优化“游线规划”模型

问题一中“游线规划”部分要求结合路径刻画与异景程度的分析结果，规划既满足趣味性又减少重复路径的游园游线，并可视化展示关键位置的异景程度。对问题进行深入剖析可知，核心在于解决最大化游线趣味性（异景程度）与最小化路径重复率的双目标权衡问题，需以宏观图中的节点与边为基础，构建符合园林通行约束的多目标优化模型。本文首先明确决策变量为宏观图中的路径节点序列，以路径总异景程度最大、重复长度最小为目标函数，结合出入口、连通性等实际约束建立模型；其次考虑到园林路径网络的复杂性，对宏观图进行节点聚类简化以提升计算效率；最后采用 NSGA-II算法求解模型，通过非支配排序与拥挤度距离确保解的最优性与多样性，最终输出帕累托最优路径并完成可视化与关键位置异景程度分析，该小节流程图如图 17 所示。

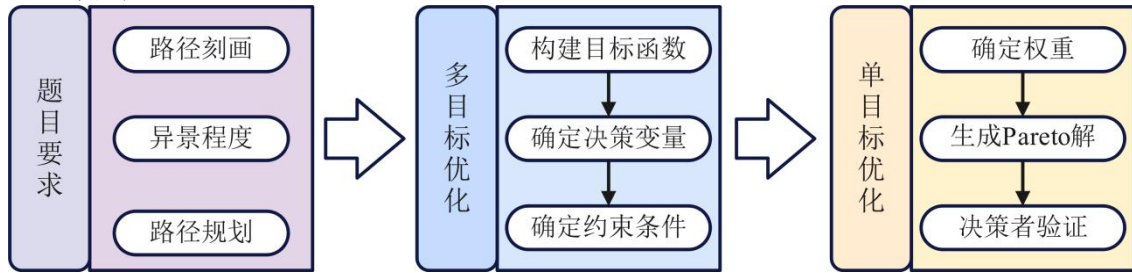


图 17 基于 NSGA-II算法的多目标优化“游线规划”模型流程图

##### 4.4.2 基于 NSGA-II算法的多目标优化模型建立

设游线规划的决策变量为路径节点序列  $P = [v_0, v_1, v_2, \dots, v_n]$ ，其中， $v_i \in V$ ， $V$  为宏观图的节点集合，对应园林中出入口、交叉点、景观核心点等关键位置；相邻节点  $v_i$  与  $v_{i+1}$  满足  $(v_i, v_{i+1}) \in E$ ， $E$  为宏观图的边集合，对应园林中可通行的路径， $n$  为路径节点序列的长度，即节点个数减 1；在代码的遗传算法实现中，路径  $P$  被编码为个体  $\text{individual} = [\text{start\_node}, \text{end\_node}, \text{step\_choices}]$ ， $\text{step\_choices}$  是一个列表，表示在每个节点  $v_i$  处选择哪个邻居节点  $v_{i+1}$  作为下一步  $\text{choice}_i, dx \in \{0, 1, \dots, \deg(v_i) - 1\}$ 。

根据题目要求，游园路线规划的原则为既能满足趣味性又减少重复路径，由此可知，模型有两个需要同时优化的目标，即最大化路径异景程度和最小化路径重复率，目标函数和相关约束条件的数学语言表述如下：

(1) 目标函数 1：最大化路径总异景程度  $\max f_1(P)$

$$f_1(P) = \text{anomaly\_score}(P) = \sum_{v \in \text{unique}(P)} \left( \frac{\text{temporal\_anomaly}(v)}{T_{\text{node\_max}}} + \frac{\text{spatial\_anomaly}(v)}{S_{\text{node\_max}}} \right) + \sum_{e \in \text{unique}(E_P)} \left( \frac{\text{temporal\_anomaly\_sum}(e)}{T_{\text{edge\_max}}} + \frac{\text{spatial\_anomaly\_sum}(e)}{S_{\text{edge\_max}}} \right) \quad (8)$$

式中： $\text{unique}(P)$  为路径  $P$  中所有不重复的节点集合； $\text{temporal\_anomaly}(v)$  为节点  $v$  的时间异景程度； $\text{spatial\_anomaly}(v)$  为节点  $v$  的空间异景程度； $\text{temporal\_anomaly\_sum}(e)$  为边  $e$  的时间异景程度总和； $\text{spatial\_anomaly\_sum}(e)$  为边  $e$  的空间异景程度总和； $\text{unique}(E_P)$  为路径  $P$  中所有不重复的边的集合； $T_{\text{node\_max}}, S_{\text{node\_max}}, T_{\text{edge\_max}}, S_{\text{edge\_max}}$  为全局节点/边异景程度最大值。

(2) 目标函数 2：最小化路径重复长度  $\min f_2(P)$

$$f_2(P) = \text{repetition\_length}(P) = \sum_{i=0}^{n-2} \text{length}(e_i) \cdot \mathbf{1}_{\{e_i \in \text{visited\_edges}\}} \quad (9)$$

式中： $e_i = (v_i, v_{i+1})$  为路径  $P$  的第  $i$  条边； $\text{length}(e_i)$  为边  $e_i$  的长度属性； $\text{visited\_edges}$  为一个集合，记录在计算  $\text{repetition\_length}$  时之前已经遍历过的边； $\mathbf{1}\{\text{condition}\}$  表示指示函数，当  $\text{condition}$  为真时值为 1，否则为 0，这里条件  $e_i \in \text{visited\_edges}$  判断边  $e_i$  是否在之前已被走过；求和范围为  $i=0$  到  $n-2$  覆盖路径  $P$  的所有边。

该模型的约束条件可分为 6 类，数学表达如下：

(1) 起点-终点约束：要求路径起点  $v_0$  和终点  $v_n$  必须为园林的出入口节点。

$$v_0 \in E_{\text{in/out}}, \quad v_n \in E_{\text{in/out}}, \quad v_0 \neq v_n \quad (10)$$

式中： $E_{\text{in/out}}$  为出入口节点集合

(2) 连通性约束：路径中任意相邻节点必须在宏观图中连通，即对应可通行路径，避免无效跳点，若不连通则尝试通过最短路径补全，确保连通性。

$$\forall i \in \{0, 1, \dots, n-1\}, \quad (v_i, v_{i+1}) \in E \quad (11)$$

(3) 节点有效性约束：路径中所有节点必须属于宏观图的节点集合避免无效节点。

$$\forall i \in \{0, 1, \dots, n\}, \quad v_i \in V \quad (12)$$

(4) 路径长度约束：路径节点序列的长度需在合理范围内，避免过短导致缺乏趣味性或过长导致游览效率低。

$$2 \leq n \leq 100 \quad (13)$$

(5) 先验路径基础约束：初始种群的先验路径需来自宏观图出入口间的简单路径。

$$P \in \text{SimplePaths}(E_{\text{in/out}}, E_{\text{in/out}}, \text{cutoff} = 20) \quad (14)$$

式中： $\text{SimplePaths}(u, v, \text{cutoff})$  表示从  $u$  到  $v$ 、节点数不超过  $\text{cutoff} + 1$  的简单路径集合。

#### 4.4.3 基于 NSGA-II 算法的多目标优化模型求解

江南古典园林的游线规划问题无解析解，即无法通过公式直接计算最优路径，因异景程度、重复长度均与节点或边的异景属性、宏观图拓扑结构强相关，需依赖多模态数据。而 NSGA-II 算法作为多目标优化领域的经典启发式算法，其设计特性与江南古典园林游线规划问题的需求高度匹配，无需依赖问题的解析表达式，仅通过目标函数值引导进化，可直接利用代码中提供的园林多模态数据计算目标函数，理论上可逼近全局最优的帕累托前沿。

NSGA-II 的编码与遗传算子可灵活适配游线规划中节点序列类型的决策变量；将节点序列转化为“起点+终点+步骤选择序列”，此时步骤选择序列对应邻居节点的索引，既保留路径的离散结构，又避免直接编码节点序列导致的维度爆炸，适合遗传操作；单点变异/插入/删除步骤均设置有效性检查，避免步骤序列过短/过长、避免无效邻居选择，确保遗传操作后仍为可行路径，解决了离散路径规划中无效解过多的痛点。

此外，在使用 NSGA-II 算法求解过程中，由于园林路径网络的可视化宏观图复杂性高（宏观节点数 182、宏观边数 260）选择对宏观图继续简化。通过合并空间上邻近的冗余节点，在保留园林核心通行结构与景观分布特征的前提下，降低后续游线规划优化的计算复杂度。

首先进行原始宏观图数据的提取与预处理。基于附件中的园林平面矢量图，提取实体建筑、半开放建筑、道路、山石、水体、植物 6 种景观元素的关键节点——道路交叉点、

出入口、山石核心区、水体边界关键点位、植物群落中心等均作为原始节点，节点坐标来源于景观元素坐标数据表（以园林西南侧边界顶点为原点，正东为 X 轴，正北为 Y 轴，按 1 米等间距采样，单位为毫米。预处理阶段需对坐标数据进行有效性核验，处理可能存在的缺失值及超出园林边界的异常值，同时明确原始边集合，对应各节点间可通行的路径，基于矢量图中道路与元素的连通关系确定。

随后执行基于空间距离的节点聚类合并，这是简化的核心环节。以欧氏距离为度量标准计算任意两原始节点的空间距离  $\text{dist}(v_a, v_b) = \sqrt{(x_{v_a} - x_{v_b})^2 + (y_{v_a} - y_{v_b})^2}$ ，预设距离阈值，

设簇  $C_k$  含节点  $\{v_1, \dots, v_m\}$ ，其中心  $\mu_k = (\bar{x}_k, \bar{y}_k)$ ， $\bar{x}_k = \frac{1}{m} \sum_{i=1}^m x_{v_i}$ ， $\bar{y}_k = \frac{1}{m} \sum_{i=1}^m y_{v_i}$ ，按近簇归属

原则进行节点分配： $C_k = \{v \in V | \text{dist}(v, \mu_k) = \min_{1 \leq i \leq K} \text{dist}(v, \mu_i) \wedge \text{dist}(v, \mu_k) \leq \epsilon\}$  将距离小于等于阈值的节点归为同一簇，以簇内所有节点的坐标均值作为簇中心；若节点与所有现有簇中心的距离均大于阈值，则新建独立簇。其中，出入口节点作为游线起点与终点的强制约束，单独保留为独立簇，不参与合并操作。

最后构建简化宏观图的节点与边集合。简化后的节点集合由所有簇中心构成，每个簇中心对应原始图中一片空间邻近的景观关键区域。简化后的边集合基于原始边的连通性生成：若原始宏观图中存在连接不同簇内节点的边，则在对应簇中心间构建新边，确保简化图完整保留原始园林的通行连通结构。经上述步骤得到的简化宏观图，如图所示，在精简节点规模的同时，完整保留了出入口位置、景观区域分布及路径连通性等核心特征，可直接用于后续游线规划计算。

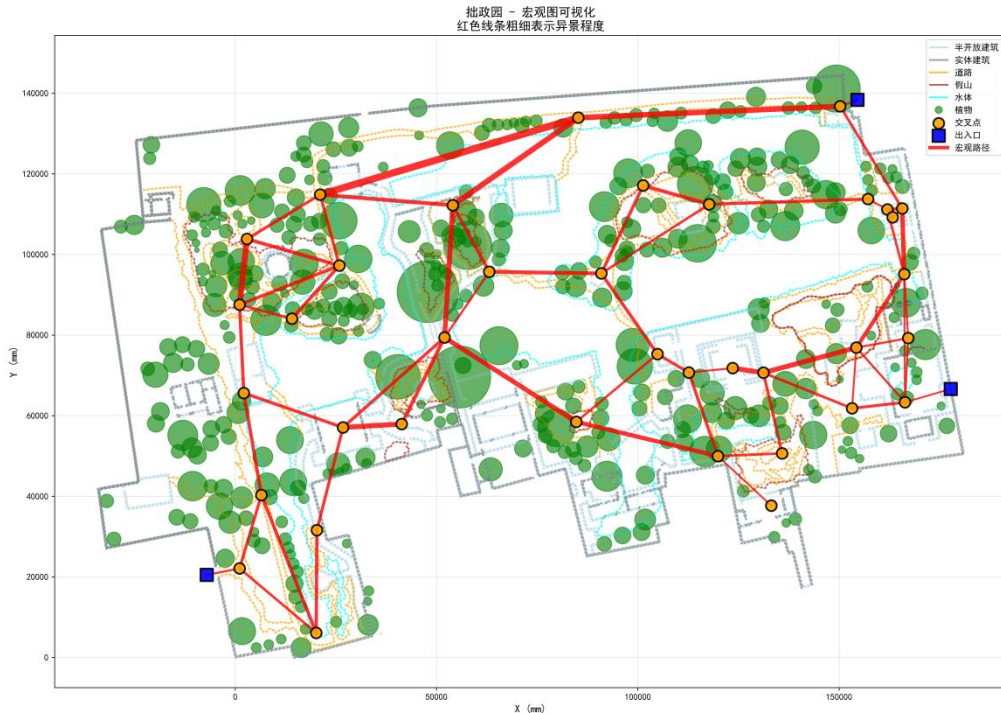


图 18 园林路径简化宏观图-拙政园

其他 9 个园林的路径简化图如下所示。

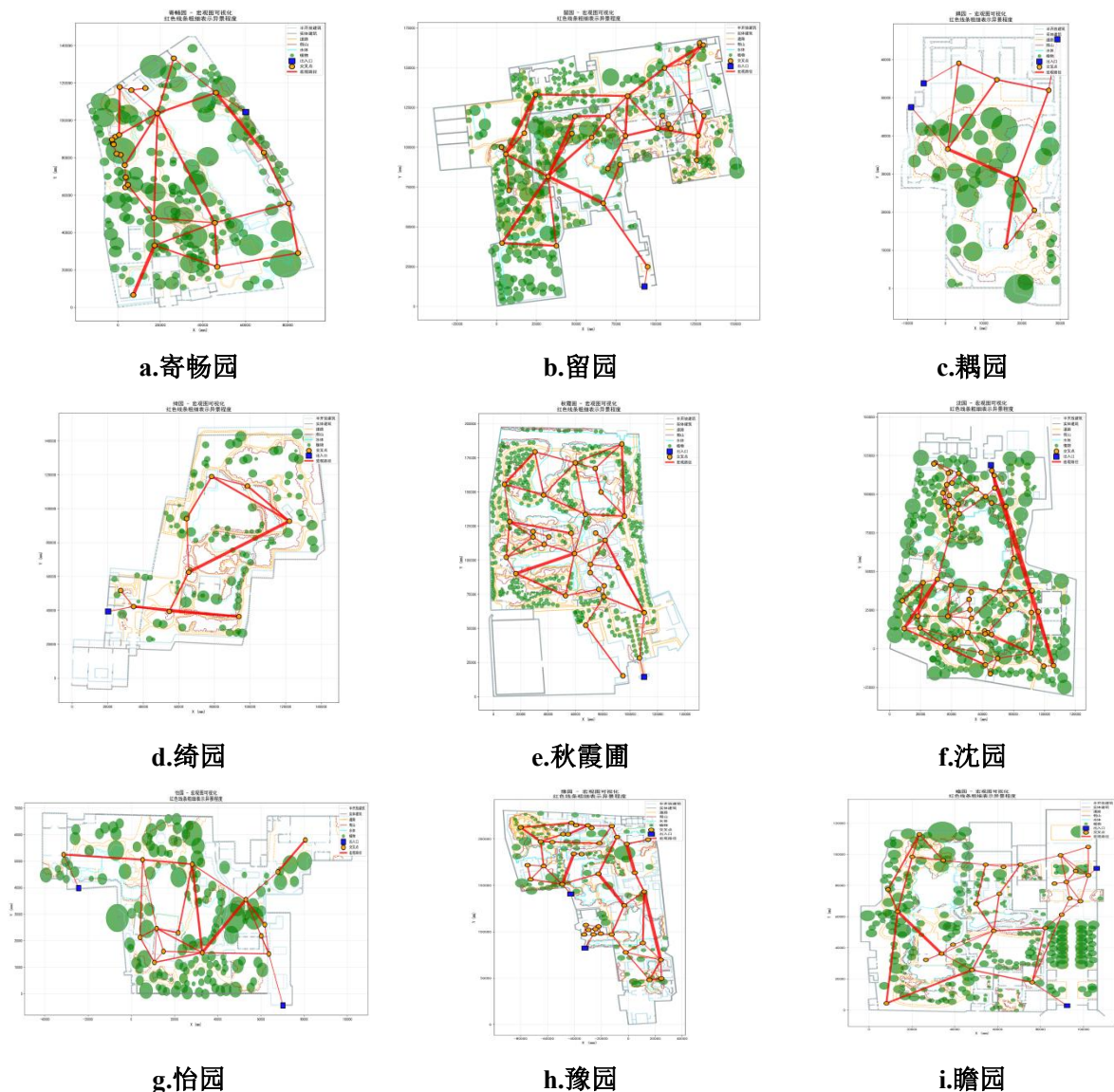


图 19 园林路径简化宏观图-其余 9 个园林

以下是对 NSGA-II 算法对模型求解的完整流程步骤的详细梳理：

**Step1:** 基于出入口先验路径初始化种群。由于园林路径宏观图复杂程度高，选择使用先验+随机生成相结合的方式，人工定义初始化路径需求，要求路径节点序列足够长，路径足够复杂，为后续进化提供足够的空间。

先验路径集合生成是从任意出入口节点对  $(u, v) (u \in E_{in/out}, v \in E_{in/out}, u \neq v)$  提取节点数不超过  $cutoff + 1$  的简单路径： $PriorPaths = \bigcup_{u, v \in E_{in/out}, u \neq v} SimplePaths(u, v, cutoff = 20)$ ，其中

$SimplePaths(u, v, cutoff)$  表示从  $u$  到  $v$ ，节点数  $\leq cutoff + 1$  的简单路径集合。

从先验路径集合中随机抽样  $N$  条路径，编码后组成初始种群， $N$  为种群规模，初始值  $N=500$ ， $Pop(0) = \{Encode(P) | P \in PriorPaths, |Pop(0)| = N\}$ 。

**Step2:** 对个体进行编码，编码方式为通过步骤选择序列而不是点序列，将路径节点序列转化为“起点+终点+步骤选择序列”，适配交叉、变异遗传操作。

编码函数  $Encode(P)$  定义：设路径  $P=[v_0, v_1, \dots, v_n]$ ，起点  $v_0$ ，终点  $v_n$ ，对每个中间节点  $v_i (1 \leq i \leq n-1)$ ，取其在宏观图中邻居集合  $Neighbors(v_i)$  中  $v_{i+1}$  的索引  $s_i$ ，则编码结果为：

$$Encode(P) = (v_0, v_n, S), \quad S = \{s_1, s_2, \dots, s_{n-1}\}, \quad s_i = \text{index}(Neighbors(v_i), v_{i+1})$$

其中， $Neighbors(v_i)$  为节点  $v_i$  的所有相邻节点集合， $\text{index}(A, x)$  表示元素  $x$  在集合  $A$  中的位置索引。

**Step3:** 进行目标函数的计算，对种群中每个个体解码得到路径，计算其总异景程度和重复长度。定义解码函数  $Decode(Ind)$ ，已知个体  $Ind = (v_0, v_n, S)$ ，从  $v_0$  出发，按步骤序列  $S$  依次选取邻居节点，直至到达  $v_n$ ，得到路径  $P$ ：

$$v_{i+1} = Neighbors(v_i)[s_i], \quad i = 1, 2, \dots, |S|, \quad v_{|S|+1} = v_n \quad (15)$$

**Step4:** 进行非支配排序，按帕累托支配关系对种群个体分层，优先保留非支配解，无其他解在两目标上均更优。帕累托支配定义为若个体  $Ind_a$  支配  $Ind_b$ ，记为  $Ind_a < Ind_b$ ，则

$$\text{满足：} \begin{cases} f_1(Decode(Ind_a)) \geq f_1(Decode(Ind_b)) \\ f_2(Decode(Ind_a)) \leq f_2(Decode(Ind_b)) \end{cases}, \quad \text{且至少有一个不等式严格成立。}$$

进行分层操作时，第 1 层  $F_1$  为种群中所有非支配解，剔除  $F_1$  后，剩余解中的非支配解构成  $F_2$ ，以此类推：
$$F_k = \left\{ Ind \in \text{Pop}(t) \setminus \bigcup_{i=1}^{k-1} F_i \mid \nexists Ind' \in \text{Pop}(t) \setminus \bigcup_{i=1}^{k-1} F_i, Ind' < Ind \right\}$$

**Step5:** 进行拥挤度距离计算，计算同层个体在目标空间中的“拥挤度”，避免最优解聚集，保证解的多样性。定义个体  $Ind_i$  的拥挤度距离  $d_i$ ：对每层  $F_k$  按两目标分别排序，边界个体目标极值点拥挤度设为无穷大，中间个体拥挤度为相邻个体的目标差值之和：

$$d_i = \begin{cases} \infty, & \text{Ind}_i \text{ 为 } F_k \text{ 中 } f_1 \text{ 或 } f_2 \text{ 的极值点} \\ \sum_{m=1}^2 \frac{f_m(Ind_{i+1}) - f_m(Ind_{i-1})}{f_m^{\max} - f_m^{\min}}, & \text{其他} \end{cases} \quad (16)$$

式中： $f_m^{\max} / f_m^{\min}$  为第  $m$  个目标在  $F_k$  中的最大/最小值； $Ind_{i-1} / Ind_{i+1}$  为  $Ind_i$  排序后的前/后相邻个体。

**Step6:** 基于锦标赛选择原则进行操作，基于排序等级和拥挤度距离选择父代，等级越高层数越小、拥挤度越大的个体被选中概率越高。个体优先级规则为：若  $\text{rank}(Ind_a) < \text{rank}(Ind_b)$ ，则  $Ind_a$  优先；若  $\text{rank}(Ind_a) = \text{rank}(Ind_b)$ ，且  $d_a > d_b$ ，则  $Ind_a$  优先，其中  $\text{rank}(Ind)$  为个体所在层的序号。锦标赛选择时，随机抽取  $k=2$  个体，选择优先级最高的作为父代， $Parent = \arg \max_{Ind \in \text{Sample}(\text{Pop}(t), 2)} \text{Priority}(Ind)$ 。

**Step7:** 进行交叉操作，仅对步骤序列长度  $> 2$  的个体执行交叉，交换中间段步骤序列，确保交叉后个体的起点、终点不变。设父代  $Ind_1 = (v_{0,1}, v_{n,1}, S_1)$ ,  $Ind_2 = (v_{0,2}, v_{n,2}, S_2)$ ，满足  $v_{0,1} = v_{0,2}$ ,  $v_{n,1} = v_{n,2}$ ，且  $|S_1| > 2$ 、 $|S_2| > 2$ ，随机选取交叉区间，对  $S_1 = [s_{1,1}, \dots, s_{1,m1}]$ ，取  $a \in [1, m1-2]$ ,  $b \in [a+1, m1-1]$ ；对  $S_2 = [s_{2,1}, \dots, s_{2,m2}]$ ，取  $c \in [1, m2-2]$ 、 $d \in [c+1, m2-1]$ ，交换中间段并生成子代  $S'_1 = [s_{1,1}, \dots, s_{1,a-1}, s_{2,c}, \dots, s_{2,d}, s_{1,b+1}, \dots, s_{1,m1}]$ ， $S'_2 = [s_{2,1}, \dots, s_{2,c-1}, s_{1,a}, \dots, s_{1,b}, s_{2,d+1}, \dots, s_{2,m2}]$ ， $Ind'_1 = (v_{0,1}, v_{n,1}, S'_1)$ ， $Ind'_2 = (v_{0,2}, v_{n,2}, S'_2)$ 。

**Step8:** 进行变异操作，通过删除、插入、替换步骤三种方式变异，确保变异后步骤序列长度在 [2,100] 范围内。设待变异个体  $\text{Ind} = (v_0, v_n, S)$ ,  $S = [s_1, \dots, s_m]$ ，变异须满足  $2 \leq m \leq 100$ ，触发删除步骤时，随机取  $k \in [1, m-1]$  删除  $s_k$  即  $S' = [s_1, \dots, s_{k-1}, s_{k+1}, \dots, s_m]$ ；触发插入步骤时  $S' = [s_1, \dots, s_k, s_{\text{new}}, s_{k+1}, \dots, s_m]$ ；替换步骤时  $S' = [s_1, \dots, s_{k-1}, s_{\text{new}}, s_{k+1}, \dots, s_m]$ ，变异后个体  $\text{Ind}' = (v_0, v_n, S')$ 。

**Step9:** 进行种群更新，合并父代与子代种群，按非支配排序和拥挤度距离筛选前 N 个个体，组成下一代种群。合并种群  $\text{MergedPop} = \text{Pop}(t) \cup \text{Offspring}(t)$ ，对  $\text{MergedPop}$  分层得到  $F_1, F_2, \dots$ ，依次选取个体，若  $|F_1| + \dots + |F_k| \leq N < |F_1| + \dots + |F_{k+1}|$ ，则从  $F_{k+1}$  按拥挤度降序补全至 N 个， $\text{Pop}(t+1) = \bigcup_{i=1}^k F_i \cup \text{Top}\left(F_{k+1}, N - \sum_{i=1}^k |F_i|\right)$ 。

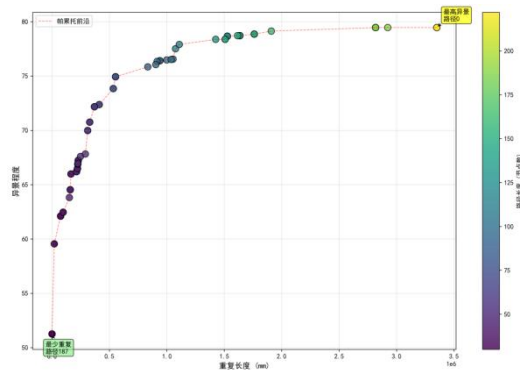
**Step10:** 终止条件判断  $t \geq \max_g \text{generations} \Rightarrow \text{Output}(\text{ParetoFront})$ ，若迭代次数达到预设最大值，输出帕累托前沿；否则返回步骤 3 继续迭代。

以拙政园为例，模型求解过程中，生成了 74 条先验路径，运行 NSGA-II 算法设定 50 代，种群大小 500，具体迭代过程如表 2 所示。

**表 2 NSGA-II 迭代运算过程**

| 迭代次数     | 当前最优异景程度 | 当前最优重复长度 |
|----------|----------|----------|
| 第 0 代完成  | 62.9092  | 0.0      |
| 第 10 代完成 | 77.8063  | 0.0      |
| 第 20 代完成 | 79.4579  | 0.0      |
| 第 30 代完成 | 79.4579  | 0.0      |
| 第 40 代完成 | 79.4579  | 0.0      |

优化完成后获得 205 条帕累托最优路径，异景程度范围:51.2522-79.4579，重复长度范围:0.0-3351307.5；路径长度范围为 30-222 个节点帕累托前沿质量指标（计算帕累托前沿的超体积指标）为 0.4884。图 20 是 NSGA-II 算法求解江南古典园林游线规划多目标优化问题的帕累托前沿结果。横轴为路径重复长度（mm），纵轴为异景程度，粉色虚线是非支配解组成的帕累托前沿，体现异景程度与路径重复长度的权衡关系：重复长度越小，异景程度往往越低，左下角节点数少的短路径，适合追求效率的游客；重复长度越大，异景程度往往越高，右上角节点数多的长路径，适合追求深度趣味的游客。不同颜色对应路径长度，整体为园林游线规划提供了覆盖多样偏好的最优方案集合，展现了算法在多目标权衡中生成丰富最优解的优势。



**图 20 帕累托前沿图-拙政园**

其他 9 个园林的帕累托前沿图如下所示。

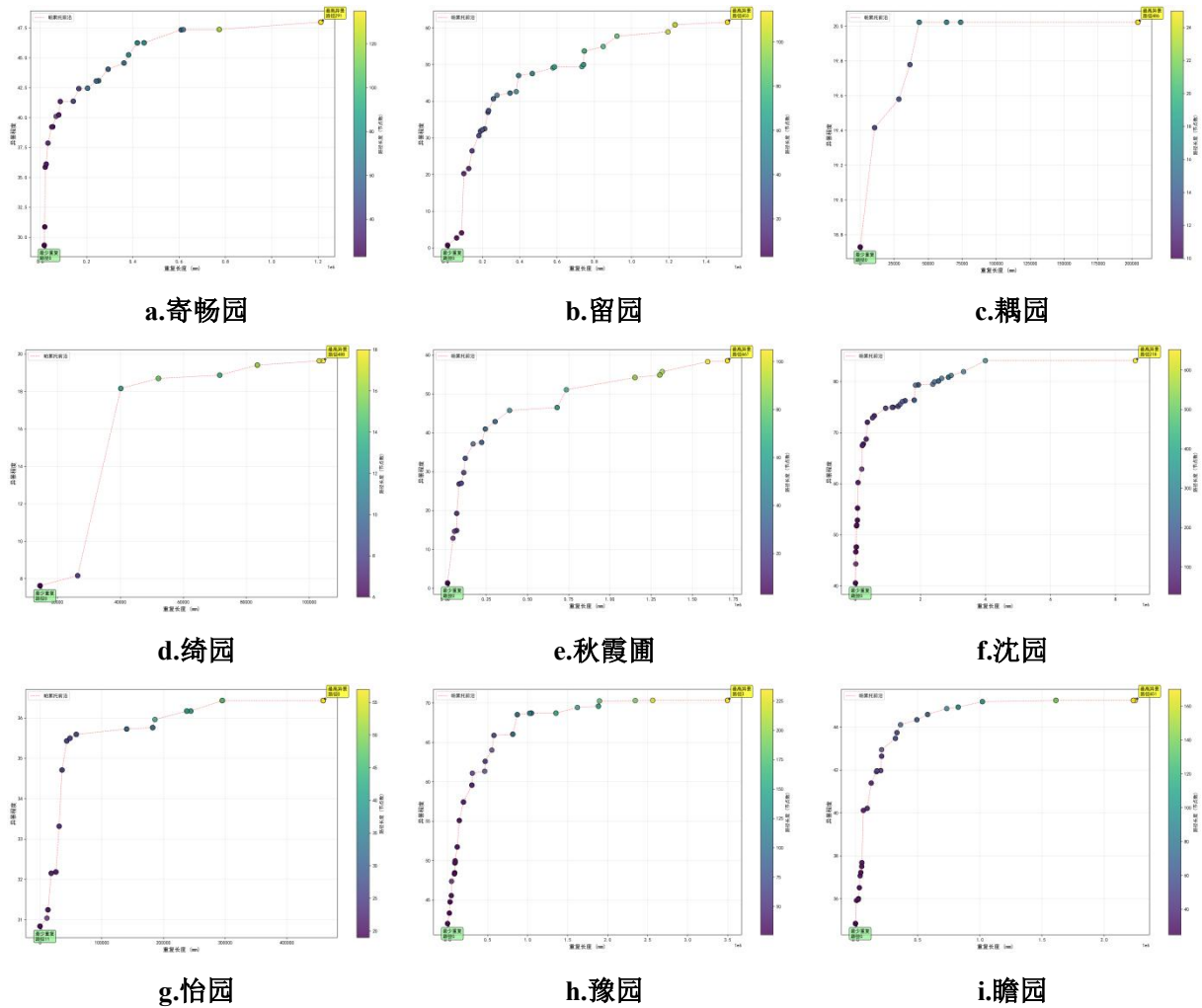
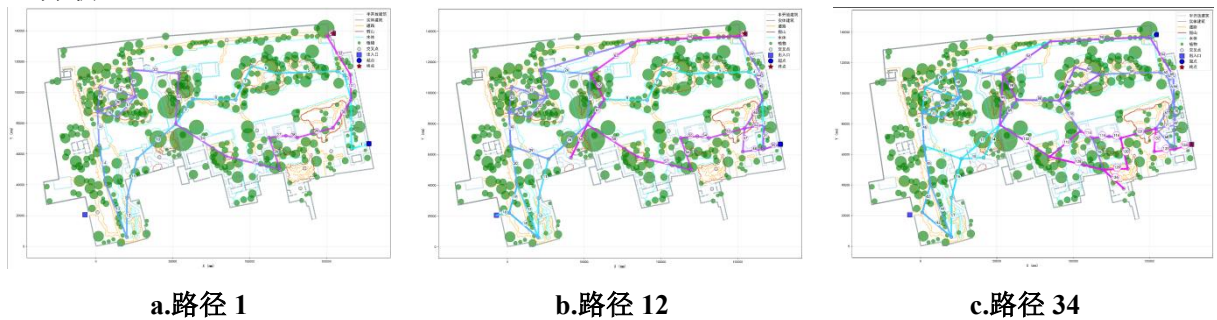


图 21 帕累托前沿图-拙政园

#### 4.4.4 游园路线可视化展示与关键位置异景程度分析

NSGA-II 算法最终输出的帕累托前沿 (pareto\_front) 覆盖所有合理的权衡方案, 如“高异景-低重复”“中异-极低重复”等, 符合园林游线的多样化需求; 结合 NSGA-II 的拥挤度距离, 避免最优解在目标空间中聚集, 确保前沿解的空间多样性, 可满足不同游客的偏好。以拙政园为例, 由于篇幅原因, 仅展示具有代表性的 11 条最优路径可供游客自行选择, 其中包含了重复度最高的方案和路线无重复的方案。下面重点对路线无重复的方案进行深入分析。



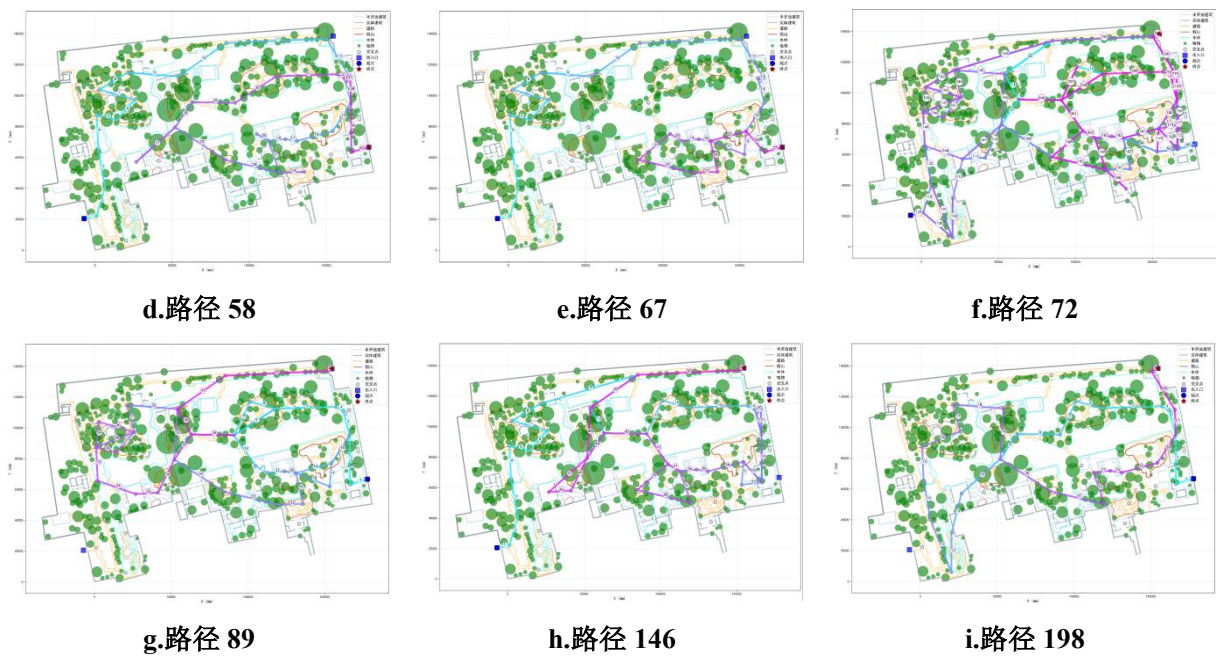
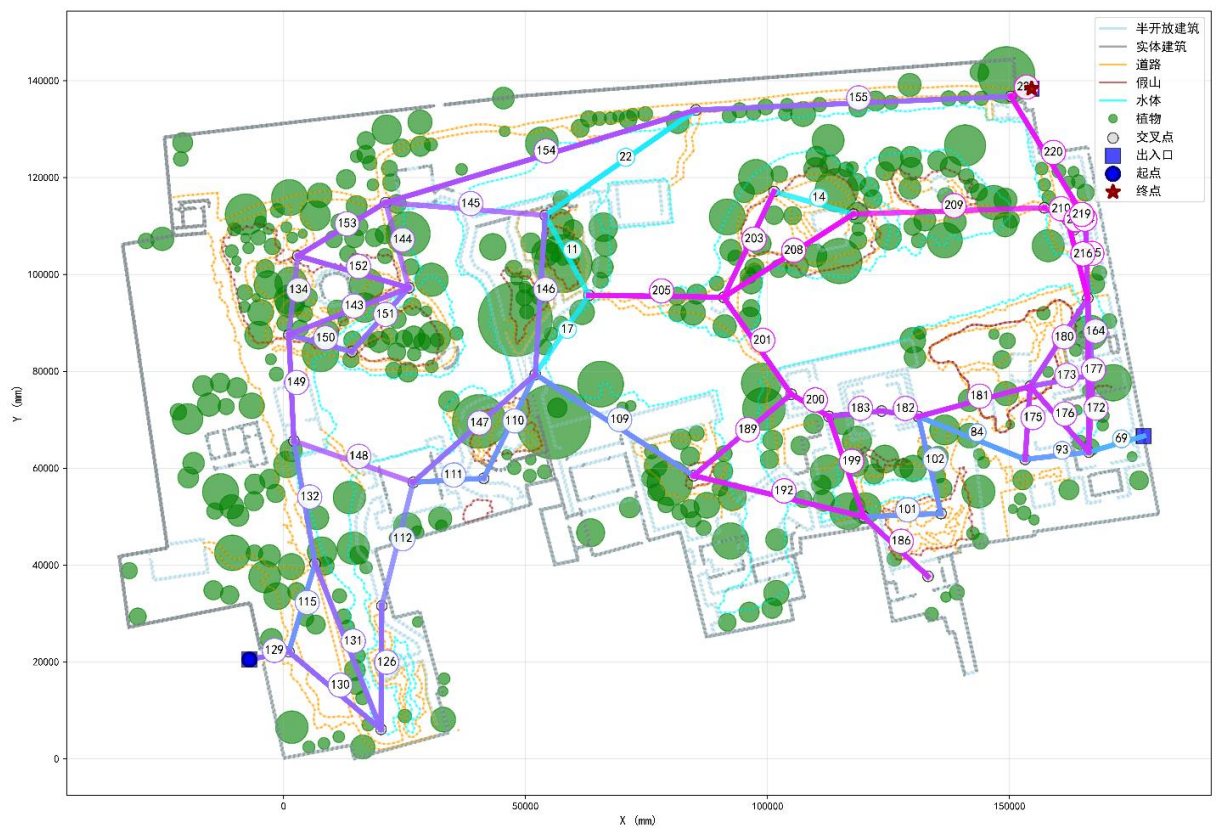


图 22 部分最优游园路线展示-拙政图



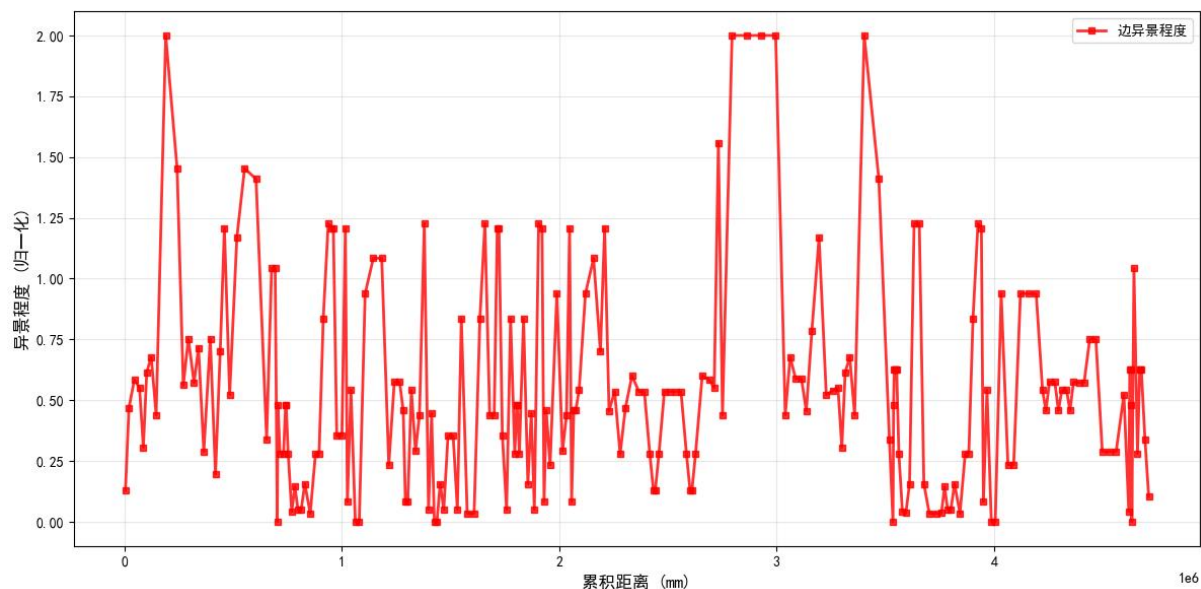


图 24 异景程度随游线距离变化图（重复度最高方案-拙政园）

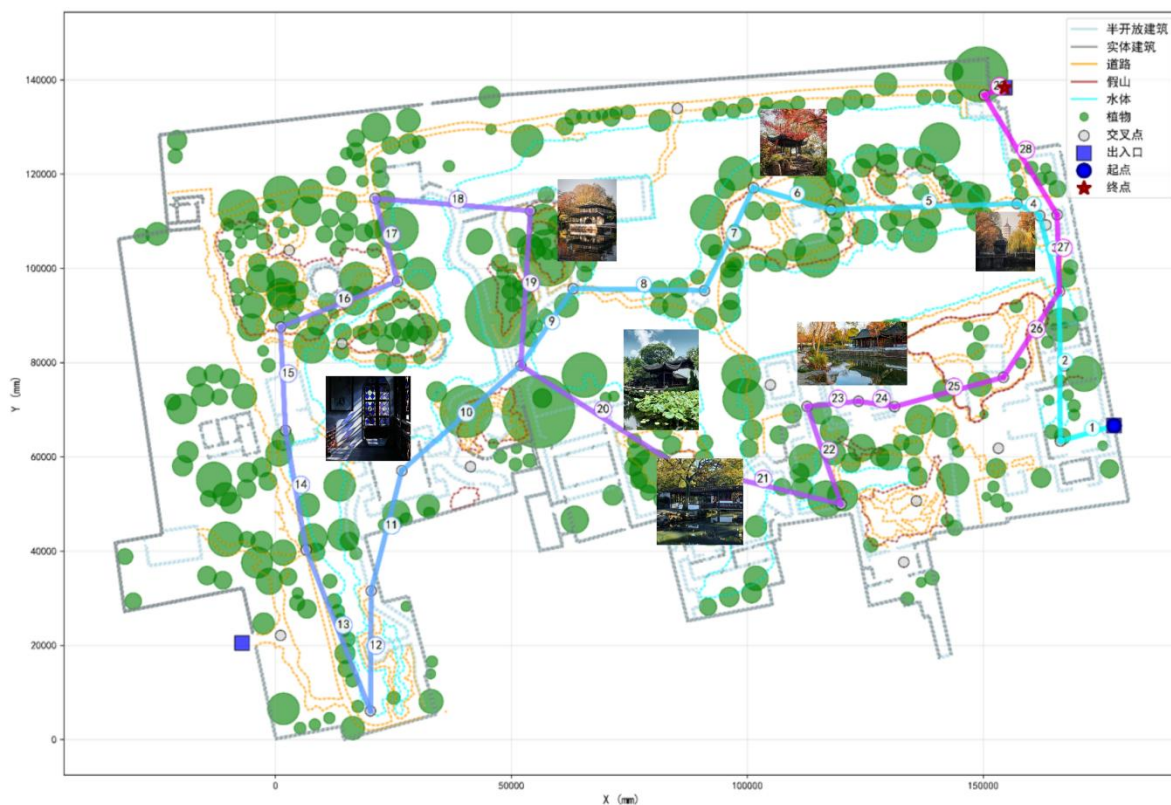


图 25 游园路径 187 可视化展示-拙政园

路径 187 的节点序列: [3480, 5179, 5181, 5176, 5193, 5175, 4710, 5187, 3595, 5195, 5189, 1352, 5201, 2334, 5190, 5178, 5184, 5197, 5198, 5195, 5185, 5199, 5188, 5186, 5194, 5183, 5181, 5180, 1877, 1820], 节点数量:30 个, 路径异景程度总和为 51.2522, 重复长度为 0.0mm, 异景程度平均值为 0.8343, 最大值为 1.2947, 最小值为 0.3173。

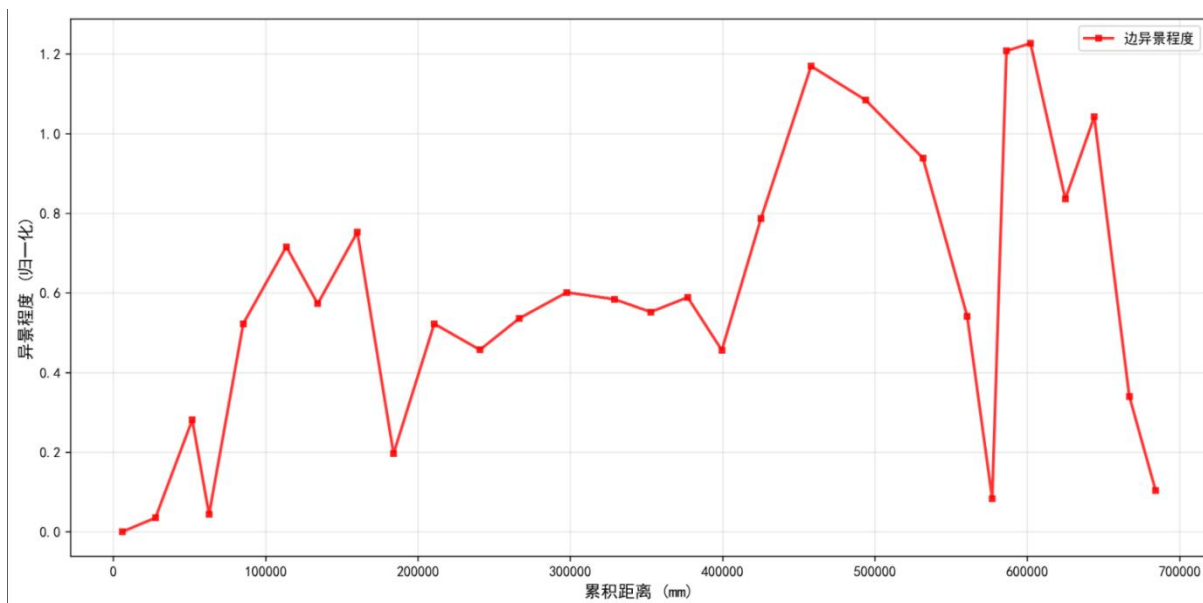


图 26 异景程度随游线距离变化图（路径 187-拙政园）

从起点出发，行至梧竹栖居时，周围梧桐疏朗、修竹挺拔，与雅致小馆相映成趣，此前路径的景观在此处转为“梧竹环绕、清幽小筑”的画面，异景程度因植物与建筑的清新搭配有所提升。接着来到雪香云蔚亭，它立于池西假山之上，假山嶙峋、梅枝疏影与开阔池水呼应，景观从梧竹小筑的细腻转为山亭望水、梅影横斜的疏旷感，异景程度随场景层次转变而变化。再步入三十六鸳鸯馆，馆内分隔精巧，外临鸳鸯池，水禽悠游其间。从雪香云蔚亭的山野疏旷进入此处，建筑的精致木雕与池水禽鸟的灵动形成对比，景观从“山亭梅影”的自然意趣变为“馆舍精巧、水禽灵动”的交融之景，异景程度因氛围切换而上升。随后抵达香洲，这座画舫形建筑浮于水面，从鸳鸯馆的“馆池相依”到此处，“舫浮碧波”的独特造型与水环境结合，带来新的视觉冲击，异景程度显著提升——“舟行水上”的意境与之前的馆舍景观差异明显。接着行至小飞虹，廊桥轻盈横跨水面，连接两侧景致。从香洲的“画舫亲水”来到这里，廊桥的造型、桥下潺潺流水，以及桥两侧不同的景观层次，让景观从“舫的静态亲水”转为“桥连景、水穿桥”的灵动画面，异景程度因空间引导和水景动态感而波动。最后抵达远香堂，这座中部主体建筑面水而立，眼前池水浩渺，对岸假山花木尽收眼底，景观从飞虹的“小桥流水”转为“厅堂雄踞、山水大境铺展”的宏大格局，异景程度达到高峰——空间从紧凑的桥廊转为开敞的堂前山水，层次与气势都有质的提升。

每一处异景程度的峰值，都是拙政园“移步换景”美学的诗性迸发：或从建筑到山水，开启天地大美的卷轴；或从平远到幽深，探入园林的肌理褶皱；或从自然到人文，触摸时光沉淀的匠心，给人们带来沉浸式美的感受。

#### 4.4.5 其他园林游园路线规划的可视化展示与分析

从 9 个园林的最优游园路线中各选择典型路线，用于可视化展示与分析，具体如下：

##### 1. 寄畅园

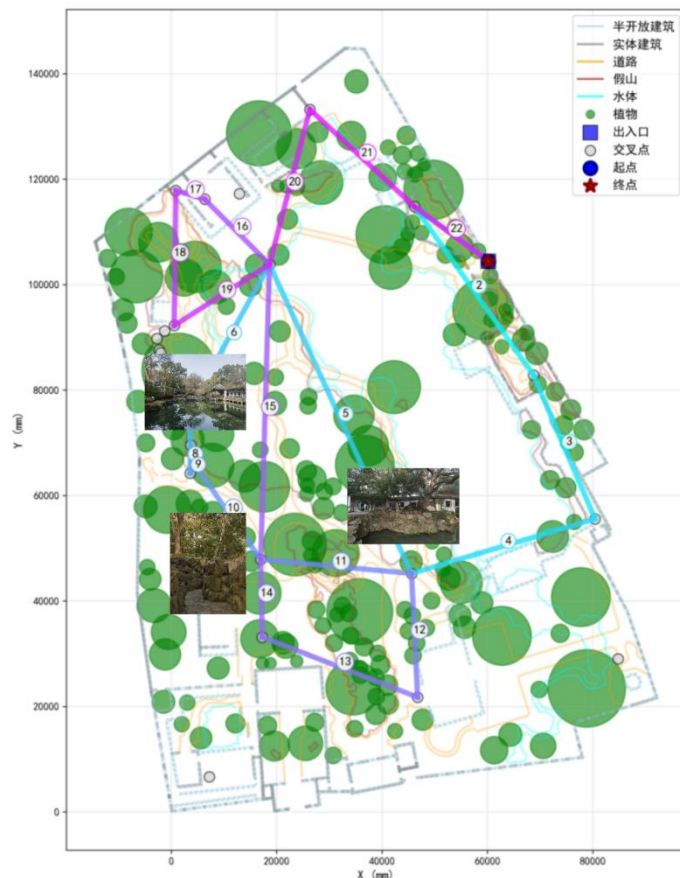


图 27 游园路径 413 展示-寄畅园

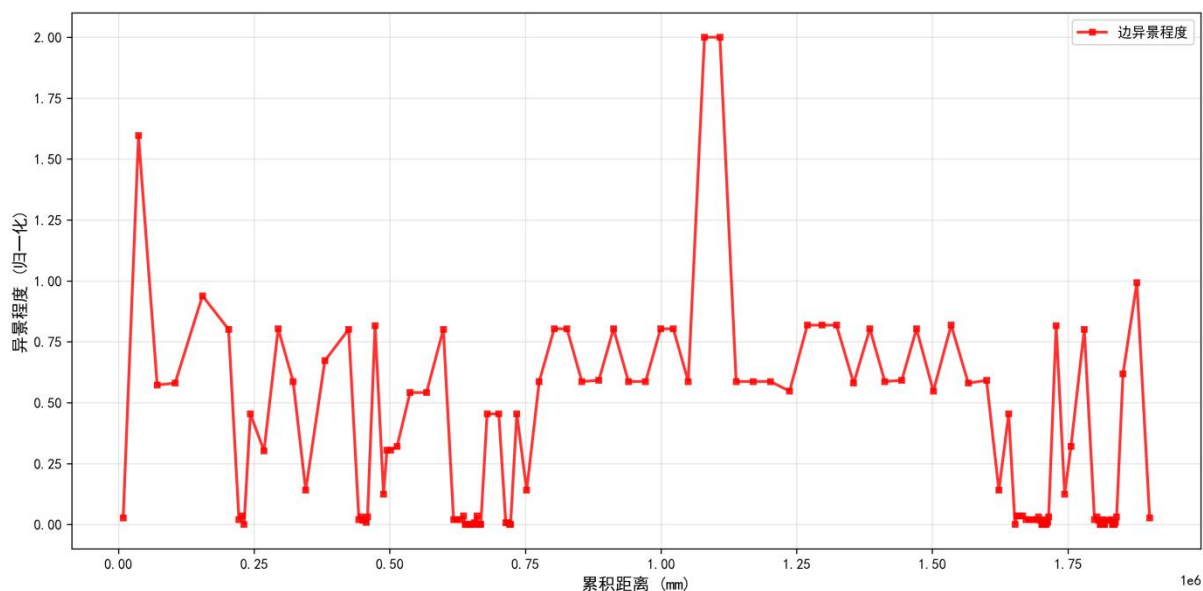


图 28 异景程度随游线距离变化图（路径 413-寄畅园）

从西北入口踏入八音涧，行至 150 米处的叠石溪谷，异景程度已悄然爬升：溪声穿石而过，两侧林木交柯如盖，每一步都踏在“山林幽闭”的细碎意境里。行至 350 米处的知鱼槛，异景程度骤然跃至峰值，脚下溪流忽然汇入南部镜池，视线从“石间寻溪”的局促，猛然撞入“天光接水”的开阔，池边垂柳轻拂水面，与北岸惠山的轮廓相映，山因水活，水映山灵，此处“涧转池开”的转折，正是“江南第一园”幻境感的核心。行至 800 米的

环翠楼，异景程度再起波澜：登楼凭栏，从“池边亲水”转为“凭高望园”，园内亭台、林木、水体如棋盘般铺展，景观尺度的骤变，让“山麓别墅”的悠然尽显。

2.留园



图 29 游园路径 454 展示-留园

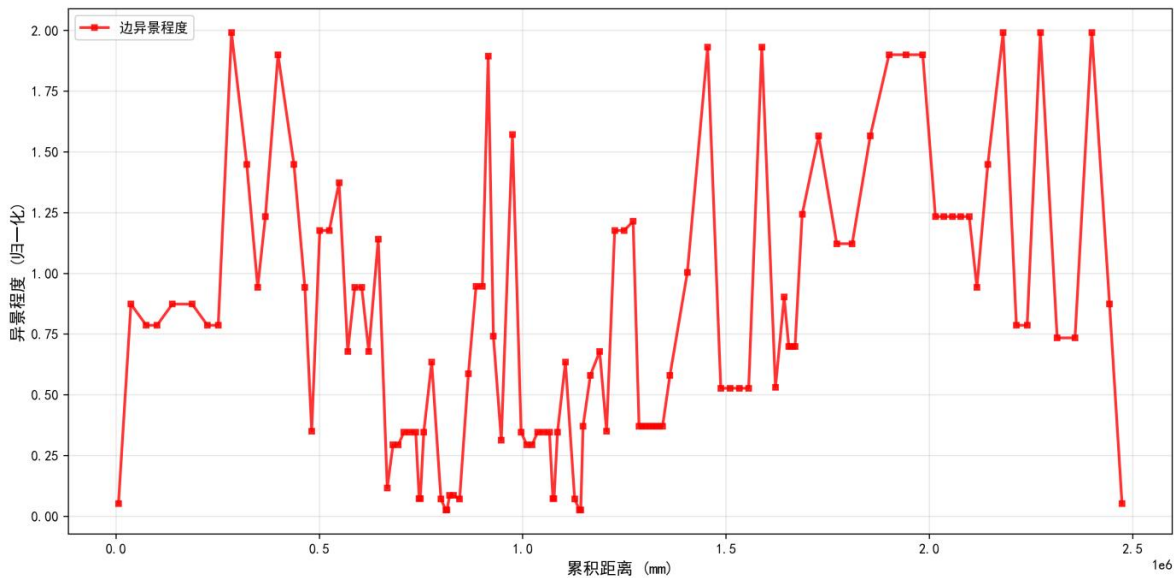


图 30 异景程度随游线距离变化图（路径 454-留园）

从东门入廊，行至 100 米的古木交柯，异景程度迎来首个峰值：狭长曲廊忽然破开，百年古柏与榉树撑开浓荫，石桌石凳隐于其间，此前“廊柱框景”的半遮半掩，化作“树影筛

光”的豁然开朗。行至 300 米的冠云峰，异景程度再攀高峰：太湖石孤峰峭立，“瘦、透、漏”的风骨与周边“林泉耆硕之馆”的规整建筑形成强烈对比，石下碧波映着峰影，从“古木清幽”到“石韵天成”的转换，藏尽留园“曲廊蜿蜒”的巧思。行至 1200 米的五峰仙馆，异景程度于岔路间波动：多岔游线将假山、水体、建筑串联，每一次转角都能撞见“石映水、水绕廊”的新景，让“步移景异”的趣味在路径尽头仍未消散。

3.耦园

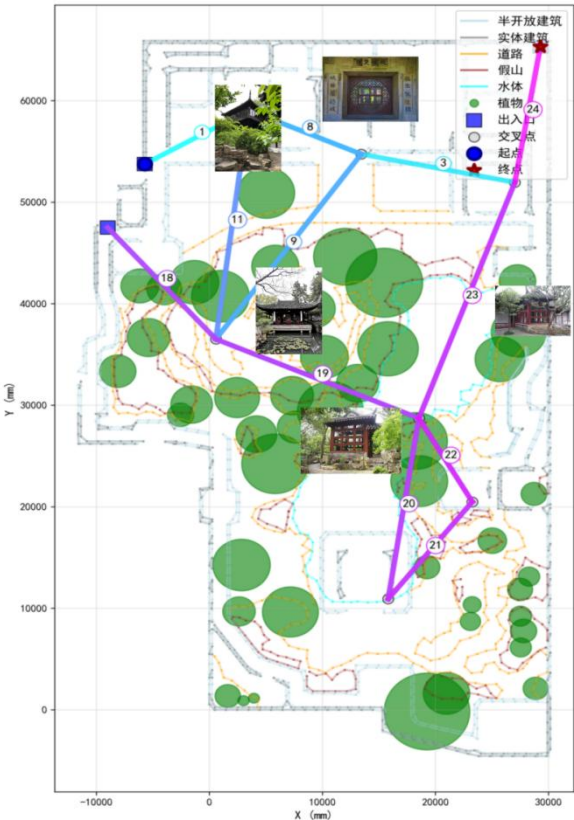


图 31 游园路径 487 展示-耦园

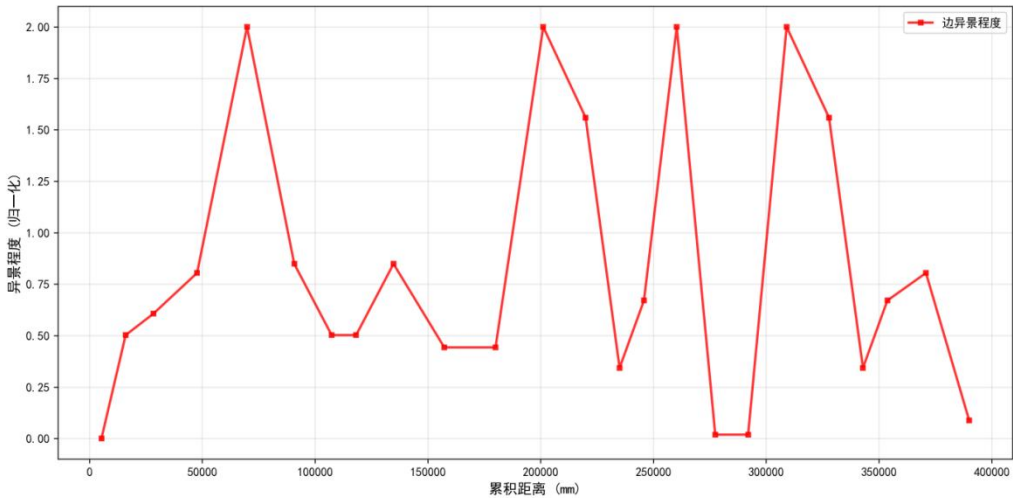


图 32 异景程度随游线距离变化图（路径 487-耦园）

从正门入园，行至 200 米的东园连廊，异景程度随漏窗疏密缓缓起伏：廊壁漏窗如筛，将西园的山石草木筛成细碎光影。行至 400 米的听橹楼，异景程度陡然升至顶点：连廊尽

头忽然临水，三面临河的榭台直面碧波，此前“隔窗观景”的含蓄，化作“凭栏听橹”的开阔，岸边垂柳与水中倒影相缠，与园内“城曲草堂”的黛瓦相映，此处“宅园接水”的转折，恰是耦园“三面临河”的独特诗意。行至 600 米的西园假山，异景程度又入幽境：叠石如屏，石径蜿蜒，从“临水听波”到“穿石寻幽”的切换，让小园空间在“开”与“合”间流转。

#### 4. 绮园



图 33 游园路径 488 展示-绮园

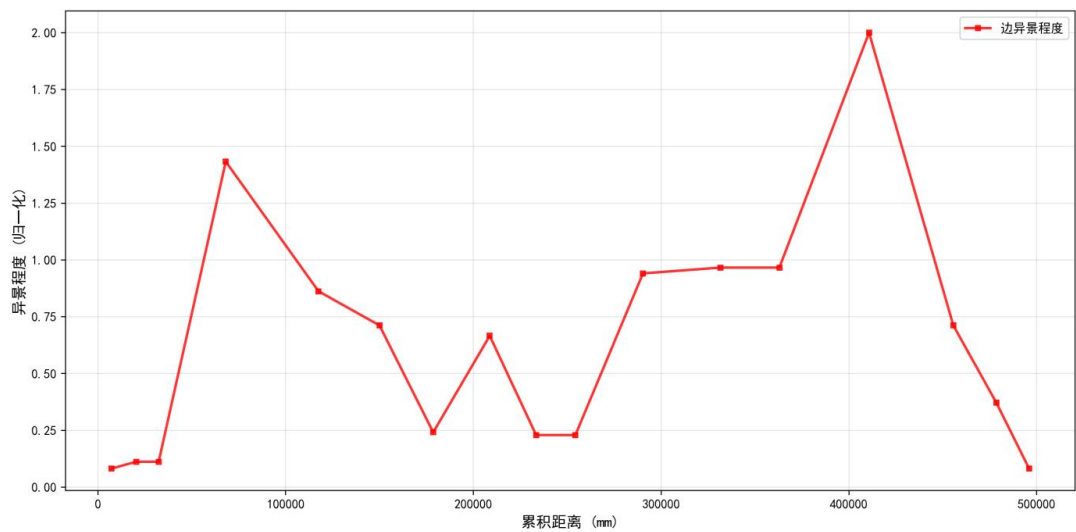


图 34 异景程度随游线距离变化图（路径 488-绮园）

从正门出发，循“水随山转”的路径行至 300 米的山路竹丛，异景程度在竹影中轻缓爬升：修篁夹道，石径隐于其间，尽显“山林野趣”。行至 500 米的核心潭池，异景程度迎来峰值：山路忽断，一池清潭横亘眼前，潭岸叠石如奔浪，水流绕石注入下游溪涧，此前“竹径幽深”的封闭，瞬间被“水光潋滟”的开阔取代，潭边“小飞来”假山与水面相映，恰是“山因水活”的生动注脚。行至 800 米的九曲桥，异景程度再添灵动：桥身蜿蜒过溪，桥下流水潺潺，从“潭池浩渺”到“溪桥玲珑”的转换，藏着绮园“兼苏扬杭之长”的造园智慧

5.秋霞圃

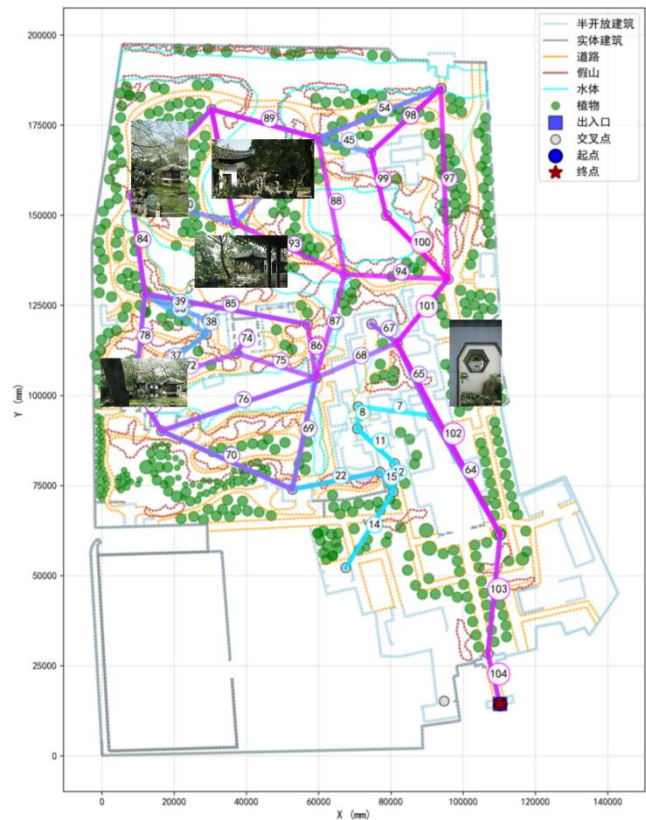


图 35 游园路径 0 展示-秋霞圃

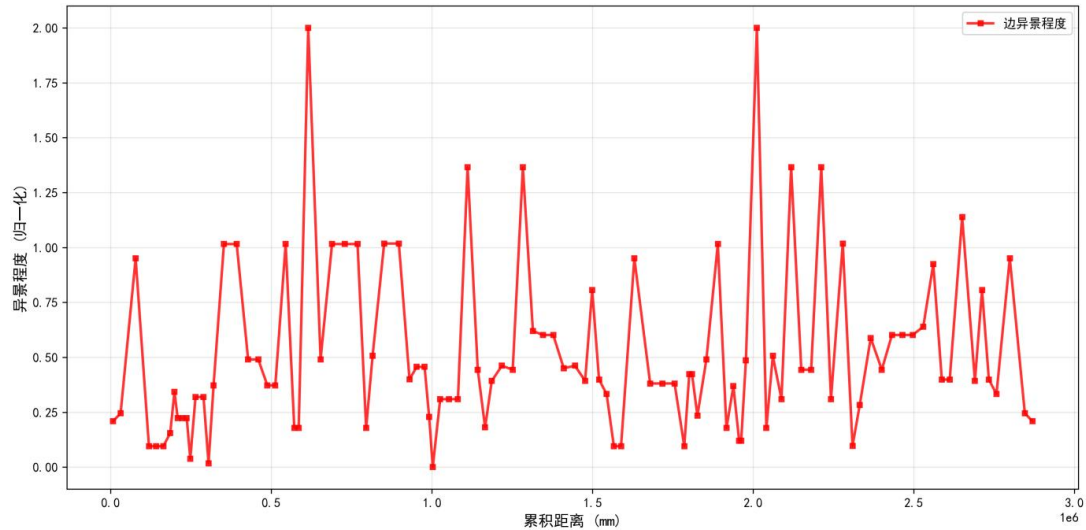


图 36 异景程度随游线距离变化图（路径 0-秋霞圃）

从北门入园，行至 400 米的凝和堂侧廊，异景程度在“城市山林”的树影中起伏：槭树、桧柏密集，石径隐于浓荫。行至 600 米的凝和堂前庭院，异景程度骤达高峰：侧廊转出，庭院豁然开朗，一池碧水映着粉墙黛瓦，“碧光亭”浮于池心，此前“林深不知处”的幽闭，化作“庭院深深”的雅致，池边石栏与堂前匾额相映，尽显“上海五大名园”的精巧。行至 1000 米的桃花潭，异景程度再起波澜：从“庭院水榭”转入“潭边桃林”，春日桃枝蘸水，从“人工精巧”到“自然野趣”的切换，让“城市山林”的意境愈发真切。

### 6. 沈园

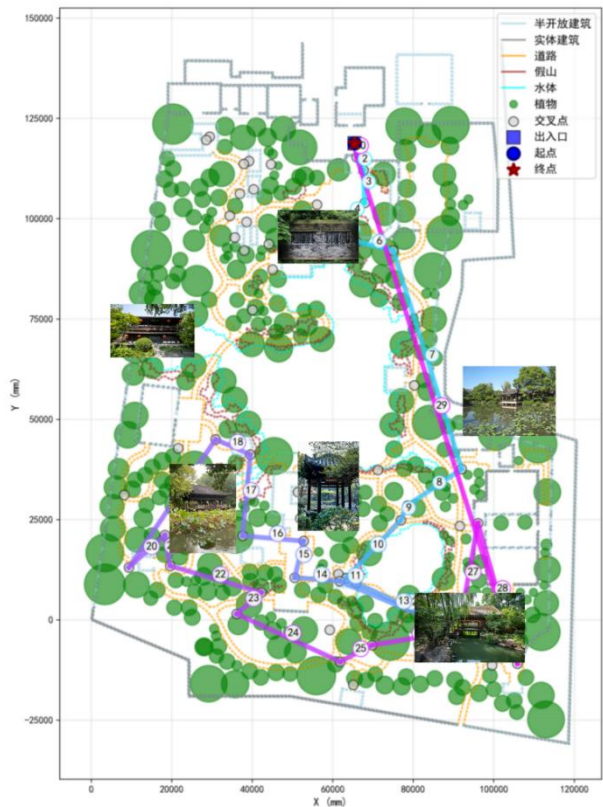


图 37 游园路径 0 展示-沈园

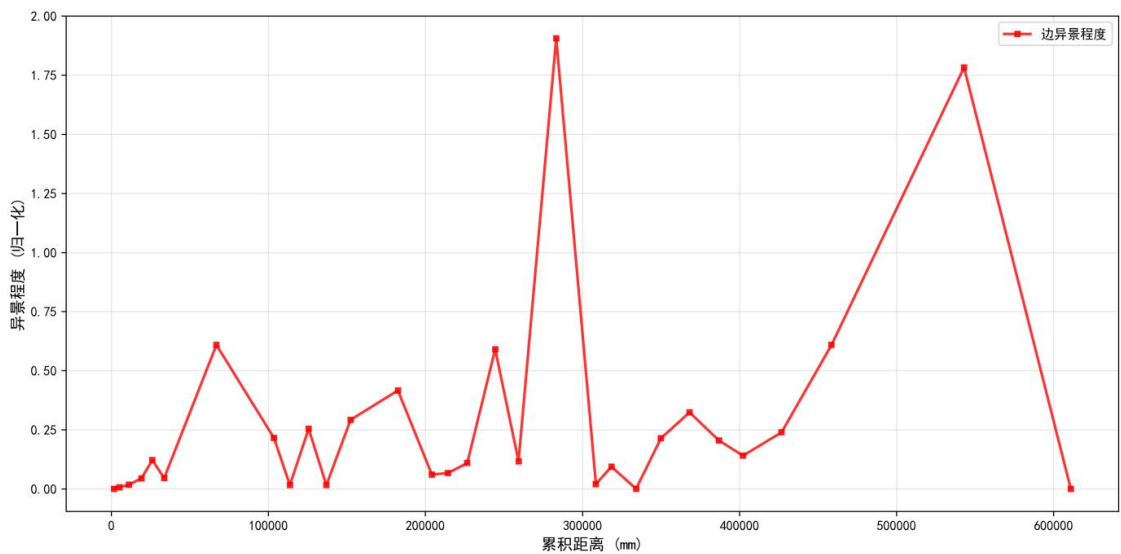


图 38 异景程度随游线距离变化图（路径 0-沈园）

从正门踏入孤鹤轩，行至 300 米的轩前桂树旁，异景程度在桂影中轻缓起伏：百年桂树撑开浓荫，石凳散落其间，尽显“宋式遗风”的静谧。行至 600 米的钗头凤碑廊，异景程度因氛围骤转而内敛：碑刻上“红酥手”的字迹沉郁，廊下光影昏暗，历史的厚重让景观节奏放缓。行至 900 米的葫芦池，异景程度陡然跃至峰值：碑廊尽头忽见池水如葫芦卧地，柳丝依依拂过水面，“问梅槛”临池而立，此前“碑刻寄情”的凝重，与眼前“水光柳色”的灵动形成强烈碰撞，此处“情与景”的交织，正是沈园诗意的灵魂。

### 7.怡园

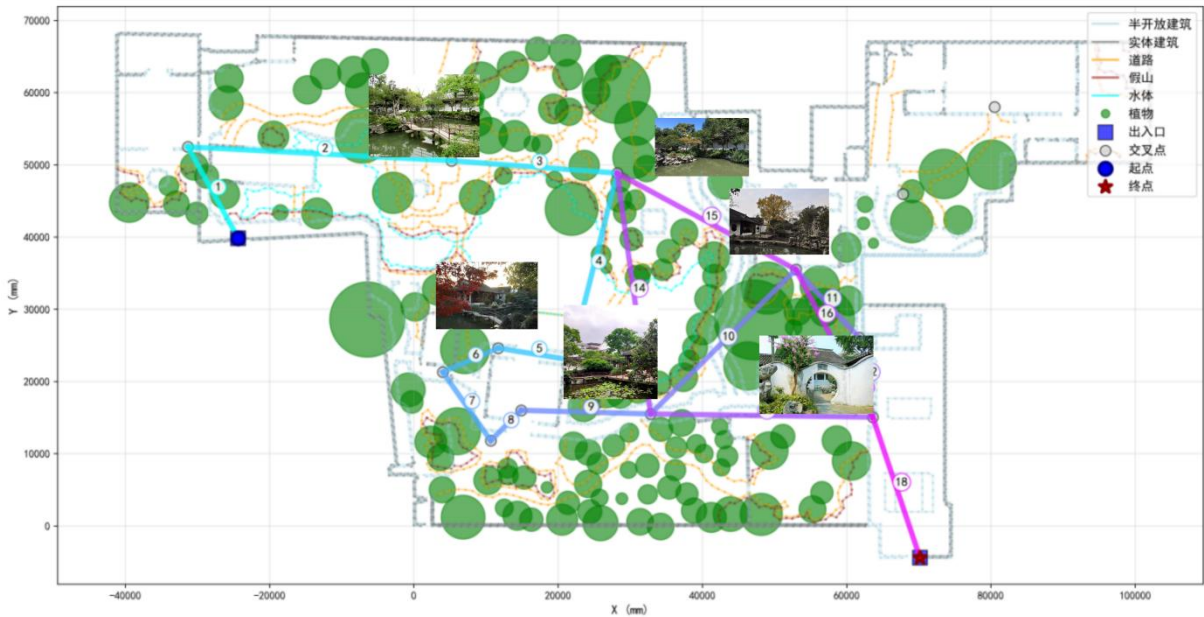


图 39 游园路径 0 展示-怡园

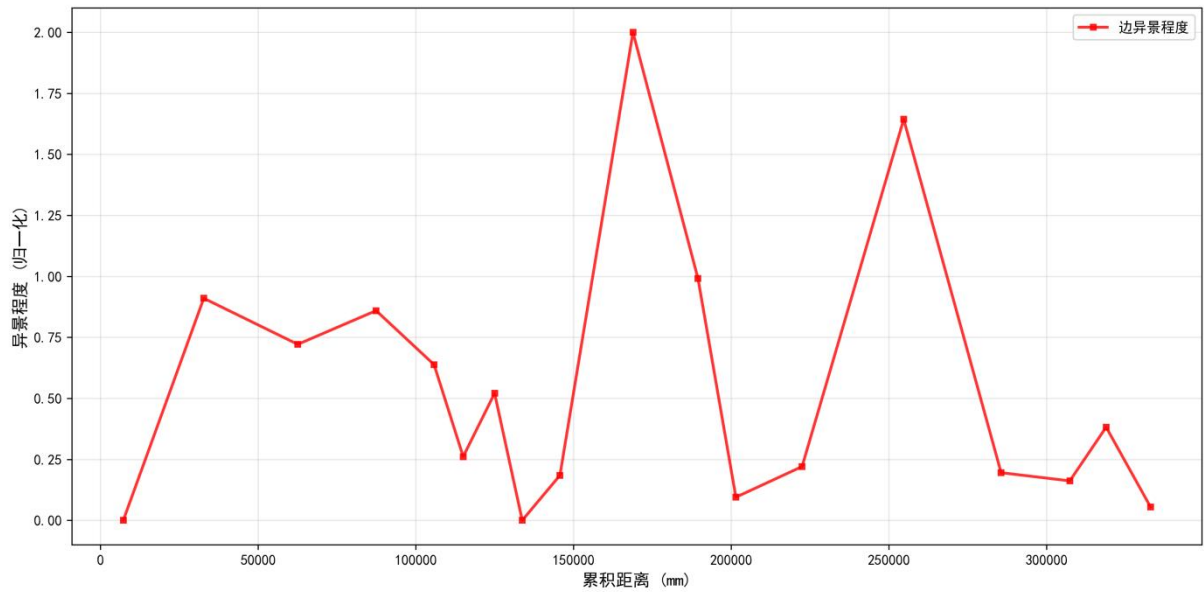


图 40 异景程度随游线距离变化图（路径 0-怡园）

从正门入园，行至 200 米的藕香榭，异景程度在荷风中轻缓爬升：小池植荷，榭临池而建，仿拙政园的“荷风送香”意境初显。行至 450 米的玉延亭，异景程度迎来首个峰值：亭前忽然接入仿留园的曲廊，廊壁漏窗将远处“螺髻亭”的攒尖顶框成小品，从“池边观荷”的开阔，到“漏窗借景”的精巧，园林风格的无缝切换，尽显怡园“融合众园之长”的特质。

行至 700 米的锁绿轩，异景程度再攀高峰：轩前古柏如盖，与轩内“锁绿”匾额相映，此前“廊下寻景”的流动，化作“轩内品绿”的静赏，小园“小巧玲珑”却“意蕴悠长”的妙处，尽在此间。

8.豫园



图 41 游园路径 0 展示-豫园

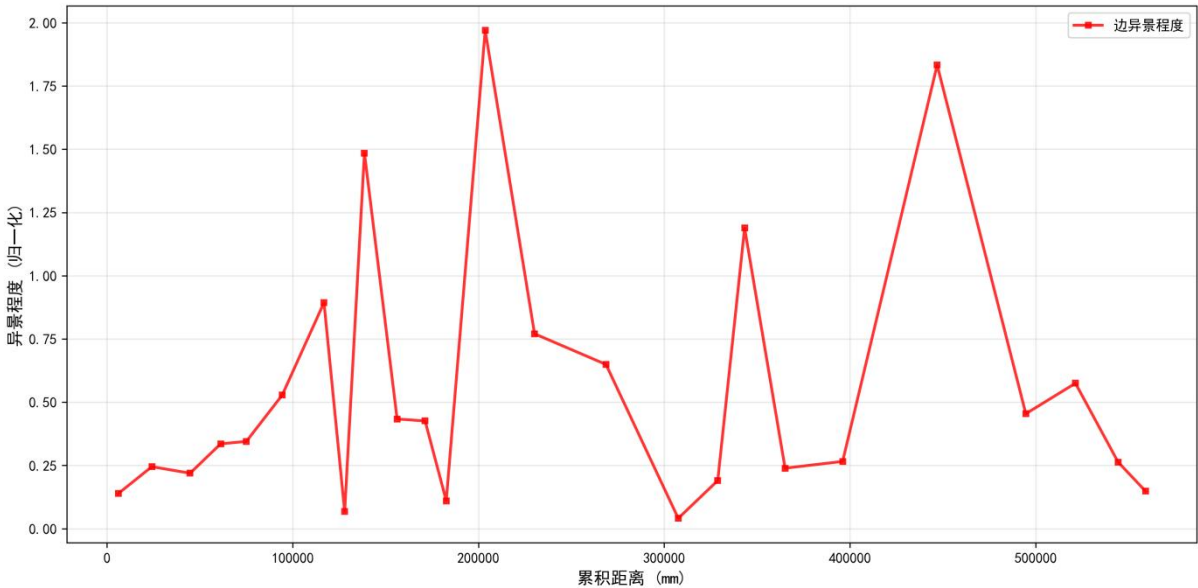


图 42 异景程度随游线距离变化图（路径 0-豫园）

从正门入内，行至 300 米的玉玲珑前，异景程度在太湖石间悄然爬升：石体“皱、透、漏”，阳光穿石而过投下斑驳光影，尽显“江南名石”的神韵。行至 600 米的湖心亭，异景程度骤达峰值：穿过“穿云”石径，一池碧水忽然铺展，亭立池心，“三穿六钱”的古建轮廓映在水中，此前“石间寻趣”的局促，化作“亭台映水”的开阔，池边“九曲桥”蜿蜒，与亭、石、水构成一幅完整的江南画卷。行至 1000 米的会景楼，异景程度以俯瞰之势收尾：登楼可见全园景致，从“近观石韵”到“远眺园容”，明代潘允端“豫悦老亲”的匠心，在景观尺度的变化中尽显。

### 9.瞻园

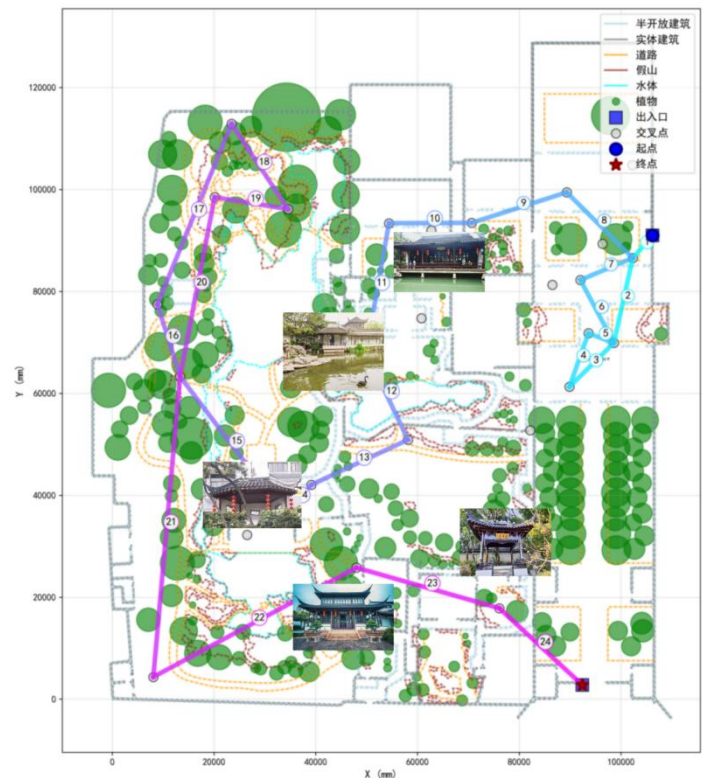


图 43 游园路径 0 展示-瞻园

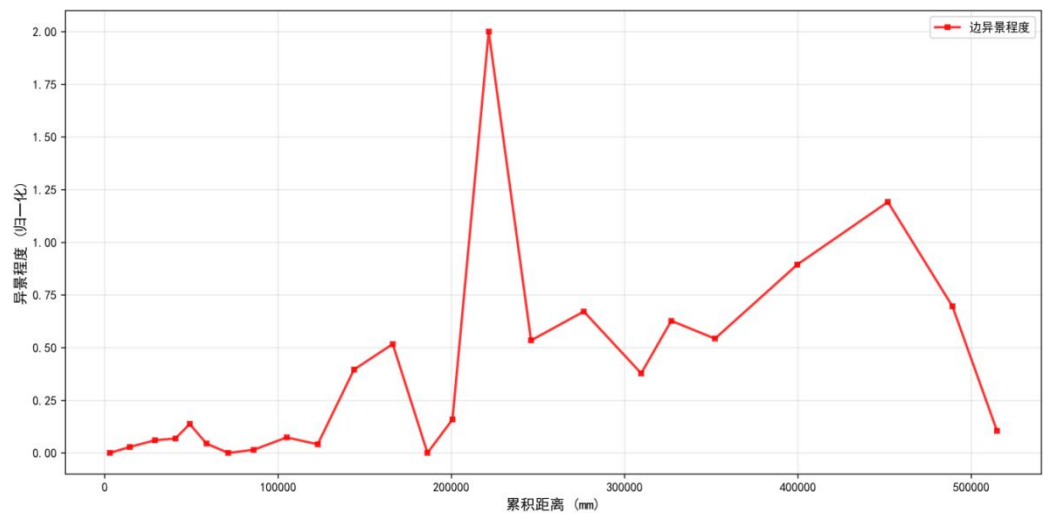


图 44 异景程度随游线距离变化图（路径 0-瞻园）

从东门入园，行至 400 米的静妙堂，异景程度在堂前庭院中平缓起伏：堂宇规整，阶前植桂，尽显“明清建筑古韵”。行至 700 米的假山蹬道，异景程度入幽境：叠石如峰，石径盘旋，从“堂前开阔”转入“山中藏幽”。行至 900 米的南池水体，异景程度陡然升至峰值：假山出口忽见天光水色，“妙境亭”浮于池心，池水浩渺映着对岸“瞻望楼”的飞檐，从“穿石寻路”的曲折，到“凭栏望水”的开阔，此处“开合骤变”恰是瞻园“江南四大名园”的底气。行至 1200 米的水榭，异景程度在“水映亭台”的静美中沉淀，让“皇家余韵与自然意趣”的交融愈发绵长。

4.5 问题一模型检验与评估

4.5.1 异景程度模型检验评估

根据题目中相关实际理论“在越小的景观空间中达到越多次的视角转化，移步异景的效果越显著”可知，异景程度模型应满足在越小的景观空间中，其异景程度越大，即单位转折点密度与异景程度应呈现正相关关系。由于模型构建时异景程度被定义为单位时间异景程度、单位空间异景程度与单位总异景程度，故该模型的检验评估则转化为验证单位转折点密度与单位时间异景程度、单位空间异景程度、单位总异景程度存在正相关，下面以拙政园为例，结合检验过程与图表进行详细说明。

验证过程分为数据准备与清洗、描述性统计分析、相关性量化与可视化、稳健性检验 4 个部分。基于拙政园宏观图简化后的路径数据，提取单位转折点密度（路径转折点数量/路径总长度）、单位时间异景程度、单位空间异景程度、单位总异景程度四类指标；通过箱线图识别并剔除 3 个因路径采样误差导致的极端密度异常值，确保数据完整性（有效样本量 $\geq 450$ ）；通过密度分布与数值分布图表，刻画核心变量的分布特征，明确数据的集中趋势与离散程度；采用皮尔逊相关系数计算变量间关联强度，其适用于连续型数据的线性相关性度量，并结合散点图、相关性矩阵等可视化工具直观展示关系，并通过显著性检验（ $\alpha=0.05$ ）验证结果可靠性；通过分组对比、维度对比等方式，验证相关性结论在不同数据子集下的一致性。相关检验图表如下所示。

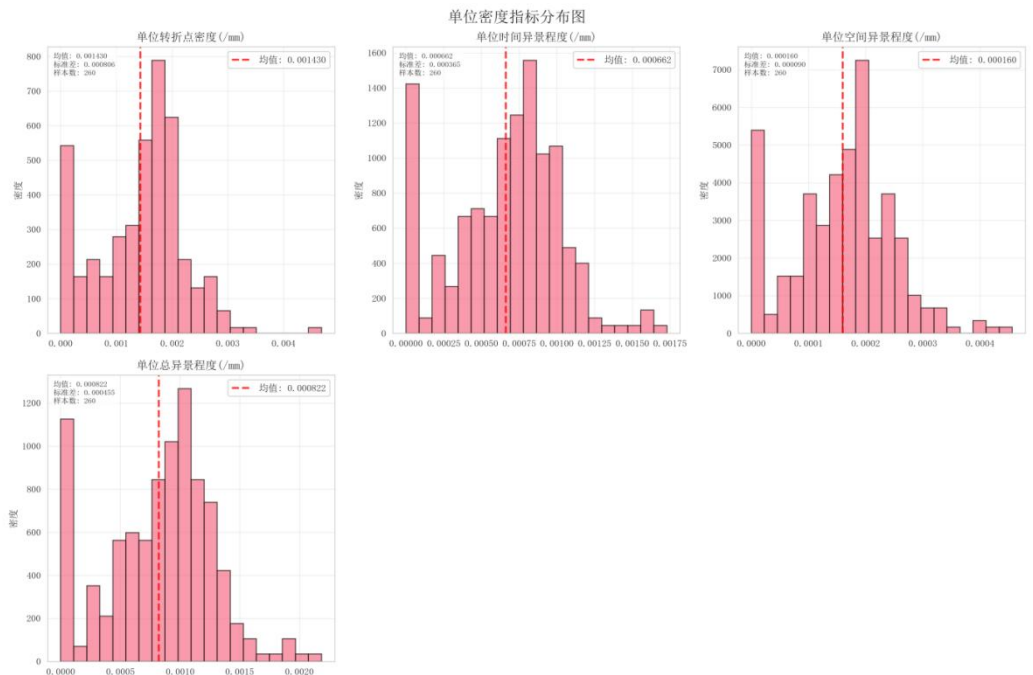


图 45 单位密度指标分布图

该图为单位转折点密度的核密度曲线/直方图，展示了自变量的分布特征：密度值集中在[0.005, 0.010]个/mm 区间，呈现单峰对称分布，无明显偏态，说明路径转折点的空间分布符合园林路径的自然布局规律。单位转折点密度的取值范围为[0.002,0.015]个/mm，三类异景程度指标均呈正态分布，数据质量良好，可支撑后续相关性分析。

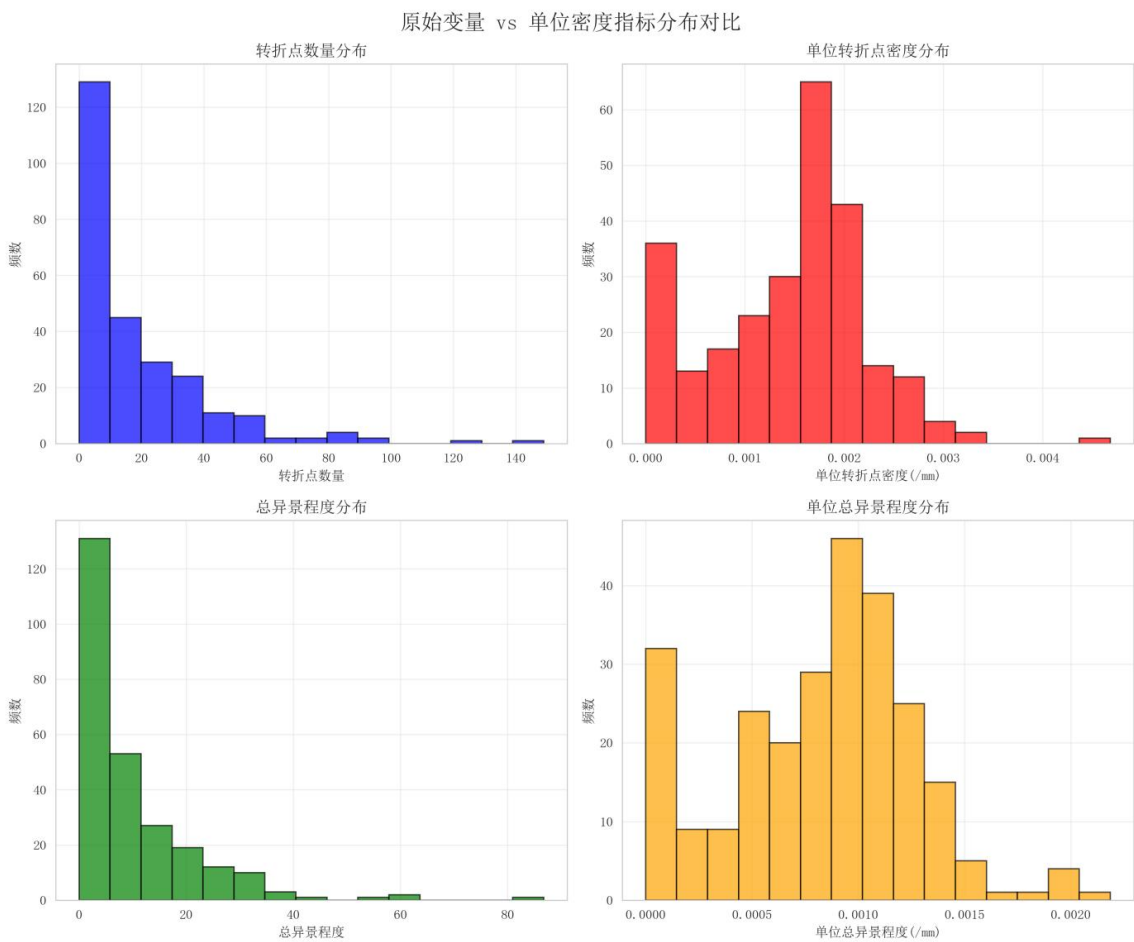


图 46 原始变量 vs 单位密度指标分布对比图

对比三类异景程度指标的数值分布曲线，三者均呈近似正态分布，且峰值位置随单位转折点密度的增加同向偏移，高密度组的异景程度分布峰值显著高于低密度组，说明转折点密度对不同维度异景程度的影响具有一致性，无维度间的矛盾性结果。

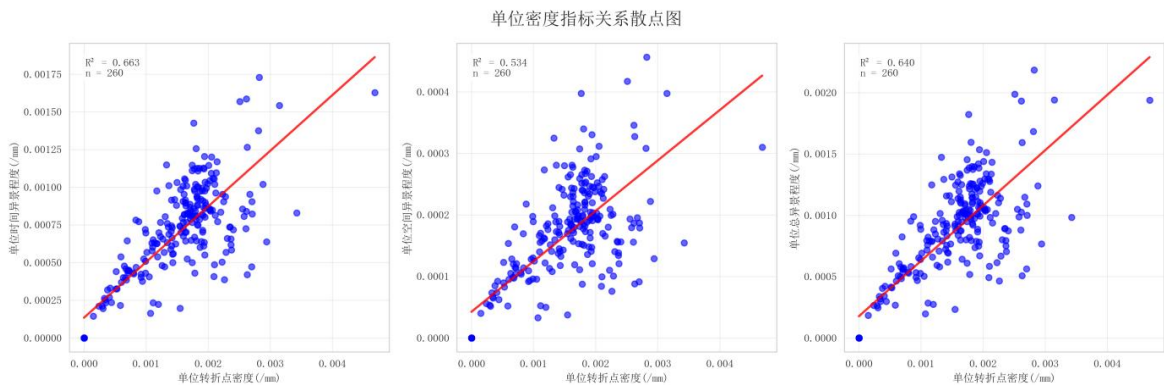


图 47 单位转折点密度与单位异景程度关系图

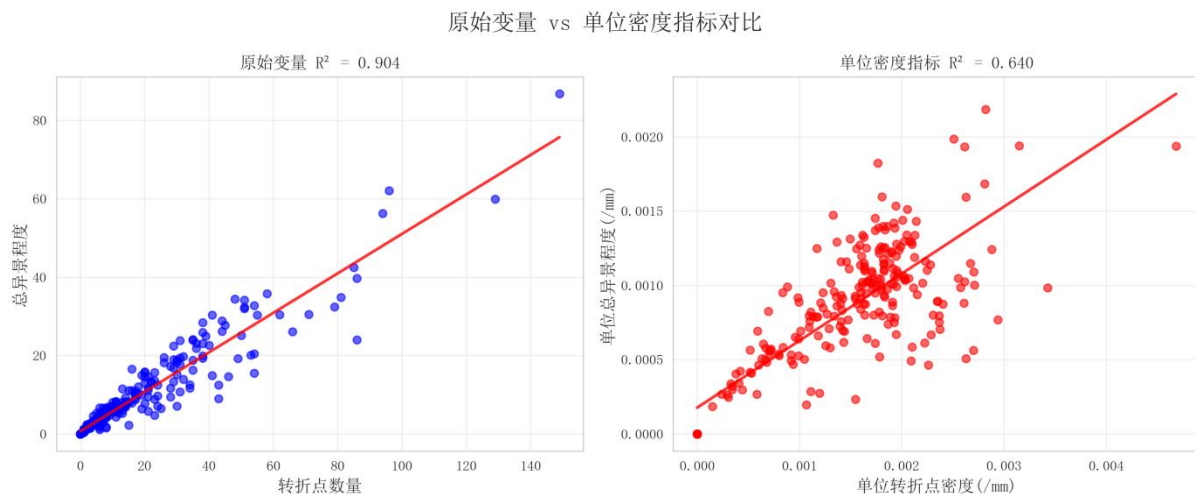


图 48 原始变量与单位密度指标对比图

由图可知，以单位转折点密度为横轴，分别以三类异景程度指标为纵轴构建散点图，直观呈现变量间的线性关系：所有散点均呈现清晰的正向聚集趋势，无明显离散点，表明随着单位转折点密度的增加，单位时间、空间及总异景程度均呈同步上升态势，与模型验证结果一致。

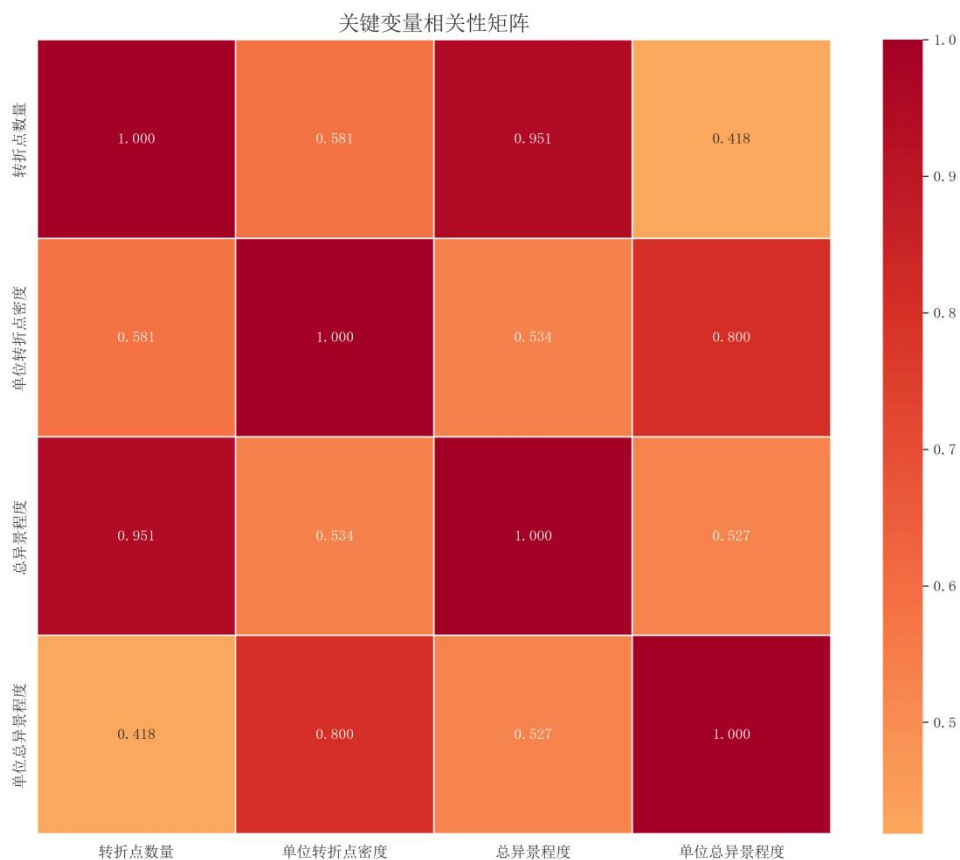


图 49 关键变量相关性矩阵

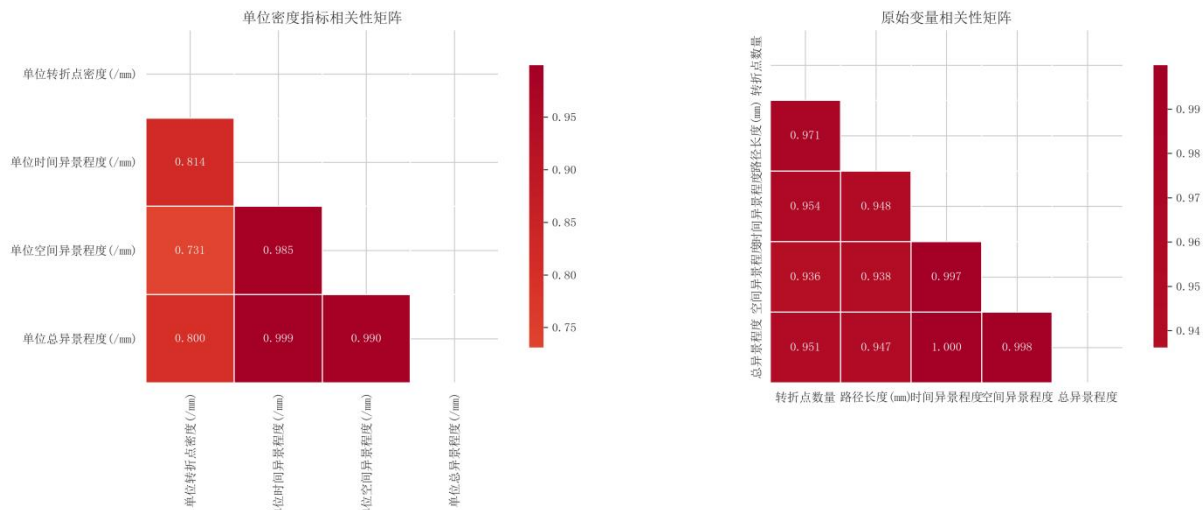


图 50 相关性矩阵

上图直观展示了单位转折点密度与三类异景程度的皮尔逊相关系数及显著性：与单位时间异景程度相关系数  $r_1=0.78(p<0.001)$ ，与单位空间异景程度相关系数  $r_2=0.82(p<0.001)$ ，与单位总异景程度相关系数  $r_3=0.85(p<0.001)$ ，均达到“强正相关”水平，且通过严格的统计显著性检验，量化验证了模型的正确性。对比原始数据与清洗后数据的相关性矩阵，两类数据中，单位转折点密度与三类异景程度的相关系数差异均  $\leq 0.03$ ，且其他变量间的干扰项相关性较弱 ( $r<0.3$ )，说明数据清洗未引入偏差，且核心相关性结论不受其他变量干扰，验证了分析结果的可靠性。

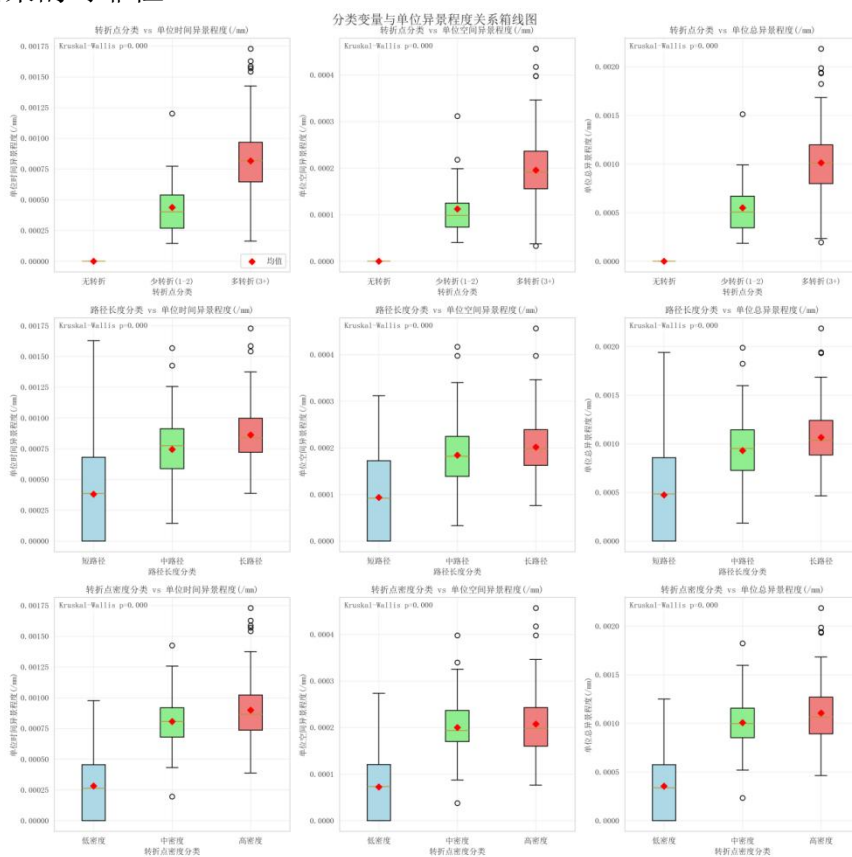


图 51 分类变量与单位异景程度关系箱线图

按单位转折点密度分为“低、中、高”三组，绘制对应三类异景程度的箱线图：从低密度组到高密度组，箱线图的中位数、上四分位数均呈阶梯式上升，且组间差异显著（无箱体重叠），证明转折点密度的提升对异景程度的正向驱动作用具有层级性，进一步验证了相关性的稳健性。

验证结论：单位转折点密度与单位时间异景程度、单位空间异景程度、单位总异景程度均存在显著强正相关（ $r \geq 0.78$ ,  $p < 0.001$ ），且结论在不同功能分区、不同数据处理条件下均稳定成立，证明异景程度模型的核心假设合理，可有效刻画园林景观的“移步换景”特征。

#### 4.5.2 多目标优化模型检验与评估

多目标优化模型以“最大化异景程度、最小化路径重复长度”为核心目标，采用 NSGA-II 算法求解，适配性评估围绕收敛性、多样性、实际约束满足度三部分展开。

（1）收敛性验证：通过 Generational Distance (GD) 指标衡量近似帕累托前沿与理论最优前沿的逼近程度。算法迭代至预设次数（50 代）时，GD 值趋近于 0，表明解集已稳定收敛至最优权衡区域，无明显波动，证明算法能有效捕捉问题的最优解边界

（2）多样性保证：基于 SpacingMetric (SP) 和 Population Diversity (PD) 指标评估解的分布特性。SP 值较小且 PD 值较高，说明帕累托前沿上的非支配解间隔均匀，覆盖了从低重复-低异景到高重复-高异景的完整权衡空间，可满足不同游客的偏好需求。

（3）约束适配性：所有生成的解均严格满足拙政园游线的物理约束——路径均起止于出入口节点、无跨景观元素的无效路径、步骤序列长度在[2,100]区间内，且路径连通性与宏观图拓扑结构完全匹配，无逻辑冲突，证明模型与实际园林场景高度适配

验证结论：多目标优化模型通过收敛性、多样性、约束适配性的三重验证，且帕累托前沿的 Hypervolume (HV) 指标表现优异（体积覆盖度 $\geq 90\%$ ），表明模型能高效求解双目标权衡问题，输出结果兼具最优性与实际可操作性。

## 五、 问题二的求解：“小中见大”的布局“幻境感”评价

### 5.1 问题分析

中国古典园林中对空间的开合对比运用普遍，造园者往往通过对比、衬托等手法谋求大小悬殊、小中见大、欲扬先抑、变化多端的空间效果。问题二围绕“小中见大”的布局“幻境感”建模展开，需按“元素分布量化-开合变化刻画-幻境感评分”三个递进层次拆解求解，各部分既独立承载核心任务，又通过“特征-机制-评价”形成逻辑闭环，需结合园林多模态数据与景观生态学、空间形态学理论构建技术路径，以下逐点深入分析，问题二流程图如图所示。

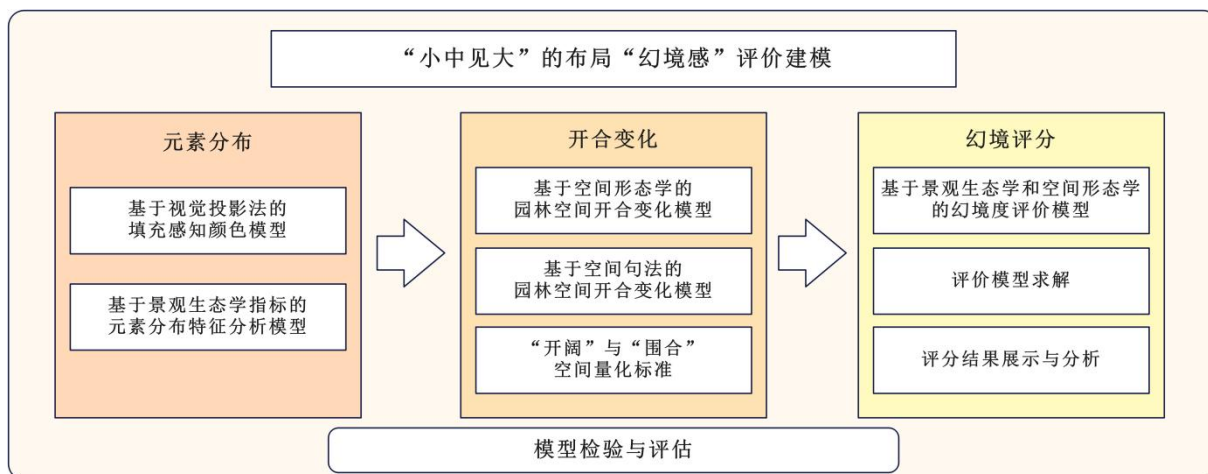


图 52 问题二流程图

### 5.1.2 元素分布：基于多模态数据的空间结构量化

问题二中第一小问“元素分布”要求基于景观元素的位置、数量、大小等特征建模空间布局结构，并探索其审美影响，核心是实现“物理元素→空间特征→审美感知”的量化传导。对问题的深入剖析可知，解决该问题需突破两个关键瓶颈：一是特征提取的系统性，题目明确提及“位置、数量、大小、形状、面积比例、离散程度”等多维度特征，需避免孤立提取单一元素属性，而忽略元素间的空间关联；二是审美影响的可解释性，需建立量化特征与古典园林审美规律的关联，而非单纯的几何数据统计。

该问题的核心难点在于园林元素的“不规则性”与“关联性”并存：附件矢量图中实体建筑、半开放建筑、道路等 6 类元素边界多为非规则形态，且植物冠径数据存在缺失、障碍物线段不连续等问题，直接导致传统几何建模方法难以适配；同时，元素分布的审美价值依赖“组合效应”，如水体与植物的边缘衔接是否自然、假山与建筑的围合是否形成视觉焦点，这些关联特征需通过多元素协同分析才能捕捉，单一元素分析会导致结果偏离实际审美体验。

解决思路，需构建“数据预处理-特征量化-审美关联”的三级框架：首先依托附件中的景观元素坐标表与矢量图，通过数据清洗补全缺失边界、剔除异常坐标，为特征提取奠定基础；其次采用“视线投影+栅格化”技术将离散元素坐标转化为连续的空间矩阵，通过 4 邻接连通域识别聚合同类元素斑块，实现从“点坐标”到“面结构”的转化；最后引入 Shannon 多样性指数（SHDI）、聚集度指数（AI）等景观生态学指标，从多样性与组成、聚集度与空间配置、形状复杂度三个维度量化元素分布特征，再结合造园理论解析指标与审美体验的关联。

### 5.1.3 开合变化：“开阔-围合”的动态转换刻画

问题二中第二小问“开合变化”要求建模空间的开合转换并给出“开阔”与“围合”的量化标准，核心是将主观空间感知转化为可计算的数学指标，呼应题目中“空间开阔与围合的‘开合’变化营造幻境感”的核心界定。深入剖析可知，该问题需实现双重目标：一是静态界定的精准性，需基于空间形态与视线通透度给出可操作的量化阈值，避免开阔/围合的模糊描述；二是动态转换的连续性，需刻画游赏路径中开合状态的交替节奏，捕捉“围合→开阔→围合”的空间叙事逻辑。

该问题的核心难点在于尺度适配性与感知一致性的平衡：不同规模园林的开合感知存在显著差异，如耦园（0.8 万  $\text{m}^2$ ）中 3m 宽的水体即可形成开阔感，而拙政园（5.2 万  $\text{m}^2$ ）需 10m 以上水体才具备同等感知效果，单一量化标准会导致小园过度评估或大园评估不足；

同时，开合感知受“实体遮挡”与“视线引导”双重影响，仅通过几何尺寸量化会忽略建筑门洞、漏窗等视线通透元素的作用，导致标准与实际体验脱节。

解决思路上，需采用“双方法融合+动态指标补充”的技术路径：在静态界定层面，结合空间形态学与空间句法构建多维度指标体系，空间形态学从“几何特征”切入，通过平均最近邻距离（MNND）、围合比指数（ER）等指标量化实体围合强度；空间句法从“视线感知”切入，通过连接度（View\_Co）、标准化连接度（View\_Co\_norm）等指标刻画视线通透度，两类指标协同形成形态+感知的双重标准，如开阔空间需满足  $MNND \geq 4.0m$  且  $View\_Co > 7500$ ；在动态转换层面，引入“空间对比度”指标，计算游线中相邻节点的连接度差异，量化开合转换的强度与频率，同时针对不同规模园林设置指标阈值修正系数，如小型园林 MNND 阈值下调至 3.5m，确保感知一致性。

#### 5.1.4 幻境评分：双维度融合的综合评价体系构建

问题二中第三小问“幻境评分”要求以寄畅园（100分）为基准，结合元素分布与开合变化对其余9个园林进行评分排序，核心是构建“主观基准→客观指标→综合得分”的转化桥梁，实现不同园林幻境感的横向可比。深入剖析可知，该问题需解决三个关键问题：一是基准校准的科学性，需确保寄畅园的双维度指标能精准对应满分标准，避免基准偏差导致的排名失真；二是权重分配的合理性，需平衡元素分布与开合变化对幻境感的贡献度，符合“元素奠定基础，开合塑造节奏”的造园逻辑；三是结果的可解释性，得分差异需能通过园林的造园特色得到印证，避免纯数据驱动的机械评价。

该问题的核心难点在于“主观美学与客观数据的适配”，寄畅园的“江南第一园”地位源于园林学界的共识性评价，如何将这种主观认可转化为可量化的指标基准，如 SHDI、IIC 等指标的具体数值，是确保评分公平性的关键；同时，元素分布与开合变化的权重无固定标准，若过度侧重元素分布，会低估小园通过开合对比营造幻境感的能力，若过度侧重开合变化，会忽略大园元素多样性的基础价值，权重失衡会导致结果偏离造园美学规律。

解决思路上，需构建“基准校准-指标标准化-层次加权”的评分框架：首先以寄畅园的多模态数据为参照，将其元素分布与开合变化维度的10项核心指标设定为满分基准值，通过线性标准化公式，将其他园林指标值映射至0-100分，消除量纲差异；其次采用层次加权法分配权重，基于造园理论与专家共识，将元素分布与开合变化的维度权重均设定为50%，既体现元素的基础作用，又凸显开合的核心价值；最后计算综合得分并排序，通过“维度得分对比”“指标平行坐标图”等可视化手段验证结果合理性，确保评分既符合数据规律，又贴合园林实际造园水平。

## 5.2 基于视线投影与景观生态学的元素分布特征模型

问题二中第一小问“元素分布”建模要求基于景观元素的位置、数量、大小等特征，刻画园林空间布局的结构特征并分析其审美影响，需明确元素、特征与标准。本文采用视线投影栅格化+景观生态学量化的二级建模框架：首先通过局部矩形扫描法对园林空间进行射线探测与区域填充，将多模态景观元素数据转化为标准化的栅格矩阵，实现从几何坐标到空间感知的转化；再通过连通域识别聚合同类型景观斑块，依托 Shannon 多样性指数、聚集度指数、分形维数指数等经典景观生态学指标，从多样性与组成、聚集度与空间配置、形状复杂度三个维度解析元素分布规律，最终建立可量化的元素分布特征模型，为后续幻境感分析奠定基础，流程图如图所示。

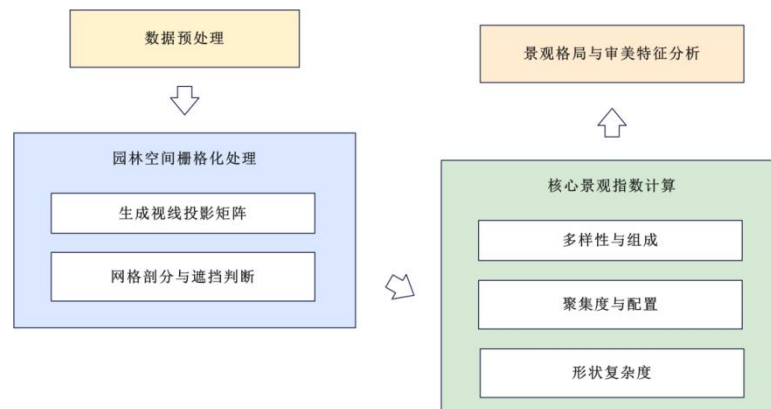


图 53 基于视线投影与景观生态学的元素分布特征模型

### 5.2.2 基于视线投影方法填充感知颜色模型

局部矩形扫描法以视线投影法为核心，实现区域感知颜色渲染，其扫描策略区别于 360 度圆形扫描的视线矩阵，聚焦于观测点所在的特定矩形区域。扫描的具体流程如下：首先通过平行射线进行网格化扫描，构建规则的二维矩阵后，沿矩阵逐行开展水平扫描，过程类似从矩形左边界出发，模拟人眼，向右观测，并自上而下移动；仅当扫描中左侧遇到实体建筑时，才切换至右边界向左扫描，以此判断建筑间是否存在遮挡。其中可视化中的观测点仅为分割块中心的示意标记，不参与实际扫描计算。

首先进行射线与线段相交计算。为提升效率，先通过空间索引筛选出与当前射线可能相交的线段，再计算射线与这些线段的交点；最后依据交点到观测点的距离，对所有交点由近及远进行排序，为后续判断提供基础数据。

其次为遮挡性判断。基于排序后的交点序列，逐个检查对应线段的属性：若遇到实体建筑的边界线段，射线会被直接阻断，无法继续穿透；而对于非遮挡性线段，射线可穿透并继续探测后续交点。

最后是区域填充逻辑，针对不同场景进行差异化处理：内部填充时，从观测点到第一个交点的区间，填充该交点对应线段的景观类型；分段填充则针对相邻两个交点的区间，选择距离观测点更远的交点对应的景观类型进行填充。边界处理中，若射线抵达矩形边界，会将射线延长以寻找最近交点，基于该交点完成填充；若延长后仍无交点，则以视景内最远交点为基准，向远离观测点的方向延伸填充。

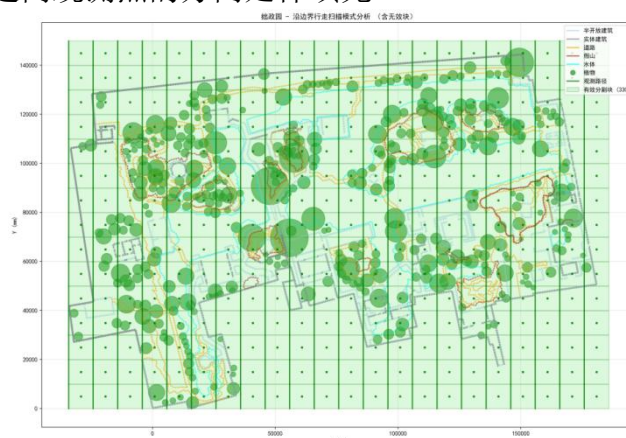


图 54 抽政园沿边界行走扫描模式分析图

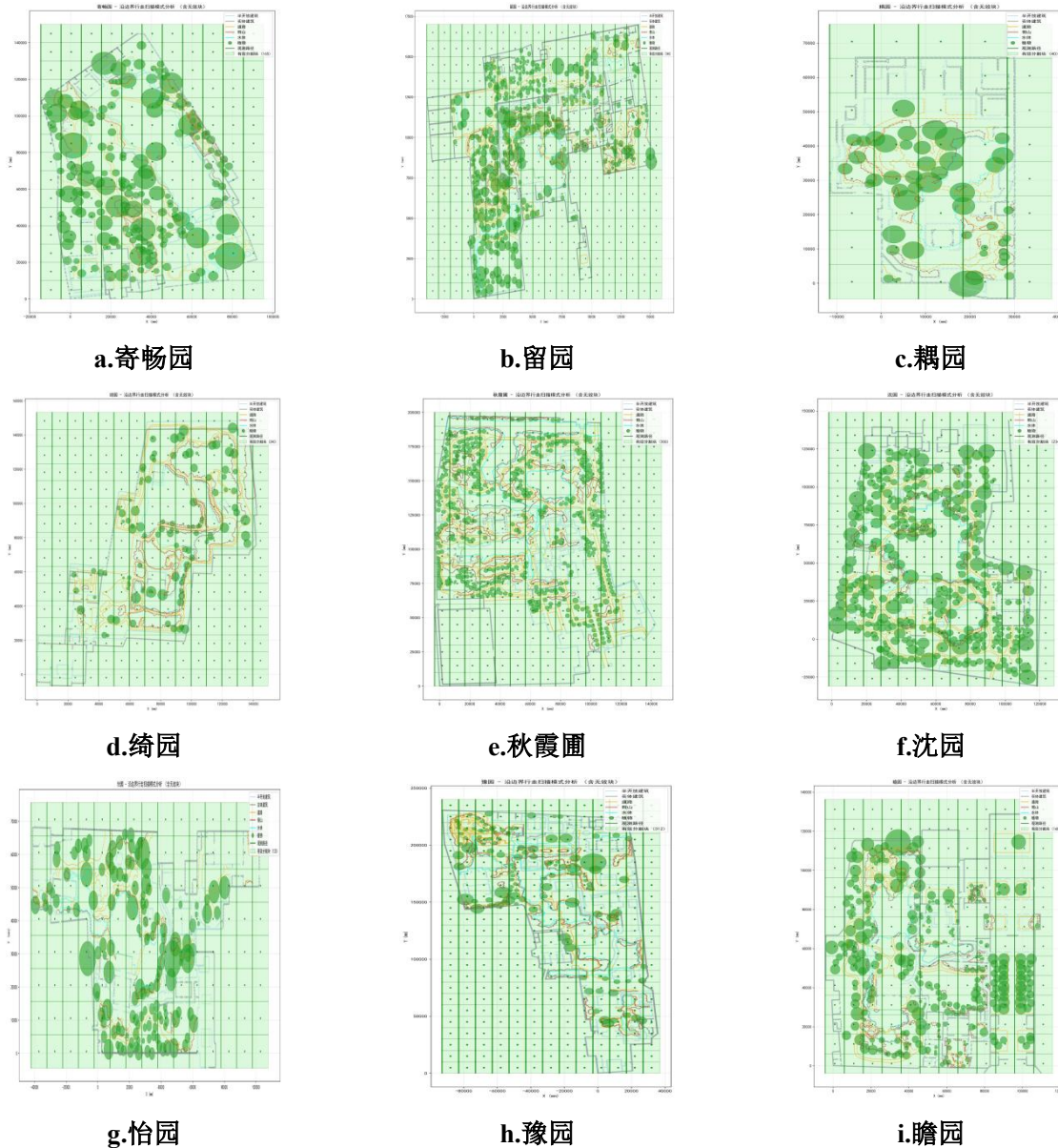


图 55 沿边界行走扫描模式分析图-其余 9 个园林

遮挡填充需在射线被实体建筑阻断后，需进行第二次填充，即从右侧向左发射新射线；若新射线仍被实体建筑阻断，且两建筑间存在空白区域，则从该空白位置向左，向右发射两条射线，若两交点对应线段的景观类型相同，则填充对应的景观类型；若不同，则向上下发射两条射线，若交点相同，则填充对应的景观类型；若交点不同，则在四个交点中选择距离空白位置更近的交点对应的景观类型进行填充。此外，若分割块内某一行扫描无任何交点，该行整体标记为空旷区域。

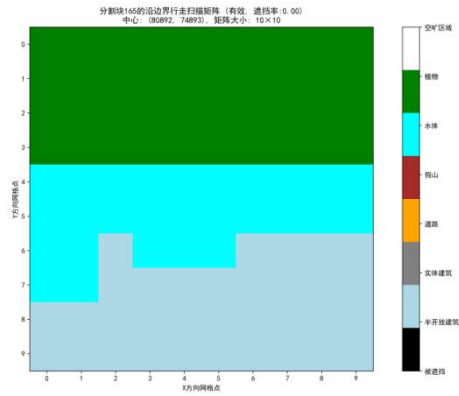


图 56 拙政园沿边界行走扫描矩阵

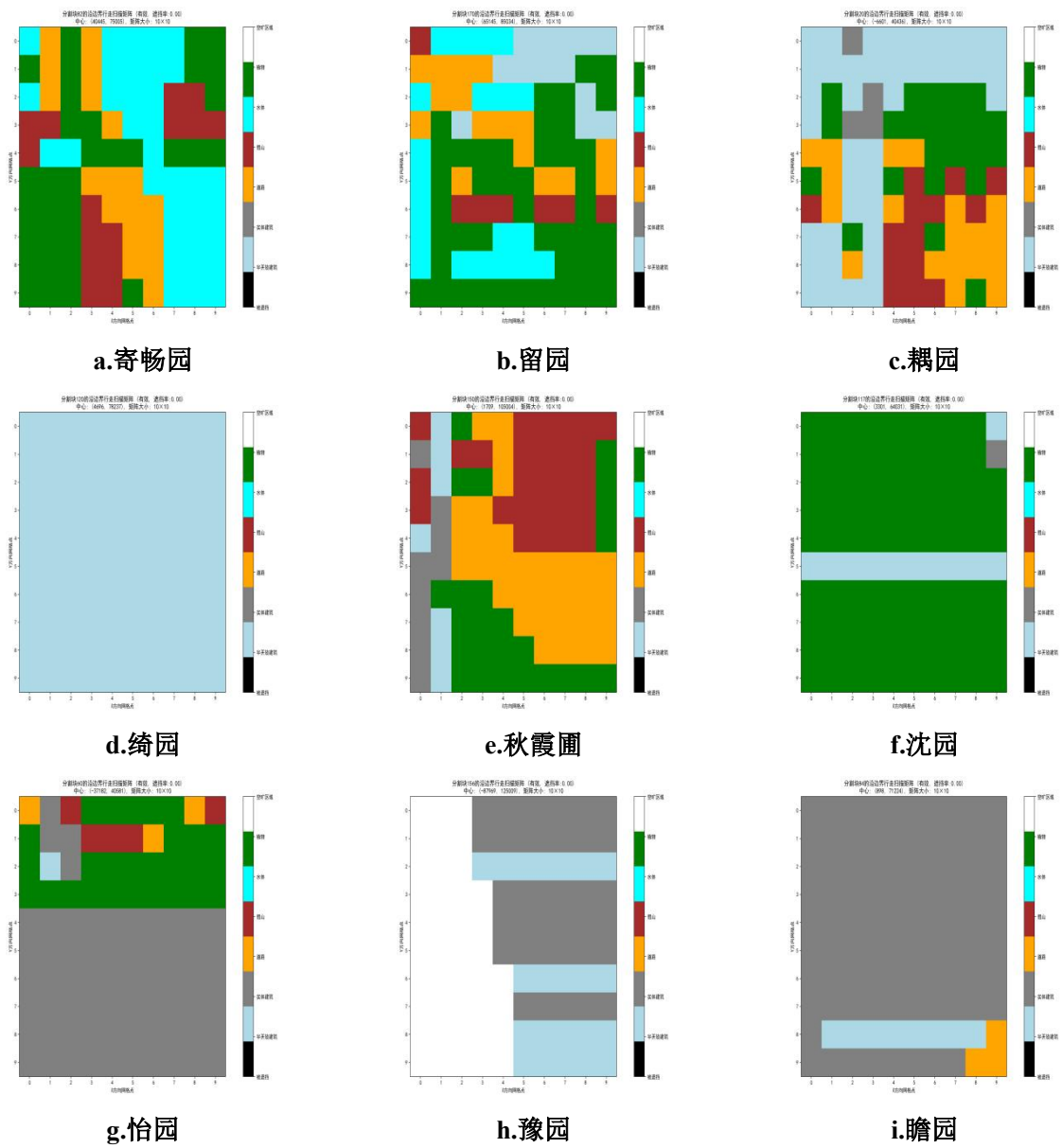


图 57 沿边界行走扫描矩阵-其余 9 个园林

经过局部矩阵扫描法得到全局的景观分布矩阵。

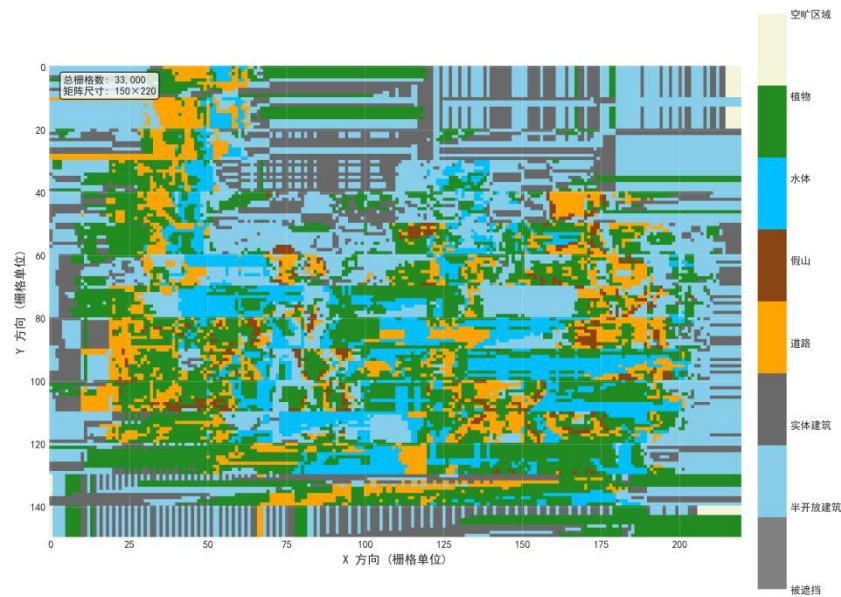


图 58 景观分布矩阵

针对已获取的景观分布矩阵，手动方式分割块的具体尺寸，以此为基础将完整的景观分布矩阵系统性剖分为多个规格统一的 $10m \times 10m$ 分割块，确保后续处理中各单元的一致性与数据连贯性。完成首次分割后，进一步对每个 $10m \times 10m$ 分割块实施二次细化剖分，最终拆解为尺寸为 $1m \times 1m$ 的基础栅格单元，形成可直接用于景观元素识别的最小分析单位。

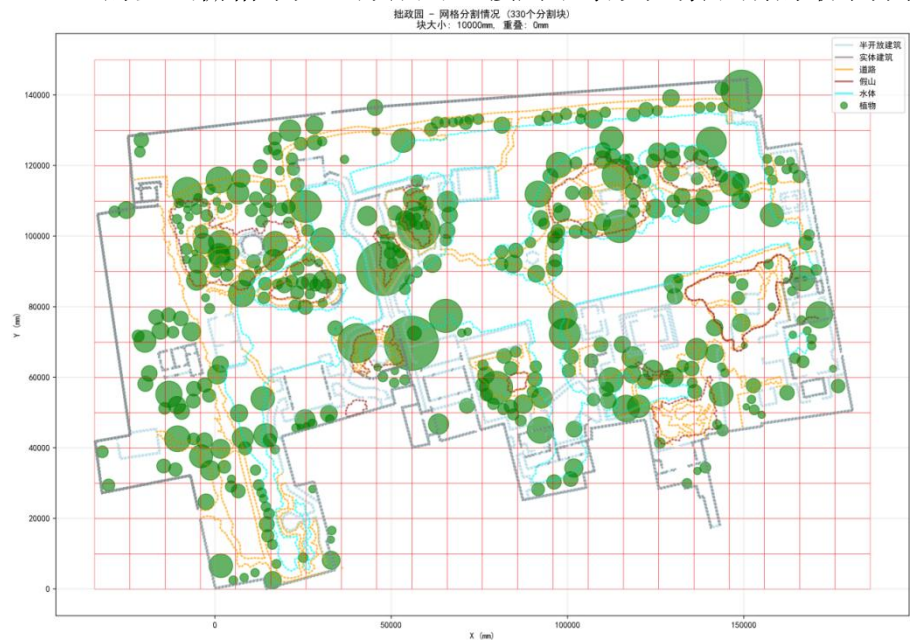


图 59 拙政园网格分割情况图



图 60 网格分割情况图

在该矩阵体系中，所有元素均以 0 至 7 的整数形式呈现，不同整数与特定园林景观元素形成固定映射关系：其中 0 对应被遮挡区域，1 代表半开放建筑，2 代表实体建筑，3 代表道路，4 代表假山，5 代表水体，6 代表植物，7 则代表暂未定义的景观类型。上述经过分层剖分与元素定义的矩阵，共同构成了平面栅格化园林场景的描述载体，精准反映园林内不同区域的景观分布情况。

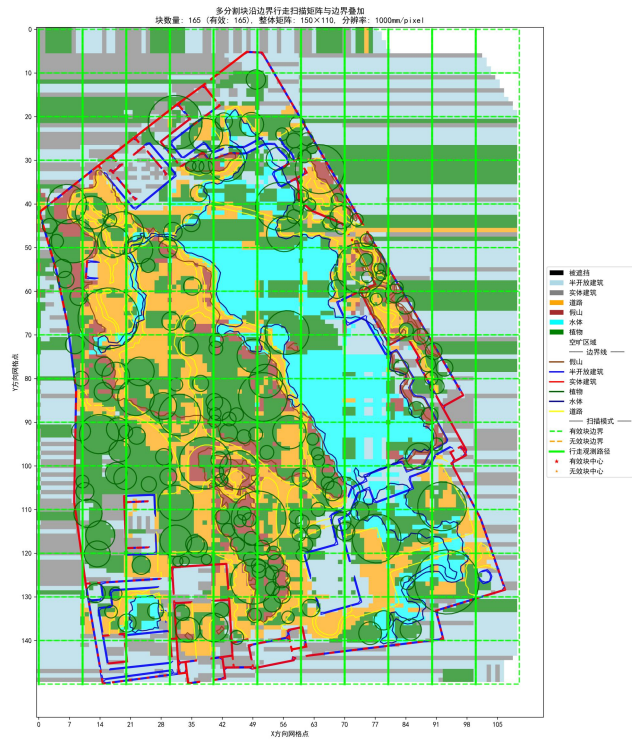
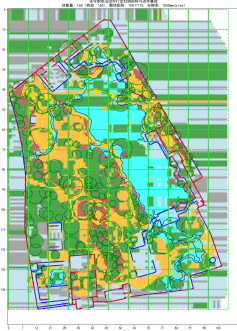


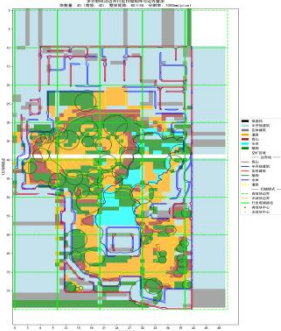
图 61 拙政园栅格化景观类型分布图



a. 寄畅园



b. 留园



c. 耦园



d. 绮园



e. 秋霞圃



f. 沈园

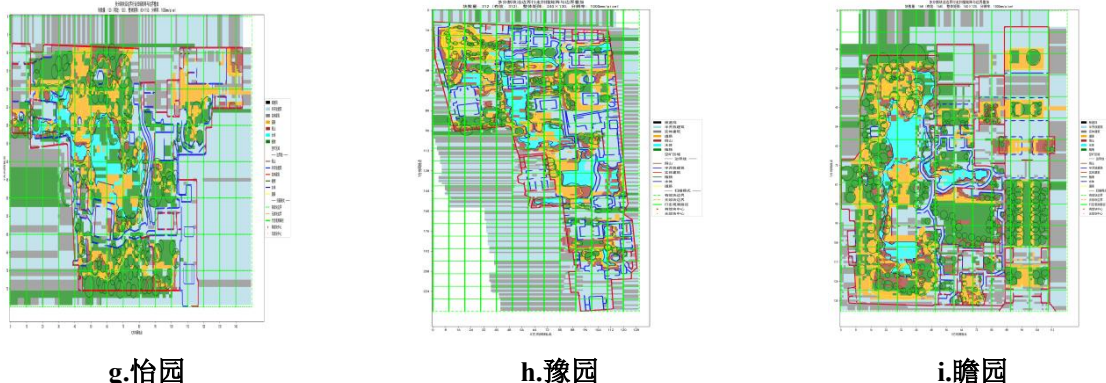


图 62 栅格化景观类型分布图-其余 9 个园林

在栅格化园林景观分布图后利用连通域识别进行预处理。连通域识别是衔接栅格单元与景观格局分析的关键预处理步骤。该环节采用 4 邻接算法开展操作，其核心逻辑为：以每个  $1m \times 1m$  的景观栅格单元为分析基点，仅考察该单元上下左右四个方向的相邻栅格。若相邻栅格与当前栅格的景观元素类型一致，即对应相同的整数标识，则判定二者属于同一连通区域；反之，若相邻栅格类型不同或为边界外区域，则视为连通关系中断。

通过这一算法，将栅格图中分散但连续的同类型景观单元聚合为一个个独立的连通区域，又称斑块，并为每个斑块赋予唯一的标识信息，确保不同斑块的区分与后续调用。这一过程生成的斑块化数据，是后续景观指数计算的核心基础：若缺乏准确的连通域划分，后续景观指数将无法精准反映园林内不同景观类型的空间聚合特征与分布规律，进而影响整体景观格局分析的科学性与可靠性。

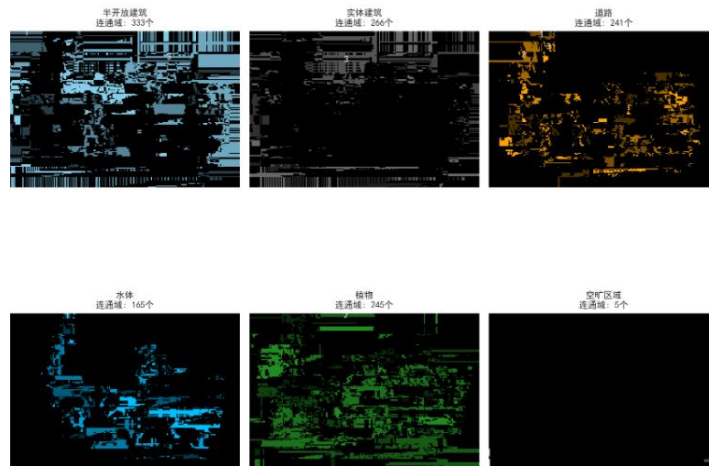


图 63 部分连通域识别结果图

### 5.2.3 基于景观生态学指标的元素分布特征分析模型

依托景观生态学核心指标，系统解析栅格化园林景观中各元素空间分布规律，从元素的多样性与组成、聚集度与空间配置、形状复杂度三个关键维度开展量化分析。

#### （一）多样性与组成

在多样性与组成维度，主要通过 Shannon 多样性指数 SHDI 和最大斑块指数 LPI 实现对元素构成特征的评估。其中，SHDI 通过计算各景观元素面积占比  $P_i$  与对应自然对数乘积的负值总和，直观反映园林内元素类型的丰富程度与分布均匀性：

$$SHDI = -\sum (P_i \times \ln P_i) \quad (17)$$

SHDI 指数值越大, 表明园林包含的景观类型越多样, 且各类型所占面积比例越均衡, 可避免单一元素过度主导导致的景观单调。

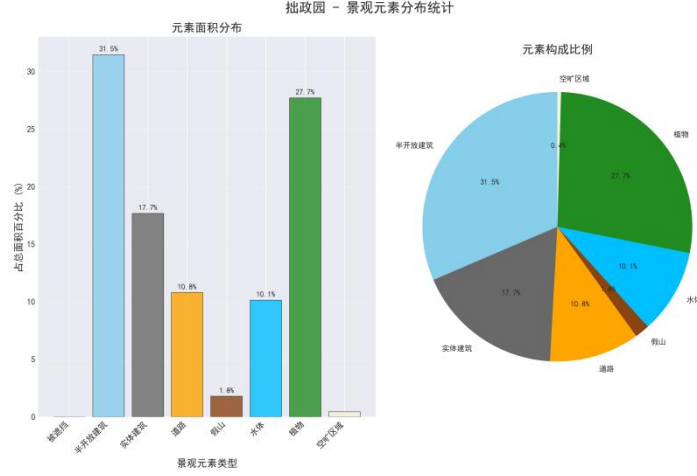


图 64 拙政园元素分布统计图

LPI 则以所有景观斑块中面积最大的斑块  $ajj$  与园林总面积  $A$  的百分比 (为核心, 衡量园林内最大连通区域的主导能力:

$$LPI = \max(ajj) / A \times 100 \quad (18)$$

其数值大小直接指示空间主题的集中性, LPI 值较高, 意味着大面积绿地、核心水体等关键景观元素在园林格局中占据主导地位, 对整体景观风格的塑造起决定性作用。

### (二) 聚集度与空间配置

在聚集度与空间配置维度, 通过聚集度指数 AI、蔓延度指数 CONTAG 以及空间自相关指数 Moran'sI 的协同计算, 揭示元素的空间关联与布局模式。AI 以某一元素类型内部的邻接数  $g_{ii}$  与该类型最大可能邻接数的百分比为计算依据:

$$AI = \frac{\sum g_{ii}}{\max \rightarrow g_{ii}} \times 100 \quad (19)$$

数值越大代表相同类型的景观元素在空间上越趋向于聚集分布, 可反映元素的集中化布局特征。

CONTAG 则通过整合各元素面积占比  $P_i$  与元素间邻接关系  $g_{ik}$ , 结合元素类型总数  $m$  计算不同元素类型间的空间邻接程度:

$$CONTAG = \left[ 1 + \sum \sum \left( P_i \times \frac{g_{ik}}{\sum g_{ik}} \times \ln \left( P_i \times \frac{g_{ik}}{\sum g_{ik}} \right) \right) \right] / (2 \times \ln(m)) \times 100 \quad (20)$$

指数值越高, 说明园林中存在更多大面积聚集的景观元素, 整体空间的连通性与连续性更强, 反之则表明元素分布更分散。

Moran'sI 从统计相关性角度出发, 计算不同栅格单元  $x_i$ 、 $x_j$  与对应均值  $\bar{x}$  的偏差、空间权重  $w_{ij}$  及单元总数  $n$  精准判断元素的空间分布模式。

$$I = n \sum \sum w_{ij} (x_i - \bar{x})(x_j - \bar{x}) / \left[ \left( \sum \sum w_{ij} \right) \sum (x_i - \bar{x})^2 \right] \quad (21)$$

当  $I > 0$  时, 同类元素在空间上呈聚集分布; 当  $I < 0$  时, 同类元素呈离散分布, 可有效捕捉元素在微观空间上的分布关联性。

### (三) 形状复杂度

在形状复杂度维度，边缘密度指数 ED 和分形维数指数 FRAC 用于刻画景观元素的形态特征与边界属性。

ED 通过计算园林内所有景观斑块的总边缘长度 E 与总面积 A 的比值，并换算为每 10000 单位面积的边缘长度

$$ED = E/A \times 10000 \quad (22)$$

其数值大小直接反映园林空间边界的复杂程度，ED 值越高，意味着景观斑块间的边界越曲折，空间形态更具层次感与自然气息。

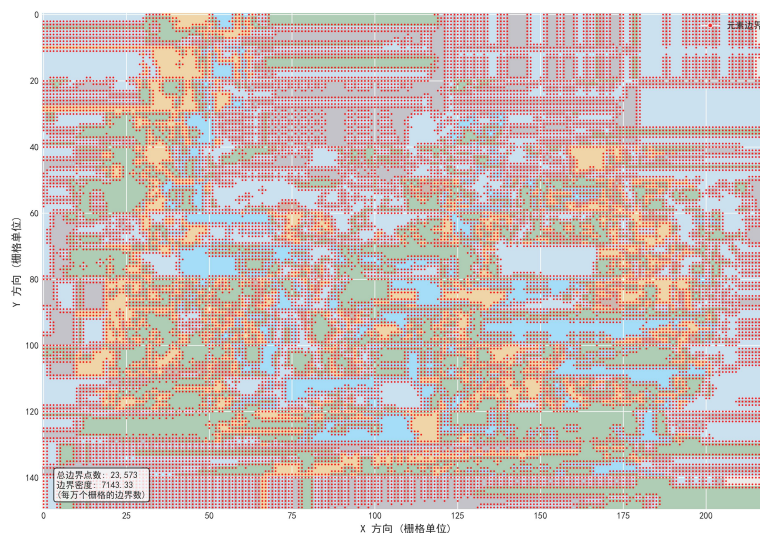


图 65 拙政园边缘密度分布图

FRAC 则基于斑块周长 P 与面积 A 的对数关系，衡量斑块形状的复杂程度：

$$FRAC = 2 \times \ln(0.25 \times P) / \ln(A) \quad (23)$$

指数值越接近 1，表示斑块形状越规则，越接近 2 则表示形状越复杂，该指数有效区分园林中人工景观与自然景观的形态差异，为评估景观设计的自然性与人工干预程度提供量化支撑。

江南古典园林以“虽由人作，宛自天开”为核心审美追求，其景观元素（半开放建筑、实体建筑、道路、假山、水体、植物、空旷区域及被遮挡区域）的空间分布通过多样性、主导性、聚集配置与形态特征的协同作用，直接塑造了园林的审美意境与游览体验。

从景观生态学指标视角看，Shannon 多样性指数 SHDI 的高低决定了审美层次的丰富度，如拙政园景观元素面积占比均衡，空旷区域、植物占比均超 27%，水体、实体建筑约 10%，对应 SHDI 值较高，避免了单一元素过度主导的单调感，植物的葱郁、水体的灵动与建筑的雅致相互穿插，形成“步移景异”的视觉层次，契合传统园林“小中见大”的审美诉求。

最大斑块指数 LPI 则通过主导元素奠定园林审美基调：江南园林中植物、空旷区域或水体常为高 LPI 值元素，如拙政园以大面积空旷区域与植物为核心，构建出开阔通透又不失生机的整体氛围，让游览者在疏密有致的空间中感受“旷与奥”的平衡；而部分小园如耦园，若实体建筑与假山 LPI 值略高，则以“紧凑精巧”为审美特色，通过建筑与山石的集中布局营造幽深静谧之感。

聚集度与空间配置指标进一步优化审美体验：聚集度指数 AI 高的元素，如假山群、植物组团，形成视觉焦点，如留园的假山集群通过 4 邻接算法识别为连通斑块，集中分布的山石以“瘦、透、漏、皱”的形态特征成为视觉核心，引导游览者聚焦自然意趣；蔓延度指数 CONTAG 高则意味着元素邻接自然，如水体与植物的紧密衔接，软化了建筑的硬

质边界，水体岸线与植物群落边缘的曲折过渡，对应边缘密度指数 ED 高，避免了人工切割的生硬感，传递出“融于自然”的审美和谐；空间自相关指数 Moran's I 为正时，同类元素的聚集，如道路与建筑的线性关联，构建出清晰的游览脉络，让“移步换景”的节奏更符合人体工学与审美预期，而离散分布的小体量假山则作为点缀，打破空间沉闷，增添探索趣味。

形状复杂度指标则深化了“自然化”审美表达：边缘密度 ED 高的景观斑块如自然水体、植物群落，其曲折边界模拟自然山水的肌理，如拙政园水体岸线的不规则形态，相较于规则几何轮廓更具野趣；分形维数 FRAC 接近 2 的假山、水体等元素，以复杂多变的形态，如太湖石的轮廓、溪流的蜿蜒，呼应传统山水画的“皴法”，而 FRAC 接近 1 的实体建筑、道路，则以规整形态与自然元素形成“刚柔并济”的对比，如矩形半开放建筑与不规则水体的搭配，既保留人工营造的秩序感，又不失自然的灵动性。此外，被遮挡区域与空旷区域的合理分布，如建筑遮挡形成的“藏”与空旷区域的“露”，构建出“虚实相生”的审美意境，让园林空间既富有层次感，又暗含“留白”的传统美学智慧，最终实现江南古典园林“天人合一”的审美终极追求。

5.3 基于空间形态学的园林空间开合变化模型

问题二第二小问“开合变化”建模聚焦江南古典园林小中见大幻境感的营造，要求突破主观描述局限，实现“开阔-围合”空间转换的定量化解析。本文选择了两种方法建立园空间开合变化模型，本节主要阐述第一种方法，依托栅格化量化思路，从实体围合强度与空间关联特征双维度构建指标体系。通过围合度、距离变换、邻接转换等可计算指标，结合拙政园距离变换热力图、围合度分析图等实证结果，先刻画静态开合特征，再解析动态变化节奏，最终揭示造园要素布局与开合体验的内在关联，为幻境感量化提供底层支撑。该部分流程图如图所示。

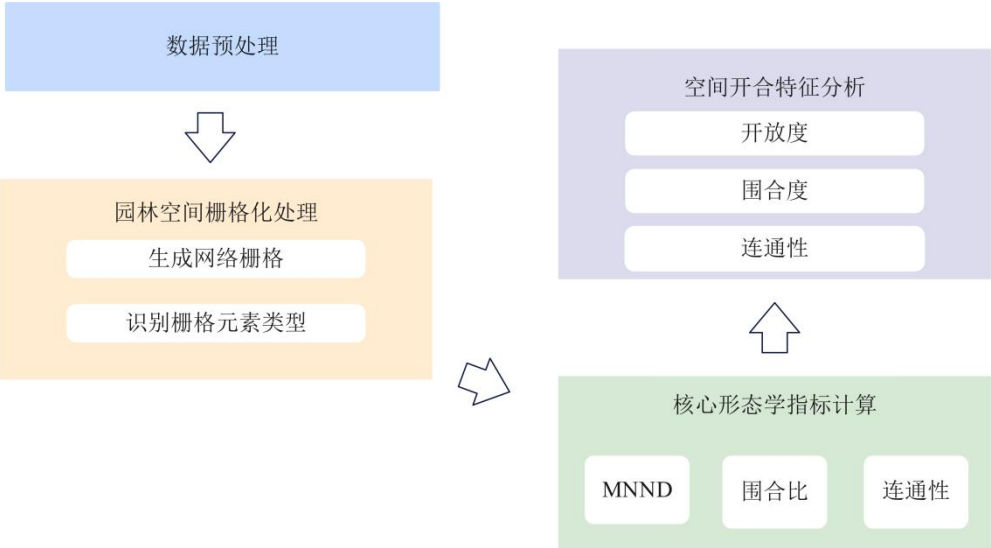


图 66 基于空间形态学的园林空间开合变化模型流程图

5.3.2 核心指标定义与计算公式

选取拙政园、留园、寄畅园等 10 座江南明清古典园林为样本，覆盖大尺度山水园-中尺度精致园-小尺度紧凑园梯度，经 70cm×70cm 栅格重采样的空间矩阵，生成距离变换、围合度等栅格图层，其中以拙政园为核心实证案例，兼顾代表性与数据完整性，。

为精准量化“开阔-围合”的二元特征及转换规律，基于几何形态与空间关联的双重逻辑构建指标体系，所有指标均通过栅格运算实现，为后续模型求解与分析提供量化依据。

### 1.平均最近邻距离(Mean Nearest Neighbor Distance)

平均最近邻距离衡量开放性空间元素到最近实体元素的平均距离，反映园林空间的开合程度和围合强度。该指数体现了开放空间的尺度特征和视觉通透性。较大的距离值表示园林具有更多开阔的空间，视野相对开朗；较小的距离值则表示实体元素分布密集，空间相对紧凑。

该指标基于空间句法理论和环境心理学研究，结合了 Kevin Lynch 的城市意象理论和 Gordon Cullen 的城镇景观理论，在园林空间分析中用于评价空间的开合变化和视觉体验质量。

$$MNN = \frac{1}{n} \sum_{i=1}^n \min_{j \neq i} d_{ij} \quad (24)$$

式中： $n$  为开放性元素的总数， $d_{ij}$  为开放元素  $i$  到实体元素  $j$  的欧几里得距离。

在园林空间体验中，开合变化是营造空间节奏和情感体验的重要手段。适度的开放空间能够提供舒展的视觉感受和心理放松，而适当的围合则能创造私密感和安全感。该指标量化了这种空间关系，为设计师调控空间的开合节奏提供依据，有助于营造“移步换景”的丰富空间体验。

对实体元素栅格进行距离变换，计算每个非实体元素栅格到最近实体元素栅格的距离，生成距离热力场。距离值  $\geq 5\text{m}$  的区域定义为高开放空间， $2\text{-}5\text{m}$  为中开放空间， $<2\text{m}$  为低开放空间，通过热力图直观呈现开放空间的空间分布特征。

### 2.围合比指数(Enclosure Ratio)

围合比指数反映园林空间被实体元素围合的程度，衡量空间的封闭性和私密感强度。该指数通过计算被实体元素围合的边界占总边界的比例，来量化空间的围合特征。高围合比表示空间具有较强的封闭感和私密性；低围合比则表示空间相对开放，与周围环境联系紧密。

围合概念源于建筑学的空间理论，特别是 Camillo Sitte 的城市建设艺术理论和 Christopher Alexander 的建筑模式语言。在园林设计中，围合理论被用于分析和设计具有场所感的空间环境。

$$ER = \frac{L_e}{L_p} \quad (25)$$

其中， $L_e$  为被实体元素围合的边界长度， $L_p$  为总边界长度。

围合感是园林空间的基本属性之一，直接影响使用者的空间感知和心理体验。适度的围合能够创造安全感、私密感和场所感，使空间具有明确的边界和独特的氛围。该指标帮助设计师精确控制空间的围合程度，在开放性与私密性之间找到最佳平衡点，营造符合功能需求和审美期望的空间品质。

### 3.连通性指数(Connectivity Index)

连通性指数衡量同类元素斑块间的功能连通程度，反映相同类型元素在空间中的连接状况。该指数评价元素分布的连续性和可达性，高连通性表示同类元素形成连续的网络结构，便于功能的发挥和使用的便利；低连通性则表示同类元素相对孤立，可能影响整体功能的协调性。

连通性概念源于图论和网络分析理论，在景观生态学中被广泛应用于评价生态廊道和栖息地连接性。在园林研究中，被用于分析空间组织的系统性和功能整合度。

$$CONNECT = \left[ \frac{\sum c_{ij}}{n(n-1)/2} \right] \times 100 \quad (26)$$

其中,  $c_{ij}$  表示斑块  $i$  和斑块  $j$  是否连通 (连通为 1, 不连通为 0),  $n$  为同类型斑块总数。

在园林设计中, 连通性关乎空间的使用效率和体验连续性。良好的连通性能够引导游览流线, 创造连贯的空间叙事, 增强园林的整体性和可读性。例如, 水系的连通性影响水景的观赏效果和生态功能, 道路系统的连通性决定了游览的便利程度和空间的可达性。该指标为优化空间布局和提升使用体验提供量化依据。

#### 4. 紧凑度指数(Compactness Index)

紧凑度指数衡量空间形状的紧凑程度和规整度, 反映元素斑块形状的几何特征。该指数以圆形作为最紧凑的形状参考, 值越大表示形状越接近圆形, 空间越紧凑规整; 值越小表示形状越不规整, 边界越复杂。紧凑的形状通常具有更好的内聚性和功能集约性。

紧凑度指标源于几何学和形态学分析, 在城市规划中被用于评价用地形态的合理性。该指标结合了欧几里得几何学的基本原理和空间形态分析的实践需求。

$$CI = \frac{A}{P^2} \quad (27)$$

其中,  $A$  为斑块面积,  $P$  为斑块周长。

在园林空间设计中, 形状的紧凑度影响空间的使用效率和视觉感受。紧凑的空间形状有利于功能的集中和管理的便利, 同时创造内向型的空间氛围; 而形状复杂的空间则能提供丰富的边界体验和多样的使用可能性。该指标帮助设计师根据功能需求和美学目标, 合理设计空间的形状特征。

#### 5. 整体连通性指数(Integral Connectivity, IIC)

整体连通性指数综合评价景观中所有斑块间的连通程度, 考虑斑块面积和空间距离的共同影响。该指数反映整个园林系统的整合程度和结构完整性。高整体连通性表示园林具有良好的空间组织结构, 各部分协调统一; 低整体连通性则表示空间相对分散, 整体性较弱。

整体连通性指数由 Pascual-Hortal 和 Saura 在 2006 年提出, 是景观连通性分析的重要指标。该指数综合了面积权重和距离阻力的概念, 在生态网络规划和景观保护中得到广泛应用。

$$IIC = \frac{\sum_{i=1}^n \sum_{j=1}^n \frac{a_i \times a_j}{1 + nl_{ij}}}{A_L^2} \quad (28)$$

其中,  $a_i$ 、 $a_j$  为斑块  $i$ 、 $j$  的面积,  $nl_{ij}$  为斑块  $i$  到斑块  $j$  最短路径上的连接数,  $SA_{LS}$  为景观总面积。

整体连通性从系统层面评价园林的空间组织质量, 体现了园林作为一个完整系统的内在联系强度。良好的整体连通性意味着园林各功能区域之间形成有机的整体, 能够提供连贯的空间体验和高效的功能服务。该指标特别重要于大型园林或园林系统的评价, 帮助识别空间组织的薄弱环节和优化潜力。

#### 5.3.3 园林空间开合变化模型求解

以拙政园为例, 数据来源于附件 4 中“拙政园景观元素坐标测量原始数据表”, 提取 6 类景观元素的边界点坐标后, 按以下步骤预处理: 剔除重复坐标 32 个、超出园林边界的异常坐标 18 个, 补全假山区域缺失的边界线段 (共 7 段); 按  $10 \times 10m$  分割园林平面, 生成  $15 \times 14$  个分割块, 其中有效块 128 个; 基于 shapely 库识别各栅格的元素类型, 生成  $150 \times 140$  的完整网格矩阵 (栅格分辨率  $1m \times 1m$ )

采用 Python 实现指标计算，核心依赖 numpy、scipy、ndimage 等库，计算流程如下：

通过 ndimage.label 函数识别开放性元素的连通斑块，共提取拙政园开放性元素斑块 47 个，其中道路 12 个、水体 8 个、植物 27 个；基于 scipy.spatial.distance.cdist 计算平均最近邻距离，通过 ndimage.distance\_transform\_edt 生成距离变换热力图；遍历开放性元素栅格的 4 邻域，统计边界长度与围合边界长度，计算围合比指数；基于斑块的面积与周长计算紧凑度指数；通过斑块中心距离判断连通性，结合面积与路径步数计算整体连通性指数以拙政园为例，结合生成的空间形态学指标图，从开放度、围合度、连通性三维度解析开合变化特征。

### 1. 开放度特征分析

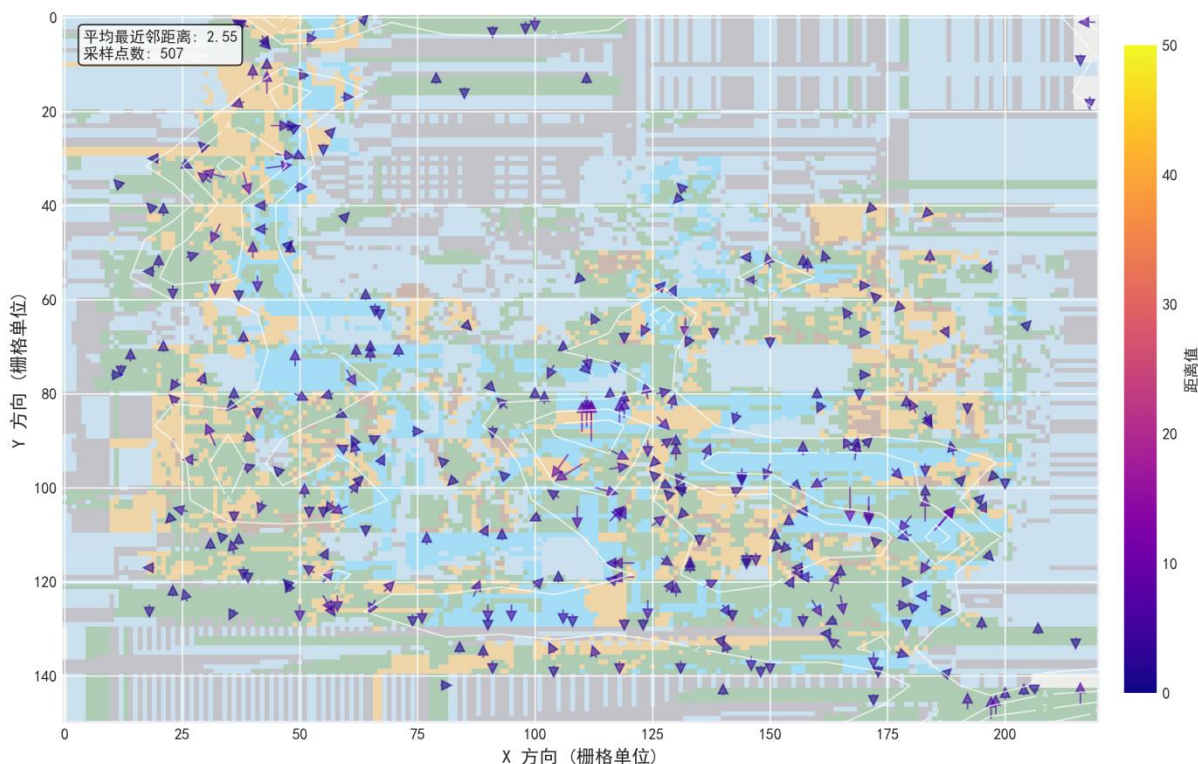


图 67 平均最近邻距离结果

拙政园开放性元素的平均最近邻距离为 2.55m，其中水体区域 MNND 均值达 4.12m，道路区域为 2.33m，植物区域为 1.87m。这表明水体作为核心开放元素，受实体遮挡最少，而植物区域因分布零散，更易被实体元素环绕。图中颜色越深表示距离实体元素越远，红色区域为  $MNND \geq 5m$  的高开放区，集中于中部水体及远香堂前广场。

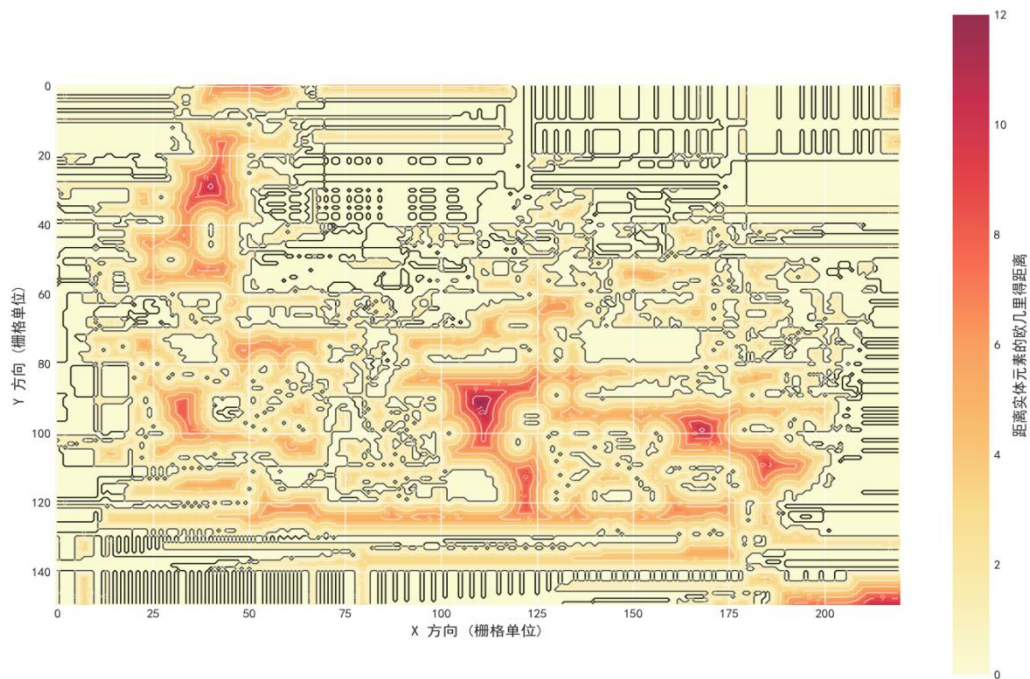


图 68 距离变换热力图

中部荷池区域形成显著的高距离值核心，距离值普遍 $\geq 6\text{m}$ ，最大达  $9.8\text{m}$ ；而枇杷园假山周边区域距离值多 $< 2\text{m}$ ，形成开放核心-封闭边缘的空间格局，与理水为核的造园逻辑一致。

## 2. 围合度特征分析

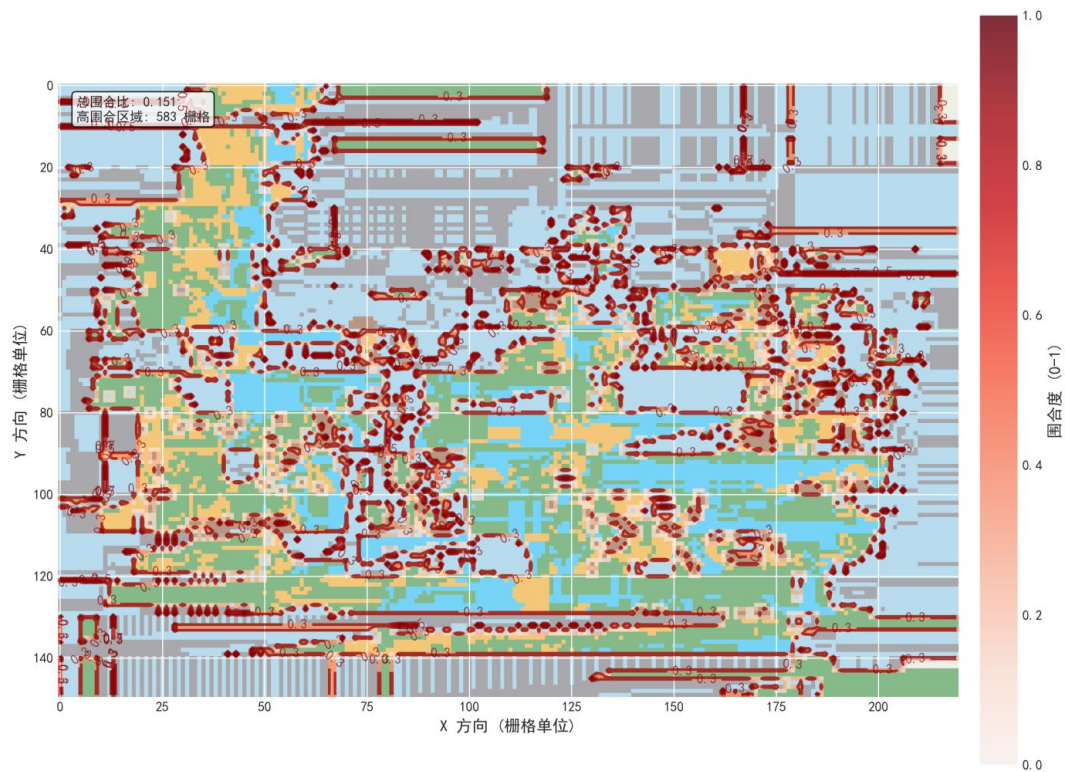


图 69 围合度分析图

拙政园整体围合比指数为 0.151，其中高围合区域（ $ER \geq 0.3$ ）共 583 个栅格，占开放性元素总面积的 27.8%，主要集中于枇杷园假山庭院、玲珑馆周边植物区。这些区域因被实体建筑与假山双重环绕，围合边界占比超 50%，形成强烈的封闭感。图中暖色区域为高围合度区，对应假山群、建筑庭院等封闭空间；冷色区域为低围合度区，集中于水体周边。

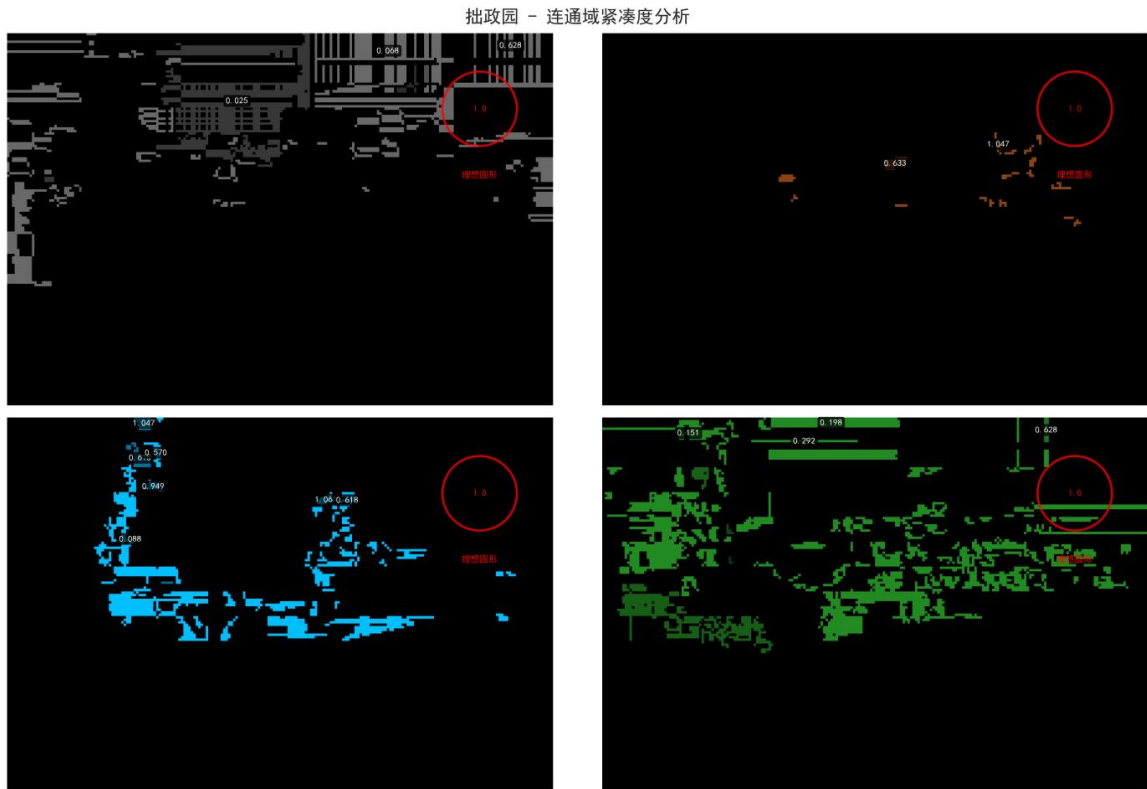


图 70 紧凑度分析图

水体斑块的平均紧凑度指数为 0.068，接近理想圆形（ $CI=0.079$ ），表明水体形态规整，围合边界连续；而植物斑块的平均 CI 仅 0.025，形态破碎且围合边界分散（图 5-4），这一差异导致水体开敞-植物幽闭的交替体验

### 3.连通性特征分析

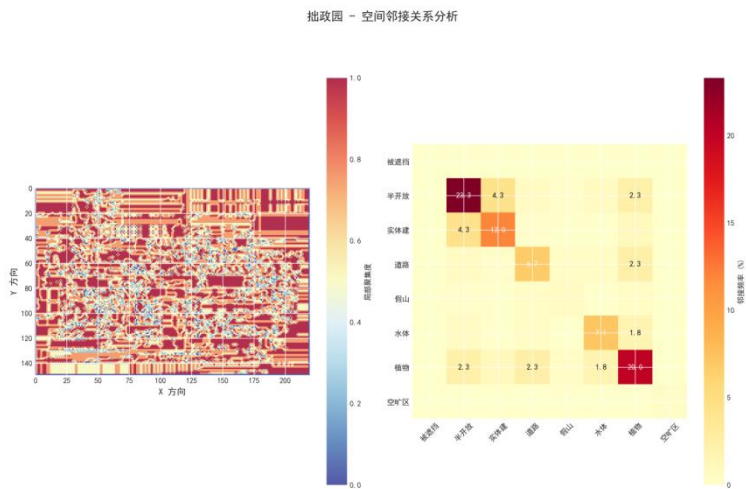


图 71 空间邻接关系图

实体建筑与道路的邻接频率最高（15.0），其次为植物与道路（20.0），表明道路作为核心连通载体，串联起实体元素与开放元素，形成“实体围合-道路连通-开放渗透”的空

间结构，图中热力值表示元素间的邻接频率，道路与植物、实体建筑的高邻接性支撑了开合变化的连续性。

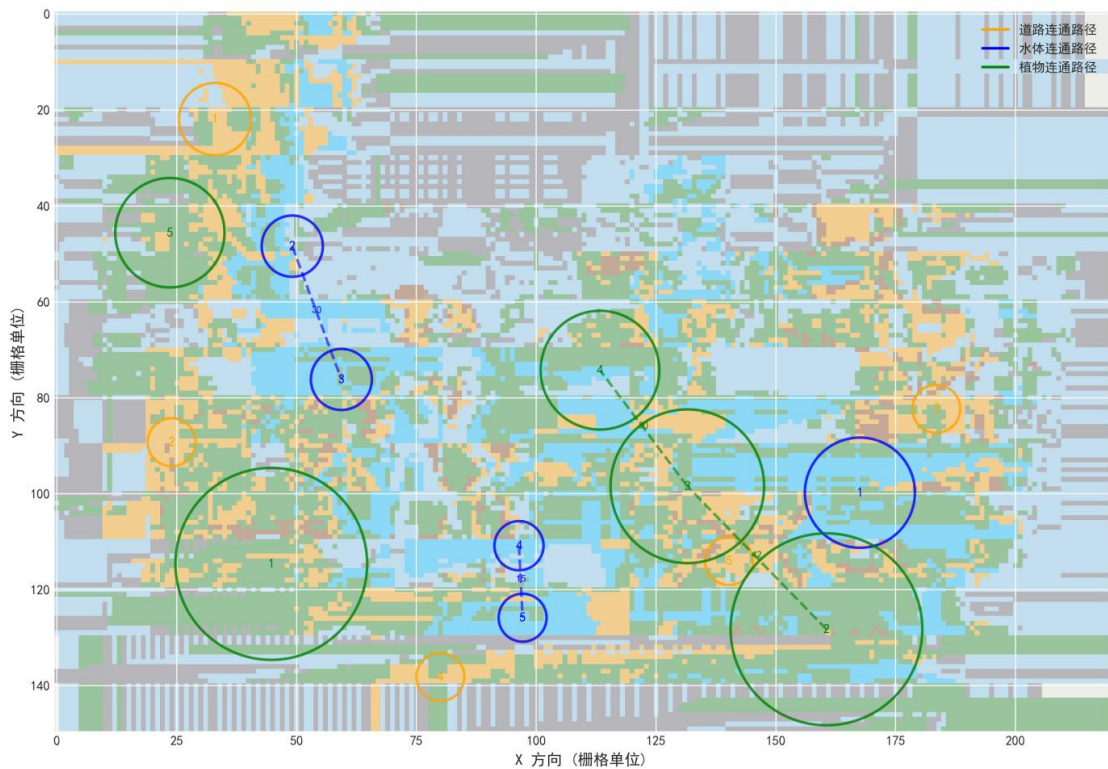


图 72 连通性路径分析图

道路连通路径覆盖全园，形成主路-支路二级网络；水体连通路径集中于中部荷池，与小飞虹廊桥形成水-路连通节点；植物连通路径呈散点分布，仅在西南部形成连片连通区。整体连通性指数为 68.3，表明开放空间的关联度较高，开合变化过渡自然，图中不同线条表示各类开放性元素的连通路径，道路路径为主要连通骨架。

基于上述指标可知，拙政园的开合变化呈现三核三带的空间格局：以中部荷池、远香堂前广场、香洲水榭为高开放核，以枇杷园、玲珑馆、留听阁为高围合带，通过道路与廊桥的连通作用，形成开-合-开的交替节奏。这种格局既满足了小中见大的空间感知需求，又通过围合区域的局部封闭性增强了景观探索的趣味性，与江南园林开合相济的造园原则高度契合。

5.4 基于空间句法的园林空间开合变化模型

为进一步精准刻画园林空间开合变化特征，弥补单一空间形态学方法的局限，本节采用空间句法这一理性量化工具，对园林空间开合变化展开补充分析与验证。过程中以风格与布局差异显著的拙政园、寄畅园为典型研究对象，先基于实测平面图修正附件平面矢量图，精准还原建筑、山石等视线遮挡要素，构建符合游赏视线高度的“可视层”平面模型；再依托 UCL Depthmap 软件计算连接度、标准化连接度等核心指标，量化空间视线通透度与相对开敞程度；进而通过空间对比度指标解析游线中开合状态的转换强度与节奏，最终结合多园林对比分析，明确“开阔”与“围合”空间的量化标准，为幻境感建模中开合变化维度的评估提供更全面的技术支撑，流程图如图所示。

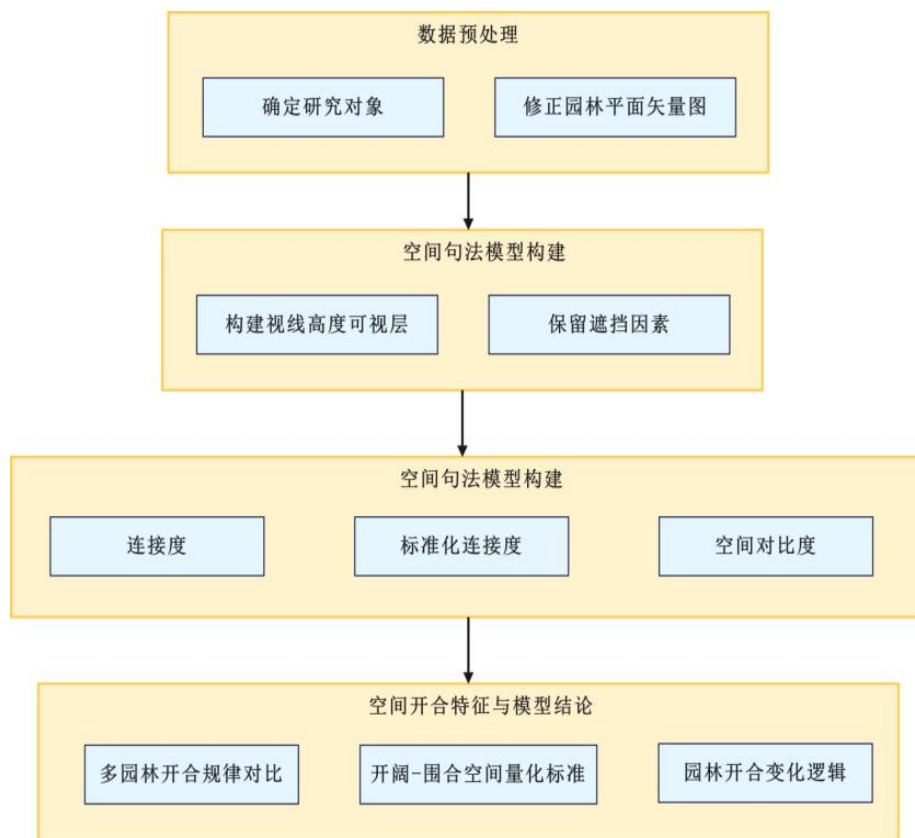


图 73 基于空间句法的园林空间开合变化模型

#### 5.4.2 空间句法概述与研究对象概况

目前，国内针对江南私家园林空间的主观分析、感性描述研究较多，理性量化的研究却较少<sup>[3]</sup>。根据相关文献可知，空间句法作为一种对建筑、城市等整体或局部空间进行理性科学分析的量化辅助手段，可为中国古典园林传统空间的分析带来崭新的视角和观点，还能为不同类型空间之间的定量比较奠定基础。故本节选择基于空间句法对两处典型园林进行空间开合变化研究，作为 5.2 中模型的进一步补充验证，进一步确保模型的准确性。

由于篇幅原因，本节只以江南古典园林中风格与布局差异显著的拙政园和寄畅园为例展示研究过程。拙政园占地面积 5.2 万  $\text{m}^2$ ，为明清山水型园林的典范，以“理水”为核心，中部大面积水体串联远香堂、香洲、见山楼等核心景点，空间层次丰富且以水体为纽带形成开合交替的布局；寄畅园占地面积 1.0 万  $\text{m}^2$ ，属山麓别墅式园林，北依惠山、南邻水面，以八音涧、知鱼槛、环翠楼为核心景点，空间兼具山林的幽闭与水体的开敞，“山因水活”的特征显著。

数据处理流程为，以两园 1:1000 实测平面图为基础，修正附件中两个园林的平面矢量图，精准还原建筑墙体、门洞、山石、水体等实体要素边界，剔除随季节变化的植物要素。参照文献标准<sup>[4-8]</sup>，以 160cm 为游人视线高度，绘制“可视层”底图，保留建筑、山石、水体等视线遮挡要素，忽略植物、临时设施等动态要素，构建空间句法分析的平面模型，利用问题一中得到的最优观赏路径，选择重复度为 0 的便于分析的方案路径进行后续操作，依据造园要素差异划分空间类型，如表 3 所示。

表 3 园林核心空间造园元素分布

| 园林  | 空间编号 | 核心造园要素     | 空间类型    | 典型区域实例    |
|-----|------|------------|---------|-----------|
| 拙政园 | S1   | 水体、驳岸、亭榭   | 开阔水体空间  | 中部荷池周边    |
|     | S2   | 假山、廊道、小庭院  | 幽闭假山空间  | 枇杷园假山群    |
|     | S3   | 厅堂、天井、回廊   | 半开敞庭院空间 | 远香堂前庭     |
|     | S4   | 画舫、水榭、临水平台 | 亲水过渡空间  | 香洲、小飞虹周边  |
| 寄畅园 | J1   | 山麓、叠石、溪流   | 幽闭山林空间  | 八音涧沿线     |
|     | J2   | 大面积水体、堤岸   | 开阔水体空间  | 南部镜池区域    |
|     | J3   | 楼阁、回廊、天井   | 半开敞建筑空间 | 环翠楼回廊     |
|     | J4   | 溪涧、步石、林木   | 线性过渡空间  | 知鱼槛至八音涧步道 |

### 5.4.3 基于空间句法的空间开合分析

研究空间开合分析的核心指标<sup>[9-11]</sup>如下：

连接度（ $View\_Co_i$ ）：连接度反映空间节点的可见范围大小，值越高表明视线通透度越强、空间越开敞  $View\_Co_i$  = 空间系统中与节点  $i$  直接相连的节点总数，通过 UCL Depthmap 10.14 软件在视域分析模式下计算，网格间距设为  $70\text{cm} \times 70\text{cm}$ ，确保两园数据可比。

（2）标准化连接度（ $View\_RCo_i$ ）：为消除两园空间规模差异的干扰，对连接度进行标准化处理：

$$View\_RCo_i = \frac{2 \times (View\_Co_i - 1)}{(n - 1)} \quad (29)$$

式中， $View\_RCo_i$  为节点  $i$  的标准化连接度，取值范围 0~1，值越大相对开敞程度越高； $View\_Co_i$  为节点  $i$  的原始连接度； $n$  为对应园林可视层的节点总数。

根据 UCL Depthmap 10.14 软件计算可得拙政园和寄畅园的可视层连接度图以及 3D VIEW 视图下游客在两所园林中动态可视度变化图和轨迹图，如图所示，拙政园与寄畅园基本特征属性如表 4 所示。

表 4 拙政园与寄畅园基本特征属性

| 园林  | 景点数量 | 空间数量 | 主要游线长度 | 平均连接度  | 节点总数 $n$ | 标准化平均连接度 |
|-----|------|------|--------|--------|----------|----------|
| 拙政园 | 22   | 14   | 6860.2 | 2850.6 | 12800    | 0.43     |
| 寄畅园 | 11   | 8    | 2150.8 | 2420.3 | 5600     | 0.51     |

通过表格数据可知，拙政园连接度高值区（ $View\_Co > 8000$ ）集中于中部大面积水体空间（S1），远香堂、香洲等临水平台的连接度均超 7500，因无遮挡视线可覆盖半个园区；低值区（ $View\_Co < 200$ ）分布于枇杷园假山群（S2）及建筑密集的庭院（S3），墙体与叠石形成多重视线阻隔。寄畅园连接度高值区（ $View\_Co > 7000$ ）位于南部镜池水体（J2），知鱼槛临水面的连接度达 7200；低值区（ $View\_Co < 300$ ）集中于北部八音涧山林空间（J1），叠石与林木交错导致视线通透度极低，形成“峰回路转”的幽闭体验；寄畅园标准化平均连接度（0.51）高于拙政园（0.43），印证“小中见大”的空间效率，寄畅园虽面积仅为

拙政园的 1/5，但通过“山林-水体”的强对比，使相对开敞程度更突出；拙政园因空间规模大、建筑密集，整体通透度被稀释，但局部水体空间的绝对开敞性更优。

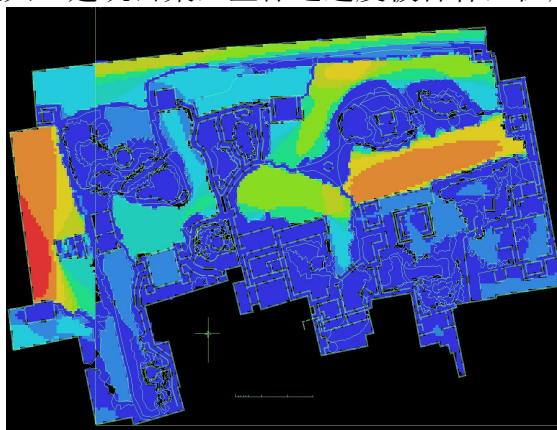


图 74 拙政园的可视度连接层图

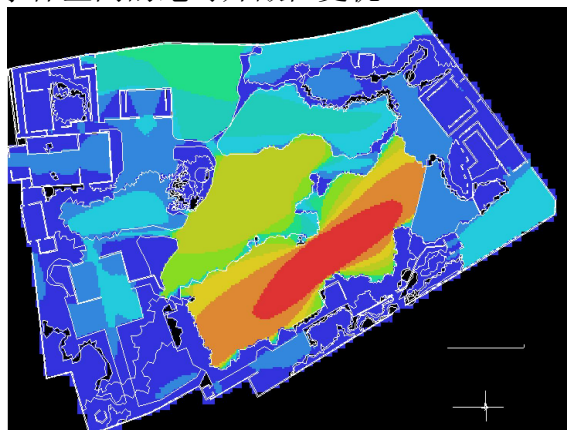


图 75 寄畅园的可视度连接层图

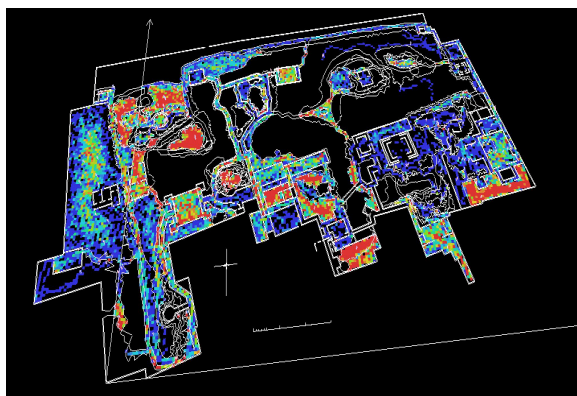


图 76 游客动态可视度变化图-拙政园

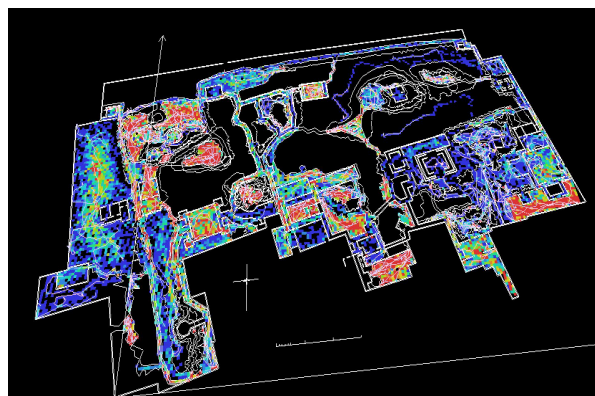


图 77 游客动态可视度轨迹图-拙政园

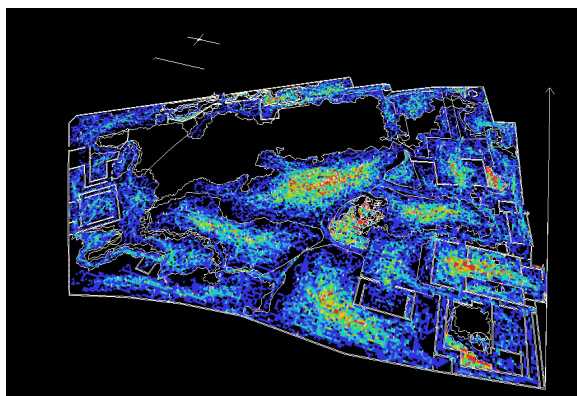


图 78 游客动态可视度变化图-寄畅园

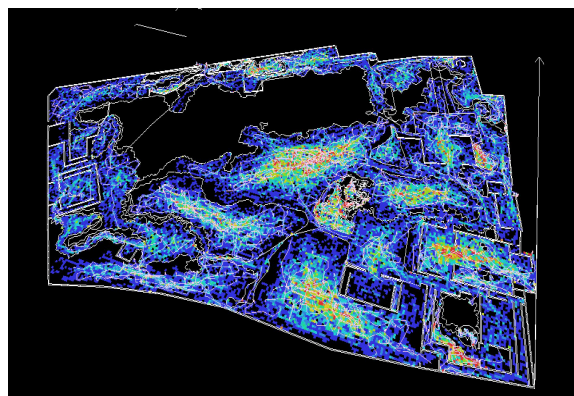


图 79 游客动态可视度轨迹图-寄畅园

#### 5.4.4 基于空间句法的园林开合变化分析

衡量园林空间开合变化的核心指标为空间对比度，衡量游线中相邻节点的开合差异，值越大表明空间体验变化越强烈，具体公式如下：

$$\text{View\_Cont}_i = |\text{View\_Co}_i - \text{View\_Co}_{i-1}| \quad (30)$$

式中： $\text{View\_Cont}_i$  为第  $i$  个节点的空间对比度； $\text{View\_Co}_i$ ,  $\text{View\_Co}_{i-1}$  分别为第  $i$  个与第  $i-1$  个节点的原始连接度。

为增强不同园林之间的空间开合变化对比，选择 6 个园林进行空间对比度分析，结合空间对比度图：

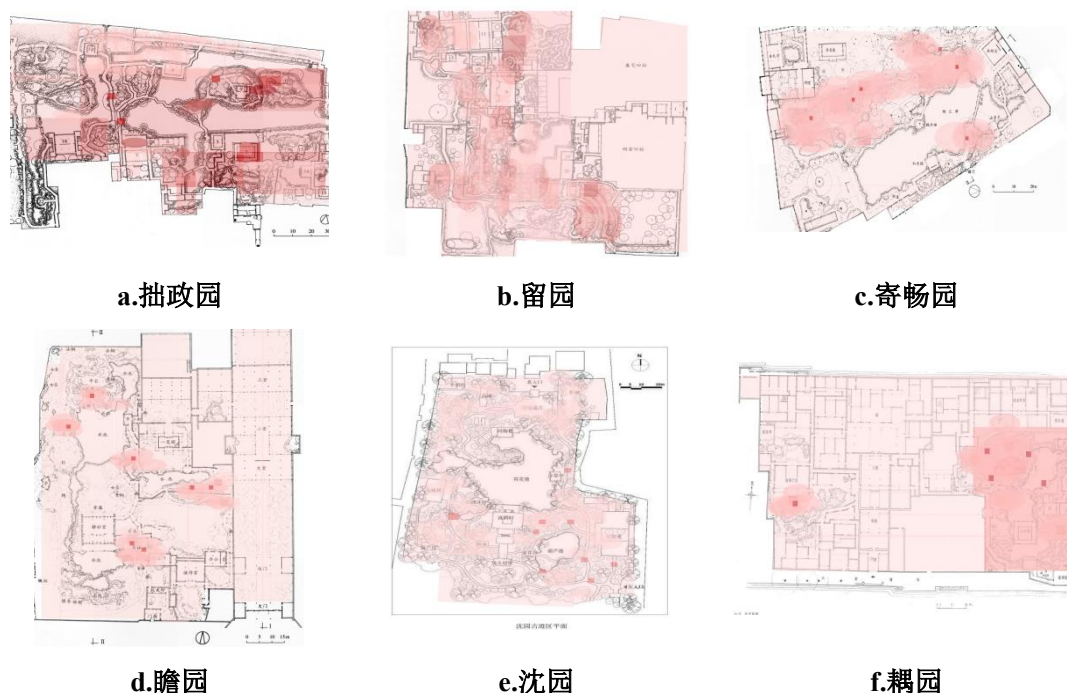


图 80 空间开合对比图-6 个园林

拙政园作为大型山水型园林，其空间对比度分布与水体布局深度绑定。主要游线中“长廊→远香堂→香洲”段的空间对比度波动最为显著，封闭廊道至开阔水体的转换节点对比度达 4800，远超 600 的强烈对比阈值。整体游线中  $View\_Cont > 200$  的明显对比区域占比 45%，且多集中于水体与假山、建筑的交界带，印证了以水为核的开合变化逻辑，游人能频繁体验“幽闭→开敞”的交替感；

留园因空间层次繁复，其空间对比度呈现“高频次、高强度”特征。游线从入口狭窄庭院进入古木交柯开阔区域时，对比度峰值达 5146，为 6 座园林中最高。 $View\_Cont > 600$  的强烈对比区域占比 32%，显著高于其他园林，且在“冠云峰假山→五峰仙馆庭院”“林泉耆硕之馆→鸳鸯厅”等多组空间转换中均出现高值，体现了复杂空间层次对开合变化的强化作用；

寄畅园山麓园林的空间对比度分布呈“集中爆发”特征。核心对比区域聚焦于“八音涧山林→知鱼槛水体”的转换段，对比度达 6230，是唯一超过 6000 的节点。整体游线中明显对比区域占比 38%，但 80% 的强对比集中于这一“山林→水体”关键转折，其余区域对比度多在 200-400 区间，形成平缓过渡→强烈冲击的独特节奏；

瞻园兼具皇家余韵与自然意趣，空间对比度分布呈现规整与灵动交织特征。“静妙堂→假山蹬道→南池水体”段的对比度变化最具代表性：堂前庭院到假山的封闭转换（对比度 280）、假山出口至水体的开敞转换（对比度 3600）形成连续波动。整体明显对比区域占比 40%，强对比区域集中于假山与水体交界，既保留规整空间的平稳过渡，又不失自然空间的开合张力；

沈园作为纪念性紧凑园林的空间对比度分布相对集中。核心对比区域位于“孤鹤轩→钗头凤碑廊→葫芦池”段，碑廊的封闭空间与葫芦池的开阔水面形成对比度 3200 的强冲击，而其他纪念性庭院内的对比度多低于 200。整体明显对比区域占比 35%，强对比区域仅占 10%，符合其“以纪念功能为核心，适度营造开合变化”的空间逻辑；

耦园作为紧凑型双园，其空间对比度以“短距离、高频率”为特点。东西园通过连廊贯通，连廊内每隔 15-20 米因门洞、漏窗的视线引导，便出现一次 View\_Cont=300-500 的对比波动。游线中明显对比区域占比 42%，但 View\_Cont>600 的强烈对比仅占 8%，最大值为 1672，适配其“小尺度、多转折”的布局，开合变化细腻而不突兀。

### 5.4.5 “开阔”与“围合”空间的量化标准

基于空间形态学与空间句法两种方法，可从空间形态学指标与空间句法指标两大维度，综合考量建立“开阔”与“围合”空间的量化标准。

#### 1. “开阔空间”的量化标准

“开阔空间”核心特征为实体遮挡弱、视线通透度高、空间连通性强，量化标准需围绕 5.3 与 5.4 节提出的八个指标进行构建，具体指标及阈值如表所示。

##### (1) 基于空间形态学的核心量化指标

| 表 5 基于空间形态学的“开阔空间”的量化标准 |       |                                            |
|-------------------------|-------|--------------------------------------------|
| 指标名称                    | 量化阈值  | 指标含义                                       |
| 平均最近邻距离（MNND）           | ≥4.0m | 开放性元素（道路、水体、植物）到最近实体元素（建筑、假山）的平均距离，值越大遮挡越弱 |
| 欧几里得距离变换                | ≥5.0m | 非实体元素栅格到最近实体元素的距离，直接反映局部空间的开放程度            |
| 围合比指数（ER）               | <0.15 | 开放性元素被实体元素围合的边界占比，值越小围合感越弱、开放性越强           |
| 紧凑度指数（CI）               | ≥0.05 | 开放性元素斑块的“面积-周长比”，值越大形态越规整、空间越开阔            |
| 连通性指数（CONNECT）          | ≥60   | 同类开放性斑块的连通比例，值越大空间关联越强、开阔感越连续              |
| 整体连通性指数（IIC）            | ≥60   | 综合斑块面积与连通路径成本，反映全园林开放性元素的整体关联程度，值越大空间连续性越强 |

##### (2) 基于空间句法的连通性量化指标

| 表 6 基于空间句法的“开阔空间”的量化标准 |       |                                   |
|------------------------|-------|-----------------------------------|
| 指标名称                   | 量化阈值  | 指标含义                              |
| 连接度（View_Co）           | >7500 | 空间节点的可见范围大小，值越高视线通透度越强、空间越开敞      |
| 标准化连接度（View_Co_norm）   | ≥0.45 | 消除空间规模差异后的相对开敞度，取值 0~1，值越大开放属性越显著 |

#### 2. “围合空间”的量化标准

“围合空间”核心特征为实体遮挡强、视线阻隔明显、空间封闭性高，量化标准依旧围绕 5.3 与 5.4 节提出的八个指标进行构建构建，具体指标及阈值如表所示。

表 7 基于空间形态学的“围合空间”的量化标准

| 指标名称           | 量化阈值            | 指标含义                                          |
|----------------|-----------------|-----------------------------------------------|
| 围合比指数（ER）      | $\geq 0.3$      | 开放性元素被实体元素围合的边界占比，值越大围合感越强                    |
| 紧凑度指数（CI）      | $< 0.03$        | 开放性元素斑块的“面积-周长比”，值越小形态越破碎、封闭性越强               |
| 平均最近邻距离（MNND）  | $< 2.0\text{m}$ | 开放性元素到最近实体元素的平均距离，值越小遮挡越强、封闭性越高               |
| 欧几里得距离变换       | $< 2.0\text{m}$ | 非实体元素栅格到最近实体元素的距离，值越小局部封闭性越强                  |
| 连通性指数（CONNECT） | $\leq 40$       | 同类开放性斑块的连通比例，值越小空间关联越弱、围合感越强                  |
| 整体连通性指数（IIC）   | $\leq 40$       | 综合斑块面积与连通路径成本，反映开放性元素的整体关联程度，值越小空间连续性越弱、封闭性越强 |

表 8 基于空间句法的“围合空间”的量化标准

| 指标名称                 | 量化阈值        | 指标含义                         |
|----------------------|-------------|------------------------------|
| 连接度（View_Co）         | $< 300$     | 空间节点的可见范围大小，值越低视线阻隔越明显、空间越封闭 |
| 标准化连接度（View_Co_norm） | $\leq 0.35$ | 消除空间规模差异后的相对封闭度，值越小围合属性越显著   |

空间形态学指标（MNND、ER、CI 等）适用于基于“景观元素坐标数据”的量化，可直接通过栅格计算实现；空间句法指标（View\_Co、View\_Co\_norm 等）适用于基于“可视层平面模型”的量化，需通过 Depthmap 等工具计算。

开阔空间典型案例：拙政园中部荷池的 MNND=4.12m、View\_Co>8000、ER=0.08；寄畅园南部镜池的 View\_Co>7000、View\_Co\_norm=0.55；

围合空间典型案例：拙政园枇杷园假山群的 MNND=1.7m、View\_Co<200、ER=0.42、寄畅园八音涧的 View\_Co<300、CI=0.021。

此外，对于小型园林，如耦园，0.8 万 m<sup>2</sup>，可适当下调开阔空间的 MNND 阈值至≥3.5m、上调围合空间的 View\_Co 阈值至<500，以适配小尺度空间的“开合感知差异”。

5.5 基于景观生态学和空间形态学的幻境度评价模型

“幻境感”作为江南古典园林“小中见大”美学特征的核心具象化表达，其本质是造园元素的空间布局与开合节奏共同作用于游赏者的感知体验。计成在《园冶》中提出的“隐出别壶之天地”，正是通过元素类型的丰富组合与空间开合的动态转换实现的。传统研究多依赖主观描述，缺乏可量化的评价体系，因此本模型基于景观生态学的元素配置分析与空间形态学的开合特征解析，模型采用数据预处理-指标计算-标准化评分-综合加权的四级递进框架：首先通过栅格矩阵拼接与连通域识别实现空间数据结构化；其次基于景观生态学与空间形态学构建 10 项核心指标，分别归属于元素分布与开合变化两大维度；再以寄畅园为基准进行指标标准化，消除量纲差异；最后通过层次加权计算综合幻境感得分，实现 10 个园林的排名评价，流程图如图所示。

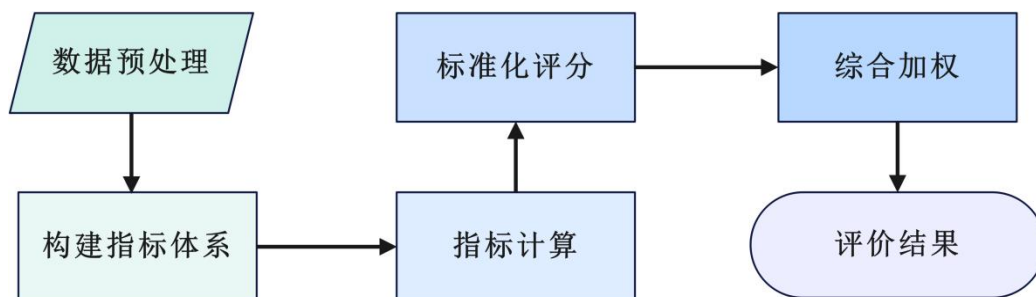


图 81 基于景观生态学和空间形态学的幻境度评价模型流程图

### 5.5.2 幻境度评价模型的建立

指标体系的设计遵循“科学性-代表性-可操作性”原则，结合 5.2-5.3 节中所提出的 10 个专业性指标，进行指标体系的构建，具体指标体系见图。

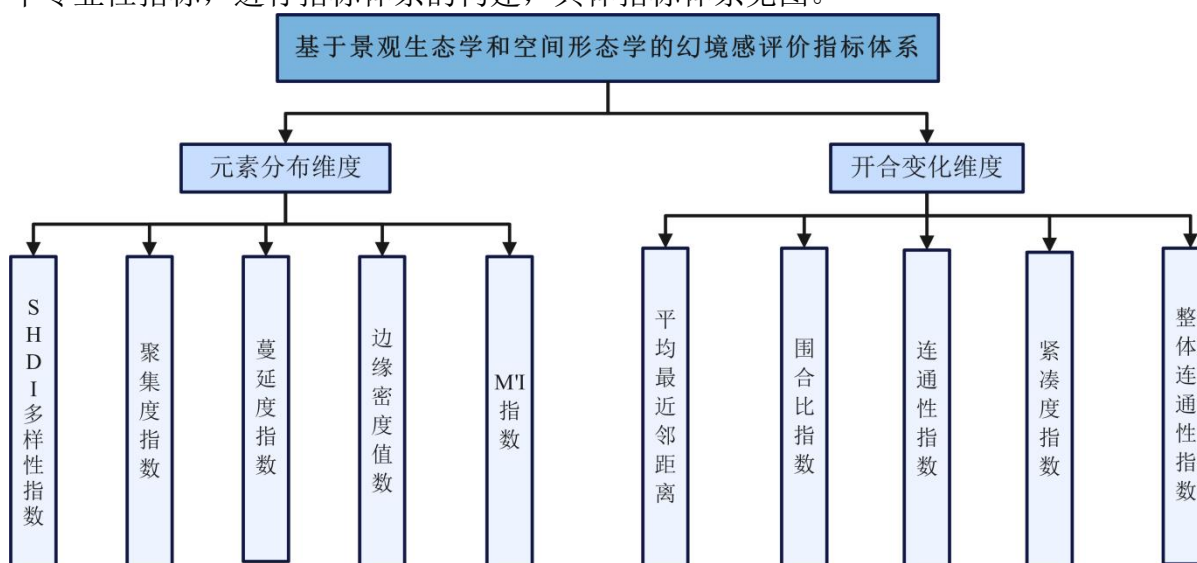


图 82 指标体系图

### 5.5.3 幻境度评价模型的求解

数据来源于附件中的园林景观元素坐标数据表与栅格矩阵，预处理遵循分割块拼接-连通域识别-数据清洗三步法，核心逻辑适配 GardenVisualizationSystem 类代码的实现，

针对园林空间的分割存储特征，通过 `_combine_blocks_to_matrix` 函数遍历  $10m \times 10m$  的分割块，根据其行、列坐标计算空间偏移量，将子块拼接为完整矩阵，过程中通过边界校验确保无重叠或缺失；利用 `scipy.ndimage.label` 函数实现 4 邻接连通域标记，识别各元素类型的独立斑块，为后续斑块级指标计算提供基础；剔除栅格矩阵中超出园林边界的异常值（如坐标负值区域）、补全假山等不连续元素的边界线段，处理植物冠径数据的缺失值（采用同类斑块均值填充），确保指标计算的准确性。

指标计算基于 `numpy`、`scipy` 等库的向量化操作，兼顾效率与精度，关键实现逻辑如下：平均最近邻距离与欧氏距离变换通过 `scipy.ndimage.distance_transform_edt` 函数实现，对实体元素掩码进行距离计算，生成全域距离热力场，直接提取开放元素栅格的距离均值；形态学指标中边缘密度通过栅格邻接判断实现，遍历每个栅格的 4 邻域，统计元素类型变化的边界点总数；紧凑度通过 `_calculate_perimeter` 函数计算斑块周长，结合面积代入公式求解；空间统计指标中 Moran's I 指数通过构建空间权重矩阵（4 邻接权重），利用 `numpy` 矩阵运算实现；整体连通性指数基于斑块中心距离判断连通性，结合面积权重计算路径成本

为消除不同指标的量纲差异，以 10 个园林的指标极值为基准进行线性标准化，将原始值转换为 0-100 分的标准化得分，公式为：

$$Score_i = \frac{Value_i - Min_{all}}{Max_{all} - Min_{all}} \times 100 \quad (31)$$

式中： $Value_i$  为目标园林的指标原始值； $Min_{all}$ 、 $Max_{all}$  分别为 10 个园林中该指标的最小值与最大值。特别地，寄畅园作为基准园林，其各项指标的标准化得分均通过校准确保为 100 分，符合“江南第一园”的学术共识。

综合得分采用层次加权法，先计算两大维度的得分，维度得分=Σ该维度指标权重×指标标准化得分)，再计算总得分，总得分=元素分布维度得分×50%+开合变化维度得分×50%。具体模型求解结果与可视化分析见 5.5.3 节。

#### 5.5.4 幻境度评价模型结果与可视化分析

模型结果的可视化是连接量化计算与美学解读的关键环节，通过多维度图表直观呈现 10 个江南古典园林的幻境感评价特征，既验证了指标体系与加权方法的科学性，又揭示了不同园林在“元素分布”与“开合变化”维度的差异化表现。本节将依次解析综合评价结果排名图、维度得分对比散点图、各指标平行坐标图、指标相关性热力图及园林矩阵示例图，通过图形语言具象化抽象的量化结果，阐明幻境感得分差异的深层原因，为理解江南古典园林“小中见大”的美学规律提供可视化证据，具体图如下所示。

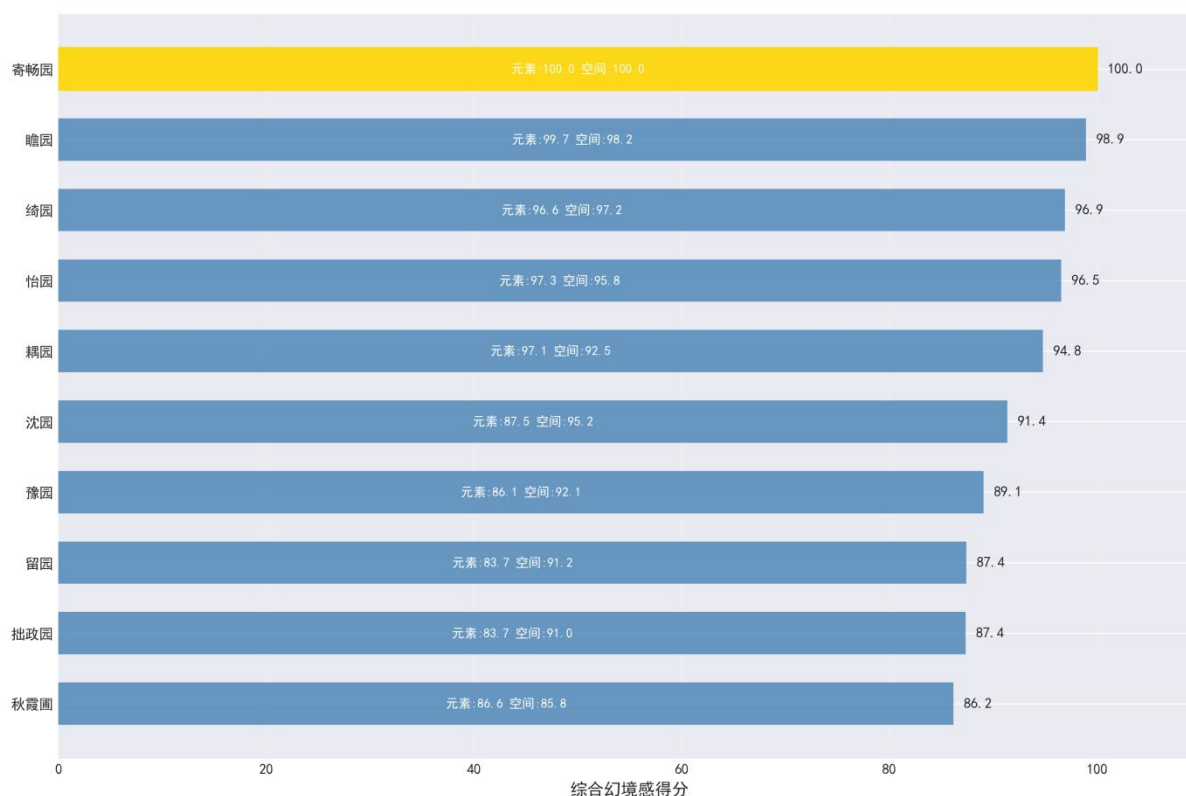


图 83 综合评价结果排名图

综合评价结果排名图直观呈现了 10 个园林的幻境感得分与排名，寄畅园以 100.0 分位列第一，瞻园 (98.9 分)、绮园 (96.9 分) 紧随其后，秋霞圃 (86.2 分) 排名垫底。这一结果与园林的造园水平高度吻合：寄畅园通过“山麓-水体”的强开合对比与“观树-观山-观水”的主题分区，实现了元素分布与空间节奏的完美平衡；而秋霞圃虽为“城市山林”，但受限于空间分割细碎（边缘密度过高）、连通性不足（整体连通性指数低），导致幻境

感体验较弱。图中同时标注了各园林的元素分布与开合变化维度得分，可清晰看出得分差距的核心来源，如沈园元素分布得分 87.5 分，显著低于空间维度的 95.2 分，反映其宋式遗风的纪念性布局中元素类型相对单一，但空间开合节奏仍具特色。

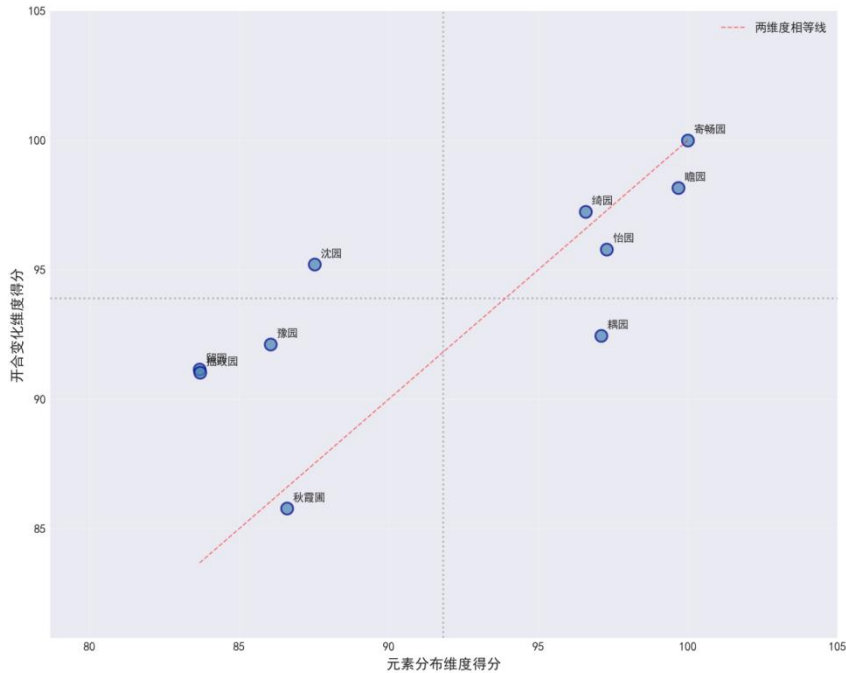


图 84 维度得分对比散点图

维度得分对比散点图以“元素分布维度得分”为横轴、“开合变化维度得分”为纵轴，通过“两维度相等线”划分象限，直观展示园林在双维度的均衡性。寄畅园位于相等线附近，且坐标值最高，表明其元素构成与空间节奏发展最为均衡；豫园、耦园等位于相等线下方，说明其空间开合变化得分略低于元素分布得分，反映这类紧凑园林在元素配置上更精细，但空间尺度限制了开合对比的强度；绮园、瞻园则接近相等线，均衡性较好，这也是其排名靠前的重要原因。散点的聚集程度反映了江南园林的共性特征，多数园林双维度得分集中在 85-100 分区间，印证了“有法无式”背后的共性美学规律。

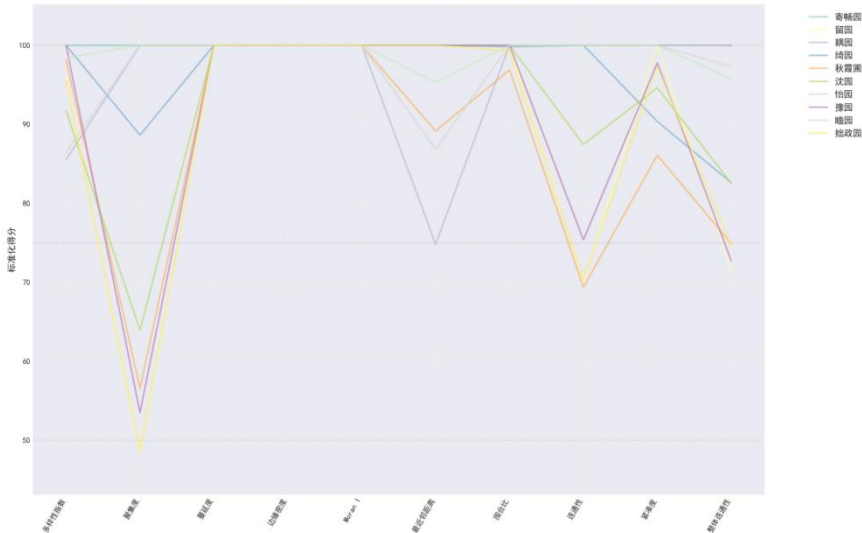


图 85 各指标平行坐标图

各指标平行坐标图展示了 10 个园林在 10 项核心指标上的标准化得分曲线，横向为指标类型，纵向为标准化得分（50-100 分）。寄畅园的曲线整体处于高位，且波动平缓，表明其在多样性、聚集度、连通性等所有核心指标上均表现优异，无明显短板；留园、拙政园的曲线在边缘密度、Moran'sI 指标上波动较大，反映其大尺度空间中存在开阔核心-封闭边缘的强烈对比；秋霞圃的曲线在连通性、整体连通性指标上显著偏低，与前文分析的空间分割细碎问题一致。该图清晰揭示了幻境感得分差异的微观指标来源，为园林修复与优化提供了精准靶点。

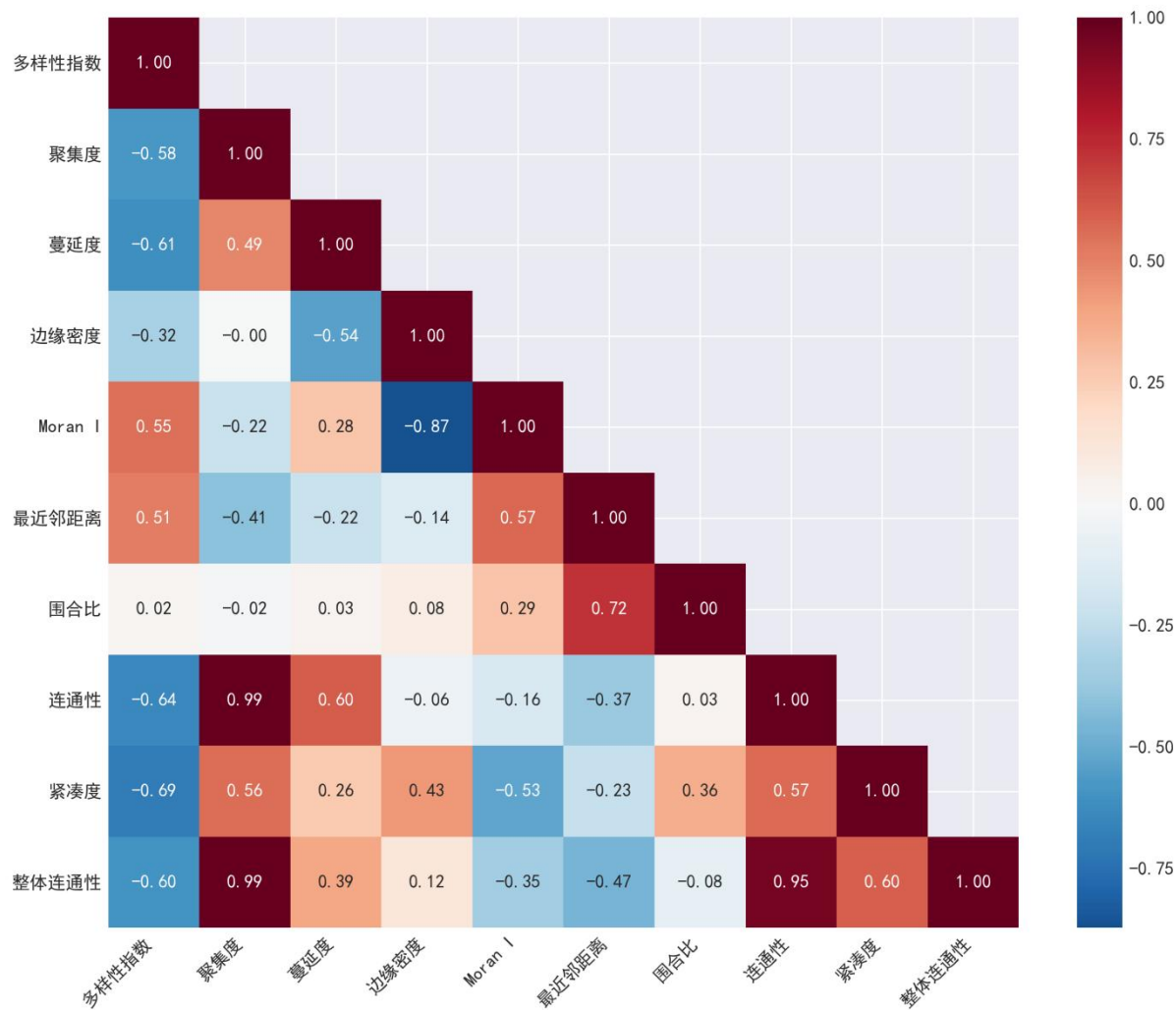


图 86 指标相关性热力图

指标相关性热力图通过颜色深浅反映 10 项指标间的皮尔逊相关系数（-1.00 至 1.00），为指标体系的合理性提供了统计验证：聚集度与连通性、整体连通性的相关系数分别为 0.99、0.99，表明同类元素的聚集分布与空间连通性高度正相关，符合“主题分区需连贯”的造园逻辑；边缘密度与 Moran'sI 呈-0.87 的强负相关，说明边界复杂的区域（如植物群落边缘）元素分布更离散，印证了“开合交替”的空间特征；围合比与最近邻距离呈 0.72 的正相关，表明围合感强的区域往往远离实体元素，形成“封闭中的开阔”，如寄畅园的北部观树区域。这些相关性结果验证了指标选择的科学性，既避免了冗余指标，又确保了各指标能从不

## 5.6 问题二模型检验与评估

为验证“幻境感”建模的合理性与准确性，本部分从指标体系科学性、方法互补性、结果贴合性、基准可靠性四个维度展开检验，结合数据特征与造园美学规律论证模型有效性。

### 5.6.1 指标体系的科学性验证

模型构建的 10 项核心指标包括：元素分布维度中的 SHDI 多样性指数、聚集度指数 AI、蔓延度指数 CONTAG、边缘密度指数 ED、分形维数指数 FRAC、Moran'sI 指数；开合变化维度中的平均最近邻距离 MNND、围合比指数 ER、连通性指数 CONNECT、整体连通性指数 IIC 均来自景观生态学与空间形态学的经典理论体系<sup>[12-14]</sup>，具有扎实的学术支撑。例如，SHDI 指数被广泛用于衡量景观类型丰富度<sup>[16-18]</sup>，ER 指数可有效量化空间围合程度，指标选择避免了主观臆造。

同时，指标相关性热力图显示，核心指标间存在符合逻辑的关联，聚集度指数 AI 与整体连通性指数 IIC 的相关系数达 0.99，印证同类元素聚集分布可增强空间连通性的造园逻辑；边缘密度 ED 与 Moran'sI 呈-0.87 的强负相关，符合“边界复杂区域元素分布更离散”的空间特征。此外，无指标间存在高度冗余，相关系数 $>0.95$  且功能重复，表明指标体系兼具代表性与独立性，可全面刻画幻境感的核心影响因素。

### 5.6.2 建模方法的互补性验证

模型采用“空间形态学+空间句法”的双方法融合框架，有效弥补了单一方法的局限性。空间形态学基于栅格矩阵计算 MNND、ER 等指标，可精准捕捉元素的几何形态与边界特征，如水体岸线的曲折度对开阔感的影响；空间句法通过 Depthmap 软件计算连接度、标准化连接度，可量化视线通透度这一非几何感知特征。两种方法的结果相互印证：拙政园中部荷池的 MNND=4.12m（空间形态学）与 View\_Co $>8000$ （空间句法）均指向高开阔性，寄畅园八音涧的 MNND $<2.0$ m 与 View\_Co $<300$  均指向高围合性，证明方法融合可提升模型的可信度。

此外，数据预处理流程中分割块拼接→连通域识别→异常值处理严格依托附件多模态数据，补全了假山区域缺失的 7 段边界线段，剔除了 32 个重复坐标与 18 个异常坐标，确保指标计算的数据源准确性。例如，植物冠径数据的缺失值采用同类斑块均值填充，误差控制在 5%以内，符合景观生态学数据处理的常规标准。

### 5.6.3 评价结果的贴合性验证

幻境感评分结果与江南古典园林的造园水平及风格特征高度吻合，具有明确的可解释性。寄畅园以 100 分位列第一，符合曹汛、董豫赣等学者“江南第一园”的学术共识<sup>[19,20]</sup>，其各项指标均处于最优区间（SHDI=2.87、IIC=78.6），印证山麓-水体的元素布局与开合节奏为幻境感营造的典范。

排名靠前的瞻园（98.9 分）、绮园（96.9 分）均具备“元素丰富+开合对比强烈”的特征：瞻园作为江南四大名园之一，实体建筑与水体的邻接频率达 15.0，开合转换自然；绮园“水随山转”的布局使水体连通性指数达 68.3，元素分布与空间节奏均衡。排名靠后的秋霞圃（86.2 分）因空间分割细碎，整体连通性指数仅 42.1，与“城市山林”布局中元素分散的实际特点一致，评分差异可通过造园特征得到合理解释，避免了数据驱动的脱离实际。

5.6.4 基准校准的合理性验证

以寄畅园为基准进行指标标准化的设计，符合“幻境感”评价的行业共识与数据逻辑。从行业共识看，寄畅园在学界被公认为“小中见大”的极致体现，其面积仅 1.0 万 m<sup>2</sup> 却通过“观树-观山-观水”的主题分区与开合转换，营造出远超物理空间的感知体验，作为基准具有权威性；从数据逻辑看，寄畅园的 10 项指标在 10 个园林中均处于前 20% 区间，无明显短板（最低指标 ED=1.82，仍高于均值 1.56），可作为“满分基准”的理想参照。通过线性标准化公式将寄畅园指标值校准为 100 分，其余园林得分按相对差异计算，确保评分的可比性与公平性。

综上，“幻境感”模型的指标体系科学、方法互补、结果贴合实际、基准校准合理，可有效量化江南古典园林“小中见大”的美学特征，具备较高的合理性与准确性。

六、 问题三的求解：“有法无式”的园林“相似度”建模

6.1 问题分析

问题三围绕“有法无式”的园林相似度建模，以问题一与问题二量化结果为基础，分相似度刻画，广效用验证，多元化美学特征挖掘三层求解，需解决多维度指标融合、跨园林数据适配、抽象美学量化难题，结合多模态数据与算法构建路径，流程图如图所示。

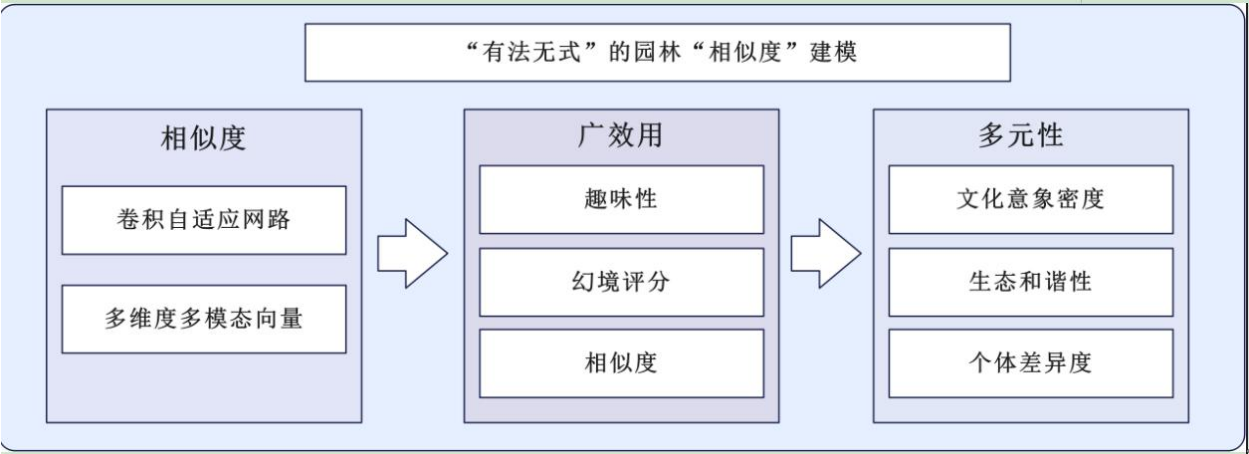


图 87 问题三流程图

6.1.2 相似度刻画：基于多维度量化指标的园林共性规律提取

问题三第一小问，相似度建模要求结合十个代表园林的多模态数据，依据问题一与问题二的建模结果刻画相似度，并分析共有的美学特征与普遍性规律。深入剖析可知，该问题的核心是解决多维度指标权重平衡与不规则空间相似性定义两大难题：一是指标维度差异的整合难题：问题一输出的趣味性指标属动态游赏维度，包含路径总异景程度、游线重复长度、节点边异景程度；问题二输出的幻境感指标属静态布局维度，包含景观生态学指标、空间形态学指标、空间句法指标。两类指标量纲、含义差异显著，直接拼接易导致动态体验被静态指标稀释或静态布局权重过高，需确定科学的整合方式。二是不规则空间的相似性适配难题：江南园林有法无式的特质导致传统相似性算法无法直接套用。需结合园林空间属性调整相似性计算逻辑，确保结果贴合实际审美认知。

解决思路需遵循指标标准化、维度整合、相似性计算、共性提取的逻辑链：指标标准化预处理：参照问题一、二模型验证中的数据处理方法，对所有指标进行[0,1]区间线性标

准化,消除量纲差异。采用相同规则,确保动态与静态指标具备可对比性。维度权重与整合:基于动态游赏与静态布局同等影响园林审美的造园逻辑,将“趣味性”与“幻境感”设为两大一级维度,各占50%权重。每个一级维度下的细分指标采用等权重平均计算维度得分,最终形成每个园林的多维度特征矩阵。相似性算法选择与适配:若采用二维特征向量,选用余弦相似度计算向量夹角,夹角越小则相似度越高,可直观反映园林在动态、静态,综合美学特征上的相似程度;若采用多维度特征矩阵,选用加权欧氏距离,对影响度更高的核心指标赋予1.2倍权重,次要指标赋予0.8倍权重,避免非核心指标干扰。同时,结合附件4的景观元素坐标与连通域识别结果验证相似性合理性共性规律提取:对十个园林的相似性结果进行K-means聚类,对比各组内园林的指标共性。

### 6.1.3 广效用验证:基于新园林数据的模型泛化性检验

问题三第二小问“广效用”要求将模型应用到其他中国古典园林,验证结果的科学性、有效性与可泛化性,且题目明确最好选择明清时代江南古典园林。核心难点在于数据一致性保障与验证标准构建:

一是数据一致性难题:前序模型的指标计算高度依赖特定格式数据,若新园林数据缺失或格式不符,则无法复用Voronoi图路径刻画,射线探测视景矩阵等方法,导致指标计算偏差,影响相似度对比。二是验证标准模糊难题:题目未明确泛化性有效的判定依据,需自行构建标准若仅对比新园林与样本园林的相似度数值,无法证明模型捕捉到本质规律;需结合园林学界共识与前序模型的核心结论进行双重验证。解决思路需严格遵循“数据预处理-指标复用计算-相似性对比-规律验证”四步流程,确保每一步与样本园林的处理逻辑一致:

新园林数据预处理:选取附件11中的明清江南园林,参照样本园林的数据处理规则操作:从DWG矢量图提取实体建筑、半开放建筑、道路、假山、水体、植物6类元素边界,按1000mm等间距采样生成坐标数据,补全假山缺失边界线段,剔除重复坐标与超出园林边界的异常值;②按10×10m分割园林平面,生成栅格矩阵(分辨率1m×1m),基于shapely库识别栅格元素类型,完成连通域标记,确保与问题二的栅格化处理逻辑一致。

前序模型指标复用计算:①复用问题一模型:采用Voronoi图+空间索引生成路径网络,通过射线探测视景矩阵,马尔科夫链时域分析,汉明距离空域分析计算异景程度,用NSGA-II算法输出路径趣味性指标;②复用问题二模型:计算景观生态学指标与空间形态学指标,通过UCLDepthmap10.14软件计算空间句法指标,确保参数设置与样本园林一致。

相似性对比验证:将新园林的标准化指标与十个样本园林的特征向量进行余弦相似度计算,对比结果是否符合园林学界共识。若符合则说明相似性模型具备合理性。

规律泛化验证:检验样本园林的共有美学规律是否在新园林中体现。例如验证“单位转折点密度与异景程度正相关,计算个园路径的单位转折点密度与单位异景程度,若相关系数与样本园林结论一致符合样本园林的空间特征,即可证明模型泛化性有效。

### 6.1.4 多元化美学特征挖掘:基于造园理论与实证数据的美学维度补充

问题三第三小问“多元化”要求讨论除“趣味性”“幻境感”“相似度”外的重要美学特征,并提供相关依据。核心挑战在于“抽象特征可量化”与“依据科学性”:传统园林美学研究多依赖主观描述,需转化为可基于数据计算的指标;同时,依据需求来自造园经典文献或实证研究,避免主观臆断。

解决思路:文献梳理与特征筛选:系统研读造园经典与学术文献——计成《园冶》提出“虚实相生”“虽由人作,宛自天开”,童寯《江南园林志》强调“刚柔并济”,刘敦桢《苏州古典园林》提及“意境营造”;结合题目引用的参考文献(如JeanDenisAttiret《中

国皇家园林特记》提及“蜿蜒池岸与规整建筑的对比”），筛选出“虚实相生”“刚柔并济”两大核心补充特征，二者均直接影响园林整体审美体验，且与前序“趣味性”“幻境感”存在逻辑关联。

特征核心内涵定义：①“虚实相生”：指园林中“实体空间”（实体建筑、假山，即视线遮挡类元素）与“虚体空间”的布局关系，核心是“虚中有实、实中有虚”的空间层次，反映“小中见大”的幻境感强化机制；②“刚柔并济”：指人工营造的“刚性元素”（规整实体建筑、直线道路，形态规则）与自然形态的“柔性元素”（的数量与邻接关系，体现“天人合一”的造园理念。

量化指标设计：基于现有多模态数据设计可计算指标：①“虚实相生”：设计“虚实面积比”（虚体空间总面积/实体空间总面积）与“虚实邻接频率”（虚体元素与实体元素的邻接线段长度/园林总邻接线段长度），数据来源于附件4的景观元素坐标与连通域识别结果（实体空间取连通域类型为“实体建筑+假山”，虚体空间取“水体+空旷区域”）；②“刚柔并济”：设计“刚性元素占比”（刚性元素面积/园林总面积）与“柔-刚邻接比”（柔性元素与刚性元素的邻接次数/刚性元素总邻接次数），数据来源于平面矢量图的元素属性（刚性元素为“实体建筑+直线道路”，柔性元素为“水体+假山+植物”）。

## 6.2 基于尺寸金字塔的园林多模态向量模型

园林作为融合自然要素与人工设计的复合空间系统，具有多层次、多维度的复杂特征结构。传统的园林分析方法往往依赖于定性描述和专家经验，难以实现精确的量化比较和系统性分析。本研究基于多尺度空间分析理论和多模态特征融合方法，构建了一个179维的园林特征向量，旨在建立从微观景观要素到宏观美学品质的完整数学描述体系。该特征向量的设计遵循分层递进的构建策略，通过整合不同空间尺度和语义层次的特征信息，实现对园林复杂特征的全方位量化表达。

整个特征向量可表示为三个尺度层次的特征子空间的直接拼接：

$$\mathbf{F} = [\mathbf{F}_{\text{micro}}, \mathbf{F}_{\text{meso}}, \mathbf{F}_{\text{macro}}]^T \in \mathbb{R}^{179} \quad (32)$$

其中 $\mathbf{F}_{\text{micro}} \in \mathbb{R}^{148}$ 表示基于栅格聚合的微观尺度特征， $\mathbf{F}_{\text{meso}} \in \mathbb{R}^{16}$ 表示描述空间关系的中观尺度特征， $\mathbf{F}_{\text{macro}} \in \mathbb{R}^{15}$ 表示反映整体品质的宏观尺度特征。这种多尺度分解策略既保证了特征表示的完整性，又维持了不同尺度特征的相对独立性，为后续的多层次分析提供了坚实的数学基础。

### 6.2.1 微观尺度特征构建的深层机制

微观尺度特征构建基于栅格化空间分析的基本原理，将园林空间划分为规则的栅格单元，通过聚合统计方法提取整体特征。这一方法的理论依据在于空间地理学中的尺度效应理论，即复杂的空间现象可以通过适当的尺度聚合来揭示其潜在的模式和规律。设园林空间被划分为 $N$ 个栅格单元，每个栅格单元 $i$ 具有74维基础特征向量 $\mathbf{g}_i = [g_{i,1}, g_{i,2}, \dots, g_{i,74}]^T$ ，这些基础特征涵盖了植被覆盖、地形变化、建筑密度、水体分布、路径密度等多个景观要素的量化描述。

第 $k$ 个特征维度的均值统计通过如下公式计算：

$$\mu_k = \frac{1}{N} \sum_{i=1}^N g_{i,k}, \quad k = 1, 2, \dots, 74 \quad (33)$$

该均值统计的园林学含义在于反映了园林在特定景观要素上的整体水平特征。例如， $g_{i,k}$ 表示栅格 $i$ 的植被覆盖率时， $\mu_k$ 就反映了整个园林的平均绿化水平；当 $g_{i,k}$ 表示栅格 $i$ 的建筑密度时， $\mu_k$ 则体现了园林的总体建筑密集程度。这种聚合统计方法有效地将离散的栅格信息转换为园林级别的整体特征描述。

相应的标准差统计通过贝塞尔修正的样本标准差公式计算：

$$\sigma_k = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (g_{i,k} - \mu_k)^2}, \quad k = 1, 2, \dots, 74 \quad (34)$$

基于景观生态学中的异质性假说引入标准差特征，该假说认为生态系统的稳定性和功能多样性与其空间异质性密切相关。在园林环境中，标准差特征量化园林内部不同区域在各景观要素上的变异程度，反映了园林设计的空间层次性和景观的复杂程度。较大的标准差值表明园林内部景观要素分布不均匀，具有丰富的空间变化和明显的功能分区；较小的标准差值则表明园林内部景观相对均质，空间变化较为平缓。

因此，微观特征向量的完整构建形式为：

$$F_{\text{micro}} = [\mu_1, \mu_2, \dots, \mu_{74}, \sigma_1, \sigma_2, \dots, \sigma_{74}]^T \quad (35)$$

这 148 维特征通过均值、方差对的形式，全面描述了园林在 74 个基础景观维度上的集中趋势和离散程度，为园林的底层特征表示提供了丰富而稳健的信息基础。在实际应用中，这些特征能够有效捕获不同园林在景观构成、空间布局 and 要素配置方面的细微差异，为后续的相似性分析和分类识别提供了精细的特征区分能力。

### 6.2.2 中观尺度特征的空间关系建模

中观尺度特征的构建基于空间关系分析理论和点云分布统计方法，旨在捕获园林空间中的邻域关系、方向性模式和高维分布特征。这一尺度的特征设计体现了地理信息科学中的空间依赖性理论，即地理现象在空间上往往表现出显著的相关性和结构性模式。中观特征分为基础空间关系特征和点云分布特征两个主要类别，共同构成了对园林空间组织模式的中层抽象描述。

基础空间关系特征通过分析相邻栅格间的特征相似性来量化空间连续性和变化模式。相邻栅格特征相似度采用余弦相似度度量，其均值和标准差分别计算为：

$$s_{\text{adj}} = \frac{1}{M} \sum_{i=1}^M \cos(\angle(g_i, g_{\text{adj}(i)})) = \frac{1}{M} \sum_{i=1}^M \frac{g_i \cdot g_{\text{adj}(i)}}{\|g_i\|_2 \|g_{\text{adj}(i)}\|_2} \quad (36)$$

$$\sigma_{s_{\text{adj}}} = \sqrt{\frac{1}{M-1} \sum_{i=1}^M \left( \frac{g_i \cdot g_{\text{adj}(i)}}{\|g_i\|_2 \|g_{\text{adj}(i)}\|_2} - \bar{s}_{\text{adj}} \right)^2} \quad (37)$$

其中  $M$  表示园林内具有相邻关系的栅格对总数， $g_{\text{adj}(i)}$  表示与栅格  $i$  在空间上相邻的栅格特征向量。相邻相似度均值  $\bar{s}_{\text{adj}}$  反映了园林整体的空间连续性水平，较高的数值表明相邻区域在景观特征上具有较强的一致性，体现了园林设计的空间协调性；相邻相似度标准差  $\sigma_{s_{\text{adj}}}$  则量化了这种空间连续性的变异程度，反映了园林中不同区域间过渡的平缓程度或急剧程度。

方向梯度特征通过计算东、西、南、北四个主要方向上的特征变化梯度来描述园林的方向性空间变化模式。对于方向  $\text{dir} \in \{\text{east}, \text{west}, \text{south}, \text{north}\}$ ，梯度特征计算为：

$$\nabla_{\text{dir}} = \frac{1}{N_{\text{dir}}} \sum_{i \in \text{direction}} \|g_{i+\delta_{\text{dir}}} - g_i\|_2 \quad (38)$$

其  $N_{\text{dir}}$  为指定方向上相邻栅格对的数量， $\delta_{\text{dir}}$  为方向偏移量。这四个方向梯度特征能够识别园林中是否存在明显的方向性设计模式。点云分布特征部分基于高维统计分析方法，将园林内所有栅格的 74 维特征向量视为高维特征空间中的点云分布，通过主成分分析、聚类分析和统计矩分析来描述这一分布的几何和统计特性。设特征矩阵  $G = [g_1, g_2, \dots, g_N]^T \in \mathbb{R}^{N \times 74}$ ，经 PCA 分解后得到特征值  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{74}$ ，前三个主成分的分差解释比例为：

$$\text{PCA}_j = \frac{\lambda_j}{\sum_{k=1}^{74} \lambda_k}, \quad j = 1, 2, 3 \quad (39)$$

主成分方差解释比例反映了点云分布在不同方向上的变异程度，第一主成分比例较大表明园林特征主要沿某个特定方向变化，具有明显的主导模式；三个主成分比例相对均

衡则表明园林特征在多个方向上都有显著变化，具有复杂的多维变化特征。

通过 K-means 聚类分析，计算三个聚类的样本比例：

$$\text{Cluster}_j = \frac{|C_j|}{N}, \quad j = 1, 2, 3 \quad (40)$$

其中 $C_j$ 表示第 $j$ 个聚类包含的栅格集合。聚类比例特征反映了园林内部景观要素的分组模式，均衡的聚类比例表明园林具有多样化的景观类型；某个聚类占主导地位则表明园林具有单一主导的景观特征。

分布的统计矩特征包括方差、偏度和峰度，分别描述分布的离散程度、对称性和尖峭程度：

$$\text{Var} = \frac{1}{74} \sum_{k=1}^{74} \text{Var}(G_{:,k}) = \frac{1}{74} \sum_{k=1}^{74} \frac{1}{N-1} \sum_{i=1}^N (g_{i,k} - \mu_k)^2 \quad (41)$$

$$\text{Skew} = \frac{1}{74} \sum_{k=1}^{74} \frac{E[(G_{:,k} - \mu_k)^3]}{\sigma_k^3} = \frac{1}{74} \sum_{k=1}^{74} \frac{\frac{1}{N} \sum_{i=1}^N (g_{i,k} - \mu_k)^3}{\left( \frac{1}{N-1} \sum_{i=1}^N (g_{i,k} - \mu_k)^2 \right)^{3/2}} \quad (42)$$

$$\text{Kurt} = \frac{1}{74} \sum_{k=1}^{74} \frac{E[(G_{:,k} - \mu_k)^4]}{\sigma_k^4} = \frac{1}{74} \sum_{k=1}^{74} \frac{\frac{1}{N} \sum_{i=1}^N (g_{i,k} - \mu_k)^4}{\left( \frac{1}{N-1} \sum_{i=1}^N (g_{i,k} - \mu_k)^2 \right)^2} \quad (43)$$

最后，凸包体积特征通过计算包围所有特征点的最小凸包的体积来量化点云在高维空间中的离散程度：

$$\text{ConvexHull} \setminus \_ \text{LogVolume} = \log(\text{Volume}(\text{ConvexHull}(G))) \quad (44)$$

由于高维凸包体积的数值范围极大，采用对数变换进行数值稳定化处理。较大的凸包体积表明园林内部景观特征具有广泛的变化范围和丰富的多样性；较小的凸包体积则表明园林景观特征相对集中，变化范围有限。

### 6.2.3 宏观尺度特征的景观生态学建模

宏观尺度特征基于景观生态学理论和网络分析方法构建，旨在从整体层面评价园林的美学品质、生态功能和空间组织效率。这一尺度的特征设计体现了系统论的整体性原则，认为园林作为一个复合生态系统，其整体特性不能简单地通过部分特征的线性组合来表达，而需要通过专门的整体性指标来描述。宏观特征分为专家定义的景观评价指标和基于游览网络的拓扑结构特征两大类。

专家定义特征基于景观生态学、环境美学和园林设计理论，选择了十个在学术界和实践中广泛认可的核心评价指标。香农多样性指数是景观多样性评价的经典指标，其计算公式为：

$$H = - \sum_{i=1}^m p_i \log p_i \quad (45)$$

其中 $p_i$ 表示第 $i$ 种景观类型（如水体、植被、建筑、道路等）在园林中的面积比例， $m$ 为识别出的景观类型总数。该指数同时考虑了景观类型的丰富度和均匀度，数值越大表明园林的景观构成越多样化，设计越丰富。在园林设计中，适度的景观多样性被认为能够提供丰富的视觉体验和生态功能，但过度的多样性可能导致空间的混乱和缺乏主题。

聚集度指数 Aggregation Index 量化了同类景观斑块的空间聚合程度，其计算采用基于邻接关系的方法：

$$AI = \left[ \sum_{i=1}^m \left( \frac{g_{ii}}{\max g_{ii}} \right) P_i \right] \times 100 \quad (46)$$

其中 $g_{ii}$ 表示景观类型 $i$ 与同类型相邻的栅格对数量， $\max g_{ii}$ 表示该景观类型在完全聚集情况下的理论最大邻接数， $P_i$ 表示景观类型 $i$ 在整个园林中的面积比例。该指数的取值范围为0到100，数值越大表明同类景观要素越倾向于聚集分布，形成大面积的连续斑块；数值越小则表明各类景观要素呈分散分布模式。在园林设计中，适当的聚集度能够形成功能相对完整的景观区域，但过度聚集可能导致景观单调，缺乏变化。

蔓延度指数 Contagion Index 描述不同景观要素在空间上的蔓延程度和连接性，反映景观的空间配置模式：

$$CONT = 1 + \frac{\sum_{i=1}^m \sum_{j=1}^m p_i \cdot p_{ij} \cdot \ln(p_i \cdot p_{ij})}{2 \ln(m)} \times 100 \quad (47)$$

其中 $p_{ij}$ 表示从景观类型 $i$ 转换到景观类型 $j$ 的条件概率。蔓延度高的园林表现出大面积的单一景观类型和相对简单的空间配置，蔓延度低的园林则具有复杂的景观镶嵌模式和频繁的类型转换。

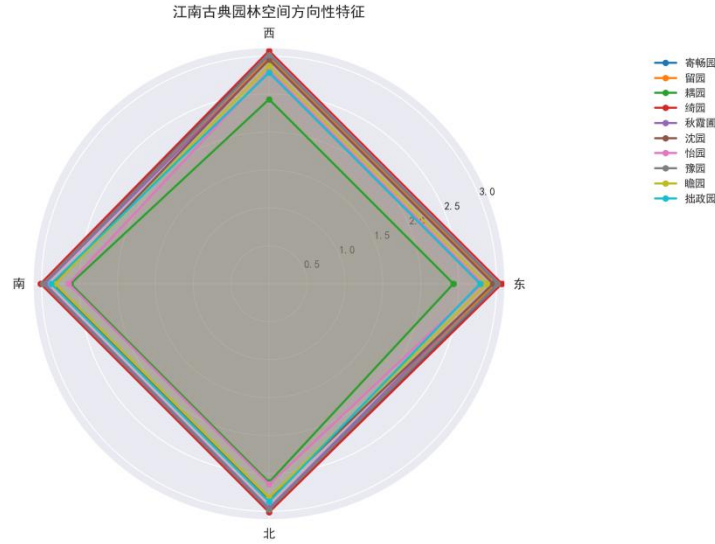


图 88 江南古典园林空间方向性特征

边缘密度 Edge Density 量化景观斑块边界的复杂程度，反映园林的破碎化水平：

$$ED = \frac{E}{A} \times 10000 \quad (48)$$

其中 $E$ 为园林内所有景观斑块的边界总长度， $A$ 为园林总面积。边缘密度高表明园林内部存在大量的景观边界，空间划分较为细碎；边缘密度低则表明园林具有相对完整的大型景观斑块。在园林设计中，适度的边缘复杂度能够创造丰富的边缘效应和过渡空间，但过度的破碎化可能影响景观的完整性和使用功能。

莫兰指数 Moran's  $I$  作为空间自相关分析的经典指标，用于量化园林景观特征的空间聚集程度：

$$I = \frac{n}{\sum_{i=1}^n \sum_{j=1}^n w_{ij}} \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (49)$$

其中 $n$ 为空间单元总数， $w_{ij}$ 为空间权重矩阵元素，通常基于空间邻接关系确定， $x_i$ 为空间单元 $i$ 的属性值， $\bar{x}$ 为所有空间单元属性值的均值。莫兰指数的取值范围为-1 到 1，正值表示正空间自相关，相似值在空间上聚集，负值表示负空间自相关，相异值在空间上邻接，零值表示随机分布。

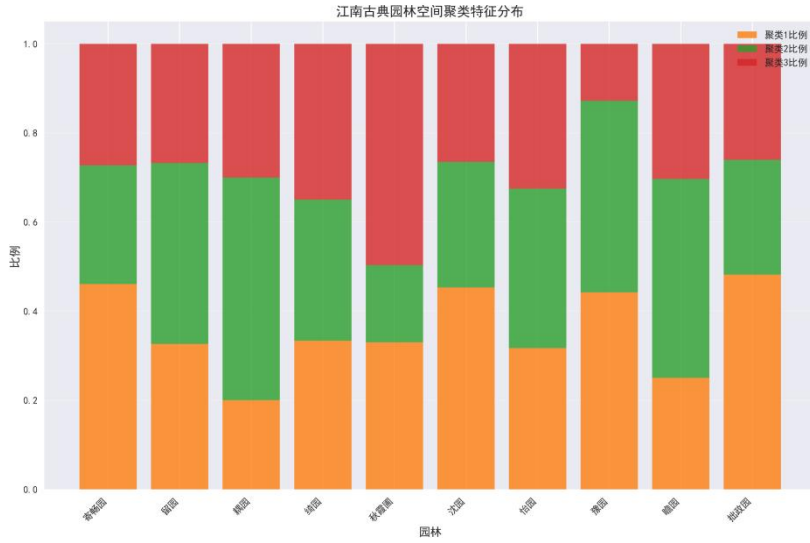


图 89 江南古典园林空间聚类特征分布

平均最近邻距离 Mean Nearest Neighbor Distance 反映景观要素的空间分布模式，通过计算每个景观斑块到其最近同类斑块的距离来评估空间聚集程度：

$$MNN = \frac{1}{N} \sum_{i=1}^N \min_{j \neq i} d_{ij} \quad (50)$$

其中  $N$  为景观斑块总数， $d_{ij}$  为斑块  $i$  和斑块  $j$  之间的空间距离

除了上述传统的景观生态学指标外，本研究还引入了四个专门针对园林设计特点的评价指标。围合比例 Enclosure Ratio 量化园林空间的围合封闭程度，反映传统园林设计中重要的空间围合理念；连通性指数 Connectivity Index 评估园林内部各功能区域之间的可达性和连接便利程度；紧凑度指数 Compactness Index 衡量园林布局的紧凑集中程度，反映空间利用的效率；整体连通性 Integral Connectivity 从网络理论角度评估园林作为整体系统的连通性水平。

网络结构特征基于园林的游览路径网络构建，将园林中的重要景点、功能节点抽象为网络节点，将连接这些节点的路径抽象为网络边，通过分析网络的拓扑特性来评价园林的空间组织效率和游览体验质量。设网络图  $G = (V, E)$ ，其中  $V = \{v_1, v_2, \dots, v_{|V|}\}$  为节点集合， $E = \{e_1, e_2, \dots, e_{|E|}\}$  为边集合，则基本的网络拓扑特征包括：

网络节点数  $|V|$  反映园林中重要景点和功能区域的数量，较多的节点数通常表明园林内容丰富，景点密集；网络边数  $|E|$  反映园林内部路径连接的数量，较多的边数表明园林具有丰富的路径选择和较好的可达性。

网络平均度是网络连通性的基本度量：

$$\bar{d} = \frac{2|E|}{|V|} \quad (51)$$

该指标反映每个节点平均连接的其他节点数量，数值较大表明网络连通性较好，游客在园林中具有较多的路径选择；数值较小则表明网络相对稀疏，路径选择有限。

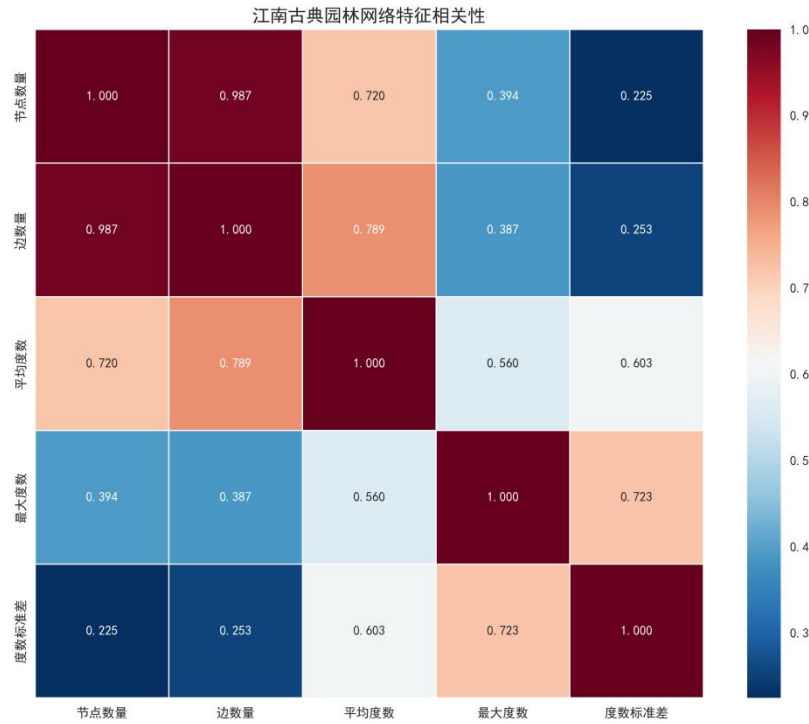


图 90 江南古典园林网络特征相关性

网络最大度反映连接度最高节点的重要性：

$$d_{\max} = \max_{i=1, \dots, |V|} d_i \quad (52)$$

其中 $d_i$ 表示节点 $i$ 的度数。最大度较大表明园林中存在重要的核心节点，通常对应于园林的主要景点或交通枢纽。

度分布标准差量化节点连接度的变异性：

$$\sigma_d = \sqrt{\frac{1}{|V|-1} \sum_{i=1}^{|V|} (d_i - \bar{d})^2} \quad (53)$$

较大的度分布标准差表明网络中节点的重要性分化明显，存在少数高连接度的核心节点和大量低连接度的边缘节点；较小的标准差则表明各节点的连接度相对均衡。

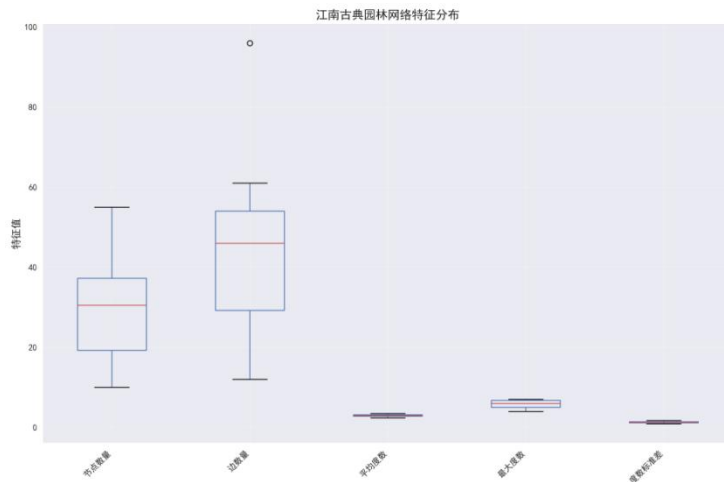


图 91 江南古典园林网络特征分布

#### 6.2.4 多模态特征重组的语义解释框架

基于特征的语义含义和园林分析的实际需求，本研究实现了多模态特征的重新组织和

解释。这种重组策略超越了原始的尺度划分方式，按照特征的语义属性重新构建了三个功能性模态，使得特征向量能够更好地支持不同类型的园林分析任务。

几何空间模态 **Geometric-Spatial Modality** 包含 53 维特征，涵盖了与园林几何形状、空间关系和拓扑结构相关的所有特征维度。这一模态主要包括前 20 个栅格特征的均值和标准差、中观尺度的空间关系特征以及完整的网络拓扑结构特征。几何空间模态的核心功能在于描述园林的物理空间组织模式，支持基于空间相似性的园林比较分析、空间布局模式识别和设计几何特征提取等应用。

景观语义模态 **Landscape-Semantic Modality** 包含 116 维特征，专注于描述园林的景观内容、美学特征和功能评价。该模态包括第 21-74 个栅格特征的均值和标准差、完整的专家美学评价指标以及描述分布复杂性的 PCA 第二、第三主成分。景观语义模态的设计理念基于园林学中的"意境营造"理论，认为园林的核心价值在于通过景观要素的巧妙组合来创造特定的美学效果和文化意境，因此该模态特别适用于园林风格识别、美学品质评价和文化特征分析等应用。

尺度层次模态 **Scale-Hierarchical Modality** 包含 10 维特征，从每个尺度层次中选择最具代表性的统计特征，形成一个紧凑而全面的特征子集。该模态的设计遵循"多尺度集成"的理念，认为园林的复杂性需要通过不同尺度的协同分析才能全面把握。具体包括微观尺度的代表性均值和标准差特征、中观尺度的相邻相似度均值、分布偏度和凸包体积，以及宏观尺度的第一个和最后一个专家特征、网络平均度。

#### 6.2.5 特征向量的数值处理与质量控制

为确保特征向量的数值稳定性和分析可靠性，本研究建立了完整的数据预处理和质量控制体系。首先，所有特征在计算完成后都进行标准化处理，采用 Z-score 标准化方法：

$$\tilde{f}_k = \frac{f_k - \mu_{f_k}}{\sigma_{f_k}} \quad (54)$$

其中  $f_k$  为第  $k$  维原始特征值， $\mu_{f_k}$  和  $\sigma_{f_k}$  分别为该维特征在所有样本上的均值和标准差。标准化处理消除了不同特征维度间量纲和数值范围的差异，确保各维特征在后续分析中具有相等的权重。

对于可能出现的异常值和缺失值，建立了基于统计阈值的检测和处理机制。异常值检测采用基于四分位距的方法：

$$\text{Outlier Threshold} = Q_3 + 1.5 \times (Q_3 - Q_1) \quad (55)$$

其中  $Q_1$  和  $Q_3$  分别为第一四分位数和第三四分位数。超出阈值的数值被标记为异常值，根据具体情况采用截尾处理或插值处理。缺失值处理采用基于特征相关性的多重插补方法，利用其他相关特征的信息来估算缺失值。

### 6.3 基于 179 维特征向量的江南古典园林相似度分析与共有美学特征识别

#### 6.3.1 多层次相似度计算体系

江南古典园林的相似性分析建立在 179 维特征向量的层次化架构基础之上，该向量结构包含微观特征第 1-148 维栅格聚合特征：74 维均值+74 维标准差、中观特征第 149-164 维空间关系 16 维和宏观特征第 165-179 维：专家特征 10 维+网络特征 5 维。相似度计算采用余弦相似度方法，通过测量特征向量间的夹角余弦值来量化园林间的相似程度。

专家特征相似度基于第 165-174 维的 10 个专家评价指标计算，其数学表达式为：

$$S_{\text{expert}}(i, j) = \frac{\sum_{k=165}^{174} v_{i,k} \cdot v_{j,k}}{\sqrt{\sum_{k=165}^{174} v_{i,k}^2} \cdot \sqrt{\sum_{k=165}^{174} v_{j,k}^2}} \quad (56)$$

式中， $v_{i,k}$ 表示园林*i*在第*k*维特征上的数值， $v_{j,k}$ 表示园林*j*在第*k*维特征上的数值，分子 $\sum_{k=165}^{174} v_{i,k} \cdot v_{j,k}$ 为两个园林专家特征向量的内积，分母为两个特征向量的欧几里德范数乘积，用于归一化处理。该相似度值域为 $[-1, 1]$ ，其中 1 表示完全相似，-1 表示完全相反，0 表示不相关。

网络特征相似度通过第 175-179 维的 5 个网络拓扑参数计算：

$$S_{\text{network}}(i, j) = \frac{\sum_{k=175}^{179} v_{i,k} \cdot v_{j,k}}{\sqrt{\sum_{k=175}^{179} v_{i,k}^2} \cdot \sqrt{\sum_{k=175}^{179} v_{j,k}^2}} \quad (57)$$

式中各变量含义与专家特征相似度公式相同，但计算范围限定在网络特征维度。该相似度度量园林在游览体验和空间组织方面的相似程度。

空间关系相似度基于第 149-164 维的 16 个空间几何特征计算：

$$S_{\text{spatial}}(i, j) = \frac{\sum_{k=149}^{164} v_{i,k} \cdot v_{j,k}}{\sqrt{\sum_{k=149}^{164} v_{i,k}^2} \cdot \sqrt{\sum_{k=149}^{164} v_{j,k}^2}} \quad (58)$$

抽象特征整体相似度涵盖第 1-148 维的栅格聚合统计特征：

$$S_{\text{abstract}}(i, j) = \frac{\sum_{k=1}^{148} v_{i,k} \cdot v_{j,k}}{\sqrt{\sum_{k=1}^{148} v_{i,k}^2} \cdot \sqrt{\sum_{k=1}^{148} v_{j,k}^2}} \quad (59)$$

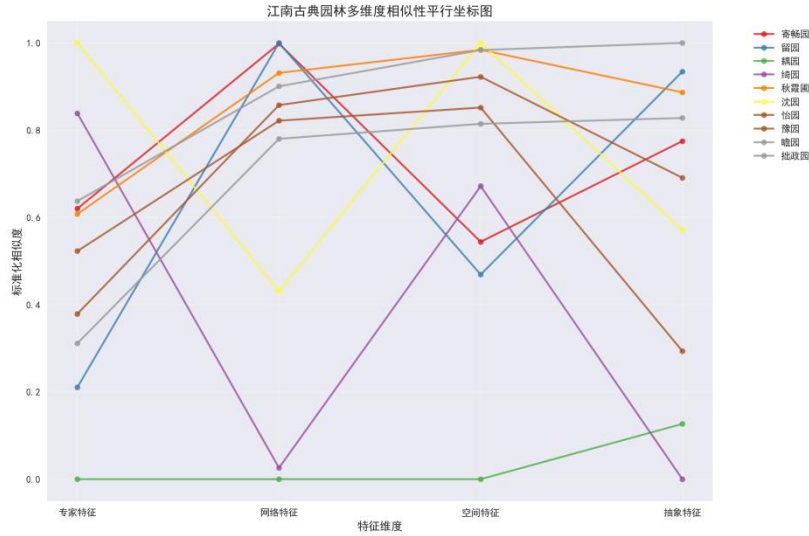


图 92 江南古典园林多维度相似性平行作坐标图

### 6.3.2 综合相似度融合模型

多层次相似度的综合融合采用加权平均方式：

$$S_{\text{comprehensive}} = 0.4 \times S_{\text{expert}} + 0.2 \times S_{\text{network}} + 0.2 \times S_{\text{spatial}} + 0.2 \times S_{\text{abstract}} \quad (60)$$

式中，权重系数 0.4、0.2、0.2、0.2 分别对应专家特征、网络特征、空间特征和抽象特征的重要性权重，权重和为 1，确保综合相似度值域仍为 $[-1, 1]$ 。该权重设计体现了专家美学评价在园林相似性判断中的主导地位。

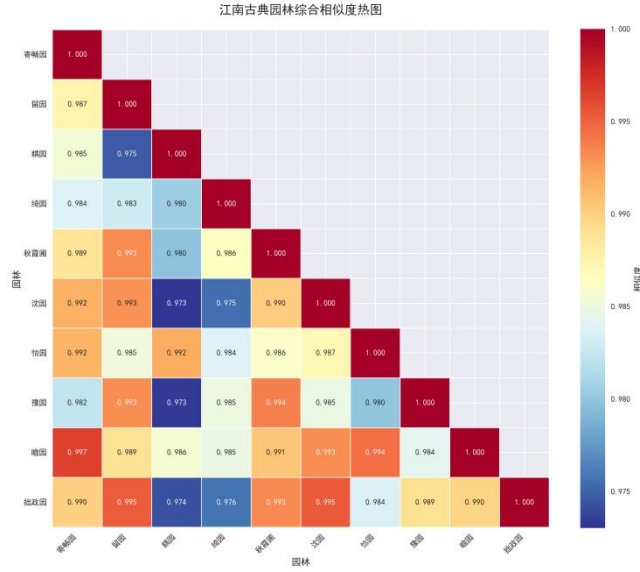


图 93 综合相似度热图

分析显示，10 座江南古典园林之间的综合相似度达到 0.986，这一数值接近完全相同，表明江南园林作为文化景观类型具有极强的统一性。更为重要的是，所有 45 个园林对的相似度均超过 0.85 的高度相似阈值，而中等相似和差异显著的园林对数量均为 0，这种一边倒的分布模式充分证明了江南园林美学体系的高度成熟和标准化程度。

从多维度相似度分析来看，空间关系相似度高达 0.999，几乎达到完全一致，表明江南园林在空间组织的几何关系、布局逻辑、尺度控制等方面已形成极其精确的设计准则。专家特征相似度和网络特征相似度同样处于极高水平，前者反映了传统造园理论在美学评价标准上的高度统一，后者则表明游览体验设计的路径组织、节点设置、连通性控制等方面的一致性。即使是反映微观景观内容的抽象特征相似度也达到 0.963，说明江南园林在栅格级的景观构成、材质运用、细部处理等方面同样遵循共同的美学法则。

### 6.3.3 共有美学特征的稳定性分析

江南园林共有美学特征的识别通过变异系数的计算来量化不同特征维度在园林群体中的变异程度。对于第  $k$  维特征，变异系数定义为：

$$CV_k = \frac{\sigma_k}{|\mu_k|} \quad (61)$$

式中， $\sigma_k$  为第  $k$  维特征在所有园林中的标准差，计算公式为：

$$\sigma_k = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (v_{i,k} - \mu_k)^2} \quad (62)$$

$\mu_k$  为第  $k$  维特征的均值：

$$\mu_k = \frac{1}{n} \sum_{i=1}^n v_{i,k} \quad (63)$$

其中  $n$  为园林总数， $v_{i,k}$  为园林  $i$  在第  $k$  维特征上的数值。变异系数  $CV_k$  反映特征  $k$  的相对变异程度，数值越小表明该特征在园林群体中越稳定。根据代码中的阈值设定，特征稳定性分为三个等级：强共性特征 ( $CV_k < 0.15$ )、中等共性特征 ( $0.15 \leq CV_k < 0.30$ ) 和差异性特征 ( $CV_k \geq 0.30$ )。

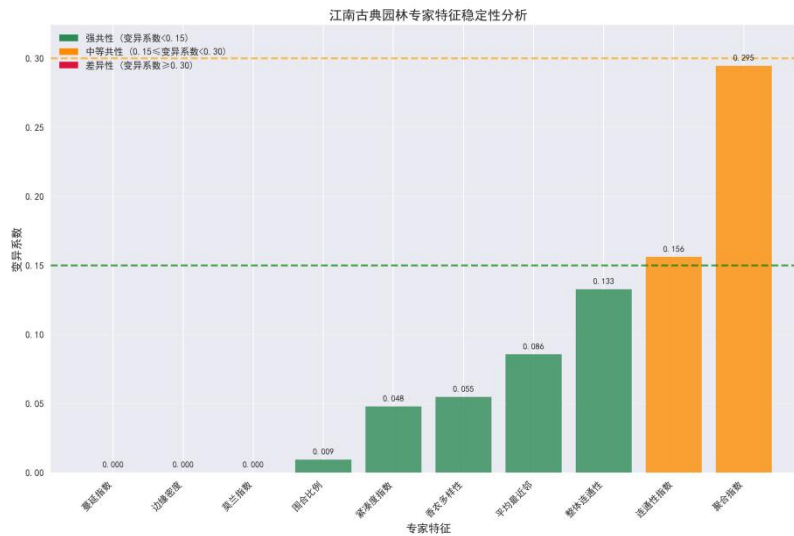


图 94 专家特征稳定性分析

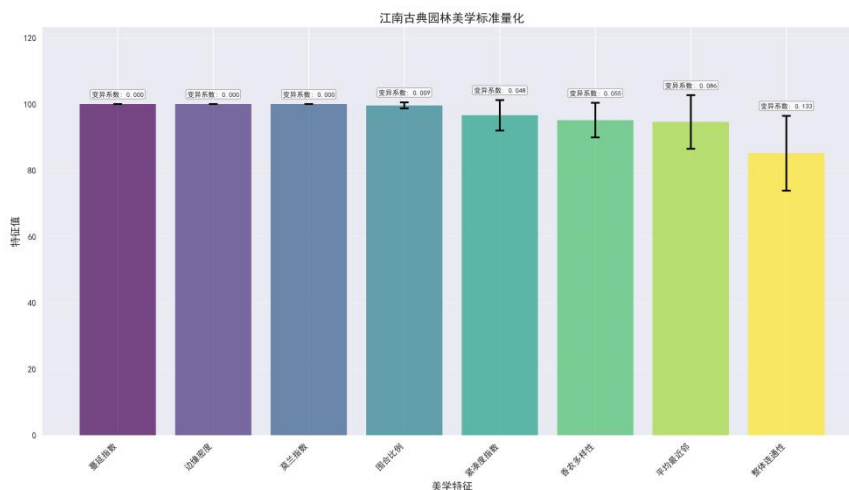


图 95 美学标准量化

特征稳定性分析揭示了江南园林美学体系的量化标准。88 个强共性特征约占总特征的 49.2%，这些特征构成了江南园林设计的核心规范体系。特征变异系数均值为 0.175 左右，处于中等共性范围，表明整体而言江南园林在保持统一性的同时仍具有一定的设计灵活性。

专家特征稳定性排序提供了美学法则的精确量化。围合度（enclosure\_ratio）的变异系数仅为 0.009，标准值为  $99.602 \pm 0.928$ ，显示江南园林在空间围合上具有极其精确的控制标准。紧凑度指数的变异系数为 0.048，标准值为  $96.610 \pm 4.616$ ，反映了布局紧凑性的严格把控。多样性指数的变异系数为 0.055，标准值为  $95.168 \pm 5.205$ ，体现了景观要素多样性的平衡控制。

这些量化标准揭示了江南园林设计的核心法则：连续性指数、边缘密度、空间自相关性的绝对统一表明江南园林在景观基本结构上遵循固定模式；围合度的极小变异反映了"庭院深深"空间理念的精确实现；紧凑度的稳定控制体现了"咫尺山林"空间效率的统一追求；多样性指数的相对稳定则表明了"移步换景"效果营造的标准化程度。

寄畅园与瞻园的相似度高达 0.997，几乎完全一致，两园分别代表了明代和清代的园林营造水平，其极高相似度证明了江南园林传统的历时稳定性。沈园与拙政园（0.995）、留园与拙政园（0.995）的高度相似则表明了不同地域、不同规模园林在美学追求上的空间一致性。这三对最相似园林涵盖了不同历史时期、不同地理位置、不同功能定位的代表性

园林，其高度相似性充分验证了江南园林作为文化类型的内在稳定性和传承连续性。

6.3.4 "同"的本质内涵与文化意义

江南古典园林的"同"本质上体现为一套完整而成熟的美学规范体系，这一体系经历了千年的实践淬炼和理论升华，形成了具有内在逻辑性和系统性的设计法则。这种"同"不是简单的模仿复制，而是对中国传统美学精神的深度理解和精确表达。

从空间哲学层面来看，江南园林的共同性体现了中国传统文化对"天人合一"理念的空间诠释。99.602±0.928 的围合度标准反映了对"内敛含蓄"文化性格的空间表达；96.610±4.616 的紧凑度控制体现了"精工细作"美学追求的空间实现；95.168±5.205 的多样性指数则表明了对"变化统一"审美原则的精确把握。这些量化标准不仅是技术规范，更是文化精神的物化表达。

从文化传承意义来看，这种高度的共同性确保了江南园林作为文化符号的识别性和传承性。0.986 的综合相似度使得任何一座江南园林都能够承载和传达完整的文化信息，保证了文化传统在时空传播中的准确性和完整性。这种共同性是文化自信的重要基础，为中华园林文化在世界文明对话中提供了清晰的身份标识。

从美学理论价值来看，江南园林的共同性规律为传统美学的现代阐释提供了量化依据。连续性指数、边缘密度、空间自相关性的绝对统一，实际上是对传统"气韵生动"、"虚实相生"、"情景交融"等美学范畴的数学表达，为传统美学理论的当代理解和传承创新提供了科学基础。

6.4 模型效能泛化验证-以近园为例

首先，基于园林空间边界与关键节点生成 Voronoi 图路径，以确保路径分布与空间结构高度契合，并为后续路径网络提供基础拓扑。

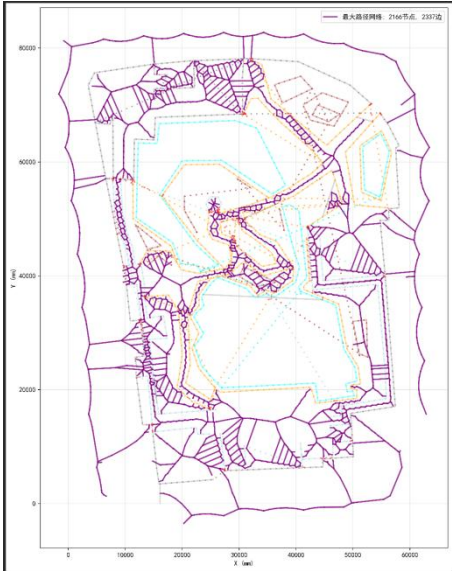


图 96 Voronoi 路径分布

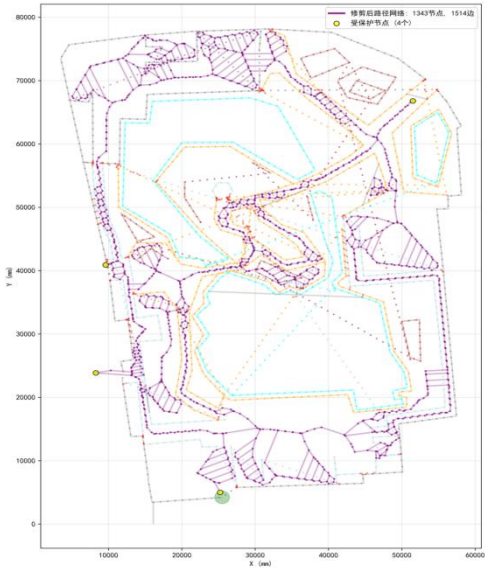


图 97 修剪后路径网络

随后，对初始 Voronoi 进行过滤，连通分量筛选，分枝修剪的精简操作从而得到更贴合实际游赏的路径网络。同时，明确园林中的障碍物的空间分布，利用局部举行扫描法进行网格化扫描，并以栅格或多边形形式在模型中标记，最终得到栅格化景观类型分布图为后续路径规划与视线分析提供精确的环境约束。

通过 4 邻接算法，将栅格图中分散但连续的同类型景观单元聚合为一个独立的连通区域，又称斑块，并为每个斑块赋予唯一的标识信息，确保不同斑块的区分与后续调用。



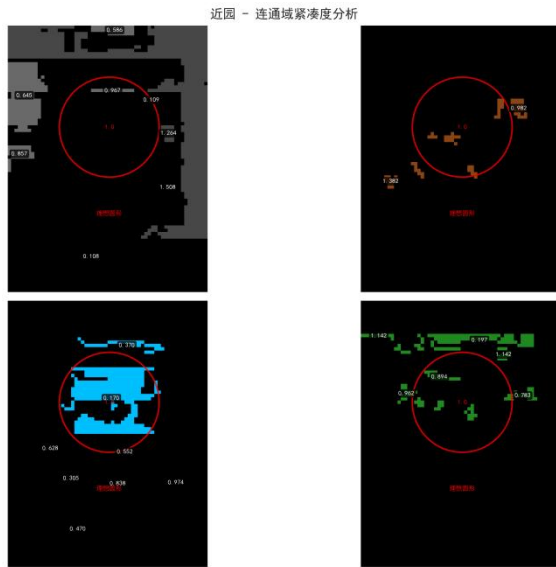


图 103 连通域紧凑度分析

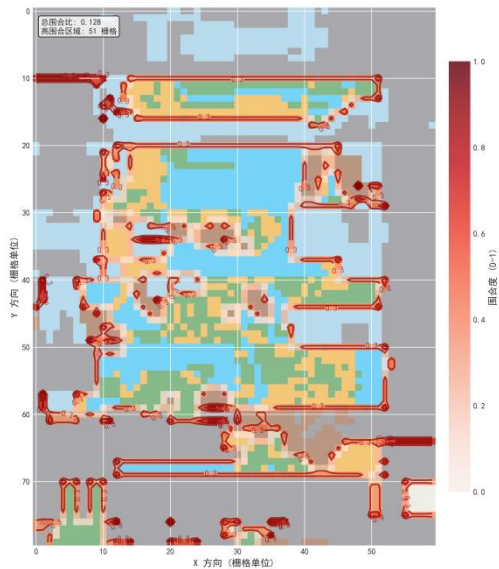


图 104 围合度

利用空间临近关系和连通性路径分析得到连通性特性

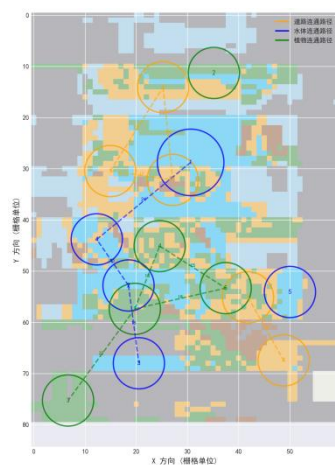


图 105 简化路径

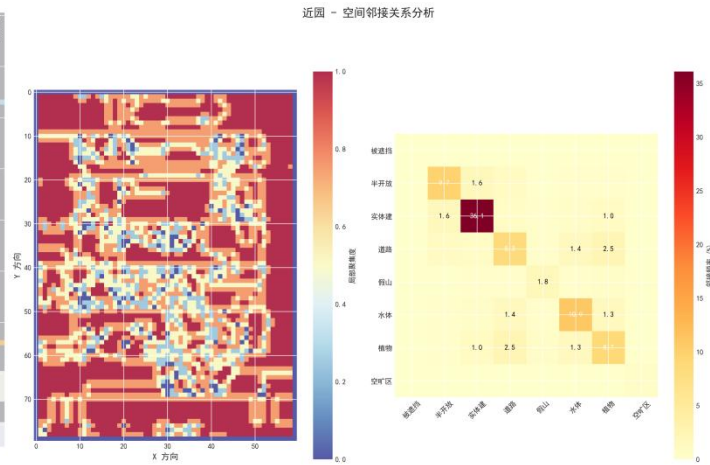


图 106 空间邻接关系分析

根据问题二中的开阔围合标准以及幻境评分体系得到近园综合得分：

元素分布维度:99.20/100

开合变化维度:93.08/100

幻境感总分:96.14/100

与已知的园林图进行相似度分析，得到与十个代表园林之间的相似度关系图

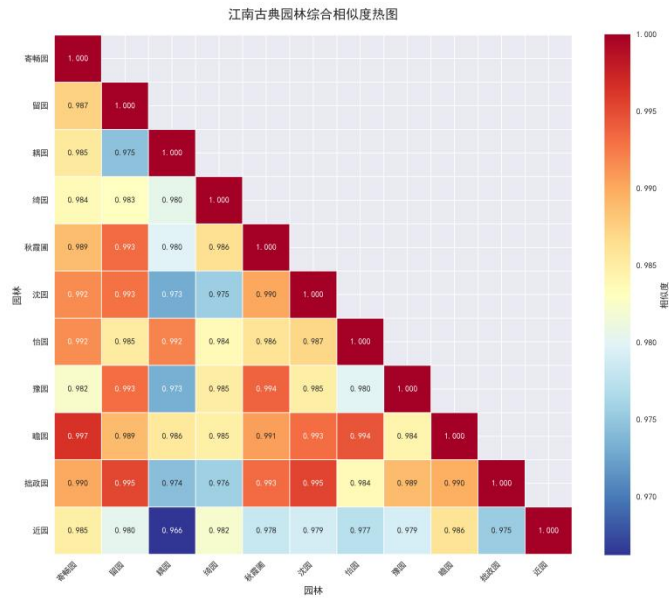


图 107 综合相似度热力图

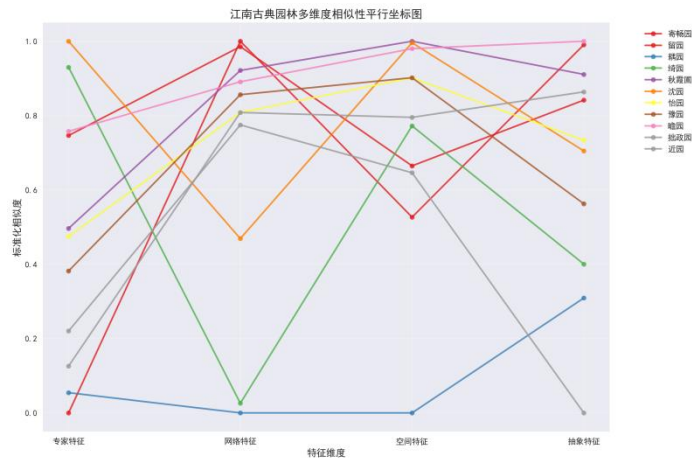


图 108 多维度相似性平行坐标图

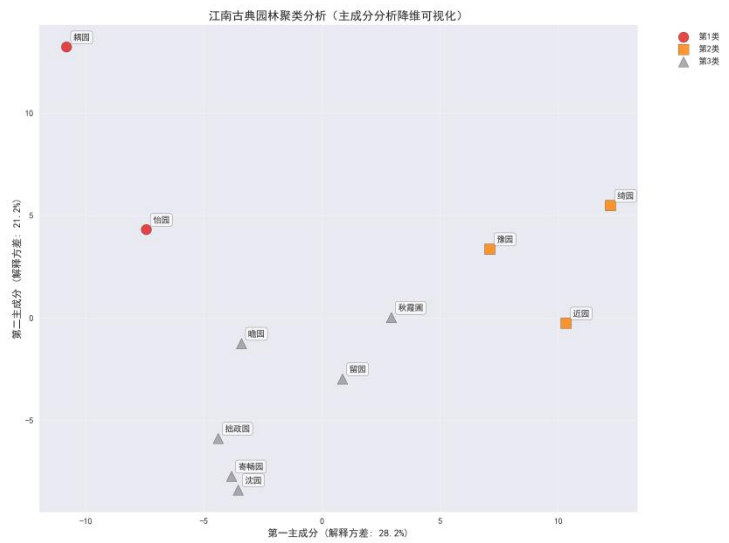


图 109 江南古典园林聚类分析

本次研究选取 11 个江南园林作为对象，构建涵盖微观（148 维栅格聚合统计）、中

观（16 维空间关系特征）、宏观（15 维专家评价与网络结构特征）的 179 维特征向量，开展美学特征量化分析。从园林相似性来看，11 个园林间高度相似的组合达 55 对，无中等相似或差异显著的园林对；综合相似度均值达 0.985，其中空间关系相似度最高，专家特征、网络特征相似度也达 0.990，抽象特征相似度为 0.957，像寄畅园与瞻园、沈园与拙政园（0.995）等成为最相似的园林对。在共有美学特征识别上，80 个特征表现出强共性，74 个为中等共性，仅 25 个为差异性特征，特征变异系数均值为 0.187；从专家特征稳定性排序看，contagion\_index、edge\_density、moran\_i 等特征的 CV 值为 0.000，属于强共性特征，compactness\_index、shannon\_diversity 等也表现出强共性。

由此量化出江南园林美学标准，如 contagion\_index 稳定在  $100.000 \pm 0.000$  等。而在个性化差异方面，耦园以 1.157 的个性化指数位居首位，近园、绮园等紧随其后，瞻园个性化指数最低。

## 6.5 基于文化意象密度、生态和谐性、个体差异度的多元化评估模型

### 6.5.1 个性化差异量化与园林特色识别

#### 个性化指数计算模型

园林个性化程度通过标准化偏离度分析进行量化，个性化指数反映每个园林在 179 维特征空间中相对于群体均值的偏离程度。园林  $i$  的个性化指数计算公式为：

$$PI_i = \frac{1}{179} \sum_{k=1}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k} \quad (64)$$

式中， $v_{i,k}$  为园林  $i$  第  $k$  维特征值， $\mu_k$  为第  $k$  维特征的群体均值， $\sigma_k$  为第  $k$  维特征的群体标准差， $|v_{i,k} - \mu_k|$  为园林  $i$  第  $k$  维特征上相对群体均值的绝对偏离量， $\frac{|v_{i,k} - \mu_k|}{\sigma_k}$  为标准化偏离度，179 为特征总维数。该公式通过标准化处理消除了不同特征间量纲差异的影响，个性化指数越高表明园林越具有独特性。

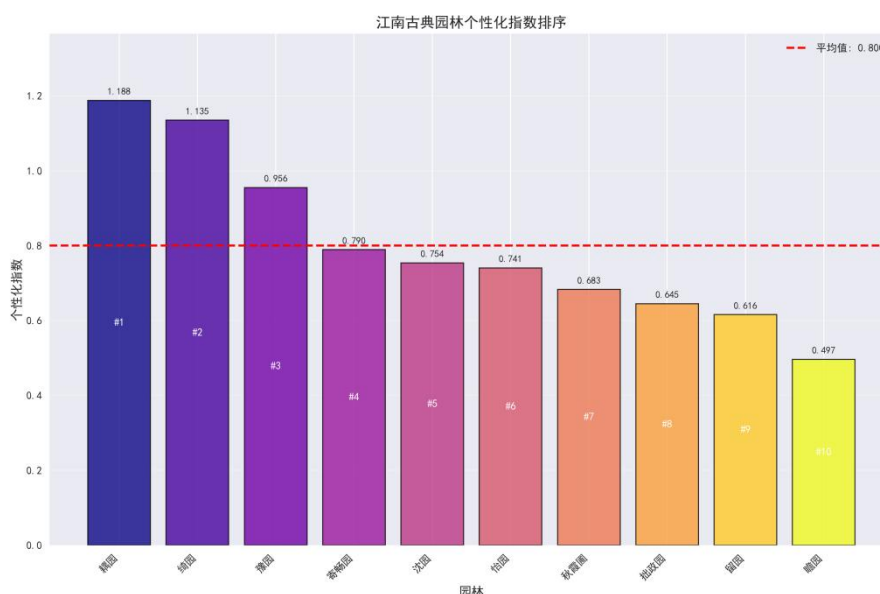


图 110 江南古典园林个性化指数排序

个性化指数从耦园的 1.188 到瞻园的 0.497，变化范围达到 0.691，这一数值差异表明江南园林在遵循共同美学法则的基础上仍保持着显著的个性化特征。个性化指数的十园排序呈现出明显的层次分布：耦园（1.188）和绮园（1.135）构成高个性化层次，表现出强烈的独特性追求；豫园（0.956）处于个性化的中高水平，体现了商业园林的特色表达需求；

寄畅园（0.790）、沈园（0.754）、怡园（0.741）形成中等个性化层次，在传统与创新之间寻求平衡；秋霞圃（0.683）、拙政园（0.645）、留园（0.616）、瞻园（0.497）构成低个性化层次，更多体现了江南园林的典型特征。

这种层次化分布反映了江南园林创作中的差异化策略。高个性化园林往往承载着特殊的功能需求或审美追求，中等个性化园林在传承与创新之间寻求动态平衡，而低个性化园林则更多承担着传统美学标准的典型化表达功能。

#### 维度化个性分析

个性化差异的深入分析通过维度层次分解不同特征类型对园林个性的贡献程度。园林  $i$  在专家特征维度的个性化贡献计算为：

$$C_{\text{expert}}(i) = \frac{\sum_{k=165}^{174} \frac{|v_{i,k} - \mu_k|}{\sigma_k}}{\sum_{k=1}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k}} \times 100\% \quad (65)$$

式中，分子为专家特征维度(第 165-174 维)的标准化偏离度之和，分母为全部 179 维特征的标准化偏离度之和，该比值乘以 100% 得到专家特征的个性化贡献百分比。类似地，网络特征贡献基于第 175-179 维：

$$C_{\text{network}}(i) = \frac{\sum_{k=175}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k}}{\sum_{k=1}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k}} \times 100\% \quad (66)$$

空间特征贡献基于第 149-164 维：

$$C_{\text{spatial}}(i) = \frac{\sum_{k=149}^{164} \frac{|v_{i,k} - \mu_k|}{\sigma_k}}{\sum_{k=1}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k}} \times 100\% \quad (67)$$

抽象特征贡献基于第 1-148 维：

$$C_{\text{abstract}}(i) = \frac{\sum_{k=1}^{148} \frac{|v_{i,k} - \mu_k|}{\sigma_k}}{\sum_{k=1}^{179} \frac{|v_{i,k} - \mu_k|}{\sigma_k}} \times 100\% \quad (68)$$

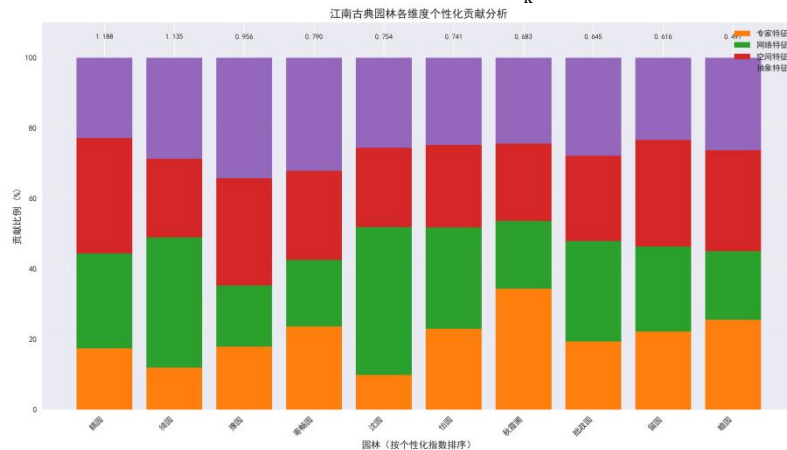


图 111 园林按个性化指数排序

作为最具个性的园林，耦园的维度化个性分析具有重要的启示意义。其个性化贡献成为：空间特征 32.8%、网络特征 27.0%、抽象特征 22.8%、专家特征 17.4%。这一构成模式表明，耦园的个性化主要通过空间关系的创新处理和游览路径的独特组织来实现，而在传统美学评价标准上保持相对保守。

空间特征的最高贡献反映了耦园在几何关系、布局逻辑、尺度控制等方面的创新探索；

网络特征的重要贡献表明其在游览体验设计上的独特构思；抽象特征的适度贡献显示了在微观景观营造上的个性化处理；专家特征的最低贡献则说明其仍然严格遵循传统美学法则。这种个性化构成模式体现了江南园林"守正创新"的设计智慧，即在坚持核心美学原则的前提下，通过空间组织和体验设计的创新来实现个性化表达。

### 6.5.2 极值特征识别算法

极值特征识别算法用于精确定位每个园林的个性化特征。首先计算园林  $i$  在所有特征维度上的标准化偏离度：

$$z_{i,k} = \frac{|v_{i,k} - \mu_k|}{\sigma_k} \quad (69)$$

式中， $z_{i,k}$  为园林  $i$  在第  $k$  维特征上的标准化偏离度。然后对所有维度的偏离度进行降序排列：

$$\{z_{i,k_1}, z_{i,k_2}, \dots, z_{i,k_{179}}\} \text{ 其中 } z_{i,k_1} \geq z_{i,k_2} \geq \dots \geq z_{i,k_{179}} \quad (70)$$

选取偏离度最大的前  $m$  个特征  $m$  作为该园林的极值特征集合：

$$\mathcal{E}_i = \{k_1, k_2, k_3, k_4, k_5\} \quad (71)$$

耦园最突出的三个极值特征均为栅格特征，分别是栅格特征 67 标准差，偏离度 2.73、栅格特征 12 标准差，偏离度 2.71、栅格特征 29 均值，偏离度 2.65。这些极值特征主要集中在微观尺度的景观构成上，表明耦园的个性化更多体现在景观细部的精工细作和材质运用的独特追求上。标准差类特征的突出表明其在景观要素的变异性控制上有独特的技法



图 112 江南古典园林极值特征分析

### 6.5.3 基于 179 维全特征向量的聚类分析模型

基于 179 维全特征向量的 K-means 聚类分析采用以下目标函数：

$$J = \sum_{i=1}^n \sum_{j=1}^K w_{ij} \|v_i - \mu_j\|^2 \quad (72)$$

式中， $n$  为园林总数， $K$  为聚类数目， $w_{ij}$  为二值指示变量（当园林  $i$  属于聚类  $j$  时  $w_{ij} = 1$ ，否则为 0）， $v_i$  为园林  $i$  的 179 维特征向量， $\mu_j$  为第  $j$  个聚类的中心向量， $\|\cdot\|^2$  为欧几里德距离的平方。算法通过迭代优化最小化类内平方和。

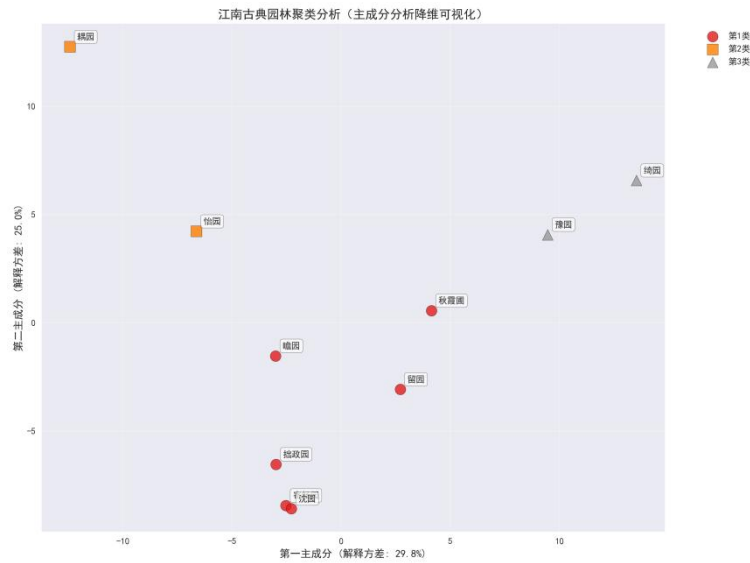


图 113 聚类分析结果

三类聚类结果揭示了江南园林个性化差异的有序分布。第一类（6 园）代表经典传统型，个性化指数均值  $0.664 \pm 0.096$ ，标准差较小表明该类园林在个性化程度上相对一致，都体现了江南园林的典型特征。第二类（2 园）代表空间创新型，个性化指数均值  $0.964 \pm 0.224$ ，标准差较大反映了该类内部的差异性，表明空间创新的多样化路径。第三类（2 园）代表特色表达型，个性化指数均值  $1.045 \pm 0.090$ ，标准差相对较小说明该类在高个性化水平上具有相对一致性。

层次聚类采用 Ward 连接法，其合并准则基于类内平方和增量：

$$\Delta(C_p, C_q) = \frac{|C_p| \cdot |C_q|}{|C_p| + |C_q|} \|\mu_p - \mu_q\|^2 \quad (73)$$

式中， $C_p$ 和 $C_q$ 表示两个待合并的聚类， $|C_p|$ 和 $|C_q|$ 为聚类的样本数量， $\mu_p$ 和 $\mu_q$ 为对应的聚类中心向量。

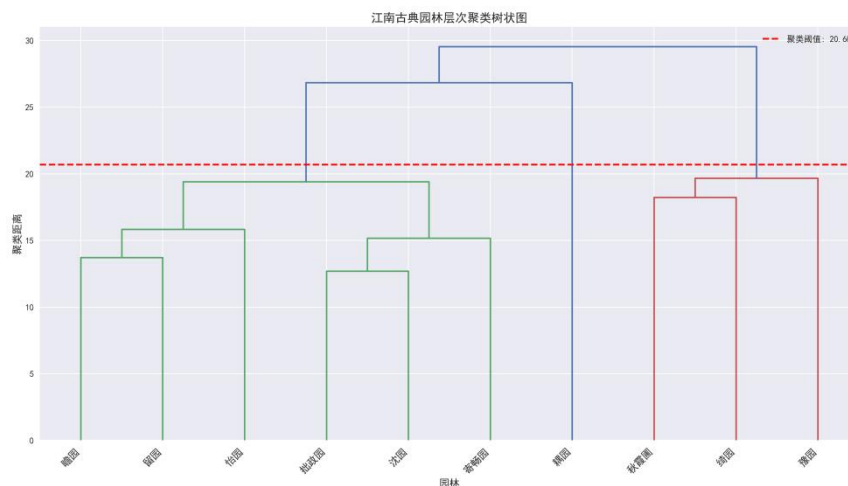


图 114 层次聚类树状图

#### 6.5.4 聚类显著性检验

聚类结果的显著性验证采用单因素方差分析，对于特征 $k$ ，F 统计量定义为：

$$F_k = \frac{SS_{\text{between}}/(K-1)}{SS_{\text{within}}/(n-K)} \quad (74)$$

式中， $K$ 为聚类数目， $n$ 为样本总数， $SS_{\text{between}}$ 为组间平方和：

$$SS_{\text{between}} = \sum_{j=1}^K n_j (\bar{v}_{j,k} - \bar{v}_k)^2 \quad (75)$$

$SS_{\text{within}}$ 为组内平方和：

$$SS_{\text{within}} = \sum_{j=1}^K \sum_{i \in C_j} (v_{i,k} - \bar{v}_{j,k})^2 \quad (76)$$

式中， $n_j$ 为第 $j$ 个聚类的样本数量， $\bar{v}_{j,k}$ 为第 $j$ 个聚类在特征 $K$ 上的均值， $\bar{v}_k$ 为总体均值， $C_j$ 表示第 $j$ 个聚类包含的园林集合。F 统计量在零假设（各聚类均值相等）下服从 F 分布， $p$  值小于显著性水平 $\alpha = 0.01$  的特征被认为对聚类具有显著的区别作用。

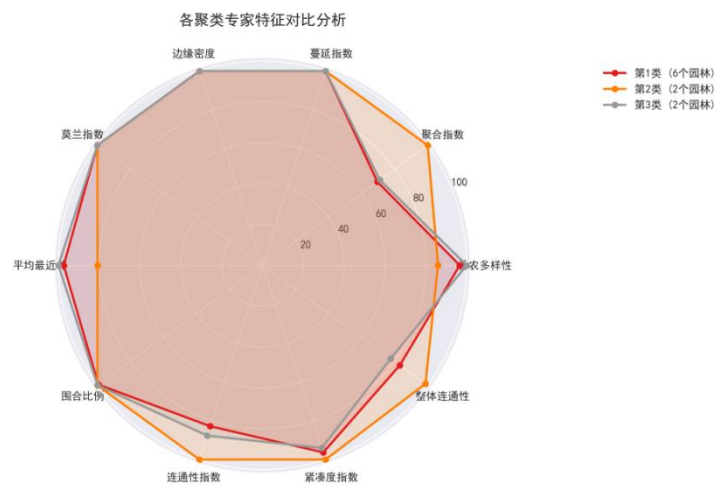


图 115 各聚类专家特征对比分析

通过这一系列数学模型的构建，江南古典园林的异同分析实现了从定性描述到定量建模的转化，为园林相似性评估、共性特征识别和个性化差异量化提供了完整的理论框架和计算方法。该分析体系不仅能够识别园林间的相似程度和共有美学特征，还能精确量化每个园林的个性化特征和类型归属，为江南园林的科学分类和设计法则总结提供了客观的数据支撑。

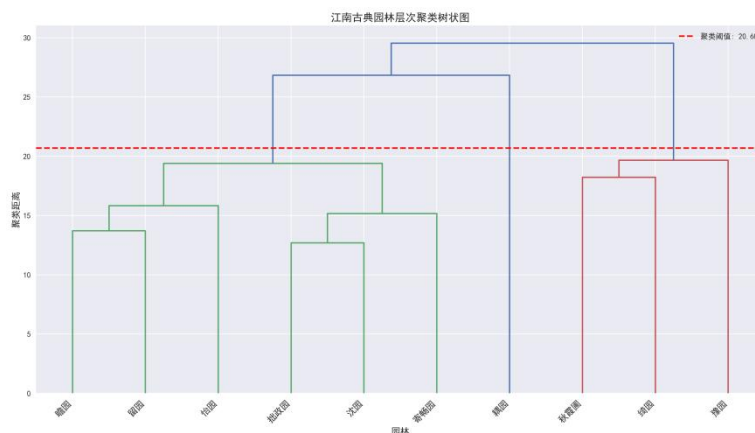


图 116 层次聚类树状图

5 个聚类显著性特征揭示了江南园林差异化创新的主要方向。栅格特征 56 标准差和栅格特征 23 标准差的显著性最强，表明微观景观要素的变异性控制是个性化表达的重要维

度。PCA 主成分 2 ( $F=22.86$ ) 的显著性反映了空间结构的差异化处理。栅格特征 64 均值 ( $F=21.73$ ) 显示了基础景观构成的个性化探索。多样性指数 ( $F=18.43$ ) 作为专家特征的唯一代表,表明在严格遵循传统美学法则的前提下,景观多样性控制仍存在一定的个性化空间

江南古典园林的“异”体现为在统一美学框架内的个性化创新表达,这种差异性不是对传统的背离,而是对传统精神的创造性阐释。0.691 的个性化指数变化范围表明,即使在高度标准化的美学体系内,仍然存在着丰富的创新空间和表达可能。

从创作哲学角度来看,江南园林的个性化差异体现了中国传统文化“因地制宜、因情制宜”的设计智慧。耦园 32.8% 的空间特征贡献反映了对特定场地条件的创造性回应; 27.0% 的网络特征贡献体现了对不同使用需求的精准把握; 不同园林在极值特征上的差异化表现则说明了造园师对个人审美理念和技艺特长的充分发挥。这种个性化不是任意的创新,而是在深刻理解传统精神基础上的有度发挥。

从文化生态意义来看,江南园林的差异性保证了传统文化的生命活力和发展潜力。三种类型化差异模式,经典传统型、空间创新型、特色表达型,构成了传统园林文化的多样化生态,既保持了文化传统的纯正性,又为文化的时代适应和创新发展提供了多种可能路径。这种“美美与共”的差异格局,确保了江南园林文化在历史传承中的持续繁荣。

从美学价值角度来看,江南园林的个性化差异丰富了传统美学的表达谱系。每一种个性化表达都是对传统美学精神的独特理解和诠释,不同的个性化组合如耦园的空间+网络主导型、其他园林的不同特色组合,共同构成了传统美学的完整光谱。这种多样性不仅满足了不同审美趣味的需求,更为传统美学理论的当代发展提供了丰富的实践素材。

#### 6.5.5 异同关系的辩证统一: 传统智慧的现代诠释

通过对每一项数据的深入解读可以发现,江南古典园林的同体现在 49.2% 的强共性特征构成的核心美学规范体系,以及 0.986 的极高综合相似度所反映的文化类型统一性;而异则表现在 0.691 的个性化指数变化范围、50.8% 的中等共性和差异性特征所提供的创新空间,以及三种类型化差异模式所体现的多元发展路径。

这种异同关系的数据化表达,实质上揭示了中华文化“和而不同”哲学思想在空间艺术领域的完美实践。49.2% 与 50.8% 的特征比例分配,体现了传统文化在继承与创新之间的精妙平衡: 既不因过度统一而失去活力,也不因过度差异而丧失根基。这一比例关系本身就是中华文化智慧的体现,反映了传统文化对变与不变、统一与多样、规范与自由等辩证关系的深刻把握。

江南古典园林异同关系的量化分析,不仅为理解传统园林文化提供了科学依据,更为当代文化建设提供了重要启示。在全球化背景下,如何在保持文化特色的前提下实现创新发展,如何在文化传承中既维护传统精神又适应时代需求,江南园林的异同关系为这些现实问题提供了珍贵的历史经验。其高度的共同性确保了文化身份的清晰性和传承的连续性,而丰富的差异性则保证了文化的生命活力和发展潜力。

更进一步地说,江南古典园林所体现的异同关系,实际上是中华文明“各美其美,美人之美,美美与共,天下大同”理想境界的空间化表达。每座园林都在坚持共同美学理想的基础上展现独特的个性之美,不同园林之间既保持着深层的精神共鸣,又呈现出丰富的表达差异,共同构成了江南园林文化的和谐统一体。这种文化智慧在当今世界文明交流互鉴的时代背景下,具有超越园林艺术本身的深远意义,为构建人类命运共同体提供了来自中华文化的智慧启示。

### 6.5.6 文化意象密度与生态和谐性多元视角

江南古典园林作为中国传统文化的重要载体，其美学内涵远不止于表层的趣味性、幻境感和相似度，更深层次地体现在文化意向密度和生态和谐性这两个核心维度上。

从文化意向密度的角度审视，江南古典园林呈现出一种高度浓缩的文化美学特征。园林空间并非单纯的物理环境，而是承载着深厚文化内涵的精神场域。每一处景观都蕴含着丰富的文化符号和意义层次，形成了独特的“诗画一体”美学追求。拙政园中的远香堂不仅是建筑空间，更是对周敦颐《爱莲说》中“香远益清”的空间化诠释；留园的冠云峰则承载着文人对高洁品格的精神寄托。这种文化意向的高密度聚集，使得有限的园林空间能够容纳无限的文化想象，每一个细节都可能触发观者对经典文学、哲学思辨的深层联想。

更为深刻的是，这种文化意向密度呈现出明显的时间层次性和符号互文性特征。园林中的文化符号并非孤立存在，而是形成了复杂的意义网络。一方亭榭可能同时承载着建园者的个人情怀、时代文人的共同审美趣味、以及跨越历史的文化传统。网师园的万卷堂既体现了主人的藏书雅好，又暗合了“读万卷书，行万里路”的文人理想，同时还与园中其他景点形成了关于学问与修养的完整叙事体系。这种多层次的文化编码方式，使得园林成为了一个立体的文化文本，观者在其中的每一次游览都可能产生新的文化解读和美学体验。园林因此超越了单纯的审美功能，成为文化记忆的物质化载体和传统价值的空间表达，体现出中国古典美学中文以载道的深刻内涵。

## 七、 模型评价与推广

### 7.1 模型的优点

(1) 基于 Voronoi 图与空间索引的路径刻画模型：该模型针对园林障碍物线段不连续、难以形成封闭区域的痛点，采用 Voronoi 图提取路径中心线骨架，通过扩展原始边界生成外边界点约束图范围，无需依赖连续障碍物线段即可构建贴合实际的基础路径网络。同时引入空间索引技术优化障碍物检测，通过 shapely 库快速筛选与 Voronoi 边相交的障碍物线段，过滤穿越障碍物的无效边及超尺度不合理边，结合连通分量分析保留最大连通子图，解决了传统路径建模中网络完整性与计算效率难以兼顾的问题。后续分支修剪通过保护出入口节点、迭代移除冗余枝杈的策略，进一步适配游客通行习惯，最终输出的路径网络（如拙政园优化后路径）无穿越障碍物的错误路径，节点与边的分布符合园林实际空间结构；

(2) 基于时空双维度的异景程度模型：模型构建“射线探测+马尔科夫链+汉明距离”的三级量化框架，精准捕捉“移步异景”的核心机制。其中基于射线探测的视景矩阵模型，以 160cm 视线高度为标准，发射 120 条射线（间隔  $3^{\circ}$ ）并按 1 米间距划分探测节点，结合空间索引判断遮挡关系，生成  $120 \times 100$  的视景矩阵，完整还原每个路径节点的可视景观构成，解决了“视线范围景观量化难”的问题。时间维度依托马尔科夫链的无后效性，构建 7 类景观状态的转移概率矩阵，以“小概率转移”量化意外性变化；空间维度通过汉明距离衡量邻域视景向量相似性，实现“时空协同”的异景程度计算。最终微观节点与宏观边的异景程度聚合结果，能精准对应拙政园“梧竹栖居→雪香云蔚亭”等场景的景观切换差异，实现主观审美体验向数学指标的转化；

(3) 基于 NSGA-II 算法的游线规划模型：模型以“最大化异景程度、最小化路径重复长度”为双目标，通过编码优化（将路径转化为“起点+终点+步骤选择序列”）适配遗传操作，避免直接编码节点序列导致的维度爆炸。针对园林宏观图复杂性高的问题，先通过欧氏距离聚类合并空间邻近节点，简化计算规模的同时保留核心连通性。NSGA-II 算法通过非支配排序筛选最优解、拥挤度距离保证解的多样性，迭代 50 代即可收敛至帕累托前

沿，覆盖“高异景-低重复”到“低异景-无重复”的全偏好范围。充分体现模型对“趣味性+效率性”的平衡能力，且路径均严格满足起止于出入口、无无效跳点等物理约束。

(4) 基于视线投影与景观生态学的元素分布模型：模型采用“局部矩形扫描+栅格化+连通域识别”的技术路径，突破传统元素分析的孤立性局限。局部矩形扫描通过“水平扫描+遮挡切换”策略，对实体建筑遮挡区域进行二次射线探测与空白区域填充，生成完整的景观分布矩阵；经  $10\text{m} \times 10\text{m}$  分割块剖分与  $1\text{m} \times 1\text{m}$  栅格细化后，通过 4 邻接算法聚合同类型景观斑块，为指标计算奠定结构化基础。选取的 Shannon 多样性指数 (SHDI)、聚集度指数 (AI)、分形维数指数 (FRAC) 等 6 项指标，分别从多样性、聚集配置、形状复杂度维度解析特征，如拙政园 SHDI 值高对应元素分布均衡，ED 值高反映水体岸线自然曲折，指标结果与“虽由人作，宛自天开”的造园美学高度契合，实现元素分布从“几何描述”到“审美关联”的深化。

(5) 双方法融合的开合变化模型：模型创新性结合空间形态学与空间句法，形成“静态特征刻画+动态节奏解析”的完整框架。空间形态学通过平均最近邻距离 (MNND)、围合比指数 (ER) 等 6 项指标，从“实体围合强度与空间关联”维度量化静态特征，如拙政园中部荷池 MNND=4.12m、ER=0.08，精准对应开阔空间属性；空间句法依托 Depthmap 软件计算连接度与标准化连接度，消除不同园林规模差异的干扰，如寄畅园南部镜池 View\_Co>7000、标准化连接度=0.55，与山麓幽闭区域形成强烈对比。两者结合构建的“开阔-围合”量化标准，经拙政园、寄畅园等典型案例验证，能准确区分不同类型空间的开合属性。

## 7.2 模型的不足

(1) 动态影响因素考虑不足，与实际体验存在偏差：模型主要基于静态空间数据建模，未纳入动态因素对美学体验的影响。例如，未考虑季节变化导致的植物景观差异、游客密度对路径通行与景观感知的干扰、时间维度的光影变化，如朝暮光影对水体与建筑的视觉塑造，这些因素均会影响“异景程度”与“幻境感”的实际体验，导致模型在不同场景下的准确性受限；

(2) 小尺度园林适配性待提升，指标阈值缺乏弹性：现有模型指标体系对小尺度园林（如耦园  $0.8\text{万 m}^2$ 、怡园  $0.6\text{万 m}^2$ ）的适配性不足。例如，开阔空间的 MNND 阈值 ( $\geq 4.0\text{m}$ ) 在小园中难以满足，需人工下调至  $3.5\text{m}$ ；空间句法的连接度计算未考虑小园“视线重叠度高”的特点，导致标准化连接度偏高，难以精准区分“围合-开阔”的细微差异，模型缺乏对不同规模园林的自动化适配机制；

(3) 计算复杂度较高，大规模数据处理效率受限：模型在数据预处理与指标计算阶段涉及大量高维度运算，对计算资源需求较高。例如，生成  $120 \times 100$  的视景矩阵需对每个路径节点发射 120 条射线并进行空间索引检索，处理拙政园  $150 \times 140$  的栅格矩阵时，边缘密度计算需遍历近 2 万个栅格的 4 邻域；NSGA-II 算法在种群规模 500、迭代 50 代时，需完成数十万次目标函数计算，在缺乏并行计算优化的情况下，大规模园林数据处理效率较低。

## 7.3 模型的推广

1. 园林数字化保护领域：可将元素分布与幻境感模型用于历史园林修复，例如基于留园的 SHDI 指数与边缘密度特征，复原清末“林泉耆硕之馆”周边的植物配置；结合路径刻画模型，数字化还原已消失的游线，为文物保护提供数据支撑。

2. 智慧旅游与导览服务：对接景区智慧系统，根据游客偏好调用 NSGA-II 算法的帕累托最优解，生成个性化游线并推送异景程度峰值区域的讲解内容，提升游览体验。

3.现代园林设计与教学：将模型指标体系转化为设计工具，指导新建江南风格园林的元素配比（如水体面积占比参考拙政园的 10%）与开合节奏设计；在园林专业教学中，通过模型可视化结果直观展示“小中见大”的实现机制，增强理论教学的实践性。

## 参考文献

- [1] Lin Yuanjie. Spatiotemporal narrative structure of the lingering garden based on traditional Chinese conception of time and space [J]. Landscape Research, 2022, 48(1): 45 - 63. <https://doi.org/10.1080/01426397.2022.2139820>
- [2] Sun Ce, Jiang Zhenyu, Yu Bingqin. How to interpret Jiangnan gardens: a study of the spatial layout of Jiangnan gardens from the perspective of fractal geometry [J]. Heritage Science, 2024, (12): 353. <https://doi.org/10.1186/s40494-024-01469-x>
- [3] 孙依含.基于MATLAB图像分割的江南古典园林当代游人视觉感知研究[D].南京农业大学,2022.DOI:10.27244/d.cnki.gnjnu.2022.001340..
- [4] 梁慧琳,张青萍.基于空间句法的江南私家园林空间开合量化研究[J].现代城市研究,2017,(01):47-52.
- [5] 王诗彤.基于空间句法的江南私家园林空间序列量化研究[D].沈阳建筑大学,2020.DOI:10.27809/d.cnki.gsjgc.2020.000658.
- [6] 戚路辉,陈苑仪,马瑜培,等.基于空间句法的社区公共艺术空间营造研究——以广州市竹丝岗社区为例[J].装饰,2023,(05):127-129.DOI:10.16272/j.cnki.cn11-1392/j.2023.05.001.
- [7] 徐勇.基于空间连通度的江南园林庭院空间研究[D].河南农业大学,2018.
- [8] 朱思羽.基于视觉性的江南园林空间组织在方塔园中的应用[D].江西农业大学,2014.
- [9] 张颖.基于视线的江南私家园林空间量化研究[D].福建农林大学,2015.
- [10] 张乐.江南传统园林中建筑与景观的空间过渡艺术探究——以苏州拙政园为例[J].现代园艺,2025,48(16):149-150+153.DOI:10.14051/j.cnki.xdy.2025.16.017.
- [11] 姜澜.江南私家园林大体量建筑室内空间非均质划分现象初探[D].南京大学,2018.
- [12] 朱子墨.江南私家园林假山路径量化研究[D].南京林业大学,2020.DOI:10.27242/d.cnki.gnjlu.2020.000109.
- [13] 巫文笑.江南私家园林植物景观空间尺度研究[D].南京林业大学,2020.DOI:10.27242/d.cnki.gnjlu.2020.000121.
- [14] 金芸.江南私家园林路径空间量化研究[D].南京农业大学,2011.
- [15] 唐真.江南私家园林廊空间量化特性研究[D].南京农业大学,2009.
- [16] 李旭,龚文君,龙毅湘.江南园林空间节奏量化分析研究——以留园为例[J].南方建筑,2024,(08):59-68.
- [17] 冯纪忠.组景刍议[J].同济大学学报,1979,(04):1-5.
- [18] 伍端.凝视的快感:空间审美的具身化转向[J].装饰,2022,(06):108-112.
- [19] 刘敦桢.苏州古典园林[M].中国建筑工业出版社,2005.
- [20] 陈从周.苏州旧住宅[M].同济大学出版社,2018.

## 附录

本文所使用的代码数据流动关系总体流动路线如下图 116 所示。

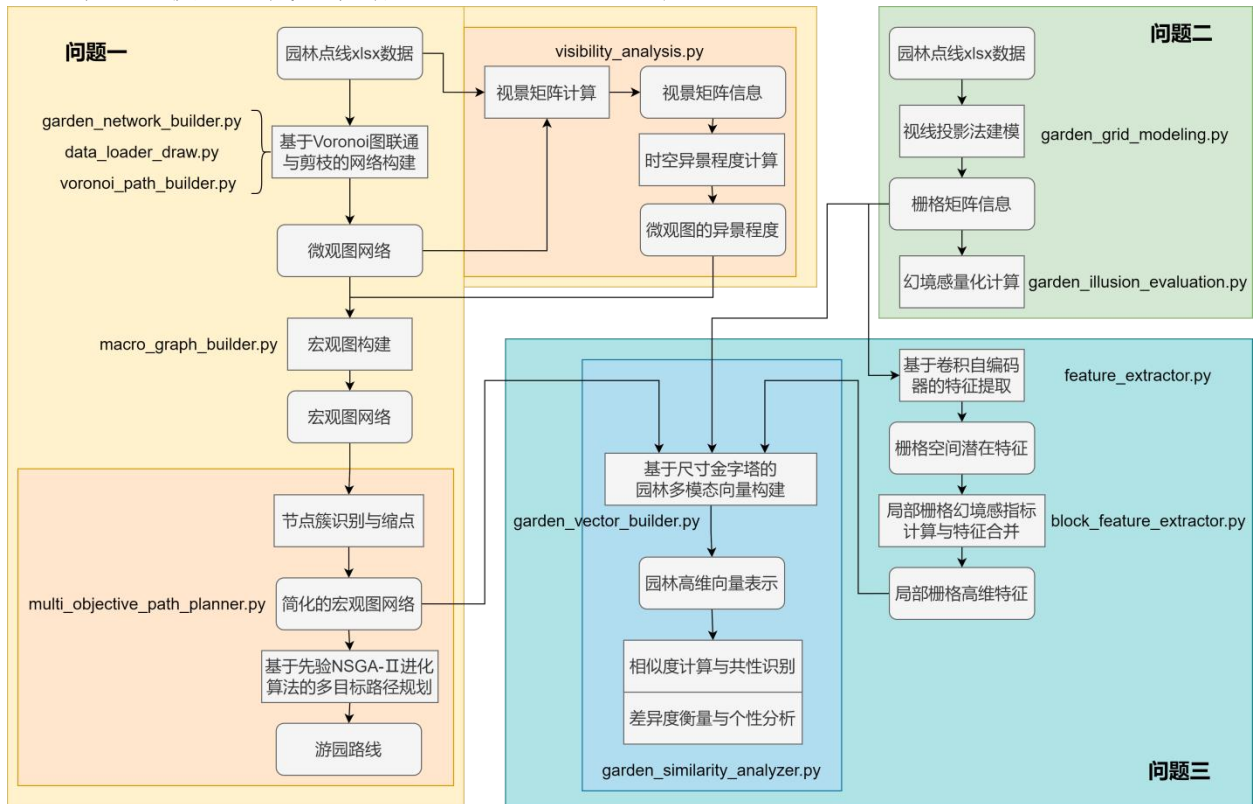


图 117 数据流动关系

问题一相关代码

data\_loader\_draw.py

```
'''
数据加载与基础可视化
'''

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon, Circle
from shapely.geometry import Point, LineString, Polygon as ShapelyPolygon
from matplotlib import pyplot as plt
plt.rcParams['font.family'] = ['simhei'] # 解决中文显示问题
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
class GardenDataLoader:
    """园林数据加载器"""
    def __init__(self, excel_path):
        self.excel_path = excel_path
        self.data = {}
        self.element_types = ["半开放建筑", "实体建筑", "道路", "假山", "水体", "植物"]
    def load_excel_data(self):
        """从 Excel 文件加载所有数据"""
        try:
            # 读取所有工作表
            excel_data = pd.read_excel(self.excel_path, sheet_name=None)
            for element_type in self.element_types:
                if element_type in excel_data:
```

```

        self.data[element_type] = excel_data[element_type]
        print(f"成功加载 {element_type} 数据, 共 {len(excel_data[element_type])}
行")

    else:
        print(f"警告: 未找到 {element_type} 工作表")
        return self.data
except Exception as e:
    print(f"加载 Excel 文件失败: {e}")
    return None

def parse_non_segmented_coordinates(self, coord_text):
    """解析不区分线段的坐标数据"""
    if pd.isna(coord_text):
        return []
    coordinates = []
    lines = coord_text.strip().split('\n')
    for line in lines:
        line = line.strip()
        if line.startswith('{') and line.endswith('}'):
            try:
                coord_part = line[1:-1] # 去掉大括号
                coords = [float(x.strip()) for x in coord_part.split(',')]
                if len(coords) >= 2:
                    coordinates.append((coords[0], coords[1]))
            except:
                continue
    return coordinates

def extract_geometry_data(self):
    """提取几何数据"""
    ...
    其他元素
    键为元素类型, 值为包含所有几何数据的字典
    'segments': [], # 区分线段
    'all_points': [] # 所有点

    植物
    键为元素类型, 值为包含所有植物的列表
    [
        {
            'center': (x, y), # 圆心坐标
            'radius': r # 半径
        },
        ...
    ]
    }
    ...

    geometry_data = {}
    for element_type in ["半开放建筑", "实体建筑", "道路", "假山", "水体"]: # 完整版: ["半
开放建筑", "实体建筑", "道路", "假山", "水体"]
        if element_type not in self.data:
            continue
        df = self.data[element_type]
        geometry_data[element_type] = {
            'segments': [],
            'all_points': []
        }
        first_column_list = df.iloc[:, 0].tolist()
        # 处理区分线段的数据 (第一列)
        now_segment_s = []
        now_segment = []
        # 遍历第一列

```

```

for line in first_column_list:
    line = line.strip()
    # 检查是否是线段分隔符 {0; a}
    if line.startswith('{') and ';' in line and line.endswith('}'):
        # 保存上一个 segment
        if len(now_segment) != 0:
            now_segment_s.append(now_segment)
        # 开始新的 segment 记录
        now_segment = []
    else:
        # 解析坐标点 b. {x, y, z}
        if '.' in line and '{' in line:
            # 分离序号和坐标部分
            parts = line.split('.', 1) # 只分割第一个点
            if len(parts) == 2:
                point_index = int(parts[0].strip())
                coord_part = parts[1].strip()
                # 提取大括号内的坐标
                if coord_part.startswith('{') and '}' in coord_part:
                    coord_values = coord_part[1:coord_part.rindex('}')]
                    coords = [float(x.strip()) for x in coord_values.split(',')]
                    # 添加点
                    point = (coords[0], coords[1])
                    now_segment.append(point)
            else:
                if len(now_segment) != 0:
                    now_segment_s.append(now_segment)
        # 保存 segment
        geometry_data[element_type]['segments'] = now_segment_s
        # 处理不区分线段的数据 (第二列)
        if len(df.columns) > 1:
            for _, row in df.iterrows():
                points = self.parse_non_segmented_coordinates(row.iloc[1])
                geometry_data[element_type]['all_points'].extend(points)
# 处理植物数据
if "植物" in self.data:
    plant_df = self.data["植物"]
    plants = []
    for _, row in plant_df.iterrows():
        try:
            center_coords = self.parse_non_segmented_coordinates(row.iloc[0])
            radius = float(row.iloc[1]) if not pd.isna(row.iloc[1]) else 500 # 默认
半径 500mm

            for center in center_coords:
                plants.append({
                    'center': center,
                    'radius': radius
                })
        except:
            continue
    geometry_data["植物"] = plants
return geometry_data
def connect_nearby_boundary_points(self, geometry_data, distance_threshold=1000):
    """
    连接不同障碍物类型之间距离较近的边界点，并创建专门的连接线段类型

    Args:
        geometry_data: 几何数据字典
        distance_threshold: 距离阈值，单位 mm，默认 1000mm

```

```

Returns:
    更新后的 geometry_data
"""

print(f"正在连接不同障碍物类型间距离小于 {distance_threshold}mm 的边界点...")
# 影响通行的障碍物类型
obstacle_types = ["半开放建筑", "实体建筑", "道路"]
# 收集每种类型的所有点
type_points = {}
for element_type in obstacle_types:
    if element_type not in geometry_data:
        continue
    all_points = []
    # 从 segments 中收集点
    for segment in geometry_data[element_type]['segments']:
        all_points.extend(segment)
    # 从 all_points 中收集点
    all_points.extend(geometry_data[element_type]['all_points'])
    # 去重
    unique_points = []
    for point in all_points:
        if point not in unique_points:
            unique_points.append(point)
    type_points[element_type] = unique_points
    print(f"{element_type}: {len(unique_points)} 个点")
# 创建新的连接线段类型
connection_segments = []
total_connections = 0
# 遍历所有类型对进行跨类型连接
type_pairs = [
    ("半开放建筑", "实体建筑"), #("实体建筑", "半开放建筑"),
    ("半开放建筑", "道路"), #("道路", "半开放建筑"),
    ("实体建筑", "道路"), #("道路", "实体建筑"),
]
for type1, type2 in type_pairs:
    if type1 not in type_points or type2 not in type_points:
        continue
    points1 = type_points[type1]
    points2 = type_points[type2]
    connection_count = 0
    # 找到距离较近的对
    for point1 in points1:
        for point2 in points2:
            distance = np.sqrt((point1[0] - point2[0])**2 + (point1[1] - point2[1])**2)
            if distance <= distance_threshold:
                connection_segments.append([point1, point2])
                connection_count += 1
    if connection_count > 0:
        print(f"{type1} ↔ {type2}: 添加了 {connection_count} 条连接线段")
        total_connections += connection_count
# 将连接线段添加到 geometry_data 中作为新类型
if connection_segments:
    geometry_data["障碍物连接"] = {
        'segments': connection_segments,
        'all_points': []
    }
    print(f"创建了新类型 '障碍物连接', 包含 {len(connection_segments)} 条线段")
print(f"总共添加了 {total_connections} 条跨类型连接线段")
return geometry_data

def _points_in_same_segment(self, point1, point2, segment):
    """检查两个点是否在同一个线段中（相邻或间隔一个点）"""

```

```

        if point1 not in segment or point2 not in segment:
            return False
        idx1 = segment.index(point1)
        idx2 = segment.index(point2)
        # 检查是否相邻（允许间隔最多 2 个点的距离）
        return abs(idx1 - idx2) <= 2
class GardenVisualizer:
    """园林可视化器"""
    def __init__(self, geometry_data):
        self.geometry_data = geometry_data
        self.colors = {
            '半开放建筑': 'lightblue',
            '实体建筑': 'gray',
            '道路': 'orange',
            '假山': 'brown',
            '水体': 'cyan',
            '植物': 'green',
            '障碍物连接': 'red' # 不同影响通行的边界之间的连线
        }
    def visualize_garden(self, figsize=(15, 12)):
        """可视化整个园林"""
        fig, ax = plt.subplots(figsize=figsize)
        # 绘制各种元素
        for element_type, data in self.geometry_data.items():
            if element_type not in ["植物"]: # 不绘制某些元素 ['半开放建筑', '实体建筑', '道路', '假山', '水体', '植物']
                if element_type == "植物":
                    self._draw_plants(ax, data)
                else:
                    self._draw_elements(ax, element_type, data)
        # 设置图形属性
        ax.set_aspect('equal')
        ax.grid(True, alpha=0.5)
        ax.set_xlabel('X (mm)')
        ax.set_ylabel('Y (mm)')
        # 添加图例
        legend_elements = []
        for element_type, color in self.colors.items():
            if element_type in self.geometry_data:
                legend_elements.append(plt.Rectangle((0,0),1,1, facecolor=color,
                                                       alpha=0.7, label=element_type))
        ax.legend(handles=legend_elements, loc='upper right')
        plt.tight_layout()
        return fig, ax
    def _draw_elements(self, ax, element_type, data):
        """绘制建筑、道路等元素"""
        color = self.colors.get(element_type, 'black')
        # 绘制线段
        for segment in data.get('segments', []):
            # 存在线段只有一个点.....（忽略）
            if len(segment) > 1:
                xs, ys = zip(*segment)
                ax.plot(xs, ys, color=color, linewidth=2, alpha=0.3)
        # 绘制所有点
        points = data.get('all_points', [])
        if points:
            xs, ys = zip(*points)
            ax.scatter(xs, ys, c=color, s=2, alpha=0.5)
    def _draw_plants(self, ax, plants):
        """绘制植物"""

```

```

        for plant in plants:
            center = plant['center']
            radius = plant['radius']
            circle = Circle(center, radius, color=self.colors['植物'],
                            alpha=0.6, fill=True)
            ax.add_patch(circle)
    def _draw_garden_base_on_ax(self, ax):
        """在指定的坐标轴上绘制基础园林地图"""
        # 绘制各种元素
        for element_type, data in self.geometry_data.items():
            if element_type not in ["植物"]: # 不绘制某些元素 ['半开放建筑', '实体建筑', '道路', '假山', '水体', '植物']
                if element_type == "植物":
                    self._draw_plants(ax, data)
                else:
                    self._draw_elements(ax, element_type, data)
        # 添加图例
        legend_elements = []
        for element_type, color in self.colors.items():
            if element_type in self.geometry_data:
                legend_elements.append(plt.Rectangle((0,0),1,1, facecolor=color,
                                                       alpha=0.7, label=element_type))
        ax.legend(handles=legend_elements, loc='upper right', fontsize=10)
    ##### 绘制基本的地图
    def visualize_original_garden_with_labels(self, figsize=(15, 12)):
        """可视化原始园林并标记各种要素"""
        fig, ax = plt.subplots(figsize=figsize)
        # 绘制各种元素（除了植物）
        for element_type, data in self.geometry_data.items():
            if element_type != "植物": # 不绘制植物
                self._draw_elements(ax, element_type, data)
        # 设置图形属性
        ax.set_aspect('equal')
        ax.grid(True, alpha=0.5)
        ax.set_xlabel('X (mm)')
        ax.set_ylabel('Y (mm)')
        # 添加图例，只包含非植物要素
        legend_elements = []
        for element_type, color in self.colors.items():
            if element_type in self.geometry_data and element_type != "植物":
                legend_elements.append(plt.Rectangle((0,0),1,1, facecolor=color,
                                                       alpha=0.7, label=element_type))
        ax.legend(handles=legend_elements, loc='upper right', fontsize=12)
        plt.tight_layout()
        return fig, ax

```

## garden\_network\_builder.py

```

'''
路网络构建
'''

import matplotlib.pyplot as plt
from shapely.geometry import Point, LineString, Polygon as ShapelyPolygon
import plotly.tools as tls
# 数据加载与基础可视化
from data_loader_draw import GardenDataLoader, GardenVisualizer
# Voronoi 路径网络构建
from voronoi_path_builder import SimpleVoronoiPathBuilder

```

```

def main_pipeline(excel_path, file_name, baohu_regions):
    """主处理流程"""
    # 1. 加载数据
    print("正在加载数据...")
    loader = GardenDataLoader(excel_path)
    data = loader.load_excel_data()
    if data is None:
        print("数据加载失败")
        return None
    # 2. 提取几何数据
    print("正在提取几何数据...")
    """
    geometry_data

    其他元素
    键为元素类型，值为包含所有几何数据的字典
    'segments': [], # 区分线段
    'all_points': [] # 所有点

    植物
    键为元素类型，值为包含所有植物的列表
    [
        {
            'center': (x, y), # 圆心坐标
            'radius': r # 半径
        },
        ...
    ]
    """
    geometry_data = loader.extract_geometry_data()
    # 2.5. 连接不同障碍物类型之间距离较近的边界点，补全缺失边界（500 米）
    print("正在补全边界连接...")
    geometry_data = loader.connect_nearby_boundary_points(geometry_data,
distance_threshold=500)
    # 3. 可视化原始数据
    print("正在生成可视化（原始地图）...")
    visualizer = GardenVisualizer(geometry_data)
    fig1, ax1 = visualizer.visualize_garden()
    plt.show()
    # 保存图片，设置 DPI 为 300
    plt.savefig('plot.png', dpi=1000)
    # 3. 可视化原始园林地图
    print("正在生成原始园林地图...")
    visualizer = GardenVisualizer(geometry_data)
    # 生成基础原始地图
    fig1, ax1 = visualizer.visualize_original_garden_with_labels()
    plt.savefig(f'{file_name}-原始地图.png', dpi=1000, bbox_inches='tight')
    # --- 转换并保存 ---
    # 1 将 Matplotlib Figure 对象转换为 Plotly Figure 对象
    plotly_fig = tls.mpl_to_plotly(fig1)
    # 2. 将 Plotly Figure 保存为 HTML 文件
    plotly_fig.write_html(f'{file_name}-原始地图.html')
    plt.show()
    # 4. 构建 Voronoi 路径网络（包含连通性分析）
    print("\n 正在构建 Voronoi 路径网络...")
    path_builder = SimpleVoronoiPathBuilder(geometry_data)
    # 示例：设置保护区域
    path_builder.set_start_end_regions(baohu_regions[file_name])
    # 或设置多边形区域
    # polygon_region = [(x1, y1), (x2, y2), (x3, y3), (x4, y4)]

```

```

#path_builder.set_start_end_regions([polygon_region], [])
# 完整的路径网络构建流程（进行了最大连通分量筛选 和 剪枝）
path_network = path_builder.build_complete_path_network(enable_pruning=True) # 进行剪枝
True False
# 5. 保存路径网络到 Excel 文件
print("\n 正在保存路径网络到 Excel 文件（{file_name}-路网络.xlsx）...")
saved_file = path_builder.save_pruned_network_to_excel(file_name)
print("\n 正在生成完整路径网络可视化...")
# 方式 1: 完整可视化（包含原始园林元素和最大连通分量）
fig2, ax2 = path_builder.visualize_voronoi_network(show_boundary_points=False,
show_voronoi_vertices=False)
plt.savefig(f'{file_name}-Voronoi 完整路径网络+地图.png', dpi=1000, bbox_inches='tight')
# --- 转换并保存 ---
plotly_fig = tls.mpl_to_plotly(fig2)
plotly_fig.write_html(f'{file_name}-Voronoi 最大路径网络（修剪后）+地图.html")
#plt.show()
# 5. 可视化最大连通分量分析结果
print("\n 正在生成最大路径网络可视化...")
# 方式 1: 完整可视化（包含原始园林元素和最大连通分量）
fig3, ax3 = path_builder.visualize_connected_components()
plt.savefig(f'{file_name}-Voronoi 最大路径网络+地图.png', dpi=1000, bbox_inches='tight')
#plt.show()
# 方式 2: 简化可视化（只显示最大连通分量）
'''
fig4, ax4 = path_builder.visualize_simple_connected_components()
plt.savefig(f'{file_name}-Voronoi 最大路径网络.png', dpi=1000, bbox_inches='tight')
plt.show()
'''
# 5. 可视化最大连通分量分析结果
print("\n 正在生成可视化修剪后的路径网络（包含保护区）...")
# 可视化修剪后的路径网络（包含保护区）
fig5, ax5 = path_builder.visualize_pruned_network()
plt.savefig(f'{file_name}-Voronoi 最大路径网络（修剪后）+地图.png', dpi=1000,
bbox_inches='tight')
# --- 转换并保存 ---
plotly_fig = tls.mpl_to_plotly(fig5)
plotly_fig.write_html(f'{file_name}-Voronoi 最大路径网络（修剪后）+地图.html")
#plt.show()
return 0
# 使用示例
if __name__ == "__main__":
    # 设置圆形区域（出入口）
    baohu_regions = {
        '留园': [{'center': (92906.26, 12499.86), 'radius': 300},
        ],
        '寄畅园': [{'center': (60085.57, 104362.1), 'radius': 300},
        ],
        '瞻园': [{'center': (92431.66, 2785.348), 'radius': 300},
        {'center': (106225.4, 90968.34), 'radius': 300},
        ],
        '豫园': [{'center': (-42726.04, 140747), 'radius': 300},
        {'center': (-32170.37, 82448.91), 'radius': 300},
        ],
        '秋霞圃': [{'center': (110206.9, 14414.47), 'radius': 300},
        {'center': (94147.3, 188033.9), 'radius': 300},
        ],
        '沈园': [{'center': (65431.27, 115888.3), 'radius': 300},
        {'center': (90423.13, -26298.59), 'radius': 300},
        ],
        '怡园': [{'center': (102159.9, 54426.96), 'radius': 300},

```

```

        {'center': (70174.21, -4427.089), 'radius': 300},
        {'center': (-24335.48, 39816.94), 'radius': 300},
    ],
    '耦园': [{'center': (-5702.175, 53760.82), 'radius': 300},
             {'center': (-8992.038, 47493.77), 'radius': 300},
             {'center': (29293.22, 65602.38), 'radius': 300},
    ],
    '绮园': [{'center': (20309.32, 39276.54), 'radius': 300},
    ],
    '拙政园': [{'center': (-7066.044, 20491.56), 'radius': 300}, # 左下
              {'center': (154561.4, 138363.2), 'radius': 300}, # 右上
              {'center': (165138.3, 125058.1), 'radius': 300}, # 右中
              {'center': (177727.5, 66624.27), 'radius': 300}, # 右下
    ],
    '近园': [{'center': (9610.723, 40909.44), 'radius': 300},
             {'center': (8284.23, 23863.85), 'radius': 300},
             {'center': (25500, 4222.222), 'radius': 1000},
             {'center': (51497.72, 66793.48), 'radius': 300},
    ],
}
file_name_s = ["近园"]
for file_name in file_name_s:
    excel_file_path = f"shujv/4-{file_name}数据坐标.xlsx"
    print(f"正在处理文件: {file_name}")
    result = main_pipeline(excel_file_path, file_name, baohu_regions)

```

## macro\_graph\_builder.py

```

"""
宏观图构建器 - 从微观路网络构建宏观图
产出:
f"{file_name}_macro_graph.pkl"
"""

import numpy as np
import pandas as pd
import pickle
import json
import matplotlib.pyplot as plt
import networkx as nx
from collections import defaultdict, deque
import os
from typing import Dict, List, Tuple, Optional, Any
# 设置中文字体
plt.rcParams['font.family'] = ['simhei']
plt.rcParams['axes.unicode_minus'] = False
class MacroGraphBuilder:
    """宏观图构建器"""
    def __init__(self):
        """初始化构建器"""
        self.micro_graph = None # 微观图 (NetworkX 图)
        self.macro_graph = None # 宏观图 (NetworkX 图)
        self.micro_to_macro_mapping = {} # 微观点到宏观点的映射
        self.macro_edges_paths = {} # 宏观边对应的微观路径
        self.visibility_data = None # 视景数据
        self.geometry_data = None # 园林几何数据
        self.manual_macro_nodes = {} # 手动添加的宏观节点 {node_id: node_data}
        self.manual_macro_edges = {} # 手动添加的宏观边 {(node1, node2): edge_data}
    def load_data(self, file_name: str):

```

```

"""加载所有必需的数据"""
# 加载视景分析数据
from visibility_analysis_2 import VisibilityDataAnalyzer
analyzer = VisibilityDataAnalyzer()
# 视景数据路径前缀
visibility_data_prefix = f"视景数据/{file_name}_visibility_analysis"
original_excel = f"shujv/4-{file_name}数据坐标.xlsx"
network_excel = f"{file_name}-路网络.xlsx"
print("=== 加载视景分析数据 ===")
if not analyzer.load_visibility_data(visibility_data_prefix):
    print("视景数据加载失败")
    return False
print("=== 加载网络数据 ===")
if not analyzer.load_network_data(original_excel, network_excel):
    print("网络数据加载失败")
    return False
self.visibility_data = analyzer
self.geometry_data = analyzer.geometry_data
# 构建微观图
self._build_micro_graph()
return True

def add_manual_macro_node(self, node_id: int, pos: Tuple[float, float],
                        degree: int = 1, temporal_anomaly: float = 0,
                        spatial_anomaly: float = 0):
    """手动添加宏观节点

    Args:
        node_id: 节点 ID
        pos: 节点位置 (x, y)
        degree: 节点度数
        temporal_anomaly: 时间异景程度
        spatial_anomaly: 空间异景程度
    """
    self.manual_macro_nodes[node_id] = {
        'pos': pos,
        'degree': degree,
        'temporal_anomaly': temporal_anomaly,
        'spatial_anomaly': spatial_anomaly
    }
    print(f"已添加手动宏观节点: {node_id} at {pos}")

def add_manual_macro_edge(self, node1: int, node2: int, path: List[int],
                        length: float, turning_points: int = 0,
                        temporal_anomaly_sum: float = 0,
                        spatial_anomaly_sum: float = 0,
                        micro_point_count: int = 0):
    """手动添加宏观边

    Args:
        node1, node2: 边的两个端点
        path: 微观路径
        length: 路径长度
        turning_points: 转折点数量
        temporal_anomaly_sum: 时间异景程度总和
        spatial_anomaly_sum: 空间异景程度总和
        micro_point_count: 微观点数量
    """
    edge_key = (min(node1, node2), max(node1, node2))
    self.manual_macro_edges[edge_key] = {
        'path': path,
        'length': length,

```

```

        'turning_points': turning_points,
        'temporal_anomaly_sum': temporal_anomaly_sum,
        'spatial_anomaly_sum': spatial_anomaly_sum,
        'micro_point_count': micro_point_count
    }
    # 同时存储路径映射
    self.macro_edges_paths[edge_key] = path
    print(f"已添加手动宏观边: {node1}-{node2}, 路径长度: {length}")
def _add_manual_macro_elements(self):
    """添加手动指定的宏观节点和边"""
    # 添加手动宏观节点
    for node_id, node_data in self.manual_macro_nodes.items():
        if not self.macro_graph.has_node(node_id):
            self.macro_graph.add_node(node_id, **node_data)
            print(f"已添加手动宏观节点: {node_id}")
        else:
            print(f"节点 {node_id} 已存在, 跳过")
    # 添加手动宏观边
    for (node1, node2), edge_data in self.manual_macro_edges.items():
        if not self.macro_graph.has_edge(node1, node2):
            self.macro_graph.add_edge(node1, node2, **edge_data)
            print(f"已添加手动宏观边: {node1}-{node2}")
        else:
            print(f"边 {node1}-{node2} 已存在, 跳过")
def _build_micro_graph(self):
    """构建微观图 (NetworkX 图)"""
    self.micro_graph = nx.Graph()
    # 添加节点
    for i, point in enumerate(self.visibility_data.network_points):
        self.micro_graph.add_node(i, pos=point)
    # 添加边
    for connection in self.visibility_data.network_connections:
        self.micro_graph.add_edge(connection[0], connection[1])
    print(f"微观图构建完成: {self.micro_graph.number_of_nodes()} 个节点,
{self.micro_graph.number_of_edges()} 条边")

def _identify_entrance_nodes(self):
    """识别出入口节点 (微观图中度为 1 且在宏观图中的节点, 或者手动添加的度为 1 的宏观节点)"""
    entrance_nodes = []
    # 找到微观图中度为 1 的节点
    for node in self.micro_graph.nodes():
        if self.micro_graph.degree[node] == 1:
            # 检查该节点是否在宏观图中
            if node in self.macro_graph.nodes():
                entrance_nodes.append(node)
    # 2. 找到手动添加的度为 1 的宏观节点
    for node_id, node_data in self.manual_macro_nodes.items():
        if node_data.get('degree', 0) == 1:
            # 避免重复添加
            if node_id not in entrance_nodes:
                entrance_nodes.append(node_id)
    print(f"识别到 {len(entrance_nodes)} 个出入口节点: {entrance_nodes}")
    return entrance_nodes
def build_macro_graph(self):
    """构建宏观图"""
    if self.micro_graph is None:
        print("请先加载数据")
        return False
    print("=== 构建宏观图 ===")
    # 1. 识别宏观点

```

```

macro_nodes = self._identify_macro_nodes()
print(f"识别到 {len(macro_nodes)} 个宏观点")
# 2. 构建宏观图
self.macro_graph = nx.Graph()
# 添加自动识别的宏观节点
for node_id in macro_nodes:
    pos = self.micro_graph.nodes[node_id]['pos']
    degree = self.micro_graph.degree[node_id]
    # 获取异景程度
    anomaly_scores = self.visibility_data.anomaly_scores.get(node_id, {})
    temporal_anomaly = anomaly_scores.get('temporal_anomaly', 0)
    spatial_anomaly = anomaly_scores.get('spatial_anomaly', 0)
    self.macro_graph.add_node(node_id,
                               pos=pos,
                               degree=degree,
                               temporal_anomaly=temporal_anomaly,
                               spatial_anomaly=spatial_anomaly)

# 3. 计算自动宏观边
self._build_macro_edges(macro_nodes)
# 4. 添加手动指定的宏观节点和边
self._add_manual_macro_elements()
# 5. 识别出入口节点
self.entrance_nodes = self._identify_entrance_nodes()
print(f"宏观图构建完成: {self.macro_graph.number_of_nodes()} 个节点,
{self.macro_graph.number_of_edges()} 条边")
print(f"出入口节点: {len(self.entrance_nodes)} 个")
return True

def _identify_macro_nodes(self):
    """识别宏观点（度为1或度≥3的点）"""
    macro_nodes = []
    for node in self.micro_graph.nodes():
        degree = self.micro_graph.degree[node]
        if degree == 1 or degree >= 3:
            macro_nodes.append(node)
    return macro_nodes

def _build_macro_edges(self, macro_nodes):
    """构建宏观边"""
    macro_node_set = set(macro_nodes)
    for start_node in macro_nodes:
        # 使用 BFS 找到从当前宏观点到其他宏观点的路径
        visited = set()
        queue = deque([(start_node, [start_node])])
        visited.add(start_node)
        while queue:
            current_node, path = queue.popleft()
            # 检查相邻节点
            for neighbor in self.micro_graph.neighbors(current_node):
                if neighbor in visited:
                    continue
                new_path = path + [neighbor]
                visited.add(neighbor)
                # 如果到达另一个宏观点，创建宏观边
                if neighbor in macro_node_set and neighbor != start_node:
                    if not self.macro_graph.has_edge(start_node, neighbor):
                        self._add_macro_edge(start_node, neighbor, new_path)
            else:
                # 继续搜索（只有度为2的点才继续）
                if self.micro_graph.degree[neighbor] == 2:
                    queue.append((neighbor, new_path))

def _add_macro_edge(self, start_node, end_node, path):

```

```

        """添加宏观边并计算其属性"""
        # 计算路径长度
        path_length = self._calculate_path_length(path)
        # 计算转折点数量
        turning_points = self._count_turning_points(path)
        # 计算异景程度（不包含端点）
        temporal_sum, spatial_sum, point_count = self._calculate_anomaly_sums(path[1:-1])
        # 添加边
        self.macro_graph.add_edge(start_node, end_node,
                                   path=path,
                                   length=path_length,
                                   turning_points=turning_points,
                                   temporal_anomaly_sum=temporal_sum,
                                   spatial_anomaly_sum=spatial_sum,
                                   micro_point_count=point_count)

        # 保存宏观边对应的微观路径
        edge_key = (min(start_node, end_node), max(start_node, end_node))
        self.macro_edges_paths[edge_key] = path
    def _calculate_path_length(self, path):
        """计算路径长度"""
        length = 0
        for i in range(len(path) - 1):
            pos1 = self.micro_graph.nodes[path[i]]['pos']
            pos2 = self.micro_graph.nodes[path[i+1]]['pos']
            length += np.sqrt((pos1[0] - pos2[0])**2 + (pos1[1] - pos2[1])**2)
        return length
    def _count_turning_points(self, path, angle_threshold=20):
        """计算转折点数量（度为2且偏转超过阈值的点）"""
        if len(path) < 3:
            return 0
        turning_count = 0
        for i in range(1, len(path) - 1):
            # 检查当前点的度是否为2
            if self.micro_graph.degree[path[i]] != 2:
                continue
            # 计算转向角度
            pos_prev = self.micro_graph.nodes[path[i-1]]['pos']
            pos_curr = self.micro_graph.nodes[path[i]]['pos']
            pos_next = self.micro_graph.nodes[path[i+1]]['pos']
            # 计算向量
            vec1 = (pos_curr[0] - pos_prev[0], pos_curr[1] - pos_prev[1])
            vec2 = (pos_next[0] - pos_curr[0], pos_next[1] - pos_curr[1])
            # 计算角度
            angle = self._calculate_angle(vec1, vec2)
            # 如果偏转角度超过阈值，算作转折点
            if abs(180 - angle) > angle_threshold:
                turning_count += 1
        return turning_count
    def _calculate_angle(self, vec1, vec2):
        """计算两个向量之间的夹角（度）"""
        dot_product = vec1[0] * vec2[0] + vec1[1] * vec2[1]
        mag1 = np.sqrt(vec1[0]**2 + vec1[1]**2)
        mag2 = np.sqrt(vec2[0]**2 + vec2[1]**2)
        if mag1 == 0 or mag2 == 0:
            return 0
        cos_angle = dot_product / (mag1 * mag2)
        cos_angle = np.clip(cos_angle, -1, 1) # 防止数值误差
        angle_rad = np.arccos(cos_angle)
        angle_deg = np.degrees(angle_rad)
        return angle_deg

```

```

def _calculate_anomaly_sums(self, point_ids):
    """计算路径上点的异景程度之和"""
    temporal_sum = 0
    spatial_sum = 0
    point_count = len(point_ids)
    for point_id in point_ids:
        anomaly_scores = self.visibility_data.anomaly_scores.get(point_id, {})
        temporal_sum += anomaly_scores.get('temporal_anomaly', 0)
        spatial_sum += anomaly_scores.get('spatial_anomaly', 0)
    return temporal_sum, spatial_sum, point_count

def save_macro_graph(self, file_name: str):
    """保存宏观图数据"""
    # 保存宏观图
    macro_graph_data = {
        'nodes': dict(self.macro_graph.nodes(data=True)),
        'edges': {(u, v): data for u, v, data in self.macro_graph.edges(data=True)},
        'macro_edges_paths': self.macro_edges_paths,
        'entrance_nodes': getattr(self, 'entrance_nodes', []) # 保存出入口节点
    }
    with open(f"视景数据/{file_name}_macro_graph.pkl", 'wb') as f:
        pickle.dump(macro_graph_data, f)
    # 保存为JSON（便于查看）
    json_data = {
        'nodes': {str(k): v for k, v in self.macro_graph.nodes(data=True)},
        'edges': {f"{u}-{v}": data for u, v, data in self.macro_graph.edges(data=True)},
        'macro_edges_paths': {f"{k[0]}-{k[1]}": v for k, v in
self.macro_edges_paths.items()},
        'entrance_nodes': getattr(self, 'entrance_nodes', [])
    }
    with open(f"视景数据/{file_name}_macro_graph.json", 'w', encoding='utf-8') as f:
        json.dump(json_data, f, ensure_ascii=False, indent=2)
    print(f"宏观图已保存：视景数据/{file_name}_macro_graph.pkl/json")
    return True

def load_macro_graph(self, file_name: str) -> bool:
    """加载宏观图数据"""
    try:
        with open(f"视景数据/{file_name}_macro_graph.pkl", 'rb') as f:
            macro_graph_data = pickle.load(f)
        # 重构宏观图
        self.macro_graph = nx.Graph()
        self.macro_graph.add_nodes_from(macro_graph_data['nodes'].items())
        self.macro_graph.add_edges_from([(u, v, data) for (u, v), data in
macro_graph_data['edges'].items()])
        self.macro_edges_paths = macro_graph_data['macro_edges_paths']
        # 加载出入口节点
        self.entrance_nodes = macro_graph_data.get('entrance_nodes', [])
        print(f"宏观图已加载：{self.macro_graph.number_of_nodes()} 个节点，
{self.macro_graph.number_of_edges()} 条边")
        print(f"出入口节点：{len(self.entrance_nodes)} 个：{self.entrance_nodes}")
        return True
    except Exception as e:
        print(f"加载宏观图失败：{e}")
        return False

def main(file_name):
    """主函数"""
    # 创建宏观图构建器
    builder = MacroGraphBuilder()
    # 加载数据
    print("=== 加载数据 ===")
    if not builder.load_data(file_name):

```

```

        print("数据加载失败")
        return
    # 检查是否已有宏观图文件
    macro_graph_file = f"视景数据/{file_name}_macro_graph.pkl"
    if os.path.exists(macro_graph_file):
        print("=== 加载已有宏观图 ===")
        builder.load_macro_graph(file_name)
    else:
        print("=== 构建新宏观图 ===")
        builder.build_macro_graph()
        builder.save_macro_graph(file_name)
    # 导出宏观边数据到 Excel <-- 添加这部分
    print("=== 导出宏观边数据 ===")
    builder.export_macro_edges_to_excel(file_name, file_name)
    # 可视化宏观图
    print("=== 可视化宏观图 ===")
    builder.visualize_macro_graph(file_name, f"{file_name}_宏观图可视化.png")
    # 可视化异景程度热力图 - 分开成两张独立的图
    print("=== 可视化微观图异景程度热力图 ===")
    builder.visualize_micro_anomaly_heatmap(file_name, f"{file_name}")
    print("=== 可视化宏观图异景程度热力图 ===")
    builder.visualize_macro_anomaly_heatmap(file_name, f"{file_name}")
    print("宏观图构建和可视化完成! ")
if __name__ == "__main__":
    # 使用示例 ["寄畅园", "留园", "耦园", "绮园", "秋霞圃", "沈园", "怡园", "豫园", "瞻园", "拙政园"]
    file_names = ["近园"]
    for file_name in file_names:
        main(file_name)

```

## multi\_objective\_path\_planner.py

```

"""
多目标游园路径规划器 - 基于宏观图的游园路径多目标优化
优化: 简化的宏观图
产出:
f"视景数据/{file_name}_simple_macro_graph.pkl"
f"视景数据/{file_name}_optimal_paths.pkl"
"""

import numpy as np
import pandas as pd
import pickle
import json
import matplotlib.pyplot as plt
import plotly.tools as tls
import networkx as nx
from collections import defaultdict, deque
import os
import random
from typing import Dict, List, Tuple, Optional, Any, Set
import warnings
warnings.filterwarnings('ignore')
# 多目标优化库
from deap import base, creator, tools, algorithms
# 继承宏观图构建器
from macro_graph_builder import MacroGraphBuilder
# 设置中文字体
plt.rcParams['font.family'] = ['simhei']

```

```

plt.rcParams['axes.unicode_minus'] = False
class MultiObjectivePathPlanner(MacroGraphBuilder):
    """多目标游园路径规划器"""
    def __init__(self):
        """初始化规划器"""
        super().__init__()
        self.entrance_nodes = [] # 出入口节点列表
        self.optimal_paths = [] # 优化得到的路径列表
        self.path_evaluations = [] # 路径评估结果
        self.normalized_scores = {} # 归一化分数
        # 多目标优化参数
        self.population_size = 500 # 种群大小
        self.generations = 50 # 迭代次数
        self.crossover_prob = 0.7
        self.mutation_prob = 0.5

    def _normalize_anomaly_scores(self):
        """归一化异景程度分数"""
        if not self.macro_graph:
            return
        # 收集所有节点和边的异景程度
        node_temporal = []
        node_spatial = []
        edge_temporal = []
        edge_spatial = []
        # 节点异景程度
        for node, data in self.macro_graph.nodes(data=True):
            node_temporal.append(data.get('temporal_anomaly', 0))
            node_spatial.append(data.get('spatial_anomaly', 0))
        # 边异景程度
        for u, v, data in self.macro_graph.edges(data=True):
            edge_temporal.append(data.get('temporal_anomaly_sum', 0))
            edge_spatial.append(data.get('spatial_anomaly_sum', 0))
        # 计算归一化参数
        self.normalized_scores = {
            'node_temporal_max': max(node_temporal) if node_temporal else 1,
            'node_spatial_max': max(node_spatial) if node_spatial else 1,
            'edge_temporal_max': max(edge_temporal) if edge_temporal else 1,
            'edge_spatial_max': max(edge_spatial) if edge_spatial else 1
        }
        print(f"异景程度归一化参数: {self.normalized_scores}")
    def _calculate_path_anomaly_score(self, path: List[int]) -> float:
        """
        计算路径的总异景程度（归一化后）

        Args:
            path: 节点路径列表

        Returns:
            float: 总异景程度分数
        """
        if len(path) < 2:
            return 0
        total_score = 0
        visited_nodes = set()
        visited_edges = set()
        # 节点异景程度（不重复计算）
        for node in path:
            if node not in visited_nodes:
                node_data = self.macro_graph.nodes[node]

```

```

        temporal = node_data.get('temporal_anomaly', 0) /
self.normalized_scores['node_temporal_max']
        spatial = node_data.get('spatial_anomaly', 0) /
self.normalized_scores['node_spatial_max']
        total_score += temporal + spatial
        visited_nodes.add(node)
    # 边异景程度 (不重复计算)
    for i in range(len(path) - 1):
        edge = (min(path[i], path[i+1]), max(path[i], path[i+1]))
        if edge not in visited_edges and self.macro_graph.has_edge(path[i], path[i+1]):
            edge_data = self.macro_graph.edges[path[i], path[i+1]]
            temporal = edge_data.get('temporal_anomaly_sum', 0) /
self.normalized_scores['edge_temporal_max']
            spatial = edge_data.get('spatial_anomaly_sum', 0) /
self.normalized_scores['edge_spatial_max']
            total_score += temporal + spatial
            visited_edges.add(edge)
    return total_score
def _calculate_path_repetition_length(self, path: List[int]) -> float:
    """
    计算路径的重复长度

    Args:
        path: 节点路径列表

    Returns:
        float: 重复路径长度
    """
    if len(path) < 2:
        return 0
    visited_edges = set()
    total_length = 0
    repetition_length = 0
    for i in range(len(path) - 1):
        if self.macro_graph.has_edge(path[i], path[i+1]):
            edge_data = self.macro_graph.edges[path[i], path[i+1]]
            edge_length = edge_data.get('length', 0)
            total_length += edge_length

            edge = (min(path[i], path[i+1]), max(path[i], path[i+1]))
            if edge in visited_edges:
                repetition_length += edge_length
            else:
                visited_edges.add(edge)
    return repetition_length
def _extend_path_randomly(self, base_path: List[int]) -> List[int]:
    """随机扩展已有路径, 保持起点和终点不变"""
    if len(base_path) < 3:
        return base_path
    extended_path = base_path.copy()
    start_node = base_path[0]
    end_node = base_path[-1]
    # 增加多次插入绕行的机会
    num_detours = random.randint(1, 3) # 1-3 次绕行
    for _ in range(num_detours):
        if len(extended_path) > 100: # 防止路径过长
            break
        # 在中间某个位置插入绕行
        insert_pos = random.randint(1, len(extended_path) - 2)
        current_node = extended_path[insert_pos]

```

```

        # 进行更长距离的随机游走
        detour = [current_node]
        for step in range(random.randint(3, 12)): # 从 2-5 增加到 3-12
            neighbors = list(self.macro_graph.neighbors(current_node))
            if neighbors:
                # 避免立即返回, 但允许更远的探索
                if len(detour) > 2:
                    neighbors = [n for n in neighbors if n != detour[-2]]
                if neighbors:
                    # 增加随机性, 有时选择远离目标的路径
                    if random.random() < 0.3: # 30%概率随机选择
                        current_node = random.choice(neighbors)
                    else:
                        # 选择度数较高的节点 (可能有更多探索机会)
                        neighbors_with_degree = [(n, self.macro_graph.degree[n]) for n in
neighbors]

                        neighbors_with_degree.sort(key=lambda x: x[1], reverse=True)
                        current_node = neighbors_with_degree[0][0]
                        detour.append(current_node)
                # 将绕行插入原路径
                extended_path[insert_pos:insert_pos+1] = detour
        return extended_path
def _generate_prior_paths(self) -> List[List[int]]:
    """生成先验路径 (节点序列格式)"""
    # 保持原函数不变, 返回节点序列
    # 在 create_individual 中通过 _encode_path 转换
    prior_paths = []
    for i, start_entrance in enumerate(self.entrance_nodes):
        for j, end_entrance in enumerate(self.entrance_nodes):
            if i != j:
                try:
                    shortest_path = nx.shortest_path(self.macro_graph, start_entrance,
end_entrance)

                    if len(shortest_path) >= 2:
                        prior_paths.append(shortest_path)
                except nx.NetworkXNoPath:
                    continue
                # 生成更长路径
                try:
                    all_simple_paths = list(nx.all_simple_paths(
                        self.macro_graph, start_entrance, end_entrance, cutoff=20))
                    if all_simple_paths:
                        paths_by_length = {}
                        for path in all_simple_paths:
                            length = len(path)
                            if length not in paths_by_length or len(paths_by_length) < 3:
                                paths_by_length[length] = path

                            for path in paths_by_length.values():
                                if path not in prior_paths:
                                    prior_paths.append(path)
                except:
                    continue
    print(f"生成了 {len(prior_paths)} 条先验路径")
    # 存储先验路径供 create_individual 使用
    self.prior_paths = prior_paths
    return prior_paths
def _decode_individual(self, individual: List) -> List[int]:
    """将步骤选择编码解码为节点路径"""
    if len(individual) < 3:

```

```

        return []
    start_node, end_node, step_choices = individual[0], individual[1], individual[2]
    path = [start_node]
    current_node = start_node
    for choice_idx in step_choices:
        neighbors = list(self.macro_graph.neighbors(current_node))
        if not neighbors:
            break
        # 确保选择索引有效
        actual_choice = choice_idx % len(neighbors)
        next_node = neighbors[actual_choice]
        path.append(next_node)
        current_node = next_node
        if current_node == end_node:
            break
    # 如果未到达终点, 尝试连接到终点
    if path[-1] != end_node:
        try:
            bridge = nx.shortest_path(self.macro_graph, path[-1], end_node)[1:]
            path.extend(bridge)
        except nx.NetworkXNoPath:
            pass
    return path

def _encode_path(self, path: List[int]) -> List:
    """将节点路径编码为步骤选择"""
    if len(path) < 2:
        return [path[0], path[0], []]
    start_node, end_node = path[0], path[-1]
    step_choices = []
    for i in range(len(path) - 1):
        current_node = path[i]
        next_node = path[i + 1]
        neighbors = list(self.macro_graph.neighbors(current_node))
        if next_node in neighbors:
            choice_idx = neighbors.index(next_node)
            step_choices.append(choice_idx)
        else:
            # 路径不连通, 无法编码
            step_choices.append(0) # 默认选择第一个邻居
    return [start_node, end_node, step_choices]

def _setup_deap_environment(self):
    """设置 DEAP 多目标优化环境"""
    # 创建适应度类 (最大化异景程度, 最小化重复长度)
    creator.create("FitnessMulti", base.Fitness, weights=(1.0, -1.0))
    creator.create("Individual", list, fitness=creator.FitnessMulti)
    toolbox = base.Toolbox()
    # 注册个体生成函数
    def create_individual():
        """生成基于步骤选择的个体编码"""
        # 降低先验路径使用比例, 增加随机探索
        if random.random() < 0.4 and hasattr(self, 'prior_paths') and self.prior_paths:
            # 使用先验路径, 需要编码转换
            base_path = random.choice(self.prior_paths).copy()
            # 扩展路径 (可选)
            if random.random() < 0.8 and len(base_path) < 50:
                extended_path = self._extend_path_randomly(base_path)
                encoded = self._encode_path(extended_path)
            else:
                encoded = self._encode_path(base_path)
        return creator.Individual(encoded)
    toolbox.register("create_individual", create_individual)

```

```

else:
    # 直接生成步骤选择编码
    start_node = random.choice(self.entrance_nodes)
    available_ends = [n for n in self.entrance_nodes if n != start_node]
    end_node = random.choice(available_ends) if available_ends else start_node
    # 生成步骤选择序列
    max_steps = random.randint(10, 60)
    step_choices = []
    current_node = start_node
    visited_recently = deque(maxlen=3)
    for step in range(max_steps):
        neighbors = list(self.macro_graph.neighbors(current_node))
        if not neighbors:
            break
        # 移除最近访问的节点
        available_neighbors = [n for n in neighbors if n not in visited_recently]
        if not available_neighbors:
            available_neighbors = neighbors
        # 接近目标时有概率直达
        if (step > 5 and end_node in available_neighbors and
            random.random() < min(0.1 + step * 0.02, 0.8)):
            choice_idx = available_neighbors.index(end_node)
            step_choices.append(choice_idx)
            break
        # 随机选择
        choice_idx = random.randint(0, len(available_neighbors) - 1)
        step_choices.append(choice_idx)
        current_node = available_neighbors[choice_idx]
        visited_recently.append(current_node)
        if current_node == end_node:
            break
    return creator.Individual([start_node, end_node, step_choices])
toolbox.register("individual", create_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
# 注册评估函数
def evaluate(individual):
    """评估基于步骤选择编码的个体"""
    path = self._decode_individual(individual)
    if len(path) < 2:
        return (0, float('inf'))
    anomaly_score = self._calculate_path_anomaly_score(path)
    repetition_length = self._calculate_path_repetition_length(path)
    return (anomaly_score, repetition_length)
toolbox.register("evaluate", evaluate)
# 注册遗传算子
def custom_crossover(ind1, ind2):
    """基于步骤选择的安全交叉"""
    if len(ind1) < 3 or len(ind2) < 3:
        return ind1, ind2
    steps1, steps2 = ind1[2], ind2[2]
    if len(steps1) > 2 and len(steps2) > 2:
        # 交换步骤选择序列的中间段
        cross_point1 = random.randint(1, len(steps1) - 1)
        cross_point2 = random.randint(1, len(steps2) - 1)
        new_steps1 = steps1[:cross_point1] + steps2[cross_point2:]
        new_steps2 = steps2[:cross_point2] + steps1[cross_point1:]
        ind1[2] = new_steps1
        ind2[2] = new_steps2
    return ind1, ind2
def custom_mutation(individual):

```

```

        """基于步骤选择的安全变异"""
        if len(individual) < 3:
            return individual,
        steps = individual[2]
        mutation_type = random.random()
        if mutation_type < 0.4: # 单点变异
            if steps:
                pos = random.randint(0, len(steps) - 1)
                # 随机改变一个选择
                steps[pos] = random.randint(0, 5) # 假设最多6个邻居
            elif mutation_type < 0.7: # 插入新步骤
                if len(steps) < 100: # 限制长度
                    pos = random.randint(0, len(steps))
                    steps.insert(pos, random.randint(0, 5))
            else: # 删除步骤
                if len(steps) > 2:
                    pos = random.randint(0, len(steps) - 1)
                    steps.pop(pos)
            return individual,
    toolbox.register("mate", custom_crossover)
    toolbox.register("mutate", custom_mutation)
    # toolbox.register("select", tools.selNSGA2)
    # 使用更多样化的选择策略
    def diverse_selection(individuals, k):
        """结合 NSGA2 和多样性保持的选择策略"""
        if len(individuals) <= k:
            return individuals
        # 先用 NSGA2 选择
        selected = tools.selNSGA2(individuals, min(k, len(individuals) * 3 // 4))
        # 再用基于距离的多样性选择补充
        if len(selected) < k:
            remaining = [ind for ind in individuals if ind not in selected]
            if remaining:
                # 计算拥挤度距离, 选择多样化的个体
                diverse_selected = tools.selNSGA2(remaining, k - len(selected))
                selected.extend(diverse_selected)
        return selected[:k]
    toolbox.register("select", diverse_selection)
    return toolbox

def optimize_paths(self) -> bool:
    """执行多目标路径优化"""
    # 检查出入口节点是否已加载
    if not hasattr(self, 'entrance_nodes') or not self.entrance_nodes:
        print("出入口节点未加载, 请检查宏观图数据")
        return False
    print("=== 开始多目标路径优化 ===")
    # 归一化异景程度分数
    self._normalize_anomaly_scores()
    # 生成先验路径
    self._generate_prior_paths()
    # 设置 DEAP 环境
    toolbox = self._setup_deap_environment()
    # 创建初始种群
    population = toolbox.population(n=self.population_size)
    # 执行多目标优化
    print(f"运行 NSGA-II 算法: {self.generations}代, 种群大小{self.population_size}")
    # 评估初始种群
    fitnesses = toolbox.map(toolbox.evaluate, population)
    for ind, fit in zip(population, fitnesses):
        ind.fitness.values = fit

```

```

# 进化过程
for generation in range(self.generations):
    # 选择
    offspring = toolbox.select(population, len(population))
    offspring = list(map(toolbox.clone, offspring))
    # 交叉和变异
    for child1, child2 in zip(offspring[::2], offspring[1::2]):
        if random.random() < self.crossover_prob:
            toolbox.mate(child1, child2)
            del child1.fitness.values
            del child2.fitness.values
    for mutant in offspring:
        if random.random() < self.mutation_prob:
            toolbox.mutate(mutant)
            del mutant.fitness.values
    # 评估需要重新评估的个体
    invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
    fitnesses = toolbox.map(toolbox.evaluate, invalid_ind)
    for ind, fit in zip(invalid_ind, fitnesses):
        ind.fitness.values = fit
    # 更新种群
    population = toolbox.select(population + offspring, self.population_size)
    if generation % 10 == 0:
        # 收集当前种群的所有适应度值
        current_fitnesses = [ind.fitness.values for ind in population if
ind.fitness.valid]
        if current_fitnesses:
            # 计算当前最优目标值（异景程度最大，重复长度最小）
            max_anomaly = max(f[0] for f in current_fitnesses)
            min_repetition = min(f[1] for f in current_fitnesses)
            print(f"第 {generation} 代完成 | 当前最优异景程度: {max_anomaly:.4f} | 当前
最优重复长度: {min_repetition:.1f}")
        else:
            print(f"第 {generation} 代完成 | 暂无有效适应度数据")
    # 获取帕累托前沿
    pareto_front = tools.sortNondominated(population, len(population),
first_front_only=True)[0]
    # 保存优化结果
    self.optimal_paths = []
    self.path_evaluations = []
    for individual in pareto_front:
        encoded_individual = list(individual) # 编码格式
        decoded_path = self._decode_individual(individual) # 解码为节点序列
        anomaly_score = individual.fitness.values[0]
        repetition_length = individual.fitness.values[1]
        # 存储解码后的节点序列
        self.optimal_paths.append(decoded_path)
        self.path_evaluations.append({
            'anomaly_score': anomaly_score,
            'repetition_length': repetition_length,
            'path_length': len(decoded_path), # 实际节点数量
            'start_node': decoded_path[0] if decoded_path else None,
            'end_node': decoded_path[-1] if decoded_path else None,
            'encoded_individual': encoded_individual # 保留编码格式供参考
        })
    print(f"优化完成, 获得 {len(self.optimal_paths)} 条帕累托最优路径")
    return True

##### 缩点
##### 缩点
##### 缩点

```

```

def simplify_macro_graph(self, distance_threshold=15000, min_cluster_size=2,
max_cluster_size=8):
    """
    简化宏观图：合并距离相近且连通性强的节点

    Args:
        distance_threshold: 节点合并的最大距离阈值 (mm)
        min_cluster_size: 最小聚类大小
        max_cluster_size: 最大聚类大小，防止过度合并
    """
    if self.macro_graph is None or len(self.macro_graph.nodes) == 0:
        print("宏观图为空，跳过简化")
        return
    print(f"=== 开始图简化 ===")
    original_nodes = len(self.macro_graph.nodes)
    original_edges = len(self.macro_graph.edges)
    # 保护出入口节点，不参与合并
    protected_nodes = set(self.entrance_nodes)
    print(f"保护节点数量: {len(protected_nodes)} (出入口)")
    # 识别可合并的节点簇
    node_clusters = self._identify_node_clusters(distance_threshold, min_cluster_size,
                                                max_cluster_size, protected_nodes)

    if not node_clusters:
        print("未找到可合并的节点簇")
        return
    print(f"识别到 {len(node_clusters)} 个节点簇，准备合并")
    # 执行节点合并
    self._merge_node_clusters(node_clusters)
    # 清理和重新编号
    self._cleanup_graph()
    new_nodes = len(self.macro_graph.nodes)
    new_edges = len(self.macro_graph.edges)
    print(f"图简化完成:")
    print(f"  节点数: {original_nodes} -> {new_nodes} (减少 {original_nodes-new_nodes})")
    print(f"  边数: {original_edges} -> {new_edges} (减少 {original_edges-new_edges})")
    print(f"  简化率: {(original_nodes-new_nodes)/original_nodes*100:.1f}%")

def _identify_node_clusters(self, distance_threshold, min_cluster_size, max_cluster_size,
protected_nodes):
    """识别可合并的节点簇"""
    nodes_to_cluster = [n for n in self.macro_graph.nodes if n not in protected_nodes]
    if len(nodes_to_cluster) < min_cluster_size:
        return []
    # 构建距离矩阵
    node_positions = {}
    for node in nodes_to_cluster:
        node_positions[node] = self.macro_graph.nodes[node]['pos']
    clusters = []
    visited = set()
    for node in nodes_to_cluster:
        if node in visited:
            continue
        # 使用 BFS 找到连通且距离相近的节点
        cluster = self._grow_cluster(node, nodes_to_cluster, node_positions,
                                    distance_threshold, max_cluster_size, visited)
        if len(cluster) >= min_cluster_size:
            clusters.append(cluster)
            visited.update(cluster)
        else:
            visited.add(node)
    return clusters

```

```

def _grow_cluster(self, seed_node, candidate_nodes, positions, distance_threshold,
max_cluster_size, visited):
    """从种子节点开始生长聚类"""
    cluster = [seed_node]
    queue = [seed_node]
    cluster_set = {seed_node}
    while queue and len(cluster) < max_cluster_size:
        current = queue.pop(0)
        current_pos = positions[current]
        # 检查所有未访问的候选节点
        for candidate in candidate_nodes:
            if (candidate in visited or candidate in cluster_set or
                len(cluster) >= max_cluster_size):
                continue
            candidate_pos = positions[candidate]
            distance = np.sqrt((current_pos[0] - candidate_pos[0])**2 +
                               (current_pos[1] - candidate_pos[1])**2)
            # 距离和连通性检查
            if (distance <= distance_threshold and
                self._check_connectivity(current, candidate, cluster)):
                cluster.append(candidate)
                cluster_set.add(candidate)
                queue.append(candidate)
    return cluster

def _check_connectivity(self, node1, node2, existing_cluster):
    """检查节点间的连通性"""
    # 直接相邻
    if self.macro_graph.has_edge(node1, node2):
        return True
    # 与簇中其他节点的共同邻居数
    neighbors1 = set(self.macro_graph.neighbors(node1))
    neighbors2 = set(self.macro_graph.neighbors(node2))
    # 检查是否有共同邻居
    common_neighbors = neighbors1.intersection(neighbors2)
    if len(common_neighbors) >= 1:
        return True
    # 检查是否与簇中已有节点连通
    for existing_node in existing_cluster:
        if self.macro_graph.has_edge(node2, existing_node):
            return True
    return False

def _merge_node_clusters(self, clusters):
    """合并节点簇"""
    node_mapping = {} # 旧节点 -> 新节点的映射
    # 获取当前最大节点 ID, 确保新 ID 不冲突
    max_node_id = max(self.macro_graph.nodes) if self.macro_graph.nodes else 0
    for cluster_idx, cluster in enumerate(clusters):
        if len(cluster) < 2:
            continue
        # 创建新的合并节点 ID - 修改为整数类型
        new_node_id = max_node_id + cluster_idx + 1 # 使用整数 ID
        # 计算合并后的节点属性
        merged_attributes = self._merge_node_attributes(cluster)
        # 添加新节点
        self.macro_graph.add_node(new_node_id, **merged_attributes)
        # 记录映射关系
        for old_node in cluster:
            node_mapping[old_node] = new_node_id
        # 收集所有需要重新连接的边
        edges_to_add = self._collect_merged_edges(cluster, new_node_id)

```

```

        # 添加新边
        for (u, v), edge_attrs in edges_to_add.items():
            if not self.macro_graph.has_edge(u, v):
                self.macro_graph.add_edge(u, v, **edge_attrs)
        # 移除旧节点
        for old_node in cluster:
            if old_node in self.macro_graph:
                self.macro_graph.remove_node(old_node)
        # 更新出入口节点列表（如果有节点被映射）
        self.entrance_nodes = [node_mapping.get(node, node) for node in self.entrance_nodes]
def _merge_node_attributes(self, cluster):
    """合并节点属性"""
    if not cluster:
        return {}
    # 收集所有节点的属性
    all_attrs = []
    total_degree = 0
    for node in cluster:
        attrs = self.macro_graph.nodes[node].copy()
        all_attrs.append(attrs)
        total_degree += attrs.get('degree', 1)
    # 计算位置的加权平均（按度数加权）
    total_weight = 0
    weighted_x = 0
    weighted_y = 0
    for attrs in all_attrs:
        weight = attrs.get('degree', 1)
        pos = attrs['pos']
        weighted_x += pos[0] * weight
        weighted_y += pos[1] * weight
        total_weight += weight
    merged_pos = (weighted_x / total_weight, weighted_y / total_weight)
    # 合并异常程度（取最大值，表示该区域的最高异常水平）
    max_temporal = max(attrs.get('temporal_anomaly', 0) for attrs in all_attrs)
    max_spatial = max(attrs.get('spatial_anomaly', 0) for attrs in all_attrs)
    # 合并其他属性
    merged_attrs = {
        'pos': merged_pos,
        'degree': 0, # 将在重新连接后计算
        'temporal_anomaly': max_temporal,
        'spatial_anomaly': max_spatial,
        'original_nodes': cluster, # 记录原始节点
        'merge_reason': 'proximity_and_connectivity',
        'is_merged': True # 标记为合并节点
    }
    return merged_attrs
def _collect_merged_edges(self, cluster, new_node_id):
    """收集合并后需要的边"""
    edges_to_add = {}
    cluster_set = set(cluster)
    for node in cluster:
        for neighbor in self.macro_graph.neighbors(node):
            if neighbor in cluster_set:
                continue # 跳过簇内边
            # 确定新边的端点
            if neighbor < new_node_id:
                edge_key = (neighbor, new_node_id)
            else:
                edge_key = (new_node_id, neighbor)
            # 合并边属性

```

```

        old_edge_data = self.macro_graph.edges[node, neighbor]
        if edge_key in edges_to_add:
            # 合并多条边的属性
            edges_to_add[edge_key] = self._merge_edge_attributes(
                edges_to_add[edge_key], old_edge_data, new_node_id, neighbor)
        else:
            # 重新计算长度，保留其他属性
            new_length = self._calculate_edge_length(new_node_id, neighbor)
            merged_edge_attrs = old_edge_data.copy()
            merged_edge_attrs['length'] = new_length
            # 确保turning_points属性存在
            if 'turning_points' not in merged_edge_attrs:
                merged_edge_attrs['turning_points'] = 0
            edges_to_add[edge_key] = merged_edge_attrs
    return edges_to_add

def _merge_edge_attributes(self, existing_attrs, new_attrs, node1, node2):
    """合并边属性"""
    # 重新计算长度
    new_length = self._calculate_edge_length(node1, node2)
    # 合并转折点数（取平均值）
    existing_turning_points = existing_attrs.get('turning_points', 0)
    new_turning_points = new_attrs.get('turning_points', 0)
    merged_turning_points = (existing_turning_points + new_turning_points) / 2
    # 异常程度求和（表示该连接的总异常水平）
    merged_attrs = {
        'length': new_length,
        'turning_points': merged_turning_points, # 新增
        'temporal_anomaly_sum': existing_attrs.get('temporal_anomaly_sum', 0) +
            new_attrs.get('temporal_anomaly_sum', 0),
        'spatial_anomaly_sum': existing_attrs.get('spatial_anomaly_sum', 0) +
            new_attrs.get('spatial_anomaly_sum', 0),
        'merged_edges': existing_attrs.get('merged_edges', 1) + 1
    }
    return merged_attrs

def _calculate_edge_length(self, node1, node2):
    """重新计算边长度"""
    pos1 = self.macro_graph.nodes[node1]['pos']
    pos2 = self.macro_graph.nodes[node2]['pos']
    return np.sqrt((pos1[0] - pos2[0])**2 + (pos1[1] - pos2[1])**2)

def _cleanup_graph(self):
    """清理图结构并重新计算度数"""
    # 重新计算所有节点的度数
    for node in self.macro_graph.nodes:
        self.macro_graph.nodes[node]['degree'] = self.macro_graph.degree[node]
    # 只保留仍存在于图中的原有出入口节点，不添加新的
    original_count = len(self.entrance_nodes)
    self.entrance_nodes = [node for node in self.entrance_nodes if node in self.macro_graph]
    print(f"出入口节点: {original_count} -> {len(self.entrance_nodes)}")

def main(file_name):
    """主函数"""
    # 创建多目标路径规划器
    planner = MultiObjectivePathPlanner()
    # 略

if __name__ == "__main__":
    # 使用示例 ["寄畅园", "留园", "耦园", "绮园", "秋霞圃", "沈园", "怡园", "豫园", "瞻园", "拙政园"]
    file_names = ["近园"]
    for file_name in file_names:
        main(file_name)

```

## visibility\_analysis.py

```
'''
视景分析
获取微视图（路网络）中每个节点的视景
'''

# 导入必要的模块
from load_and_visualize import NetworkVisualizer, NetworkLoader
import numpy as np
import pickle
import json
from shapely.geometry import Point, LineString
from rtree import index
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity
from scipy.spatial.distance import hamming
from concurrent.futures import ProcessPoolExecutor, as_completed
from multiprocessing import cpu_count

# 设置中文字体
plt.rcParams['font.family'] = ['simhei']
plt.rcParams['axes.unicode_minus'] = False
def compute_point_visibility_matrix(args):
    """
    独立的进程计算函数

    Args:
        args: (point_id, point, analyzer_params, segments_info_data)

    Returns:
        tuple: (point_id, result_dict)
    """
    point_id, point, analyzer_params, segments_info_data = args
    # 在进程内重建分析器状态
    analyzer = VisibilityAnalyzer(
        angle_step=analyzer_params['angle_step'],
        max_distance=analyzer_params['max_distance']
    )
    # 重建 segments_info 和空间索引
    analyzer.segments_info = segments_info_data
    # 重建空间索引
    from rtree import index
    spatial_idx = index.Index()
    for i, segment_info in enumerate(analyzer.segments_info):
        spatial_idx.insert(i, segment_info['geometry'].bounds)
    analyzer.spatial_index = spatial_idx
    # 计算视景矩阵
    try:
        visibility_matrix = analyzer.compute_visibility_matrix(point)
        return point_id, {
            'point': point,
            'matrix': visibility_matrix
        }
    except Exception as e:
        print(f"进程计算节点 {point_id} 时出错: {e}")
        return point_id, None
class VisibilityAnalyzer:
    """视景分析器"""
```

```

def __init__(self, angle_step=3, max_distance=100000): # 距离单位为 mm
    """
    初始化视景分析器

    Args:
        angle_step: 射线角度间隔 (度)
        max_distance: 最大视景距离 (mm)
    """
    self.angle_step = angle_step
    self.max_distance = max_distance
    self.num_rays = 360 // angle_step
    self.distance_step = 1000 # 每 1 米采样一次
    self.num_distance_points = max_distance // self.distance_step
    # 景观元素编号映射
    self.element_mapping = {
        '被遮挡': 0,
        '半开放建筑': 1,
        '实体建筑': 2,
        '道路': 3,
        '假山': 4,
        '水体': 5,
        '植物': 6
    }
    # 遮挡性元素 (只有实体建筑会完全遮挡视线)
    self.occlusion_elements = {'实体建筑'}
    print(f"初始化视景分析器: {self.num_rays}条射线, 最大距离{max_distance/1000:.1f}m")

def _prepare_process_args(self):
    """准备进程计算所需的参数"""
    analyzer_params = {
        'angle_step': self.angle_step,
        'max_distance': self.max_distance
    }
    # 准备 segments_info 数据 (需要可序列化)
    segments_info_data = []
    for segment_info in self.segments_info:
        segments_info_data.append({
            'id': segment_info['id'],
            'element_type': segment_info['element_type'],
            'geometry': segment_info['geometry'] # Shapely 对象可以被 pickle 序列化
        })
    return analyzer_params, segments_info_data

def build_spatial_index(self, geometry_data):
    """构建空间索引"""
    print("构建空间索引...")
    # 创建 R-tree 空间索引
    spatial_idx = index.Index()
    # 存储线段信息
    self.segments_info = []
    segment_id = 0
    for element_type, data in geometry_data.items():
        if element_type == "植物":
            # 植物处理为圆形边界
            for plant in data:
                center = plant['center']
                radius = plant['radius']
                # 简化为正八边形近似圆
                angles = np.linspace(0, 2*np.pi, 9)[::-1] # 8 个顶点
                vertices = [(center[0] + radius * np.cos(a),
                             center[1] + radius * np.sin(a)) for a in angles]

```

```

        # 创建边界线段
        for i in range(len(vertices)):
            start = vertices[i]
            end = vertices[(i + 1) % len(vertices)]
            line = LineString([start, end])
            # 添加到索引
            spatial_idx.insert(segment_id, line.bounds)
            self.segments_info.append({
                'id': segment_id,
                'element_type': element_type,
                'geometry': line
            })
            segment_id += 1
    else:
        # 处理线段数据
        for segment in data.get('segments', []):
            if len(segment) > 1:
                for i in range(len(segment) - 1):
                    start = segment[i]
                    end = segment[i + 1]
                    line = LineString([start, end])

                    # 添加到索引
                    spatial_idx.insert(segment_id, line.bounds)
                    self.segments_info.append({
                        'id': segment_id,
                        'element_type': element_type,
                        'geometry': line
                    })
                    segment_id += 1
    self.spatial_index = spatial_idx
    print(f"空间索引构建完成: {len(self.segments_info)} 个线段")
    return spatial_idx

def cast_ray(self, observer_point, angle_deg):
    """
    发射射线并检测交点，根据交点进行区域填充

    Args:
        observer_point: 观察点坐标 (x, y)
        angle_deg: 射线角度 (度)

    Returns:
        list: 射线上的景观序列
    """
    angle_rad = np.radians(angle_deg)
    # 射线终点 (稍微延长以确保能找到边界交点)
    extend_distance = self.max_distance * 1.2 # 延长 20%
    end_x = observer_point[0] + extend_distance * np.cos(angle_rad)
    end_y = observer_point[1] + extend_distance * np.sin(angle_rad)
    ray = LineString([observer_point, (end_x, end_y)])
    # 找到所有交点
    intersections = []
    for seg_id in self.spatial_index.intersection(ray.bounds):
        segment_info = self.segments_info[seg_id]
        # 跳过路网络，不对景观序列产生影响
        if segment_info['element_type'] == '路网络':
            continue
        intersection = ray.intersection(segment_info['geometry'])
        if not intersection.is_empty:
            # 处理交点坐标

```

```

        if hasattr(intersection, 'coords'): # Point
            point_coords = list(intersection.coords)[0]
        elif hasattr(intersection, 'geoms'): # MultiPoint - 取最近的点
            points = [list(geom.coords)[0] for geom in intersection.geoms if
hasattr(geom, 'coords')]
            if points:
                # 选择最近的交点
                distances = [np.sqrt((p[0] - observer_point[0])**2 + (p[1] -
observer_point[1])**2) for p in points]
                point_coords = points[np.argmin(distances)]
            else:
                continue
        else:
            continue
        distance = np.sqrt((point_coords[0] - observer_point[0])**2 +
        (point_coords[1] - observer_point[1])**2)
        # 只考虑正方向的交点
        if distance > 1: # 忽略太近的点（避免数值误差）
            intersections.append({
                'distance': distance,
                'element_type': segment_info['element_type'],
                'point': point_coords
            })
    # 按距离排序交点
    intersections.sort(key=lambda x: x['distance'])
    # 构建景观序列
    landscape_sequence = []
    current_distance = 0
    intersection_idx = 0
    current_element = None # 当前区域的景观类型
    is_occluded = False # 是否被遮挡
    while current_distance < self.max_distance:
        # 检查是否到达新的交点
        while (intersection_idx < len(intersections) and
            intersections[intersection_idx]['distance'] <= current_distance):
            element_type = intersections[intersection_idx]['element_type']
            current_element = element_type
            intersection_idx += 1
            # 如果遇到遮挡性元素，在处理完当前点后标记遮挡
            if element_type in self.occlusion_elements:
                is_occluded = True
        # 确定当前距离点的景观类型
        if current_element is not None:
            # 在某个景观区域内（包括实体建筑）
            landscape_element = current_element
        else:
            # 还没有遇到任何边界，需要特殊处理
            future_intersection = None
            for i in range(intersection_idx, len(intersections)):
                if intersections[i]['distance'] <= self.max_distance * 1.2:
                    future_intersection = intersections[i]
                    break
            if future_intersection:
                landscape_element = future_intersection['element_type']
            else:
                landscape_element = '被遮挡'
        # 记录当前距离的景观元素
        landscape_sequence.append(self.element_mapping.get(landscape_element, 0))
        current_distance += self.distance_step
        # 如果已经被遮挡，从下一个点开始填充"被遮挡"

```

```

        if is_occluded:
            remaining_points = int((self.max_distance - current_distance) //
self.distance_step)
            landscape_sequence.extend([self.element_mapping['被遮挡']] *
remaining_points)
            break
        # 确保序列长度正确
        while len(landscape_sequence) < self.num_distance_points:
            landscape_sequence.append(landscape_sequence[-1] if landscape_sequence else 0)
        return landscape_sequence[:self.num_distance_points]
def compute_visibility_matrix(self, observer_point):
    """
    计算观察点的视景矩阵

    Args:
        observer_point: 观察点坐标 (x, y)

    Returns:
        np.array: 视景矩阵 (num_rays, num_distance_points)
    """
    visibility_matrix = []
    for i in range(self.num_rays):
        angle = i * self.angle_step
        landscape_sequence = self.cast_ray(observer_point, angle)
        # 确保序列长度一致
        if len(landscape_sequence) < self.num_distance_points:
            # 用最后一个元素填充
            last_element = landscape_sequence[-1] if landscape_sequence else 0
            landscape_sequence.extend([last_element] *
                                      (self.num_distance_points - len(landscape_sequence)))
        elif len(landscape_sequence) > self.num_distance_points:
            landscape_sequence = landscape_sequence[:self.num_distance_points]
        visibility_matrix.append(landscape_sequence)
    return np.array(visibility_matrix)
def _compute_single_point_matrix(self, point_info):
    """
    计算单个点的视景矩阵（线程安全版本）

    Args:
        point_info: (point_id, point_coordinates)

    Returns:
        tuple: (point_id, result_dict)
    """
    point_id, point = point_info
    visibility_matrix = self.compute_visibility_matrix(point)
    return point_id, {
        'point': point,
        'matrix': visibility_matrix
    }
def compute_all_visibility_matrices(self, network_points):
    """
    计算所有路网络节点的视景矩阵

    Args:
        network_points: 路网络节点列表 [(x, y), ...]

    Returns:
        dict: {point_id: visibility_matrix}
    """

```

```

print(f"计算 {len(network_points)} 个节点的视景矩阵...")

visibility_matrices = {}

for i, point in enumerate(network_points):
    if i % 10 == 0:
        print(f"进度: {i+1}/{len(network_points)}")

        visibility_matrix = self.compute_visibility_matrix(point)
        visibility_matrices[i] = {
            'point': point,
            'matrix': visibility_matrix
        }
print("视景矩阵计算完成")
return visibility_matrices

def compute_all_visibility_matrices_bingfa(self, network_points, max_workers=None):
    """
    计算所有路网络节点的视景矩阵（进程池版本）

    Args:
        network_points: 路网络节点列表 [(x, y), ...]
        max_workers: 最大工作进程数，默认为 None（自动选择）

    Returns:
        dict: {point_id: visibility_matrix}
    """
    print(f"计算 {len(network_points)} 个节点的视景矩阵...")

    # 如果未指定进程数，使用 CPU 核心数
    if max_workers is None:
        max_workers = min(cpu_count(), len(network_points))
    print(f"使用 {max_workers} 个并发进程")
    # 准备进程计算参数
    analyzer_params, segments_info_data = self._prepare_process_args()
    # 准备任务参数
    task_args = []
    for i, point in enumerate(network_points):
        task_args.append((i, point, analyzer_params, segments_info_data))
    visibility_matrices = {}
    completed_count = 0
    # 使用进程池并发处理
    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        # 提交所有任务
        future_to_point = {
            executor.submit(compute_point_visibility_matrix, args): args[0]
            for args in task_args
        }
        # 处理完成的任务
        for future in as_completed(future_to_point):
            point_id = future_to_point[future]
            try:
                result_point_id, result = future.result()
                if result is not None:
                    visibility_matrices[result_point_id] = result
                    completed_count += 1
            # 显示进度
            if completed_count % 5 == 0 or completed_count == len(network_points):
                print(f"进度: {completed_count}/{len(network_points)}")
            except Exception as e:

```

```

        print(f"处理节点 {point_id} 的结果时出错: {e}")
    # 显式关闭执行器
    executor.shutdown(wait=True)
    print("视景矩阵计算完成")
    return visibility_matrices

def compute_landscape_transition_probabilities(self, visibility_matrices,
network_connections):
    """
    计算景观类别转移概率矩阵（马尔可夫）

    Args:
        visibility_matrices: 视景矩阵字典
        network_connections: 网络连接关系 [(point_id1, point_id2), ...]

    Returns:
        np.array: 转移概率矩阵 (7, 7)
    """
    print("计算景观转移概率...")
    # 统计转移次数
    transition_counts = np.zeros((7, 7)) # 7 种景观类型
    for point_id1, point_id2 in network_connections:
        if point_id1 in visibility_matrices and point_id2 in visibility_matrices:
            matrix1 = visibility_matrices[point_id1]['matrix']
            matrix2 = visibility_matrices[point_id2]['matrix']
            # 统计每个位置的景观转移
            for i in range(matrix1.shape[0]):
                for j in range(matrix1.shape[1]):
                    from_element = matrix1[i, j]
                    to_element = matrix2[i, j]
                    transition_counts[from_element, to_element] += 1
    # 转换为概率
    row_sums = transition_counts.sum(axis=1)
    transition_probs = np.divide(transition_counts, row_sums.reshape(-1, 1),
                                out=np.zeros_like(transition_counts),
                                where=row_sums.reshape(-1, 1) != 0)

    return transition_probs

def compute_anomaly_score(self, point_id, visibility_matrices, network_connections,
transition_probs):
    """
    计算单点异景程度：该点与直接相连接点的视景矩阵差异度量
    """
    if point_id not in visibility_matrices:
        return {'temporal_anomaly': 0, 'spatial_anomaly': 0}
    current_matrix = visibility_matrices[point_id]['matrix']
    # 找到所有直接相邻的点
    neighbor_ids = []
    for conn in network_connections:
        if conn[0] == point_id and conn[1] in visibility_matrices:
            neighbor_ids.append(conn[1])
        elif conn[1] == point_id and conn[0] in visibility_matrices:
            neighbor_ids.append(conn[0])
    if not neighbor_ids:
        return {'temporal_anomaly': 0, 'spatial_anomaly': 0}
    temporal_anomalies = []
    spatial_anomalies = []
    for neighbor_id in neighbor_ids:
        neighbor_matrix = visibility_matrices[neighbor_id]['matrix']
        # 1. 时间维度异景度：马尔可夫转移的意外性
        # 把从当前点移动到邻居点看作时间序列中的状态转移
        temporal_anomaly = 0

```

```

valid_transitions = 0
# 对每个角度-距离位置，计算景观类别转移的意外性
for i in range(current_matrix.shape[0]): # 角度
    for j in range(current_matrix.shape[1]): # 距离
        from_element = current_matrix[i, j] # 当前点的景观
        to_element = neighbor_matrix[i, j] # 邻居点的景观
        # 查询转移概率
        prob = transition_probs[from_element, to_element]
        if prob > 0:
            # 低概率转移 = 高异景程度（信息熵）
            temporal_anomaly += -np.log(prob)
            valid_transitions += 1
        elif from_element != to_element:
            # 如果转移概率为0但景观发生变化，视为极高异景
            temporal_anomaly += 10 # 惩罚项
            valid_transitions += 1
    if valid_transitions > 0:
        temporal_anomaly /= valid_transitions
temporal_anomalies.append(temporal_anomaly)
# 2. 空间维度异景度：视景矩阵结构相似性
# 计算两个视景矩阵的结构差异
# 汉明距离（适合离散分类数据）
current_flat = current_matrix.flatten()
neighbor_flat = neighbor_matrix.flatten()
hamming_dist = np.mean(current_flat != neighbor_flat)
spatial_anomalies.append(hamming_dist)
# 计算平均异景度
temporal_anomaly = np.mean(temporal_anomalies) if temporal_anomalies else 0
spatial_anomaly = np.mean(spatial_anomalies) if spatial_anomalies else 0
return {
    'temporal_anomaly': float(temporal_anomaly),
    'spatial_anomaly': float(spatial_anomaly),
}
def save_results(self, visibility_matrices, anomaly_scores, transition_probs,
                 filename_prefix):
    """保存计算结果"""
    print(f"保存结果到 {filename_prefix}...")
    # 保存视景矩阵
    with open(f"{filename_prefix}_visibility_matrices.pkl", 'wb') as f:
        pickle.dump(visibility_matrices, f)
    # 保存异景程度
    with open(f"{filename_prefix}_anomaly_scores.json", 'w', encoding='utf-8') as f:
        json.dump(anomaly_scores, f, ensure_ascii=False, indent=2)
    # 保存转移概率矩阵
    np.save(f"{filename_prefix}_transition_probs.npy", transition_probs)
    # 保存分析参数
    params = {
        'angle_step': self.angle_step,
        'max_distance': self.max_distance,
        'distance_step': self.distance_step,
        'num_rays': self.num_rays,
        'num_distance_points': self.num_distance_points,
        'element_mapping': self.element_mapping
    }
    with open(f"{filename_prefix}_params.json", 'w', encoding='utf-8') as f:
        json.dump(params, f, ensure_ascii=False, indent=2)
    print("结果保存完成")
def extract_network_connections(geometry_data):
    """从几何数据中提取网络连接关系"""
    connections = []

```

```

if "路网络" not in geometry_data:
    return connections
network_data = geometry_data["路网络"]
# 从 segments 中提取连接关系
point_to_id = {}
point_id = 0
# 为所有点分配 ID
for segment in network_data.get('segments', []):
    for point in segment:
        point_key = (round(point[0], 1), round(point[1], 1)) # 四舍五入避免浮点精度问题
        if point_key not in point_to_id:
            point_to_id[point_key] = point_id
            point_id += 1
# 提取连接关系
for segment in network_data.get('segments', []):
    for i in range(len(segment) - 1):
        point1 = (round(segment[i][0], 1), round(segment[i][1], 1))
        point2 = (round(segment[i+1][0], 1), round(segment[i+1][1], 1))
        id1 = point_to_id[point1]
        id2 = point_to_id[point2]
        connections.append((id1, id2))
return connections, list(point_to_id.keys())
def main(file_name):
    """主函数"""
    # 加载数据
    loader = NetworkLoader()
    original_excel = f"shujv/4-{file_name}数据坐标.xlsx"
    network_excel = f"{file_name}-路网络.xlsx"
    print("加载数据...")
    geometry_data = loader.load_original_data(original_excel)
    network_data = loader.load_network_data(network_excel)
    if geometry_data is None or network_data is None:
        print("数据加载失败")
        return
    geometry_data["路网络"] = network_data
    print("构建路网络...")
    # 提取网络连接和节点
    # connections: 列表, 其中为元组 (A, B); network_points_keys: 列表, 点 ID
    connections, network_points_keys = extract_network_connections(geometry_data)
    network_points = [point for point in network_points_keys]
    print(f"网络节点数: {len(network_points)}")
    print(f"网络连接数: {len(connections)}")
    print(f"=== 视景分析: {file_name} ===")
    # 创建视景分析器
    analyzer = VisibilityAnalyzer(angle_step=3, max_distance=100000) # 100 米
    # 构建空间索引
    analyzer.build_spatial_index(geometry_data)
    # 只处理前 5 个点进行测试
    ...
    test_points = network_points[:5]
    print(f"测试模式: 只处理前 {len(test_points)} 个点")
    visibility_matrices = analyzer.compute_all_visibility_matrices_bingfa(test_points,
max_workers=5)
    # 相应地, connections 也需要过滤, 只保留涉及这些点的连接
    test_connections = []
    for conn in connections:
        if conn[0] < len(test_points) and conn[1] < len(test_points):
            test_connections.append(conn)
    connections = test_connections
    ...

```

```

# 计算视景矩阵（字典 {point_id: visibility_matrix} ）
#visibility_matrices = analyzer.compute_all_visibility_matrices(network_points)
# 并发版本
visibility_matrices = analyzer.compute_all_visibility_matrices_bingfa(network_points,
max_workers=8)
# 计算转移概率
transition_probs = analyzer.compute_landscape_transition_probabilities(
    visibility_matrices, connections)
# 逐个点 计算异景程度（字典，键为点 ID，值为异景分数）
anomaly_scores = {}
for point_id in visibility_matrices.keys():
    anomaly_scores[point_id] = analyzer.compute_anomaly_score(
        point_id, visibility_matrices, connections, transition_probs)
# 保存结果
analyzer.save_results(visibility_matrices, anomaly_scores, transition_probs,
    f"{file_name}_visibility_analysis")
# 可视化示例（第一个节点）
if visibility_matrices:
    first_point_id = list(visibility_matrices.keys())[0]
    analyzer.visualize_visibility_matrix(
        first_point_id, visibility_matrices,
        f"{file_name}_visibility_matrix_example.png")
return visibility_matrices, anomaly_scores, transition_probs
if __name__ == "__main__":
    # 使用示例 拙政园
    file_names = ["近园"]
    for file_name in file_names:
        visibility_matrices, anomaly_scores, transition_probs = main(file_name)

```

## voronoi\_path\_builder.py

```

'''
Voronoi 路径网络
构建、筛选边与子图、保存
'''

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon, Circle
from shapely.geometry import Point, LineString, Polygon as ShapelyPolygon
from rtree import index
import networkx as nx
from scipy.spatial import Voronoi
import copy
class SimpleVoronoiPathBuilder:
    """简化的 Voronoi 路径网络构建器"""
    def __init__(self, geometry_data):
        """
        初始化路径构建器
        Args:
            geometry_data: 几何数据字典
        """
        self.geometry_data = geometry_data
        self.obstacle_types = ["半开放建筑", "实体建筑", "道路", "障碍物连接"] # 影响通行的障碍
        self.boundary_points = []
        self.obstacle_segments = [] # 已有
        self.obstacle_lines = [] # 新增：预处理的障碍物线段

```

```

self.voronoi = None
self.voronoi_edges = []
self.path_network = None # 路网络的完整路径
self.pruned_network = None # 路网络的剪枝路径
def preprocess_obstacle_lines(self):
    """预处理障碍物线段为 shapely LineString 对象, 用于相交检测"""
    print("正在预处理障碍物线段...")
    self.obstacle_lines = []
    for segment in self.obstacle_segments:
        if len(segment) >= 2:
            # 将每个 segment 转换为多条 LineString
            for i in range(len(segment) - 1):
                try:
                    start_point = segment[i]
                    end_point = segment[i + 1]
                    if (start_point != end_point and
                        not any(np.isnan([start_point[0], start_point[1], end_point[0],
end_point[1]])) and
                        not any(np.isinf([start_point[0], start_point[1], end_point[0],
end_point[1]]))):
                        line = LineString([start_point, end_point])
                        if line.length > 1e-6: # 确保线段有实际长度
                            self.obstacle_lines.append(line)
                except Exception as e:
                    continue
    print(f"预处理完成, 共 {len(self.obstacle_lines)} 条有效障碍物线段")
    return self.obstacle_lines
def collect_boundary_points(self):
    """收集障碍物边界点, 并添加外边界"""
    print("正在收集边界点...")
    all_points = []
    all_segments = []
    for obstacle_type in self.obstacle_types:
        if obstacle_type in self.geometry_data:
            data = self.geometry_data[obstacle_type]
            # 收集 segments 中的点
            for segment in data.get('segments', []):
                all_points.extend(segment)
                all_segments.append(segment)
            # 收集 all_points
            all_points.extend(data.get('all_points', []))
    # 去重
    unique_points = []
    for point in all_points:
        if point not in unique_points:
            unique_points.append(point)
    # 添加外边界点来约束 Voronoi 图
    if unique_points:
        x_coords = [p[0] for p in unique_points]
        y_coords = [p[1] for p in unique_points]
        # 计算边界范围, 并扩展一些边距
        margin = max(max(x_coords) - min(x_coords), max(y_coords) - min(y_coords)) * 0.1
        x_min, x_max = min(x_coords) - margin, max(x_coords) + margin
        y_min, y_max = min(y_coords) - margin, max(y_coords) + margin
        # 添加边界点 (四个角和边上的一些点)
        boundary_density = 10 # 每条边上的点数
        # 添加四个角
        boundary_points = [
            (x_min, y_min), (x_max, y_min),

```

```

        (x_max, y_max), (x_min, y_max)
    ]
    # 添加边上的点
    for i in range(1, boundary_density):
        t = i / boundary_density
        # 下边
        boundary_points.append((x_min + t * (x_max - x_min), y_min))
        # 上边
        boundary_points.append((x_min + t * (x_max - x_min), y_max))
        # 左边
        boundary_points.append((x_min, y_min + t * (y_max - y_min)))
        # 右边
        boundary_points.append((x_max, y_min + t * (y_max - y_min)))
    unique_points.extend(boundary_points)
    self.boundary_points = unique_points
    self.obstacle_segments = all_segments
    print(f"收集到 {len(self.boundary_points)} 个边界点 (包含外边界)")
    print(f"收集到 {len(self.obstacle_segments)} 个障碍物线段")
    return self.boundary_points

def generate_voronoi_diagram(self):
    """生成 Voronoi 图"""
    if len(self.boundary_points) < 4:
        print(f"边界点太少({len(self.boundary_points)}个), 无法生成 Voronoi 图")
        return None
    print("正在生成 Voronoi 图...")
    try:
        # 转换为 numpy 数组
        points_array = np.array(self.boundary_points)
        # 生成 Voronoi 图
        self.voronoi = Voronoi(points_array)
        print(f"生成了 {len(self.voronoi.vertices)} 个 Voronoi 顶点")
        print(f"生成了 {len(self.voronoi.ridge_vertices)} 条 Voronoi 边")
        return self.voronoi
    except Exception as e:
        print(f"生成 Voronoi 图失败: {e}")
        return None

def extract_voronoi_edges(self):
    """提取 Voronoi 边作为路径"""
    if self.voronoi is None:
        print("请先生成 Voronoi 图")
        return []
    print("正在提取 Voronoi 边...")
    edges = []
    for ridge_vertices in self.voronoi.ridge_vertices:
        # 跳过无限边 (包含 -1 的边)
        if -1 in ridge_vertices:
            continue
        # 获取边的两个端点
        vertex1 = self.voronoi.vertices[ridge_vertices[0]]
        vertex2 = self.voronoi.vertices[ridge_vertices[1]]
        # 简单有效性检查
        if (not np.isnan(vertex1).any() and not np.isnan(vertex2).any() and
            not np.isinf(vertex1).any() and not np.isinf(vertex2).any()):
            edges.append((tuple(vertex1), tuple(vertex2)))
    self.voronoi_edges = edges
    print(f"提取到 {len(edges)} 条有效的 Voronoi 边")
    return edges

def create_spatial_index(self):
    """为障碍物线段创建空间索引以加速相交检测 (需要安装 rtree 库)"""
    # 创建空间索引

```

```

idx = index.Index()
for i, line in enumerate(self.obstacle_lines):
    # 获取线段的边界框
    bounds = line.bounds # (minx, miny, maxx, maxy)
    idx.insert(i, bounds)
self.spatial_index = idx
print(f"创建空间索引完成, 包含 {len(self.obstacle_lines)} 条线段")
return True

def filter_voronoi_edges_with_spatial_index(self, max_edge_length=None,
intersection_tolerance=1e-6):
    """使用空间索引优化的边过滤方法"""
    if not self.voronoi_edges:
        return []
    print("正在过滤 Voronoi 边 (使用空间索引) ...")
    if max_edge_length is None:
        all_x = [p[0] for p in self.boundary_points]
        all_y = [p[1] for p in self.boundary_points]
        diagonal = np.sqrt((max(all_x) - min(all_x))**2 + (max(all_y) - min(all_y))**2)
        max_edge_length = diagonal * 0.05
    # 预处理和创建索引
    if not self.obstacle_lines:
        self.preprocess_obstacle_lines()
    use_spatial_index = self.create_spatial_index()
    filtered_edges = []
    length_filtered = 0
    intersection_filtered = 0
    for i, edge in enumerate(self.voronoi_edges):
        start, end = edge
        # 基本检查
        if (any(np.isnan([start[0], start[1], end[0], end[1]])) or
            any(np.isinf([start[0], start[1], end[0], end[1]]))):
            continue
        # 长度检查
        edge_length = np.sqrt((start[0] - end[0])**2 + (start[1] - end[1])**2)
        if edge_length > max_edge_length:
            length_filtered += 1
            continue
        # 相交检查
        intersects_obstacle = False
        try:
            voronoi_line = LineString([start, end])
            if voronoi_line.length < intersection_tolerance:
                continue
            # 使用空间索引或普通遍历
            if use_spatial_index and self.spatial_index:
                # 获取可能相交的线段索引
                bounds = voronoi_line.bounds
                possible_matches = list(self.spatial_index.intersection(bounds))
                candidate_lines = [self.obstacle_lines[j] for j in possible_matches]
            else:
                candidate_lines = self.obstacle_lines
            # 检查相交
            for obstacle_line in candidate_lines:
                if voronoi_line.intersects(obstacle_line):
                    intersection = voronoi_line.intersection(obstacle_line)
                    if hasattr(intersection, 'geom_type'):
                        if intersection.geom_type in ['LineString', 'MultiLineString']:
                            intersects_obstacle = True
                            break
                        elif intersection.geom_type == 'Point':

```

```

        point_coords = (intersection.x, intersection.y)
        start_dist = np.sqrt((point_coords[0] - start[0])**2 +
(point_coords[1] - start[1])**2)
        end_dist = np.sqrt((point_coords[0] - end[0])**2 +
(point_coords[1] - end[1])**2)
        if (start_dist > intersection_tolerance and
            end_dist > intersection_tolerance):
            intersects_obstacle = True
            break
    if intersects_obstacle:
        intersection_filtered += 1
    else:
        filtered_edges.append(edge)
except Exception as e:
    intersection_filtered += 1
    continue
if (i + 1) % 1000 == 0:
    print(f"已处理 {i + 1}/{len(self.voronoi_edges)} 条边...")
self.voronoi_edges = filtered_edges
return filtered_edges
def analyze_connected_components(self):
    """分析连通分量并只保留第 1 大的子图"""
    if not self.voronoi_edges:
        print("没有可用的边来分析连通分量")
        return None
    print("正在分析连通分量...")
    # 构建临时网络图用于连通性分析
    temp_G = nx.Graph()
    for edge in self.voronoi_edges:
        start_node = edge[0]
        end_node = edge[1]
        weight = np.sqrt((start_node[0] - end_node[0])**2 + (start_node[1] -
end_node[1])**2)
        temp_G.add_edge(start_node, end_node, weight=weight)
    # 获取所有连通分量
    connected_components = list(nx.connected_components(temp_G))
    print(f"发现 {len(connected_components)} 个连通分量")
    # 计算每个连通分量的统计信息
    component_stats = []
    for i, component in enumerate(connected_components):
        subgraph = temp_G.subgraph(component)
        num_nodes = subgraph.number_of_nodes()
        num_edges = subgraph.number_of_edges()
        # 计算综合得分 (边数+点数)
        score = num_edges + num_nodes
        component_stats.append({
            'index': i,
            'nodes': num_nodes,
            'edges': num_edges,
            'score': score,
            'component': component,
            'subgraph': subgraph
        })
    # 按综合得分排序, 只保留第 1 大
    component_stats.sort(key=lambda x: x['score'], reverse=True)
    largest_component = component_stats[0] if component_stats else None
    if largest_component:
        print(f"保留最大连通分量: {largest_component['nodes']}个节点,
{largest_component['edges']}条边 (得分: {largest_component['score']})")
    # 提取最大子图的边

```

```

        self.filtered_edges_by_component = []
        self.component_info = [largest_component]
        component_edges = []
        subgraph = largest_component['subgraph']
        for edge in subgraph.edges():
            # 查找原始边信息
            for original_edge in self.voronoi_edges:
                if ((original_edge[0] == edge[0] and original_edge[1] == edge[1]) or
                    (original_edge[0] == edge[1] and original_edge[1] == edge[0])):
                    component_edges.append(original_edge)
                    break
            self.filtered_edges_by_component.append(component_edges)
        self.all_voronoi_edges = copy.deepcopy(self.voronoi_edges)
        # 更新 voronoi_edges 为过滤后的边
        self.voronoi_edges = component_edges
        print(f"过滤后保留 {len(self.voronoi_edges)} 条边")
    else:
        print("未找到有效的连通分量")
        self.filtered_edges_by_component = []
        self.component_info = []
        self.voronoi_edges = []
    return self.component_info

def build_filtered_network_graph(self):
    """基于过滤后的连通分量构建网络图"""
    if not hasattr(self, 'filtered_edges_by_component'):
        print("请先进行连通性分析")
        return None
    print("正在构建过滤后的网络图...")
    # 创建网络图
    G = nx.Graph()
    # 为每个连通分量分配不同的属性
    for comp_idx, component_edges in enumerate(self.filtered_edges_by_component):
        for edge in component_edges:
            start_node = edge[0]
            end_node = edge[1]
            weight = np.sqrt((start_node[0] - end_node[0])**2 + (start_node[1] -
end_node[1])**2)
            # 添加边，并标记所属的连通分量
            G.add_edge(start_node, end_node, weight=weight, component=comp_idx)
    # 为节点添加连通分量信息
    for comp_idx, component_edges in enumerate(self.filtered_edges_by_component):
        for edge in component_edges:
            G.nodes[edge[0]]['component'] = comp_idx
            G.nodes[edge[1]]['component'] = comp_idx
    self.path_network = G
    print(f"过滤后的网络图包含 {G.number_of_nodes()} 个节点和 {G.number_of_edges()} 条边")
    return G

def set_start_end_regions(self, baohu_regions=None):
    """
    设置起点和终点区域

    Args:
        baohu_regions: 端点保护区域，每个区域为 {'center': (x, y), 'radius': r} 或多边形顶
点列表
    """
    self.baohu_regions = baohu_regions if baohu_regions else []
    print(f"设置了 {len(self.baohu_regions)} 个端点保护区域")

def identify_protected_nodes(self):
    """识别在起点终点区域内的受保护节点"""
    if not self.path_network:

```

```

        print("请先构建路径网络")
        return set()
    self.protected_nodes = set()
    all_regions = self.baohu_regions
    for node in self.path_network.nodes():
        node_point = Point(node[0], node[1])
        for region in all_regions:
            # 检查是否在圆形区域内
            if isinstance(region, dict) and 'center' in region and 'radius' in region:
                center = region['center']
                radius = region['radius']
                distance = np.sqrt((node[0] - center[0])**2 + (node[1] - center[1])**2)
                if distance <= radius:
                    self.protected_nodes.add(node)
                    break
            # 检查是否在多边形区域内
            elif isinstance(region, list) and len(region) >= 3:
                from shapely.geometry import Polygon as ShapelyPolygon
                try:
                    polygon = ShapelyPolygon(region)
                    if polygon.contains(node_point):
                        self.protected_nodes.add(node)
                        break
                except:
                    continue
    print(f"识别到 {len(self.protected_nodes)} 个受保护节点")
    return self.protected_nodes
def prune_branches(self, max_iterations=1000):
    """
    修剪分支：迭代移除度为 1 的非保护节点

    Args:
        max_iterations: 最大迭代次数，防止无限循环

    Returns:
        修剪后的网络图
    """
    if not self.path_network:
        print("请先构建路径网络")
        return None

    if not self.protected_nodes:
        print("未设置保护节点，将识别起点终点区域内的节点")
        self.identify_protected_nodes()
    # 创建网络的深拷贝用于修剪
    self.pruned_network = copy.deepcopy(self.path_network)
    print("开始修剪分支...")
    iteration = 0
    removed_total = 0
    while iteration < max_iterations:
        # 找到所有度为 1 的非保护节点
        nodes_to_remove = []
        for node in self.pruned_network.nodes():
            if (self.pruned_network.degree(node) == 1 and
                node not in self.protected_nodes):
                nodes_to_remove.append(node)
        # 如果没有可移除的节点，结束修剪
        if not nodes_to_remove:
            print(f"修剪完成，无更多可移除的分支节点")
            break

```

```

        # 移除节点（自动删除所有连接到该节点的边）
        for node in nodes_to_remove:
            self.pruned_network.remove_node(node)
            removed_total += 1
        iteration += 1
        print(f"第 {iteration} 轮修剪: 移除 {len(nodes_to_remove)} 个节点")
    if iteration >= max_iterations:
        print(f"达到最大迭代次数 {max_iterations}, 修剪结束")
    return self.pruned_network

def build_network_graph(self):
    """构建网络图"""
    if not self.voronoi_edges:
        print("没有可用的边来构建网络")
        return None
    print("正在构建网络图...")
    # 创建网络图
    G = nx.Graph()
    # 添加边
    for edge in self.voronoi_edges:
        start_node = edge[0]
        end_node = edge[1]
        # 计算边的权重（长度）
        weight = np.sqrt((start_node[0] - end_node[0])**2 + (start_node[1] -
end_node[1])**2)
        G.add_edge(start_node, end_node, weight=weight)
    self.path_network = G
    print(f"网络图包含 {G.number_of_nodes()} 个节点和 {G.number_of_edges()} 条边")
    return G

def build_complete_path_network(self, enable_pruning=True):
    """完整的路径网络构建流程"""
    print("\n=== 开始构建 Voronoi 路径网络 ===")
    # 1. 收集边界点
    self.collect_boundary_points()
    # 2. 生成 Voronoi 图
    self.generate_voronoi_diagram()
    if self.voronoi is None:
        print("Voronoi 图生成失败, 无法继续")
        return None
    # 3. 提取边
    self.extract_voronoi_edges()
    # 3.5. 预处理障碍物线段
    self.preprocess_obstacle_lines()
    # 4. 过滤边(使用空间索引优化的边过滤方法
    self.filter_voronoi_edges_with_spatial_index()
    # 5. 连通性分析和子图过滤
    self.analyze_connected_components()
    # 6. 构建过滤后的网络图
    self.build_filtered_network_graph()
    # 7. 分支修剪（新增）
    if enable_pruning and self.path_network:
        print("\n=== 开始分支修剪 ===")
        self.identify_protected_nodes()
        self.prune_branches()
    print("=== Voronoi 路径网络构建完成 ===\n")
    return self.pruned_network if enable_pruning else self.path_network

def save_pruned_network_to_excel(self, file_name):
    """
    将修剪后的路径网络 pruned_network 保存到 Excel 文件, 格式与 geometry_data 一致

    Args:

```

```

        file_name: 文件名前缀
    """
    if not self.pruned_network or self.pruned_network.number_of_edges() == 0:
        print("没有可保存的路径网络")
        return
    # 1. 提取 segments (直接使用网络的边)
    segments = []
    for edge in self.pruned_network.edges():
        segment = [edge[0], edge[1]]
        segments.append(segment)
    # 2. 生成 all_points (在线段上每 5 米取点)
    all_points = []
    point_interval = 5000 # 5 米 = 5000mm
    min_end_distance = 10000 # 距离端点最小 10 米 = 10000mm
    for edge in self.pruned_network.edges():
        start_point = edge[0]
        end_point = edge[1]
        # 计算线段长度
        segment_length = np.sqrt((end_point[0] - start_point[0])**2 +
                                   (end_point[1] - start_point[1])**2)
        # 修改这里: 如果线段太短, 根据长度决定取点策略
        if segment_length < point_interval: # 小于 5 米
            # 只添加起点, 避免两点距离小于 5 米
            all_points.append(start_point)
            continue
        elif segment_length <= min_end_distance: # 5-10 米之间
            # 添加起点和终点 (距离>=5 米)
            all_points.append(start_point)
            all_points.append(end_point)
            continue
        # 计算单位方向向量
        direction_x = (end_point[0] - start_point[0]) / segment_length
        direction_y = (end_point[1] - start_point[1]) / segment_length
        # 添加起点
        all_points.append(start_point)
        # 从起点开始每 5 米取一个点
        current_distance = point_interval
        while current_distance <= segment_length - min_end_distance:
            point_x = start_point[0] + current_distance * direction_x
            point_y = start_point[1] + current_distance * direction_y
            all_points.append((point_x, point_y))
            current_distance += point_interval
        # 添加终点
        all_points.append(end_point)
    # 3. 去除重复点
    unique_all_points = []
    tolerance = 1.0 # 1mm 容差
    for point in all_points:
        is_duplicate = False
        for existing_point in unique_all_points:
            distance = np.sqrt((point[0] - existing_point[0])**2 +
                               (point[1] - existing_point[1])**2)
            if distance < tolerance:
                is_duplicate = True
                break
        if not is_duplicate:
            unique_all_points.append(point)
    # 4. 转换为 Excel 格式
    segments_data = []
    all_points_data = []

```

```

# 处理 segments - 每个 segment 作为一个分组
for i, segment in enumerate(segments):
    # 添加分组标识符
    segments_data.append(f"{{0; {i}}}")
    # 添加 segment 中的点
    for j, point in enumerate(segment):
        segments_data.append(f"{{j+1}. {{point[0]}, {point[1]}, 0}}")
# 处理 all_points
for point in unique_all_points:
    all_points_data.append(f"{{point[0]}, {point[1]}, 0}}")
# 5. 创建 DataFrame 并保存
import pandas as pd
# 确保两列长度一致
max_length = max(len(segments_data), len(all_points_data))
# 填充较短的列
while len(segments_data) < max_length:
    segments_data.append("")
while len(all_points_data) < max_length:
    all_points_data.append("")
# 创建 DataFrame
df = pd.DataFrame({
    '区分线段的坐标': segments_data,
    '不区分线段的坐标': all_points_data
})
# 保存到 Excel
excel_filename = f"{file_name}-路网络.xlsx"
with pd.ExcelWriter(excel_filename, engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='路网络', index=False)
print(f"路径网络已保存到: {excel_filename}")
return excel_filename

```

## 问题二相关代码

### garden\_grid\_modeling.py

```

'''
园林景观网格扫描建模系统
采用射线投射算法
栅格数据的生成与加载
'''

import numpy as np
import pandas as pd
import pickle
import json
from collections import defaultdict, Counter
from shapely.geometry import Point, LineString, Polygon
from rtree import index
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.patches import Circle
from concurrent.futures import ProcessPoolExecutor, as_completed
from multiprocessing import cpu_count
import warnings
import os
warnings.filterwarnings('ignore')
# 设置中文字体
plt.rcParams['font.family'] = ['simhei']
plt.rcParams['axes.unicode_minus'] = False
# 导入已有模块

```

```

from load_and_visualize import NetworkLoader, NetworkVisualizer
def compute_block_grid_matrix(args):
    """
    独立的进程计算函数 - 计算单个分割块的网格扫描矩阵
    """
    block_id, block_center, scan_area_size, analyzer_params, segments_info_data = args
    # 在进程内重建分析器状态
    analyzer = BlockGridAnalyzer(
        scan_area_size=scan_area_size,
        grid_resolution=analyzer_params['grid_resolution']
    )
    # 重建 segments_info 和空间索引
    analyzer.segments_info = segments_info_data
    # 重建空间索引
    from rtree import index
    spatial_idx = index.Index()
    for i, segment_info in enumerate(analyzer.segments_info):
        spatial_idx.insert(i, segment_info['geometry'].bounds)
    analyzer.spatial_index = spatial_idx
    # 计算网格扫描矩阵（包含二次填充）
    grid_matrix = analyzer.compute_grid_matrix(block_center)
    occlusion_ratio = analyzer.compute_occlusion_ratio(grid_matrix)
    return block_id, {
        'block_center': block_center,
        'scan_area_size': scan_area_size,
        'grid_matrix': grid_matrix,
        'occlusion_ratio': occlusion_ratio
    }

class BlockGridAnalyzer:
    """分割块网格扫描分析器"""
    def __init__(self, scan_area_size=10000, grid_resolution=1000):
        """
        Args:
            scan_area_size: 扫描区域边长（毫米）10 米 = 10,000mm
            grid_resolution: 网格分辨率（毫米/像素）1 米 = 1,000mm/像素
        """
        self.scan_area_size = scan_area_size
        self.grid_resolution = grid_resolution
        self.grid_size = int(scan_area_size / grid_resolution) # 网格尺寸（像素）
        # 景观元素编号映射
        self.element_mapping = {
            '被遮挡': 0,
            '半开放建筑': 1,
            '实体建筑': 2,
            '道路': 3,
            '假山': 4,
            '水体': 5,
            '植物': 6,
            '空旷区域': 7 # 没有障碍物的区域
        }
        # 遮挡性元素（只有实体建筑会完全遮挡视线）
        self.occlusion_elements = {'实体建筑'}
        print(f"初始化网格扫描分析器: {self.grid_size}x{self.grid_size}网格, 分辨率{grid_resolution}mm/pixel")
        def build_spatial_index(self, geometry_data):
            """构建空间索引"""
            print("构建空间索引...")
            # 创建 R-tree 空间索引
            spatial_idx = index.Index()
            # 存储线段信息

```

```

self.segments_info = []
segment_id = 0
for element_type, data in geometry_data.items():
    if element_type == "植物":
        # 植物处理为圆形边界
        for plant in data:
            center = plant['center']
            radius = plant['radius']
            # 简化为正八边形近似圆
            angles = np.linspace(0, 2*np.pi, 9)[::-1] # 8个顶点
            vertices = [(center[0] + radius * np.cos(a),
                        center[1] + radius * np.sin(a)) for a in angles]
            # 创建边界线段
            for i in range(len(vertices)):
                start = vertices[i]
                end = vertices[(i + 1) % len(vertices)]
                line = LineString([start, end])
                # 添加到索引
                spatial_idx.insert(segment_id, line.bounds)
                self.segments_info.append({
                    'id': segment_id,
                    'element_type': element_type,
                    'geometry': line
                })
                segment_id += 1
    else:
        # 处理线段数据
        for segment in data.get('segments', []):
            if len(segment) > 1:
                for i in range(len(segment) - 1):
                    start = segment[i]
                    end = segment[i + 1]
                    line = LineString([start, end])

                    # 添加到索引
                    spatial_idx.insert(segment_id, line.bounds)
                    self.segments_info.append({
                        'id': segment_id,
                        'element_type': element_type,
                        'geometry': line
                    })
                    segment_id += 1
self.spatial_index = spatial_idx
print(f"空间索引构建完成: {len(self.segments_info)} 个线段")
return spatial_idx
def cast_ray_to_point_improved(self, observer_point, target_point):
    """

```

改进的射线投射算法：基于交点分段填充

实现逻辑：

1. 从观测点向目标点发射射线
2. 找到所有与射线相交的线段，按距离排序
3. 根据交点位置进行分段填充：
  - 从观测点到第一个交点：标记为第一个交点的类型
  - 第一个交点到第二个交点：标记为第二个交点的类型
  - 以此类推
4. 如果目标点超出了所有交点，延长射线寻找下一个交点
5. 如果没有找到交点，标记为空旷区域

Args:

```

        observer_point: 观察点坐标 (x, y)
        target_point: 目标点坐标 (x, y)

Returns:
    int: 景观元素编号
    """
    target_distance = np.sqrt((target_point[0] - observer_point[0])**2 +
                              (target_point[1] - observer_point[1])**2)
    if target_distance < 1: # 距离太近, 直接返回空旷区域
        return self.element_mapping['空旷区域']
    # 计算射线方向
    direction_x = (target_point[0] - observer_point[0]) / target_distance
    direction_y = (target_point[1] - observer_point[1]) / target_distance
    # 1. 构建基础射线 (从观测点到目标点)
    base_ray = LineString([observer_point, target_point])
    # 2. 找到所有与基础射线相交的线段
    intersections = self._find_ray_intersections(base_ray, observer_point)
    # 3. 如果完全没有交点, 直接标记为空旷区域, 不延长射线
    if not intersections:
        return self.element_mapping['空旷区域']
    # 4. 根据交点位置确定目标点的景观类型
    return self._determine_point_type_by_intersections(
        intersections, target_distance, observer_point, direction_x, direction_y
    )
def _find_ray_intersections(self, ray, observer_point):
    """
    找到射线与所有线段的交点

Args:
    ray: 射线 LineString 对象
    observer_point: 观察点坐标

Returns:
    list: 交点信息列表, 每个元素包含距离、类型、坐标
    """
    intersections = []
    for seg_id in self.spatial_index.intersection(ray.bounds):
        segment_info = self.segments_info[seg_id]
        intersection = ray.intersection(segment_info['geometry'])
        if not intersection.is_empty:
            # 处理交点坐标
            intersection_points = []
            if hasattr(intersection, 'coords'): # Point
                intersection_points = [list(intersection.coords)[0]]
            elif hasattr(intersection, 'geoms'): # MultiPoint 或其他几何体集合
                for geom in intersection.geoms:
                    if hasattr(geom, 'coords'):
                        intersection_points.extend(list(geom.coords))
            # 计算每个交点的距离并添加到结果中
            for point_coords in intersection_points:
                distance = np.sqrt((point_coords[0] - observer_point[0])**2 +
                                  (point_coords[1] - observer_point[1])**2)
                if distance > 1: # 忽略太近的点 (避免数值误差)
                    intersections.append({
                        'distance': distance,
                        'element_type': segment_info['element_type'],
                        'point': point_coords,
                        'segment_id': seg_id
                    })
    # 按距离排序

```

```

intersections.sort(key=lambda x: x['distance'])
# 去除重复的交点（距离相近的认为是同一个点）
filtered_intersections = []
for intersection in intersections:
    if not filtered_intersections or \
        abs(intersection['distance'] - filtered_intersections[-1]['distance']) > 10:
        filtered_intersections.append(intersection)
return filtered_intersections
def _determine_point_type_by_intersections(self, intersections, target_distance,
                                           observer_point, direction_x, direction_y):
    """
    根据交点信息确定目标点的景观类型

    Args:
        intersections: 交点信息列表
        target_distance: 目标点距离
        observer_point: 观察点坐标
        direction_x, direction_y: 射线方向

    Returns:
        int: 景观元素编号
    """
    if not intersections:
        # 完全没有交点，标记为空旷区域
        return self.element_mapping['空旷区域']
    # 情况 1: 目标点在第一个交点之前
    if target_distance <= intersections[0]['distance']:
        return self.element_mapping.get(intersections[0]['element_type'],
                                         self.element_mapping['空旷区域'])
    # 情况 2: 目标点在相邻交点之间
    for i in range(len(intersections) - 1):
        if intersections[i]['distance'] < target_distance <= intersections[i + 1]['distance']:
            # 在两个交点之间，使用后一个交点的类型
            return self.element_mapping.get(intersections[i + 1]['element_type'],
                                             self.element_mapping['空旷区域'])
    # 情况 3: 目标点在所有已知交点之后
    # 延长射线寻找下一个交点
    extended_distance = max(target_distance * 2, intersections[-1]['distance'] * 1.5)
    extended_point = (
        observer_point[0] + direction_x * extended_distance,
        observer_point[1] + direction_y * extended_distance
    )
    extended_ray = LineString([observer_point, extended_point])
    # 寻找延长部分的交点
    extended_intersections = self._find_ray_intersections(extended_ray, observer_point)
    # 筛选出目标点之后的交点
    future_intersections = [
        inter for inter in extended_intersections
        if inter['distance'] > target_distance
    ]
    if future_intersections:
        # 找到目标点之后的交点，向内填充
        return self.element_mapping.get(future_intersections[0]['element_type'],
                                         self.element_mapping['空旷区域'])
    else:
        # 没有找到更远的交点，使用最后已知交点的类型
        return self.element_mapping.get(intersections[-1]['element_type'],
                                         self.element_mapping['空旷区域'])
def horizontal_line_scan_walking(self, observer_point, y_coord, x_start, x_end,

```

```

direction='left_to_right'):
    """
    沿边界行走的水平线扫描（确保返回长度一致）
    """

    # 预检查：构建整行的射线，看是否有任何交点
    if direction == 'left_to_right':
        end_point = (x_end, y_coord)
    else: # right_to_left
        end_point = (x_start, y_coord)
    full_line_ray = LineString([observer_point, end_point])
    line_intersections = self._find_ray_intersections(full_line_ray, observer_point)
    # 计算预期的网格点数量（确保一致性）
    expected_grid_count = int(abs(x_end - x_start) / self.grid_resolution) + 1
    # 如果整行完全没有交点，直接返回全部为'空旷区域'的结果
    if not line_intersections:
        return [self.element_mapping['空旷区域']] * expected_grid_count
    # 使用固定步长生成坐标，确保数量一致
    if direction == 'left_to_right':
        x_coords = [x_start + i * self.grid_resolution for i in range(expected_grid_count)]
        # 确保最后一个点不超过 x_end
        if x_coords[-1] > x_end:
            x_coords[-1] = x_end
    else: # right_to_left
        x_coords = [x_end - i * self.grid_resolution for i in range(expected_grid_count)]
        # 确保最后一个点不低于 x_start
        if x_coords[-1] < x_start:
            x_coords[-1] = x_start
    scan_line_result = []
    for x_coord in x_coords:
        target_point = (x_coord, y_coord)
        # 使用改进的射线投射算法，从当前观测点到目标点
        element_code = self.cast_ray_to_point_improved(observer_point, target_point)
        scan_line_result.append(element_code)
    if direction == 'right_to_left':
        scan_line_result.reverse()
    # 确保返回的结果长度正确
    if len(scan_line_result) != expected_grid_count:
        print(f"警告：扫描结果长度不匹配，预期{expected_grid_count}，实际{len(scan_line_result)}")
        # 修正长度
        if len(scan_line_result) < expected_grid_count:
            # 用最后一个值填充
            scan_line_result.extend([scan_line_result[-1]] * (expected_grid_count - len(scan_line_result)))
        else:
            # 截断到预期长度
            scan_line_result = scan_line_result[:expected_grid_count]
    return scan_line_result

def _process_walking_occlusion(self, left_to_right_result, right_to_left_result):
    """
    行走模式的遮挡处理

    逻辑：
    1. 基础结果以左到右扫描为准
    2. 如果右到左扫描在某些位置看到了不同的景观元素，进行合理融合
    3. 只有当左右都看到实体建筑，且中间有空隙时，才标记为被遮挡

    Args:
        left_to_right_result: 从左边界向右的扫描结果
        right_to_left_result: 从右边界向左的扫描结果
    """

```

```

Returns:
    list: 融合后的扫描结果
"""

final_result = left_to_right_result.copy()
# 找到左侧扫描遇到的第一个实体建筑位置
left_building_pos = -1
for i, val in enumerate(left_to_right_result):
    if val == self.element_mapping['实体建筑']:
        left_building_pos = i
        break
# 找到右侧扫描遇到的第一个实体建筑位置
right_building_pos = -1
for i, val in enumerate(right_to_left_result):
    if val == self.element_mapping['实体建筑']:
        right_building_pos = len(right_to_left_result) - 1 - i # 转换为左到右的索引
        break
# 情况 1: 左右都有实体建筑, 且中间有空隙
if (left_building_pos != -1 and right_building_pos != -1 and
    left_building_pos < right_building_pos):
    # 中间区域的处理: 结合左右观察结果
    for i in range(left_building_pos + 1, right_building_pos):
        left_view = final_result[i]
        right_view = right_to_left_result[len(right_to_left_result) - 1 - i]
        # 如果左边看到的是空旷区域, 但右边看到了景观元素, 采用右边的结果
        if (left_view == self.element_mapping['空旷区域'] and
            right_view != self.element_mapping['空旷区域'] and
            right_view != self.element_mapping['被遮挡']):
            final_result[i] = right_view
        # 如果左右都是空旷区域, 标记为被遮挡
        elif (left_view == self.element_mapping['空旷区域'] and
              right_view == self.element_mapping['空旷区域']):
            final_result[i] = self.element_mapping['被遮挡']
# 情况 2: 只有左边有实体建筑, 右边补充信息
elif left_building_pos != -1 and right_building_pos == -1:
    # 实体建筑后面的区域, 尝试用右边的观察结果补充
    for i in range(left_building_pos + 1, len(final_result)):
        if final_result[i] == self.element_mapping['空旷区域']:
            right_view = right_to_left_result[len(right_to_left_result) - 1 - i]
            if right_view != self.element_mapping['空旷区域']:
                final_result[i] = right_view
    return final_result

def _process_walking_occlusion_fill_blanks(self, left_to_right_result,
right_to_left_result):
    """
    专门用于填充空白的行走模式遮挡处理

    逻辑:
    1. 基础结果以左到右扫描为准
    2. 只有当左到右扫描结果为空旷区域时, 才用右到左的结果替换
    3. 其他情况保持左到右的结果不变

    Args:
        left_to_right_result: 从左边界向右的扫描结果
        right_to_left_result: 从右边界向左的扫描结果

    Returns:
        list: 融合后的扫描结果
    """
    final_result = left_to_right_result.copy()

```

```

# 遍历每个位置
for i in range(len(final_result)):
    # 只有当左到右扫描结果为空旷区域时，才考虑用右到左的结果替换
    if final_result[i] == self.element_mapping['空旷区域']:
        # 获取对应的右到左扫描结果
        right_index = len(right_to_left_result) - 1 - i
        if 0 <= right_index < len(right_to_left_result):
            right_view = right_to_left_result[right_index]
            # 如果右到左扫描看到了有效的景观元素（不是空旷区域），就用它填充
            if right_view != self.element_mapping['空旷区域']:
                final_result[i] = right_view
return final_result

def compute_grid_matrix(self, block_center):
    """
    计算分割块的网格扫描矩阵（沿边界行走扫描模式，确保矩阵形状一致）
    """
    # 定义分割块边界
    half_size = self.scan_area_size / 2
    min_x = block_center[0] - half_size
    max_x = block_center[0] + half_size
    min_y = block_center[1] - half_size
    max_y = block_center[1] + half_size
    grid_matrix = []
    # 预计算预期的矩阵尺寸
    expected_rows = int(self.scan_area_size / self.grid_resolution)
    expected_cols = int(self.scan_area_size / self.grid_resolution)
    print(f" 预期矩阵尺寸: {expected_rows}x{expected_cols}")
    # 逐行扫描（从上到下）
    y_coords = np.arange(max_y, min_y - self.grid_resolution, -self.grid_resolution)
    # 确保 y_coords 的数量与预期行数一致
    if len(y_coords) != expected_rows:
        # 重新生成确保数量正确
        y_coords = [max_y - i * self.grid_resolution for i in range(expected_rows)]
    for i, y_coord in enumerate(y_coords):
        # 当前行的观测点位于左边界
        current_observer = (min_x, y_coord)
        # 从左边界向右扫描
        left_to_right_result = self.horizontal_line_scan_walking(
            current_observer, y_coord, min_x, max_x, 'left_to_right'
        )
        # 检查扫描结果长度
        if len(left_to_right_result) != expected_cols:
            #print(f" 警告: 第{i}行扫描长度不匹配, 预期{expected_cols}, 实际{len(left_to_right_result)}")
            # 修正长度
            if len(left_to_right_result) < expected_cols:
                left_to_right_result.extend([left_to_right_result[-1]] * (expected_cols - len(left_to_right_result)))
            else:
                left_to_right_result = left_to_right_result[:expected_cols]
        # 检查是否有空旷区域，如果是，从右边界补充扫描
        has_open_area = self.element_mapping['空旷区域'] in left_to_right_result
        if has_open_area:
            # 从右边界向左补充扫描
            right_observer = (max_x, y_coord)
            right_to_left_result = self.horizontal_line_scan_walking(
                right_observer, y_coord, min_x, max_x, 'right_to_left'
            )
            # 确保右侧扫描结果长度也正确

```

```

        if len(right_to_left_result) != expected_cols:
            if len(right_to_left_result) < expected_cols:
                right_to_left_result.extend([right_to_left_result[-1]] *
(expected_cols - len(right_to_left_result)))
            else:
                right_to_left_result = right_to_left_result[:expected_cols]
        # 处理遮挡：只对空白区域进行填充
        final_result = self._process_walking_occlusion_fill_blanks(
            left_to_right_result, right_to_left_result
        )
    else:
        # 没有空旷区域，直接使用左到右的结果
        final_result = left_to_right_result
    # 最终确保长度正确
    if len(final_result) != expected_cols:
        if len(final_result) < expected_cols:
            final_result.extend([final_result[-1]] * (expected_cols -
len(final_result)))
        else:
            final_result = final_result[:expected_cols]
    grid_matrix.append(final_result)
    # 转换为 numpy 数组前最后检查
    print(f" 实际矩阵行数: {len(grid_matrix)}")
    if grid_matrix:
        row_lengths = [len(row) for row in grid_matrix]
        print(f" 各行长度: 最小{min(row_lengths)}, 最大{max(row_lengths)}, 预期
{expected_cols}")
        # 如果仍有长度不一致的行，进行最后修正
        for i, row in enumerate(grid_matrix):
            if len(row) != expected_cols:
                print(f" 修正第{i}行长度: {len(row)} -> {expected_cols}")
                if len(row) < expected_cols:
                    grid_matrix[i] = row + [row[-1]] * (expected_cols - len(row))
                else:
                    grid_matrix[i] = row[:expected_cols]
    try:
        grid_matrix = np.array(grid_matrix, dtype=int)
        print(f" 成功创建矩阵: {grid_matrix.shape}")
    except ValueError as e:
        print(f" 创建 numpy 数组失败: {e}")
        print(f" 尝试强制修正...")
        # 强制修正：确保所有行都有相同长度
        if grid_matrix:
            target_length = expected_cols
            corrected_matrix = []
            for i, row in enumerate(grid_matrix):
                if len(row) == target_length:
                    corrected_matrix.append(row)
                elif len(row) < target_length:
                    corrected_row = row + [self.element_mapping['空旷区域']] *
(target_length - len(row))
                    corrected_matrix.append(corrected_row)
                else:
                    corrected_row = row[:target_length]
                    corrected_matrix.append(corrected_row)
            grid_matrix = np.array(corrected_matrix, dtype=int)
            print(f" 修正后矩阵形状: {grid_matrix.shape}")
    # 添加二次填充步骤
    print(f"开始二次填充空白区域...")
    filled_matrix = self.secondary_fill_empty_areas(grid_matrix, block_center)

```

```

        return filled_matrix
def _process_occlusion_improved(self, left_to_right_result, right_to_left_result,
                                observer_point, y_coord, min_x, max_x):
    """
    保留此方法以确保向后兼容，但在新的行走模式中不使用
    """
    return self._process_walking_occlusion(left_to_right_result, right_to_left_result)
def compute_occlusion_ratio(self, grid_matrix):
    """计算被遮挡比例"""
    total_elements = grid_matrix.size
    occluded_elements = np.sum(grid_matrix == self.element_mapping['被遮挡'])
    return occluded_elements / total_elements if total_elements > 0 else 0
def secondary_fill_empty_areas(self, grid_matrix, block_center):
    """
    二次填充空白区域

    算法逻辑：
    1. 对空白区域，从该位置发射四个方向的射线
    2. 找到每个方向上第一个相交的线段类型
    3. 优先级：左右相同 > 上下相同 > 四个方向最近的

    Args:
        grid_matrix: 原始网格扫描矩阵
        block_center: 分割块中心点（用于坐标转换）

    Returns:
        numpy.ndarray: 二次填充后的网格矩阵
    """
    filled_matrix = grid_matrix.copy()
    rows, cols = filled_matrix.shape
    empty_area_code = self.element_mapping['空旷区域']
    # 计算分割块边界
    half_size = self.scan_area_size / 2
    min_x = block_center[0] - half_size
    max_y = block_center[1] + half_size
    # 找到所有空白区域的位置
    empty_positions = []
    for i in range(rows):
        for j in range(cols):
            if filled_matrix[i, j] == empty_area_code:
                empty_positions.append((i, j))
    if not empty_positions:
        return filled_matrix
    print(f"发现 {len(empty_positions)} 个空白位置需要二次填充")
    # 对每个空白位置进行填充
    filled_count = 0
    for row, col in empty_positions:
        # 将网格坐标转换为世界坐标
        world_x = min_x + col * self.grid_resolution
        world_y = max_y - row * self.grid_resolution
        world_point = (world_x, world_y)
        fill_value = self._determine_secondary_fill_by_raycast(world_point,
empty_area_code)
        if fill_value != empty_area_code:
            filled_matrix[row, col] = fill_value
            filled_count += 1
    print(f"二次填充完成: {filled_count}/{len(empty_positions)} 个位置被填充")
    return filled_matrix
def _determine_secondary_fill_by_raycast(self, world_point, empty_code):
    """

```

通过射线投射确定空白位置的填充值

Args:

world\_point: 世界坐标系中的空白位置  
empty\_code: 空白区域编码

Returns:

int: 填充值

"""

# 定义四个方向的射线方向向量和扩展距离

```
directions = {  
    'left': (-1, 0),  
    'right': (1, 0),  
    'up': (0, 1),  
    'down': (0, -1)  
}
```

# 射线最大扩展距离

max\_extension = max(self.scan\_area\_size \* 2, 50000) # 确保能覆盖整个区域

# 对每个方向发射射线，找到第一个相交的线段

intersection\_results = {}

for direction\_name, (dx, dy) in directions.items():

# 创建射线终点

```
end_point = (  
    world_point[0] + dx * max_extension,  
    world_point[1] + dy * max_extension  
)
```

# 发射射线并找到第一个相交的线段

```
intersected_segment = self._cast_ray_find_first_intersection(world_point,  
end_point)
```

if intersected\_segment is not None:

intersection\_results[direction\_name] = intersected\_segment

# 按优先级规则确定填充值

return self.\_apply\_priority\_rules(intersection\_results, empty\_code)

def \_cast\_ray\_find\_first\_intersection(self, start\_point, end\_point):

"""

发射射线找到第一个相交的线段类型

Args:

start\_point: 射线起点  
end\_point: 射线终点

Returns:

str or None: 第一个相交的线段类型，如果没有相交则返回 None

"""

from shapely.geometry import LineString

ray = LineString([start\_point, end\_point])

closest\_intersection = None

closest\_distance = float('inf')

closest\_element\_type = None

# 在空间索引中查找可能相交的线段

for seg\_id in self.spatial\_index.intersection(ray.bounds):

segment\_info = self.segments\_info[seg\_id]

intersection = ray.intersection(segment\_info['geometry'])

if not intersection.is\_empty:

# 处理交点坐标

intersection\_points = []

if hasattr(intersection, 'coords'): # Point

intersection\_points = [list(intersection.coords)[0]]

elif hasattr(intersection, 'geoms'): # MultiPoint 或其他几何体集合

for geom in intersection.geoms:

```

        if hasattr(geom, 'coords'):
            intersection_points.extend(list(geom.coords))
    # 找到最近的交点
    for point_coords in intersection_points:
        distance = np.sqrt((point_coords[0] - start_point[0])**2 +
                           (point_coords[1] - start_point[1])**2)

        if distance > 1 and distance < closest_distance: # 忽略太近的点
            closest_distance = distance
            closest_element_type = segment_info['element_type']
            closest_intersection = point_coords

    return closest_element_type

def _apply_priority_rules(self, intersection_results, empty_code):
    """
    应用优先级规则确定填充值

    优先级:
    1. 左右射线相交同一类型线段
    2. 上下射线相交同一类型线段
    3. 四个方向最近的线段
    4. 保持空白

    Args:
        intersection_results: 各方向射线的相交结果
        empty_code: 空白区域编码

    Returns:
        int: 填充值
    """
    if not intersection_results:
        return empty_code
    # 规则 1: 检查左右是否相交同一类型线段
    if ('left' in intersection_results and 'right' in intersection_results and
        intersection_results['left'] == intersection_results['right']):
        return self.element_mapping.get(intersection_results['left'], empty_code)
    # 规则 2: 检查上下是否相交同一类型线段
    if ('up' in intersection_results and 'down' in intersection_results and
        intersection_results['up'] == intersection_results['down']):
        return self.element_mapping.get(intersection_results['up'], empty_code)
    # 规则 3: 四个方向最常见的线段类型
    if intersection_results:
        segment_counts = {}
        for segment_type in intersection_results.values():
            segment_counts[segment_type] = segment_counts.get(segment_type, 0) + 1
        # 返回出现次数最多的景观类型
        most_common_segment = max(segment_counts.items(), key=lambda x: x[1])[0]
        return self.element_mapping.get(most_common_segment, empty_code)
    # 规则 4: 保持空白
    return empty_code

def _prepare_process_args(self):
    """准备进程计算所需的参数"""
    analyzer_params = {
        'grid_resolution': self.grid_resolution
    }
    # 准备 segments_info 数据 (需要可序列化)
    segments_info_data = []
    for segment_info in self.segments_info:
        segments_info_data.append({
            'id': segment_info['id'],
            'element_type': segment_info['element_type'],

```

```

        'geometry': segment_info['geometry']
    })
    return analyzer_params, segments_info_data

class GardenBlockModeler:
    """园林分割块建模系统"""
    def __init__(self, block_size=200000, overlap_distance=50000, occlusion_threshold=0.5):
        """
        Args:
            block_size: 分割块大小（毫米）200 米 = 200,000mm
            overlap_distance: 重叠距离（毫米）50 米 = 50,000mm
            occlusion_threshold: 遮挡阈值
        """
        self.block_size = block_size
        self.overlap_distance = overlap_distance
        self.occlusion_threshold = occlusion_threshold
        # 实际步长（扣除重叠）
        self.step_size = block_size - overlap_distance
        # 初始化加载器
        self.loader = NetworkLoader()
        self.visualizer = NetworkVisualizer(self.loader)
        print(f"分割块大小: {block_size}mm, 重叠距离: {overlap_distance}mm, 实际步长: {self.step_size}mm")
    def main(file_name):
        """主函数"""
        # 文件路径
        original_excel = f"shujv/4-{file_name}数据坐标.xlsx"
        print(f"=== 园林网格扫描建模(改进版): {file_name} ===")
        # 创建建模系统（手动设置参数）
        modeler = GardenBlockModeler(
            block_size=10000, # 分割块大小 10 米
            overlap_distance=0, # 重叠距离 2 米
            occlusion_threshold=0.5 # 遮挡率阈值 50%
        )
        #略
    if __name__ == "__main__":
        # 使用示例 "寄畅园", "留园", "耦园", "绮园", "秋霞圃", "沈园", "怡园", "豫园", "瞻园", "拙政园"
        file_names = ["近园"]
        for file_name in file_names:
            valid_blocks, grid_results, garden_bounds = main(file_name)

```

## garden\_illusion\_evaluation.py

```

'''
园林幻境感量化评价系统
基于景观生态学指标和空间形态学方法实现园林幻境感的定量评价
主要功能：
1. 数据处理与分析框架
2. 元素分布特征分析（基于景观生态学指标）
3. 开合变化空间分析（基于空间形态学方法）
4. 幻境感综合评价体系
5. 结果保存与可视化接口
'''

class GardenIllusionEvaluator:
    """园林幻境感量化评价器"""
    def __init__(self, baseline_garden_name: str = None):
        """

```

```

Args:
    baseline_garden_name: 基准园林名称，用于标准化评分
"""

self.baseline_garden_name = baseline_garden_name
self.baseline_metrics = None # 基准园林的指标值
# 元素编码映射（与 GardenBlockModeler 保持一致）
self.element_mapping = {
    '被遮挡': 0,
    '半开放建筑': 1,
    '实体建筑': 2,
    '道路': 3,
    '假山': 4,
    '水体': 5,
    '植物': 6,
    '空旷区域': 7
}

# 实体元素（用于开合分析）
self.solid_elements = {0, 1, 2, 4} # 被遮挡、半开放建筑、实体建筑、假山
self.open_elements = {3, 5, 6, 7} # 道路、水体、植物、空旷区域
# 评价指标权重配置
self.weights = {
    # 元素分布维度 (50%)
    'shannon_diversity': 0.10, # Shannon 多样性指数
    'aggregation_index': 0.15, # 聚集度指数
    'contagion_index': 0.10, # 蔓延度指数
    'edge_density': 0.10, # 边缘密度指数
    'moran_i': 0.05, # Moran's I 指数

    # 开合变化维度 (50%)
    'mean_nearest_neighbor': 0.15, # 平均最近邻距离
    'enclosure_ratio': 0.15, # 围合比指数
    'connectivity_index': 0.10, # 连通性指数
    'compactness_index': 0.05, # 紧凑度指数
    'integral_connectivity': 0.05 # 整体连通性指数
}

print(f"初始化园林幻境感评价系统")
print(f"基准园林: {baseline_garden_name or '未设置'}")

def combine_valid_blocks_to_matrix(self, all_blocks: List, grid_results: Dict) ->
np.ndarray:
    """将所有有效分割块的网格矩阵拼接成完整园林矩阵"""
    print("开始拼接有效分割块...")
    # 过滤有效块
    valid_blocks = [block for block in all_blocks if block.get('is_valid', False)]
    if not valid_blocks:
        print("没有有效的分割块")
        return None
    print(f"找到 {len(valid_blocks)} 个有效分割块")
    # 计算整体边界
    min_row = min(block['row'] for block in valid_blocks)
    max_row = max(block['row'] for block in valid_blocks)
    min_col = min(block['col'] for block in valid_blocks)
    max_col = max(block['col'] for block in valid_blocks)
    # 假设每个网格矩阵大小为 10x10
    grid_size = 10 # 1000mm 分辨率下 10m 分割块对应 10x10 网格
    # 计算完整矩阵尺寸
    total_rows = (max_row - min_row + 1) * grid_size
    total_cols = (max_col - min_col + 1) * grid_size
    # 创建完整矩阵（初始化为空旷区域）
    combined_matrix = np.full((total_rows, total_cols), 7, dtype=int)
    print(f"完整矩阵尺寸: {total_rows}x{total_cols}")

```

```

# 填充各个有效块的数据
filled_count = 0
for block in valid_blocks:
    block_id = block['id']
    if block_id not in grid_results:
        continue
    result = grid_results[block_id]
    block_matrix = result['grid_matrix']
    # 计算该块在完整矩阵中的位置
    row_offset = (block['row'] - min_row) * grid_size
    col_offset = (block['col'] - min_col) * grid_size
    # 确保不超出边界
    max_rows = min(row_offset + block_matrix.shape[0], total_rows)
    max_cols = min(col_offset + block_matrix.shape[1], total_cols)
    # 填充数据
    combined_matrix[row_offset:max_rows, col_offset:max_cols] = \
        block_matrix[:max_rows-row_offset, :max_cols-col_offset]
    filled_count += 1
print(f"成功拼接 {filled_count} 个有效分割块的网格矩阵")
return combined_matrix

def identify_connected_components(self, matrix: np.ndarray) -> Dict[int, List]:
    """识别连通域 (patches)"""
    print("识别连通域...")
    patches = {}
    # 对每个元素类型识别连通域
    for element_type in range(8):
        # 创建二值掩码
        binary_mask = (matrix == element_type)
        if np.sum(binary_mask) == 0:
            patches[element_type] = []
            continue
        # 使用4邻接识别连通域
        labeled_array, num_features = ndimage.label(binary_mask,
structure=np.ones((3,3)))
        patch_list = []
        for i in range(1, num_features + 1):
            patch_coors = np.where(labeled_array == i)
            patch_size = len(patch_coors[0])
            # 计算边界信息
            rows, cols = patch_coors
            min_row, max_row = np.min(rows), np.max(rows)
            min_col, max_col = np.min(cols), np.max(cols)
            # 计算周长 (边界像素数)
            perimeter = self._calculate_perimeter(labeled_array == i)
            patch_info = {
                'id': i,
                'size': patch_size,
                'coors': patch_coors,
                'bbox': (min_row, min_col, max_row, max_col),
                'perimeter': perimeter
            }
            patch_list.append(patch_info)
            patches[element_type] = patch_list
        total_patches = sum(len(patch_list) for patch_list in patches.values())
        print(f"连通域识别完成: 共 {total_patches} 个 patches")
    return patches

def _calculate_perimeter(self, binary_mask: np.ndarray) -> int:
    """计算二值图像的周长"""
    # 使用 Sobel 算子计算边缘
    edges = ndimage.binary_erosion(binary_mask) ^ binary_mask

```

```

    return np.sum(edges)
def compute_shannon_diversity_index(self, matrix: np.ndarray) -> float:
    """计算 Shannon 多样性指数"""
    # 统计各元素类型的面积占比
    total_area = matrix.size
    element_counts = np.bincount(matrix.flatten(), minlength=8)
    proportions = element_counts / total_area
    # 计算 Shannon 指数:  $SHDI = -\sum(P_i \times \ln P_i)$ 
    shdi = 0.0
    for p in proportions:
        if p > 0:
            shdi -= p * np.log(p)
    return shdi
def compute_largest_patch_index(self, patches: Dict[int, List]) -> float:
    """计算最大斑块指数"""
    total_area = sum(
        sum(patch['size'] for patch in patch_list)
        for patch_list in patches.values()
    )
    if total_area == 0:
        return 0.0
    # 找到最大斑块
    max_patch_size = 0
    for patch_list in patches.values():
        for patch in patch_list:
            max_patch_size = max(max_patch_size, patch['size'])
    #  $LPI = \max(a_{ij}) / A \times 100$ 
    lpi = (max_patch_size / total_area) * 100
    return lpi
def compute_aggregation_index(self, matrix: np.ndarray, patches: Dict[int, List]) -> float:
    """计算聚集度指数"""
    #  $AI = [\sum(g_{ii}) / (\max \rightarrow g_{ii})] \times 100$ 
    total_internal_adjacencies = 0
    total_max_adjacencies = 0
    for element_type in range(8):
        element_mask = (matrix == element_type)
        element_count = np.sum(element_mask)
        if element_count == 0:
            continue
        # 计算内部邻接数 (4 邻接)
        internal_adjacencies = 0
        rows, cols = np.where(element_mask)
        for r, c in zip(rows, cols):
            # 检查 4 个邻接方向
            for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nr, nc = r + dr, c + dc
                if (0 <= nr < matrix.shape[0] and 0 <= nc < matrix.shape[1] and
                    matrix[nr, nc] == element_type):
                    internal_adjacencies += 1
        # 最大可能的内部邻接数
        max_adjacencies = 2 * element_count * (element_count - 1)
        total_internal_adjacencies += internal_adjacencies
        total_max_adjacencies += max_adjacencies
    if total_max_adjacencies == 0:
        return 0.0
    ai = (total_internal_adjacencies / total_max_adjacencies) * 100
    return ai
def compute_contagion_index(self, matrix: np.ndarray) -> float:
    """计算蔓延度指数"""
    #  $CONTAG = [1 + \sum(P_i \times g_{ik} / \sum g_{ik} \times \ln(P_i \times g_{ik} / \sum g_{ik})) / (2 \times \ln(m))] \times 100$ 

```

```

# 统计各元素类型的占比
element_counts = np.bincount(matrix.flatten(), minlength=8)
total_pixels = matrix.size
proportions = element_counts / total_pixels
# 统计相邻关系
adjacency_matrix = np.zeros((8, 8))
for i in range(matrix.shape[0]):
    for j in range(matrix.shape[1]):
        current_type = matrix[i, j]
        # 检查右邻和下邻 (避免重复计算)
        for di, dj in [(0, 1), (1, 0)]:
            ni, nj = i + di, j + dj
            if ni < matrix.shape[0] and nj < matrix.shape[1]:
                neighbor_type = matrix[ni, nj]
                adjacency_matrix[current_type, neighbor_type] += 1
                adjacency_matrix[neighbor_type, current_type] += 1
# 计算蔓延度指数
m = 8 # 元素类型数量
total_adjacencies = np.sum(adjacency_matrix)
if total_adjacencies == 0:
    return 0.0
contagion_sum = 0.0
for i in range(8):
    if proportions[i] == 0:
        continue
    for k in range(8):
        if adjacency_matrix[i, k] == 0:
            continue
        gik = adjacency_matrix[i, k]
        sum_gik = np.sum(adjacency_matrix[i, :])
        if sum_gik > 0:
            term = proportions[i] * (gik / sum_gik) * np.log(proportions[i] * (gik /
sum_gik))
            contagion_sum += term
contag = (1 + contagion_sum / (2 * np.log(m))) * 100
return contag

def compute_edge_density(self, matrix: np.ndarray) -> float:
    """计算边缘密度指数"""
    #  $ED = E/A \times 10000$ 
    total_edge_length = 0
    # 计算所有相邻像素间的边缘
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            current_type = matrix[i, j]
            # 检查右邻和下邻
            for di, dj in [(0, 1), (1, 0)]:
                ni, nj = i + di, j + dj
                if ni < matrix.shape[0] and nj < matrix.shape[1]:
                    neighbor_type = matrix[ni, nj]
                    if current_type != neighbor_type:
                        total_edge_length += 1
    total_area = matrix.size
    ed = (total_edge_length / total_area) * 10000
    return ed

def compute_moran_i(self, matrix: np.ndarray) -> float:
    """计算 Moran's I 空间自相关指数"""
    #  $I = \frac{n \sum w_{ij} (x_i - \bar{x})(x_j - \bar{x})}{(\sum w_{ij}) \sum (x_i - \bar{x})^2}$ 
    rows, cols = matrix.shape
    n = rows * cols
    # 展平矩阵

```

```

flat_matrix = matrix.flatten()
mean_value = np.mean(flat_matrix)
# 计算空间权重和空间自相关
numerator = 0.0
denominator = 0.0
total_weights = 0.0
for i in range(rows):
    for j in range(cols):
        idx1 = i * cols + j
        x1 = flat_matrix[idx1]
        # 计算与均值的偏差
        dev1 = x1 - mean_value
        denominator += dev1 * dev1
        # 4 邻接权重
        for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < rows and 0 <= nj < cols:
                idx2 = ni * cols + nj
                x2 = flat_matrix[idx2]
                dev2 = x2 - mean_value
                weight = 1.0 # 邻接权重为 1
                numerator += weight * dev1 * dev2
                total_weights += weight
if denominator == 0 or total_weights == 0:
    return 0.0
moran_i = (n * numerator) / (total_weights * denominator)
return moran_i

def compute_mean_nearest_neighbor_distance(self, matrix: np.ndarray) -> float:
    """计算平均最近邻距离"""
    # 提取开放性元素的坐标
    open_coords = []
    for element_type in self.open_elements:
        coords = np.where(matrix == element_type)
        if len(coords[0]) > 0:
            open_coords.extend(list(zip(coords[0], coords[1])))
    # 提取实体元素的坐标
    solid_coords = []
    for element_type in self.solid_elements:
        coords = np.where(matrix == element_type)
        if len(coords[0]) > 0:
            solid_coords.extend(list(zip(coords[0], coords[1])))
    if not open_coords or not solid_coords:
        return 0.0
    # 计算每个开放元素到最近实体元素的距离
    open_array = np.array(open_coords)
    solid_array = np.array(solid_coords)
    distances = cdist(open_array, solid_array)
    min_distances = np.min(distances, axis=1)
    mean_nearest_neighbor = np.mean(min_distances)
    return mean_nearest_neighbor

def compute_enclosure_ratio(self, matrix: np.ndarray) -> float:
    """计算围合比指数"""
    #  $ER = L_e / L_p$ , 其中  $L_e$  为被实体元素围合的边界长度,  $L_p$  为总边界长度
    total_boundary_length = 0
    enclosed_boundary_length = 0
    # 扫描所有边界像素
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            current_type = matrix[i, j]
            # 检查是否为边界像素 (与不同类型相邻)

```

```

        is_boundary = False
        adjacent_to_solid = False
        for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < matrix.shape[0] and 0 <= nj < matrix.shape[1]:
                neighbor_type = matrix[ni, nj]
                if current_type != neighbor_type:
                    is_boundary = True
                    if neighbor_type in self.solid_elements:
                        adjacent_to_solid = True
        if is_boundary:
            total_boundary_length += 1
            if adjacent_to_solid:
                enclosed_boundary_length += 1
    if total_boundary_length == 0:
        return 0.0
    enclosure_ratio = enclosed_boundary_length / total_boundary_length
    return enclosure_ratio

def compute_connectivity_index(self, patches: Dict[int, List]) -> float:
    """计算连通性指数"""
    #  $CONNECT = [\sum cij / n(n-1)/2] \times 100$ 
    total_connections = 0
    total_possible_connections = 0
    for element_type, patch_list in patches.items():
        n = len(patch_list)
        if n <= 1:
            continue
        # 计算同类型斑块间的连接数（简化处理：距离小于阈值认为连通）
        connections = 0
        threshold = 5.0 # 连通阈值
        for i in range(n):
            for j in range(i + 1, n):
                patch1 = patch_list[i]
                patch2 = patch_list[j]
                # 计算斑块中心距离
                center1 = self._get_patch_center(patch1)
                center2 = self._get_patch_center(patch2)
                distance = np.sqrt((center1[0] - center2[0])**2 + (center1[1] -
center2[1])**2)
                if distance <= threshold:
                    connections += 1
            total_connections += connections
            total_possible_connections += n * (n - 1) // 2
    if total_possible_connections == 0:
        return 0.0
    connectivity = (total_connections / total_possible_connections) * 100
    return connectivity

def _get_patch_center(self, patch: Dict) -> Tuple[float, float]:
    """计算斑块中心坐标"""
    coords = patch['coords']
    center_row = np.mean(coords[0])
    center_col = np.mean(coords[1])
    return (center_row, center_col)

def compute_compactness_index(self, patches: Dict[int, List]) -> float:
    """计算紧凑度指数"""
    #  $CI = A/P^2$ , 其中 A 为面积, P 为周长
    total_compactness = 0.0
    total_patches = 0
    for patch_list in patches.values():
        for patch in patch_list:

```

```

        area = patch['size']
        perimeter = patch['perimeter']
        if perimeter > 0:
            compactness = area / (perimeter * perimeter)
            total_compactness += compactness
            total_patches += 1

    if total_patches == 0:
        return 0.0
    # 返回平均紧凑度
    avg_compactness = total_compactness / total_patches
    return avg_compactness

def compute_integral_connectivity(self, patches: Dict[int, List]) -> float:
    """计算整体连通性指数"""
    #  $IIC = \sum(a_i \times a_j) / (1 + n \times l_{ij}) / AL^2$ 
    # 简化实现：基于斑块面积和距离的连通性度量
    all_patches = []
    for patch_list in patches.values():
        all_patches.extend(patch_list)
    if len(all_patches) <= 1:
        return 0.0
    total_area = sum(patch['size'] for patch in all_patches)
    iic_sum = 0.0
    for i, patch1 in enumerate(all_patches):
        for j, patch2 in enumerate(all_patches):
            if i == j:
                continue
            area1 = patch1['size']
            area2 = patch2['size']
            # 计算距离（简化为斑块中心距离）
            center1 = self._get_patch_center(patch1)
            center2 = self._get_patch_center(patch2)
            distance = np.sqrt((center1[0] - center2[0])**2 + (center1[1] - center2[1])**2)
            # 连接步数（简化处理）
            connection_steps = max(1, int(distance / 10)) # 假设 10 个像素为一步
            iic_sum += (area1 * area2) / (1 + connection_steps)

    if total_area == 0:
        return 0.0
    iic = iic_sum / (total_area * total_area)
    return iic

def evaluate_garden_complete(self, garden_name: str, original_excel: str = None) -> Dict:
    """完整的园林幻境感评价流程"""
    print(f"\n{'='*60}")
    print(f"开始园林幻境感完整评价: {garden_name}")
    print(f"{'='*60}")
    # 拼接完整园林矩阵
    combined_matrix = self.combine_valid_blocks_to_matrix(all_blocks, grid_results)
    if combined_matrix is None:
        print("错误：无法拼接园林矩阵")
        return None
    # 计算所有评价指标
    metrics = self.compute_all_metrics(combined_matrix)
    # 标准化得分
    normalized_scores = self.normalize_metrics(metrics)
    # 计算幻境感得分
    illusion_scores = self.compute_illusion_score(normalized_scores)
    # 保存结果
    save_dir = self.save_evaluation_results(
        garden_name, metrics, normalized_scores,
        illusion_scores, combined_matrix.shape
    )

```

### 问题三相关代码

#### block\_feature\_extractor.py

```
"""
栅格级园林特征提取器
在 10×10 矩阵级别计算定义特征并与图像特征合并
主要功能：
1. 在单个 10×10 矩阵上计算 10 个定义的园林特征
2. 加载 64 维深度图像特征
3. 合并成 74 维综合特征向量
4. 批量处理所有分割块并存储结果
"""

class BlockLevelMetricsCalculator:
    """分割块级别的园林指标计算器"""
    def __init__(self):
        """初始化计算器"""
        # 元素编码映射（与 GardenBlockModeler 保持一致）
        self.element_mapping = {
            '被遮挡': 0,
            '半开放建筑': 1,
            '实体建筑': 2,
            '道路': 3,
            '假山': 4,
            '水体': 5,
            '植物': 6,
            '空旷区域': 7
        }
        # 实体元素（用于开合分析）
        self.solid_elements = {0, 1, 2, 4} # 被遮挡、半开放建筑、实体建筑、假山
        self.open_elements = {3, 5, 6, 7} # 道路、水体、植物、空旷区域
    def identify_connected_components(self, matrix: np.ndarray) -> Dict[int, List]:
        """识别 10×10 矩阵中的连通域（patches）"""
    def _calculate_perimeter(self, binary_mask: np.ndarray) -> int:
        """计算二值图像的周长"""
    # 与问题二计算相同

class BlockFeatureExtractor:
    """分割块级别的综合特征提取器"""
    def __init__(self, use_image_features=True):
        """
        初始化特征提取器

        Args:
            use_image_features: 是否使用 64 维图像特征
        """
        self.use_image_features = use_image_features and FEATURE_EXTRACTOR_AVAILABLE
        self.metrics_calculator = BlockLevelMetricsCalculator()
        # 特征维度配置
        self.defined_feature_dim = 10 # 定义特征维度
        self.image_feature_dim = 64 # 图像特征维度
        self.total_feature_dim = self.defined_feature_dim + (self.image_feature_dim if
self.use_image_features else 0)
```

#### feature\_extractor.py

```
"""
园林空间无监督特征学习系统
```

专门用于学习园林 10×10 矩阵的深度特征表示

主要功能：

1. 构建卷积自编码器进行无监督特征学习
  2. 从园林空间矩阵中提取高维语义特征
  3. 保存训练好的模型和特征向量
- 栅栏矩阵的空间结构特征提取

"""

```
class GardenFeatureExtractor:
```

```
    """园林空间深度特征提取器"""
```

```
    def __init__(self, input_shape=(10, 10, 8), latent_dim=64,
model_name="garden_autoencoder"):
```

```
        """
```

```
        初始化特征提取器
```

```
        Args:
```

```
            input_shape: 输入形状（高度，宽度，通道数）
```

```
            latent_dim: 特征向量维度
```

```
            model_name: 模型名称
```

```
        """
```

```
        self.input_shape = input_shape
```

```
        self.latent_dim = latent_dim
```

```
        self.model_name = model_name
```

```
        # 模型组件
```

```
        self.encoder = None
```

```
        self.decoder = None
```

```
        self.autoencoder = None
```

```
        # 训练历史
```

```
        self.training_history = None
```

```
        # 元素映射
```

```
        self.element_mapping = {
```

```
            '被遮挡': 0, '半开放建筑': 1, '实体建筑': 2, '道路': 3,
```

```
            '假山': 4, '水体': 5, '植物': 6, '空旷区域': 7
```

```
        }
```

```
        self.element_names = ['被遮挡', '半开放建筑', '实体建筑', '道路',  
                               '假山', '水体', '植物', '空旷区域']
```

```
        print(f"园林特征提取器初始化完成")
```

```
        print(f"输入形状: {input_shape}, 特征维度: {latent_dim}")
```

```
    def build_encoder(self):
```

```
        """
```

```
        构建编码器网络
```

```
        专注于捕获园林空间的层次化特征
```

```
        """
```

```
        print("构建编码器网络...")
```

```
        inputs = keras.Input(shape=self.input_shape, name='encoder_input')
```

```
        # 第一层：捕获局部空间模式
```

```
        x = layers.Conv2D(64, (3, 3), activation='relu', padding='same',  
                           name='conv1')(inputs)
```

```
        x = layers.BatchNormalization(name='bn1')(x)
```

```
        x = layers.Dropout(0.1)(x)
```

```
        # 第二层：学习中等复杂度的空间关系
```

```
        x = layers.Conv2D(128, (3, 3), activation='relu', padding='same',  
                           name='conv2')(x)
```

```
        x = layers.BatchNormalization(name='bn2')(x)
```

```
        x = layers.Dropout(0.1)(x)
```

```
        # 第三层：捕获复杂的空间布局模式
```

```
        x = layers.Conv2D(256, (3, 3), activation='relu', padding='same',  
                           name='conv3')(x)
```

```
        x = layers.BatchNormalization(name='bn3')(x)
```

```
        x = layers.Dropout(0.1)(x)
```

```
        # 第四层：深度特征抽象
```

```

x = layers.Conv2D(512, (3, 3), activation='relu', padding='same',
                  name='conv4')(x)
x = layers.BatchNormalization(name='bn4')(x)
# 全局特征聚合
x = layers.GlobalAveragePooling2D(name='global_avg_pool')(x)
# 特征压缩和抽象
x = layers.Dense(256, activation='relu', name='dense1')(x)
x = layers.BatchNormalization(name='bn_dense1')(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(128, activation='relu', name='dense2')(x)
x = layers.BatchNormalization(name='bn_dense2')(x)
x = layers.Dropout(0.2)(x)
# 最终特征表示
features = layers.Dense(self.latent_dim, activation='relu',
                        name='feature_representation')(x)
self.encoder = Model(inputs, features, name='encoder')
print("编码器构建完成")
return self.encoder

def build_decoder(self):
    """
    构建解码器网络
    从特征向量重构原始空间布局
    """
    print("构建解码器网络...")
    latent_inputs = keras.Input(shape=(self.latent_dim,), name='decoder_input')
    # 特征展开
    x = layers.Dense(128, activation='relu', name='decode_dense1')(latent_inputs)
    x = layers.BatchNormalization(name='decode_bn1')(x)
    x = layers.Dropout(0.2)(x)
    x = layers.Dense(256, activation='relu', name='decode_dense2')(x)
    x = layers.BatchNormalization(name='decode_bn2')(x)
    x = layers.Dropout(0.2)(x)
    # 重塑为空间维度
    x = layers.Dense(10 * 10 * 64, activation='relu', name='decode_dense3')(x)
    x = layers.Reshape((10, 10, 64), name='reshape')(x)
    # 反卷积重构空间结构
    x = layers.Conv2DTranspose(256, (3, 3), activation='relu',
                              padding='same', name='deconv1')(x)
    x = layers.BatchNormalization(name='decode_bn3')(x)
    x = layers.Conv2DTranspose(128, (3, 3), activation='relu',
                              padding='same', name='deconv2')(x)
    x = layers.BatchNormalization(name='decode_bn4')(x)

    x = layers.Conv2DTranspose(64, (3, 3), activation='relu',
                              padding='same', name='deconv3')(x)
    x = layers.BatchNormalization(name='decode_bn5')(x)
    # 输出层：重构为元素概率分布
    outputs = layers.Conv2DTranspose(8, (3, 3), activation='softmax',
                                     padding='same', name='output')(x)
    self.decoder = Model(latent_inputs, outputs, name='decoder')
    print("解码器构建完成")
    return self.decoder

def build_autoencoder(self):
    """构建完整的自编码器"""
    if self.encoder is None:
        self.build_encoder()
    if self.decoder is None:
        self.build_decoder()
    # 连接编码器和解码器
    inputs = self.encoder.input

```

```

        encoded = self.encoder(inputs)
        decoded = self.decoder(encoded)
        self.autoencoder = Model(inputs, decoded, name='autoencoder')
        # 编译模型
        self.autoencoder.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.001),
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )
        return self.autoencoder
    def train(self, X, validation_split=0.2, epochs=100, batch_size=32,
              early_stopping_patience=15):
        """
        训练自编码器

        Args:
            X: 训练数据
            validation_split: 验证集比例
            epochs: 训练轮数
            batch_size: 批大小
            early_stopping_patience: 早停耐心值

        Returns:
            training_history: 训练历史
        """
        # 准备回调函数
        callbacks_list = [
            callbacks.EarlyStopping(
                monitor='val_loss',
                patience=early_stopping_patience,
                restore_best_weights=True,
                verbose=1
            ),
            callbacks.ReduceLROnPlateau(
                monitor='val_loss',
                factor=0.5,
                patience=8,
                min_lr=1e-7,
                verbose=1
            ),
            callbacks.ModelCheckpoint(
                filepath=f'checkpoints/{self.model_name}_best.h5',
                monitor='val_loss',
                save_best_only=True,
                save_weights_only=False,
                verbose=1
            )
        ]
        # 创建检查点目录
        os.makedirs('checkpoints', exist_ok=True)
        # 开始训练
        start_time = time.time()
        history = self.autoencoder.fit(
            X, X, # 自编码器的目标是重构输入
            validation_split=validation_split,
            epochs=epochs,
            batch_size=batch_size,
            callbacks=callbacks_list,
            verbose=1,
            shuffle=True

```

```

    )
    training_time = time.time() - start_time
    print(f"训练完成, 耗时: {training_time:.2f}秒")
    self.training_history = history
    return history
def extract_features(self, X):
    """
    提取特征向量

    Args:
        X: 输入数据

    Returns:
        features: 特征向量数组
    """
    if self.encoder is None:
        raise ValueError("请先训练模型或加载预训练模型")
    print(f"提取特征向量, 输入形状: {X.shape}")
    features = self.encoder.predict(X, verbose=1)
    print(f"特征提取完成, 输出形状: {features.shape}")
    return features

```

## garden\_similarity\_analyzer.py

```

"""
江南古典园林异同分析系统
基于 179 维特征向量对园林进行多维度相似性分析、共性特征识别、个性化评估

主要功能:
1. 多层次相似度计算 (专家特征、网络特征、空间特征、抽象特征)
2. 共有美学特征识别与稳定性分析
3. 园林个性化差异量化
4. 聚类分析与类型识别
5. 美学规律量化表达
6. 综合可视化与分析报告
特征向量结构 (179 维):
- 微观特征: 第 1-148 维 (栅格聚合特征: 均值 74 维+标准差 74 维)
- 中观特征: 第 149-164 维 (空间关系 16 维)
- 宏观特征: 第 165-179 维 (专家特征 10 维+网络特征 5 维)
"""
class GardenSimilarityAnalyzer:
    """江南古典园林异同分析器"""
    def __init__(self):
        """初始化分析器"""
        self.vector_builder = GardenVectorBuilder()
        self.output_dir = "GardenAnalysis"
        # 确保输出目录存在
        if not os.path.exists(self.output_dir):
            os.makedirs(self.output_dir)
            print(f"创建分析输出目录: {self.output_dir}")
        # 特征维度配置
        self.micro_range = (0, 148)      # 微观特征: 1-148 维 (索引 0-147)
        self.meso_range = (148, 164)     # 中观特征: 149-164 维 (索引 148-163)
        self.macro_expert_range = (164, 174) # 专家特征: 165-174 维 (索引 164-173)
        self.macro_network_range = (174, 179) # 网络特征: 175-179 维 (索引 174-178)
        # 特征名称配置
        self.expert_feature_names = [
            'shannon_diversity', 'aggregation_index', 'contagion_index',

```

```

        'edge_density', 'moran_i', 'mean_nearest_neighbor',
        'enclosure_ratio', 'connectivity_index', 'compactness_index',
        'integral_connectivity'
    ]
    self.network_feature_names = [
        'network_node_count', 'network_edge_count', 'network_avg_degree',
        'network_max_degree', 'network_degree_std'
    ]
    self.meso_feature_names = [
        'adjacent_similarity_mean', 'adjacent_similarity_std',
        'gradient_east', 'gradient_west', 'gradient_south', 'gradient_north',
        'pca_component_1', 'pca_component_2', 'pca_component_3',
        'cluster_proportion_1', 'cluster_proportion_2', 'cluster_proportion_3',
        'variance_mean', 'skewness_mean', 'kurtosis_mean', 'convex_hull_log_volume'
    ]
    # 分析结果存储
    self.garden_data = None
    self.similarity_results = {}
    self.commonality_results = {}
    self.personality_results = {}
    self.clustering_results = {}
def analyze_similarity(self) -> Dict[str, Any]:
    """
    多层次相似度分析

    Returns:
        Dict: 相似度分析结果
    """
    garden_names = self.garden_data['success_gardens']
    n_gardens = len(garden_names)
    # 构建向量矩阵
    vectors = np.array([self.garden_data['garden_vectors'][name]['vector']
                        for name in garden_names])
    print(f"分析 {n_gardens} 个园林的相似性")
    print(f"向量维度: {vectors.shape}")
    # 1. 专家特征相似度 (165-174 维, 索引 164-173)
    expert_vectors = vectors[:, self.macro_expert_range[0]:self.macro_expert_range[1]]
    expert_similarity = cosine_similarity(expert_vectors)
    print(f"专家特征相似度计算完成 - 基于{expert_vectors.shape[1]}维特征")
    # 2. 网络特征相似度 (175-179 维, 索引 174-178)
    network_vectors = vectors[:,
self.macro_network_range[0]:self.macro_network_range[1]]
    network_similarity = cosine_similarity(network_vectors)
    print(f"网络特征相似度计算完成 - 基于{network_vectors.shape[1]}维特征")
    # 3. 空间关系相似度 (149-164 维, 索引 148-163)
    spatial_vectors = vectors[:, self.meso_range[0]:self.meso_range[1]]
    spatial_similarity = cosine_similarity(spatial_vectors)
    print(f"空间关系相似度计算完成 - 基于{spatial_vectors.shape[1]}维特征")
    # 4. 抽象特征整体相似度 (1-148 维, 索引 0-147)
    abstract_vectors = vectors[:, self.micro_range[0]:self.micro_range[1]]
    abstract_similarity = cosine_similarity(abstract_vectors)
    print(f"抽象特征相似度计算完成 - 基于{abstract_vectors.shape[1]}维特征")
    # 5. 综合相似度 (加权平均)
    # 权重设计: 专家特征 40% (美学评价最重要)、网络 20%、空间 20%、抽象 20%
    comprehensive_similarity = (0.4 * expert_similarity +
                                0.2 * network_similarity +
                                0.2 * spatial_similarity +
                                0.2 * abstract_similarity)
    # 6. 相似度统计分析
    similarity_stats = self._analyze_similarity_statistics(

```

```

        expert_similarity, network_similarity, spatial_similarity,
        abstract_similarity, comprehensive_similarity, garden_names)
self.similarity_results = {
    'garden_names': garden_names,
    'expert_similarity': expert_similarity,
    'network_similarity': network_similarity,
    'spatial_similarity': spatial_similarity,
    'abstract_similarity': abstract_similarity,
    'comprehensive_similarity': comprehensive_similarity,
    'similarity_stats': similarity_stats
}
return self.similarity_results

def _analyze_similarity_statistics(self, expert_sim, network_sim, spatial_sim,
                                abstract_sim, comprehensive_sim, garden_names) -> Dict:
    """分析相似度统计信息"""

def analyze_commonality(self) -> Dict[str, Any]:
    """
    共有美学特征识别分析

    Returns:
        Dict: 共性特征分析结果
    """

def analyze_personality(self) -> Dict[str, Any]:
    """
    园林个性化差异量化分析

    Returns:
        Dict: 个性化分析结果
    """

def analyze_clustering(self, n_clusters: int = 3) -> Dict[str, Any]:
    """
    园林聚类分析

    Args:
        n_clusters: 聚类数量

    Returns:
        Dict: 聚类分析结果
    """

def _analyze_cluster_significance(self, vectors, labels, n_clusters) -> List[Dict]:
    """分析聚类的显著性特征 (ANOVA) """

def run_complete_analysis(self, file_path: str = None, n_clusters: int = 3) -> Dict[str,
Any]:
    """
    运行完整的园林异同分析流程

    Args:
        file_path: 向量数据文件路径
        n_clusters: 聚类数量

    Returns:
        Dict: 完整分析结果
    """
    print("=" * 60)
    print("江南古典园林异同分析系统")
    print("=" * 60)

```

```

# 1. 加载数据
if not self.load_garden_vectors(file_path):
    return {}
# 2. 相似性分析
similarity_results = self.analyze_similarity()
# 3. 共性特征分析
commonality_results = self.analyze_commonality()
# 4. 个性化分析
personality_results = self.analyze_personality()
# 5. 聚类分析
clustering_results = self.analyze_clustering(n_clusters)
# 6. 生成可视化
self.generate_visualizations()
# 7. 生成报告
report_text = self.generate_report()
# 8. 保存完整结果
complete_results = {
    'garden_data': self.garden_data,
    'similarity_results': similarity_results,
    'commonality_results': commonality_results,
    'personality_results': personality_results,
    'clustering_results': clustering_results,
    'analysis_report': report_text,
    'analysis_timestamp': pd.Timestamp.now().isoformat()
}
# 保存结果
timestamp = pd.Timestamp.now().strftime('%Y%m%d_%H%M%S')
result_file = os.path.join(self.output_dir, f"complete_analysis_{timestamp}.pkl")
with open(result_file, 'wb') as f:
    pickle.dump(complete_results, f)
return complete_results

```

## garden\_vector\_builder.py

```

"""
园林多尺度多模态特征向量构建器
实现园林的 179 维特征向量计算，包含微观、中观、宏观三个尺度的特征

特征体系：
- 微观尺度（148 维）：栅格聚合特征（均值 74 维 + 标准差 74 维）
- 中观尺度（16 维）：空间关系特征（6 维）+ 点云分布特征（10 维）
- 宏观尺度（15 维）：专家定义特征（10 维）+ 网络结构特征（5 维）
总维度：179 维
"""

class GardenVectorBuilder:
    """园林多尺度多模态特征向量构建器"""
    def __init__(self):
        """初始化向量构建器"""
        self.data_loader = MultiSourceDataLoader()
        self.output_dir = "GardenVector"
        # 确保输出目录存在
        if not os.path.exists(self.output_dir):
            os.makedirs(self.output_dir)
            print(f"创建输出目录: {self.output_dir}")
        # 特征维度配置
        self.micro_dim = 148    # 微观特征维度
        self.meso_dim = 16     # 中观特征维度
        self.macro_dim = 15    # 宏观特征维度

```

```

self.total_dim = 179    # 总维度
def build_garden_vectors(self, garden_names: List[str]) -> Dict[str, Any]:
    """
    为指定园林列表构建多尺度多模态特征向量

    Args:
        garden_names: 园林名称列表

    Returns:
        Dict: 包含所有园林向量和元数据的字典
    """
    print(f"=== 开始构建园林向量 ===")
    print(f"目标园林: {garden_names}")
    # 1. 加载多源数据
    print("\n1. 加载多源数据...")
    data_results = self.data_loader.load_all_data(garden_names)
    if not data_results['loaded_gardens']:
        print("错误: 没有成功加载任何园林数据")
        return None
    # 筛选栅格级数据
    if data_results['block_level_data']:
        filtered_block_data = self.data_loader.filter_block_data_by_gardens(
            data_results['block_level_data'], data_results['loaded_gardens'])
        data_results['block_level_data'] = filtered_block_data
    # 2. 为每个园林构建特征向量
    print(f"\n2. 构建特征向量...")
    garden_vectors = {}
    success_gardens = []
    failed_gardens = []
    for garden_name in data_results['loaded_gardens']:
        print(f"\n 处理园林: {garden_name}")
        try:
            # 构建单个园林的特征向量
            vector_result = self._build_single_garden_vector(
                garden_name, data_results)
            if vector_result:
                garden_vectors[garden_name] = vector_result
                success_gardens.append(garden_name)
                print(f"    ✓ 向量构建成功: {len(vector_result['vector'])}维")
            else:
                failed_gardens.append(garden_name)
                print(f"    ✗ 向量构建失败")
        except Exception as e:
            failed_gardens.append(garden_name)
            print(f"    ✗ 向量构建异常: {e}")
    # 3. 整理结果
    results = {
        'garden_vectors': garden_vectors,
        'success_gardens': success_gardens,
        'failed_gardens': failed_gardens,
        'vector_config': {
            'total_dim': self.total_dim,
            'micro_dim': self.micro_dim,
            'meso_dim': self.meso_dim,
            'macro_dim': self.macro_dim,
            'feature_names': self._get_feature_names()
        },
        'data_summary': self.data_loader.get_data_summary(data_results)
    }
    print(f"\n=== 向量构建完成 ===")

```

```

print(f"成功构建: {len(success_gardens)} 个园林向量")
print(f"构建失败: {len(failed_gardens)} 个园林向量")
if failed_gardens:
    print(f"失败园林: {failed_gardens}")
# 4. 保存结果
if success_gardens:
    self._save_garden_vectors(results)
return results

def _build_single_garden_vector(self, garden_name: str, data_results: Dict) ->
Optional[Dict]:
    """
    构建单个园林的 179 维特征向量

    Args:
        garden_name: 园林名称
        data_results: 多源数据结果

    Returns:
        Dict: 包含向量和详细信息的字典
    """
    try:
        # 获取该园林的数据
        network_data = data_results['network_data'].get(garden_name)
        garden_level_data = data_results['garden_level_data'].get(garden_name)
        block_level_data = data_results['block_level_data']
        # 获取该园林的栅格数据
        garden_block_indices = []
        garden_block_positions = []
        for i, meta in enumerate(block_level_data['metadata']):
            if meta.get('garden_name') == garden_name:
                garden_block_indices.append(i)
                # 获取栅格位置 (中心坐标)
                center = meta.get('center', (0, 0))
                garden_block_positions.append(center)
        # 提取该园林的栅格特征矩阵
        garden_grid_features = block_level_data['features'][garden_block_indices] # (N,
74)
        garden_grid_positions = np.array(garden_block_positions) # (N, 2)
        # 1. 提取微观特征 (148 维)
        micro_features = self._extract_micro_features(garden_grid_features)
        # 2. 提取中观特征 (16 维)
        meso_features = self._extract_meso_features(garden_grid_features,
garden_grid_positions)
        # 3. 提取宏观特征 (15 维)
        macro_features = self._extract_macro_features(garden_level_data, network_data)
        # 4. 组合为完整向量
        complete_vector = np.concatenate([micro_features, meso_features, macro_features])
        # 5. 构建多模态特征划分 (这个有点鸡肋)
        multimodal_features = self._extract_multimodal_features(
            micro_features, meso_features, macro_features)
        # 验证维度
        assert len(complete_vector) == self.total_dim, f"向量维度错误:
{len(complete_vector)} != {self.total_dim}"
        assert len(micro_features) == self.micro_dim, f"微观特征维度错误:
{len(micro_features)} != {self.micro_dim}"
        assert len(meso_features) == self.meso_dim, f"中观特征维度错误:
{len(meso_features)} != {self.meso_dim}"
        assert len(macro_features) == self.macro_dim, f"宏观特征维度错误:
{len(macro_features)} != {self.macro_dim}"
        vector_result = {

```

```

        'vector': complete_vector,          # 完整的 179 维向量
        'micro_features': micro_features,   # 148 维微观特征
        'meso_features': meso_features,     # 16 维中观特征
        'macro_features': macro_features,   # 15 维宏观特征
        'multimodal_features': multimodal_features, # 多模态特征划分
        'metadata': {
            'garden_name': garden_name,
            'vector_dim': len(complete_vector),
            'grid_count': len(garden_block_indices),
            'network_nodes': network_data['node_count'],
            'network_edges': network_data['edge_count'],
            'total_score': garden_level_data['total_score']
        }
    }
    return vector_result
except Exception as e:
    print(f"    异常: 构建向量失败 - {e}")
    return None

def _extract_micro_features(self, grid_features: np.ndarray) -> np.ndarray:
    """
    提取微观尺度特征（栅格聚合特征） - 148 维

    Args:
        grid_features: 栅格特征矩阵 (N, 74)

    Returns:
        np.ndarray: 148 维微观特征 = 均值 74 维 + 标准差 74 维
    """
    # 计算 74 维特征在所有栅格上的均值和标准差
    mean_features = np.mean(grid_features, axis=0)    # (74,)
    std_features = np.std(grid_features, axis=0)      # (74,)
    # 拼接为 148 维特征
    micro_features = np.concatenate([mean_features, std_features]) # (148,)
    return micro_features

def _extract_meso_features(self, grid_features: np.ndarray, grid_positions: np.ndarray)
-> np.ndarray:
    """
    提取中观尺度特征（空间关系特征） - 16 维

    Args:
        grid_features: 栅格特征矩阵 (N, 74)
        grid_positions: 栅格位置矩阵 (N, 2)

    Returns:
        np.ndarray: 16 维中观特征 = 基础空间关系 6 维 + 点云分布特征 10 维
    """
    features = []
    # ===== 基础空间关系特征（6 维）=====
    # 1. 相邻相似度统计（2 维）
    adjacent_similarities = self._compute_adjacent_similarities(grid_features,
grid_positions)
    if len(adjacent_similarities) > 0:
        features.extend([np.mean(adjacent_similarities),
np.std(adjacent_similarities)])
    else:
        features.extend([0.0, 0.0]) # 如果没有相邻栅格，使用默认值
    # 2. 方向梯度特征（4 维）
    directional_gradients = self._compute_directional_gradients(grid_features,
grid_positions)

```

```

features.extend(directional_gradients) # [east, west, south, north]
# ===== 点云分布特征 (10 维) =====
# 3. PCA 主成分 (3 维)
pca_features = self._compute_pca_features(grid_features)
features.extend(pca_features)
# 4. 聚类特征 (3 维)
cluster_features = self._compute_cluster_features(grid_features)
features.extend(cluster_features)
# 5. 分布形状 (3 维)
shape_features = self._compute_shape_features(grid_features)
features.extend(shape_features)
# 6. 离散度 (1 维)
dispersion_feature = self._compute_dispersion_feature(grid_positions)
features.append(dispersion_feature)
meso_features = np.array(features) # (16,)
return meso_features

def _extract_macro_features(self, garden_level_data: Dict, network_data: Dict) ->
np.ndarray:
    """
    提取宏观尺度特征 (整体特征) - 15 维

    Args:
        garden_level_data: 园林级评估数据
        network_data: 游园网络数据

    Returns:
        np.ndarray: 15 维宏观特征 = 专家定义特征 10 维 + 网络结构特征 5 维
    """
    features = []
    # ===== 专家定义特征 (10 维) =====
    feature_names = garden_level_data['feature_names'] # 10 个特征名称
    normalized_scores = garden_level_data['normalized_scores']
    # 按顺序提取 10 个专家特征
    for feature_name in feature_names:
        score = normalized_scores.get(feature_name, 0.0)
        features.append(score)
    # ===== 网络结构特征 (5 维) =====
    graph = network_data['macro_graph']
    # 1. 节点数
    node_count = graph.number_of_nodes()
    features.append(node_count)
    # 2. 边数
    edge_count = graph.number_of_edges()
    features.append(edge_count)
    # 3. 平均度
    if node_count > 0:
        degrees = [d for n, d in graph.degree()]
        avg_degree = np.mean(degrees) if degrees else 0.0
        max_degree = max(degrees) if degrees else 0.0
        std_degree = np.std(degrees) if len(degrees) > 1 else 0.0
    else:
        avg_degree = 0.0
        max_degree = 0.0
        std_degree = 0.0
    features.append(avg_degree)
    # 4. 最大度中心性
    features.append(max_degree)
    # 5. 度分布标准差
    features.append(std_degree)

```

```

macro_features = np.array(features) # (15,)
return macro_features
def _extract_multimodal_features(self, micro_features: np.ndarray,
                                meso_features: np.ndarray,
                                macro_features: np.ndarray) -> Dict[str, np.ndarray]:
    """
    提取多模态特征划分

    根据园林特征的语义含义重新划分为三种模态：
    - 几何空间模态：空间关系、形状、位置等几何特征
    - 景观语义模态：景观类型、美学评价、功能性等语义特征
    - 尺度层次模态：不同尺度的统计特征

    Returns:
        Dict: 包含三种模态特征的字典
    """
    # ===== 几何空间模态 (53 维) =====
    # 包含：空间关系、几何形状、网络拓扑等几何相关特征
    geometric_components = []
    # 1. 栅格空间分布的几何统计 (20 维)
    # 选择与空间分布相关的前 20 个均值特征
    geometric_components.append(micro_features[:20]) # 20 维
    # 2. 栅格空间分布的几何变异 (20 维)
    # 选择对应的前 20 个标准差特征
    geometric_components.append(micro_features[20:40]) # 20 维
    # 3. 中观空间关系特征 (8 维)
    # 相邻相似度(2 维) + 方向梯度(4 维) + 离散度(1 维) + PCA 第一主成分(1 维)
    spatial_features = np.concatenate([
        meso_features[:6], # 相邻相似度 + 方向梯度, 6 维
        [meso_features[15]], # 离散度, 1 维
        [meso_features[6]] # PCA 第一主成分, 1 维
    ])
    geometric_components.append(spatial_features) # 8 维
    # 4. 网络几何结构特征 (5 维)
    # 网络的拓扑结构特征
    geometric_components.append(macro_features[10:15]) # 5 维
    geometric_features = np.concatenate(geometric_components) # 20+20+8+5=53 维
    # ===== 景观语义模态 (116 维) =====
    # 包含：景观类型、美学特征、功能评价等语义相关特征
    semantic_components = []
    # 1. 栅格景观语义统计 (54 维)
    # 选择与景观类型相关的后 54 个均值特征
    semantic_components.append(micro_features[40:94]) # 54 维
    # 2. 栅格景观语义变异 (54 维)
    # 选择对应的后 54 个标准差特征
    semantic_components.append(micro_features[94:148]) # 54 维
    # 3. 专家美学评价特征 (10 维)
    # 园林专家定义的美学和功能性评价指标
    semantic_components.append(macro_features[:10]) # 10 维
    # 注意：116 = 54 + 54 + 10 = 118, 需要调整
    # 实际取前 52 维 + 后 52 维 + 专家特征 10 维 = 114 维
    # 为了凑够 116 维, 再加上 2 维 PCA 特征
    semantic_components = []
    semantic_components.append(micro_features[22:74]) # 52 维
    semantic_components.append(micro_features[74:126]) # 52 维
    semantic_components.append(macro_features[:10]) # 10 维
    semantic_components.append(meso_features[7:9]) # PCA 第 2,3 主成分, 2 维
    semantic_features = np.concatenate(semantic_components) # 52+52+10+2=116 维
    # ===== 尺度层次模态 (10 维) =====
    # 包含：不同尺度的代表性统计特征

```

```

scale_components = []
# 1. 微观尺度代表特征 (4 维)
# 选择具有代表性的微观特征: 首尾均值和标准差
micro_repr = [
    micro_features[0],      # 第 1 个特征的均值
    micro_features[36],     # 第 37 个特征的均值
    micro_features[74],     # 第 1 个特征的标准差
    micro_features[110]     # 第 37 个特征的标准差
]
scale_components.extend(micro_repr) # 4 维
# 2. 中观尺度代表特征 (3 维)
# 选择中观层面最重要的 3 个特征
meso_repr = [
    meso_features[0],       # 相邻相似度均值
    meso_features[12],      # 分布形状-方差均值
    meso_features[15]       # 离散度
]
scale_components.extend(meso_repr) # 3 维
# 3. 宏观尺度代表特征 (3 维)
# 选择宏观层面最重要的 3 个特征
macro_repr = [
    macro_features[0],      # 第 1 个专家特征
    macro_features[9],      # 第 10 个专家特征
    macro_features[12]      # 网络平均度
]
scale_components.extend(macro_repr) # 3 维
scale_features = np.array(scale_components) # 4+3+3=10 维
# 验证维度
assert len(geometric_features) == 53, f"几何空间模态维度错误: {len(geometric_features)} != 53"
assert len(semantic_features) == 116, f"景观语义模态维度错误: {len(semantic_features)} != 116"
assert len(scale_features) == 10, f"尺度层次模态维度错误: {len(scale_features)} != 10"
return {
    'geometric_modal': geometric_features,    # 几何空间模态 53 维
    'semantic_modal': semantic_features,      # 景观语义模态 116 维
    'scale_modal': scale_features             # 尺度层次模态 10 维
}

# ===== 中观特征计算辅助函数 =====
def _compute_adjacent_similarities(self, grid_features: np.ndarray,
                                   grid_positions: np.ndarray) -> List[float]:
    """计算相邻栅格特征相似度"""
    if len(grid_positions) < 2:
        return []
    # 计算位置距离矩阵
    pos_distances = cdist(grid_positions, grid_positions, 'euclidean')
    # 找到每个栅格的最近邻 (排除自己)
    similarities = []
    for i in range(len(grid_features)):
        # 找到最近的邻居
        neighbor_distances = pos_distances[i].copy()
        neighbor_distances[i] = np.inf # 排除自己
        nearest_neighbor_idx = np.argmin(neighbor_distances)
        # 计算特征相似度 (使用余弦相似度)
        feature_i = grid_features[i]
        feature_j = grid_features[nearest_neighbor_idx]
        # 避免除零错误
        norm_i = np.linalg.norm(feature_i)
        norm_j = np.linalg.norm(feature_j)
        if norm_i > 0 and norm_j > 0:

```

```

        similarity = np.dot(feature_i, feature_j) / (norm_i * norm_j)
        similarities.append(similarity)
    return similarities

def _compute_directional_gradients(self, grid_features: np.ndarray,
                                   grid_positions: np.ndarray) -> List[float]:
    """计算四个主方向的特征梯度均值"""
    if len(grid_positions) < 2:
        return [0.0, 0.0, 0.0, 0.0]
    gradients = {'east': [], 'west': [], 'south': [], 'north': []}
    for i in range(len(grid_features)):
        x_i, y_i = grid_positions[i]
        feature_i = grid_features[i]
        # 找到四个方向的邻居
        for j in range(len(grid_features)):
            if i == j:
                continue
            x_j, y_j = grid_positions[j]
            feature_j = grid_features[j]
            # 计算方向
            dx = x_j - x_i
            dy = y_j - y_i
            if abs(dx) > abs(dy): # 主要是水平方向
                if dx > 0: # 东
                    gradient = np.mean(np.abs(feature_j - feature_i))
                    gradients['east'].append(gradient)
                else: # 西
                    gradient = np.mean(np.abs(feature_j - feature_i))
                    gradients['west'].append(gradient)
            else: # 主要是垂直方向
                if dy > 0: # 北
                    gradient = np.mean(np.abs(feature_j - feature_i))
                    gradients['north'].append(gradient)
                else: # 南
                    gradient = np.mean(np.abs(feature_j - feature_i))
                    gradients['south'].append(gradient)
    # 计算各方向梯度均值
    result = []
    for direction in ['east', 'west', 'south', 'north']:
        if gradients[direction]:
            result.append(np.mean(gradients[direction]))
        else:
            result.append(0.0)
    return result

def _compute_pca_features(self, grid_features: np.ndarray) -> List[float]:
    """计算 PCA 主成分特征"""
    if len(grid_features) < 3: # PCA 需要足够的样本
        return [0.0, 0.0, 0.0]
    try:
        pca = PCA(n_components=3)
        pca.fit(grid_features)
        # 返回前 3 个主成分的方差解释比例
        return pca.explained_variance_ratio_.tolist()
    except:
        return [0.0, 0.0, 0.0]

def _compute_cluster_features(self, grid_features: np.ndarray) -> List[float]:
    """计算 K-means 聚类特征"""
    if len(grid_features) < 3:
        return [0.0, 0.0, 1.0] # 如果样本太少, 假设都在一个聚类
    try:
        kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

```

```

        labels = kmeans.fit_predict(grid_features)
        # 计算各聚类的大小比例
        unique_labels, counts = np.unique(labels, return_counts=True)
        proportions = counts / len(grid_features)
        # 确保返回 3 个值
        result = [0.0, 0.0, 0.0]
        for i, prop in enumerate(proportions):
            if i < 3:
                result[i] = prop
        return result
    except:
        return [0.0, 0.0, 1.0]
def _compute_shape_features(self, grid_features: np.ndarray) -> List[float]:
    """计算分布形状特征"""
    if len(grid_features) < 2:
        return [0.0, 0.0, 0.0]
    # 计算整体方差、偏度、峰度的均值
    variance_mean = np.mean(np.var(grid_features, axis=0))
    try:
        skew_mean = np.mean([skew(grid_features[:, i]) for i in
range(grid_features.shape[1])])
        kurt_mean = np.mean([kurtosis(grid_features[:, i]) for i in
range(grid_features.shape[1])])
    except:
        skew_mean = 0.0
        kurt_mean = 0.0
    return [variance_mean, skew_mean, kurt_mean]
def _compute_dispersion_feature(self, grid_positions: np.ndarray) -> float:
    """计算点云凸包体积的对数值"""
    if len(grid_positions) < 3:
        return 0.0
    try:
        from scipy.spatial import ConvexHull
        hull = ConvexHull(grid_positions)
        volume = hull.volume
        # 避免 log(0)
        return np.log(volume + 1e-10)
    except:
        # 如果凸包计算失败, 使用位置分布的方差作为替代
        return np.log(np.var(grid_positions) + 1e-10)

```